

System Design Masterclass

Study notes · 16 sessions

Written in my own words from Arpit Bhayani's session PDFs (view-only). Not a copy of the slides.
Built 2026-10-10.

Contents

1. 01-01 · Arpit Bhayani's session notes: Foundational topics in System Design

Map of the session

1. How to approach a design: it's like sketching
2. Case study: Online/Offline indicator
3. Case study: Multi-user blogging platform (Medium)

What these notes add beyond the slides

One-page memory card

2. 01-02 · Arpit Bhayani's session notes: Foundational topics (Scaling, Delegation, Communication)

Map of the session

1. Scaling
2. Delegation
3. Communication

What these notes add beyond the slides

One-page memory card

3. 02-01 · Arpit Bhayani's session notes: Databases (Pessimistic locking, Airline check-in, KV store on SQL)

Map of the session

1. Pessimistic locking
2. Airline check-in system
3. Designing a KV store on a relational database

What these notes add beyond the slides

One-page memory card

4. 02-02 · Arpit Bhayani's session notes: Non-relational databases + Slack's realtime text communication

Map of the session

1. Non-relational (NoSQL) databases
2. Designing Slack's realtime text communication

What these notes add beyond the slides

One-page memory card

5. 03-01 · Arpit Bhayani's session notes: Distributed systems, Load balancer design, Remote & distributed locks

Map of the session

1. Approaching distributed systems
2. Designing a load balancer
3. Remote locks
4. Distributed locks: Redlock

What these notes add beyond the slides

One-page memory card

6. 03-02 · Arpit Bhayani's session notes: ID generation (from a function to Snowflake)

Map of the session

1. The problem
2. Building the function step by step
3. Monotonically increasing IDs
4. Real-world approaches

What these notes add beyond the slides

One-page memory card

7. 04-01 · Arpit Bhayani's session notes: CDN, Photo uploads at scale, Private photos, Gravatar

Map of the session

1. How a CDN works
2. Image upload service
3. Gravatar

What these notes add beyond the slides

One-page memory card

8. 04-02 · Arpit Bhayani's session notes: Hashtag service + Newly-unread message indicator

Map of the session

1. Hashtag service
2. Newly-unread message indicator

What these notes add beyond the slides

One-page memory card

9. 05-01 · Arpit Bhayani's session notes: Designing a distributed cache (Redis / DiceDB internals, consistent hashing)

Map of the session

1. Requirements and brainstorm
2. Single-node cache
3. Distributed cache

What these notes add beyond the slides

One-page memory card

10. 05-02 · Arpit Bhayani's session notes: Word dictionary without a DB, a superfast KV store (Bitcask), WAL recovery

Map of the session

1. Word dictionary without any database
2. A superfast KV store
3. Log (WAL) recovery

What these notes add beyond the slides

One-page memory card

11. 06-01 · Arpit Bhayani's session notes: LSM trees, Multi-tiered order storage (Amazon), Event ingestion

Map of the session

1. LSM trees: attempting to beat Bitcask
2. Designing a cost-efficient orders system for Amazon
3. Event ingestion (e.g. YouTube video impressions)

What these notes add beyond the slides

One-page memory card

12. 06-02 · Arpit Bhayani's session notes: High-throughput systems: Live streaming (CDN + ads) and Designing S3

Map of the session

1. Live streaming
2. Designing S3

What these notes add beyond the slides

One-page memory card

13. 07-01 · Arpit Bhayani's session notes: Information retrieval: Recent searches, Cricbuzz live commentary, Elasticsearch in production

Map of the session

1. Designing "recent searches"
2. Designing Cricbuzz's live text commentary
3. Running Elasticsearch in production (Unacademy)

What these notes add beyond the slides

One-page memory card

14. 07-02 · Arpit Bhayani's session notes: Distributed task scheduler + Flash sale

Map of the session

1. Distributed task scheduler (without retries)
2. Flash sale

What these notes add beyond the slides

One-page memory card

15. 08-01 · Arpit Bhayani's session notes: Impression counting (HyperLogLog) + Remote file sync (Dropbox)

Map of the session

1. Impression counting
2. Remote file sync (Dropbox)

What these notes add beyond the slides

One-page memory card

16. 08-02 · Arpit Bhayani's session notes: Geo-proximity & ride hailing (Uber/Gojek) + User affinity (follow graph)

Map of the session

1. Geo-proximity
2. Ride hailing (Gojek / Uber)
3. User affinity (follow / following)

What these notes add beyond the slides

One-page memory card

17. Flashcards (192)

01-01 · Foundational topics

01-02 · Scaling, Delegation, Communication

02-01 · Databases: locking, check-in, KV on SQL

02-02 · NoSQL + Slack realtime

03-01 · Load balancer, remote & distributed locks

03-02 · ID generation

04-01 · CDN, photo uploads, Gravatar

04-02 · Hashtag service + unread indicator

05-01 · Distributed cache

05-02 · Dictionary without DB, Bitcask, WAL recovery

06-01 · LSM trees, tiered storage, event ingestion

06-02 · Live streaming + Designing S3

07-01 · Recent searches, Cricbuzz, ES in production

07-02 · Task scheduler + flash sale

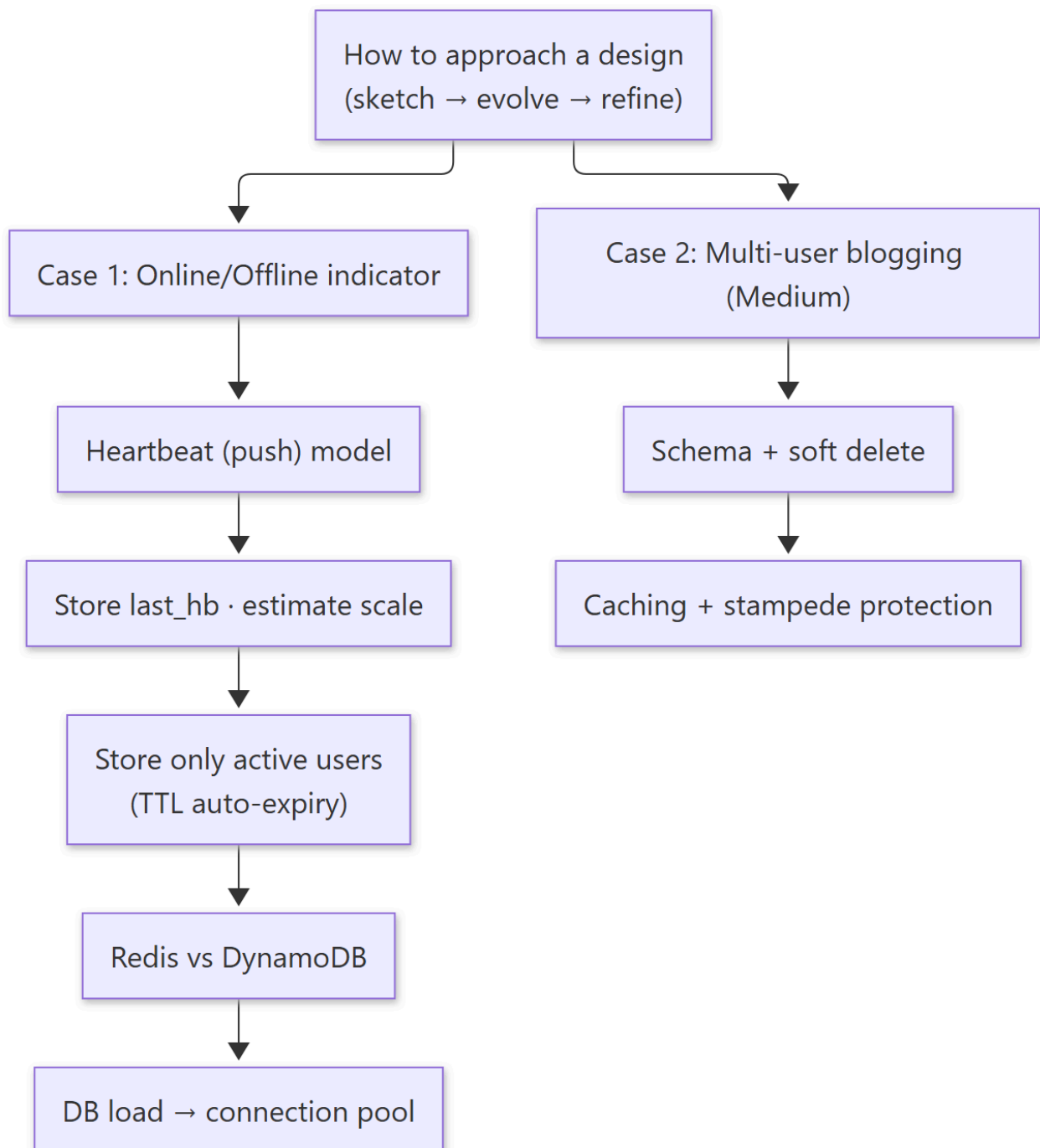
08-01 · Impression counting + Dropbox sync

08-02 · Geo-proximity, ride hailing, follow graph

01-01 · Arpit Bhayani's session notes: Foundational topics in System Design

Source: 01-01.pdf (System Design Masterclass, Google Drive, view-only). Study notes in my own words, not a copy. Pages covered: 1–19 of 19. Unreadable pages: none. Pages 2–4 are course logistics and student testimonials; they are skipped here.

Map of the session

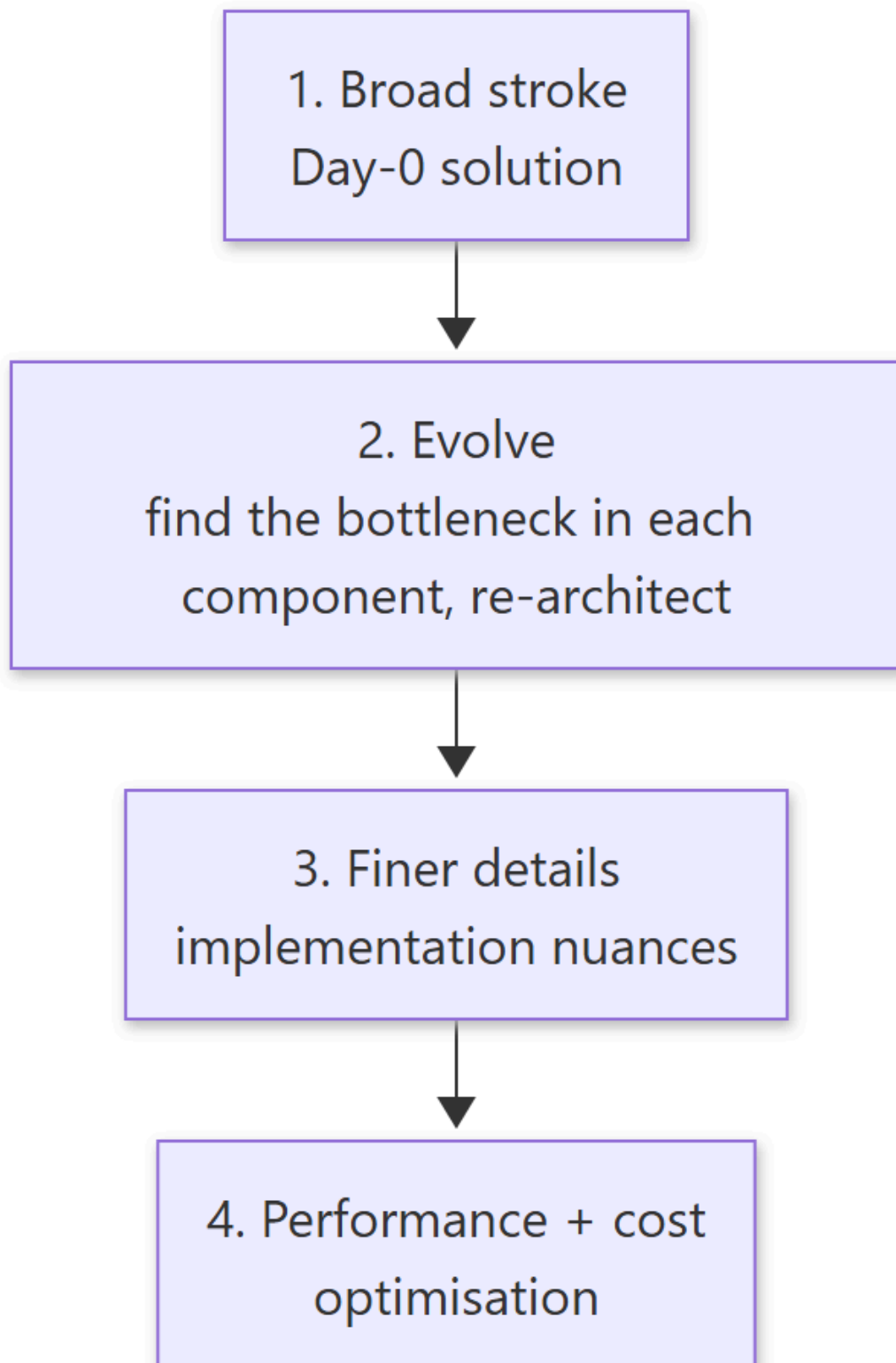


Two mental models run through the whole session:

Mental model	What it means
Identify patterns, Lego blocks	Most systems are the same few blocks (clients → service → DB). The craft is in the details, not the boxes.
Framework of the Opposites	For every decision, ask "what if we did the exact opposite?" (push vs pull, store vs don't store, do it ourselves vs offload). Evaluate all n paths, then pick.

1. How to approach a design: it's like sketching

 Anchor: **Broad strokes first, details last.** Day-0 design is a rough outline; you refine in passes.



⚙️ Mechanics

- Almost every system *looks* like one diagram: users → a fleet of API servers → a database. The difference between systems lives in the details.
- System design is **not about drawing boxes**, and it is **not templated**. Every choice is a trade-off, and you discuss the trade-off for every single decision.
- Day 1 deliberately ignores scaling, to learn the approach first.

 **So what:** In an interview, state the Day-0 design quickly, then spend your time on the bottlenecks and trade-offs. That is where the signal is.

 **Recall:** What are the four passes of "design is like sketching"?

Broad stroke (Day-0) → evolve (find each component's bottleneck, re-architect) → finer implementation details → performance and cost optimisation.

2. Case study: Online/Offline indicator

Requirement: for a list of users, show a dot that is green (online) or grey (offline). Kept primitive on purpose.

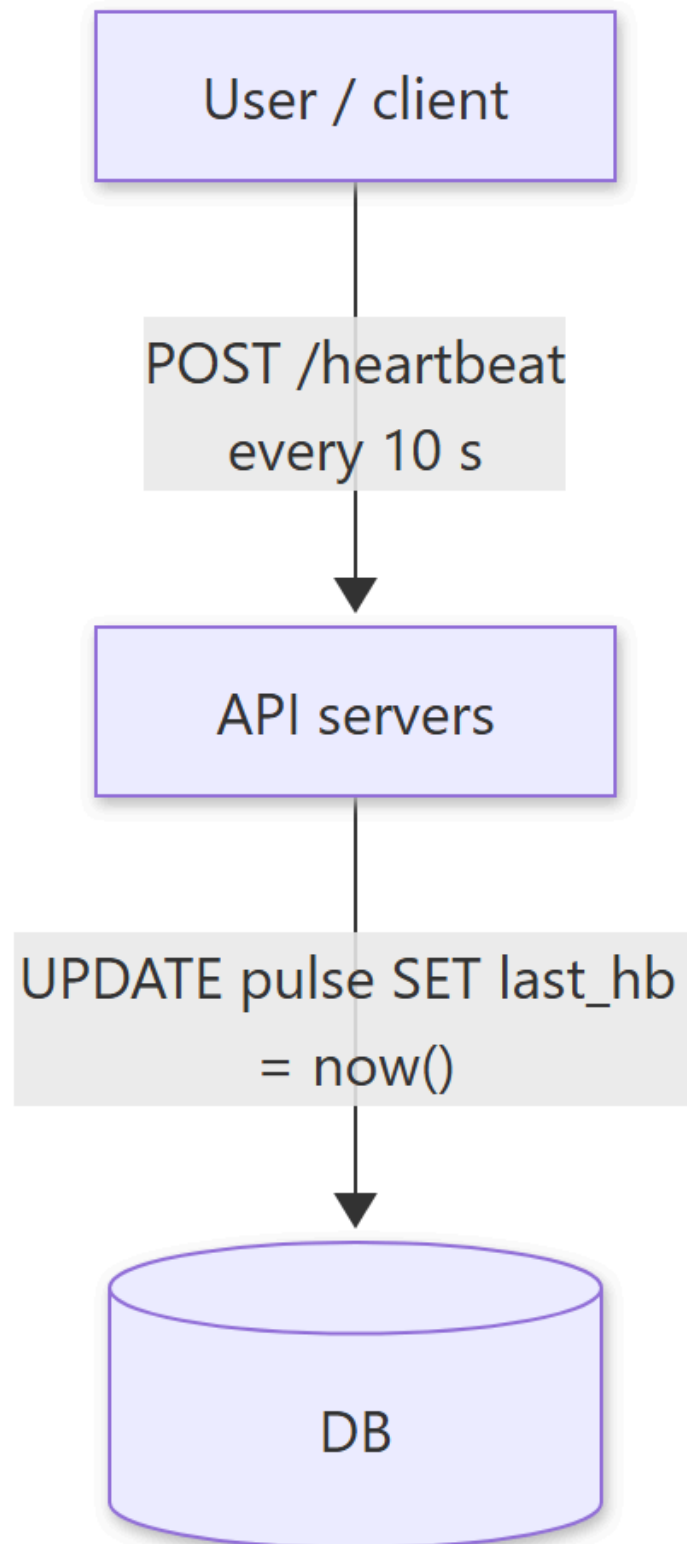
2.1 Data model and API

 Anchor: `user_id → bool` is a **key-value** access pattern. Batch reads.

- What we need is a map `user (int) → online? (bool)`. Access is a pure key lookup, so it is a KV problem. The database choice is still open at this point.
- Read API: `GET /status/users?ids=u1,u2,u3,u4`, one call for all the users on screen.
- Rule: **batch whenever and wherever possible.**

2.2 Updating status: push, not pull

 Anchor: The server **can't pull** from a client (no persistent connection), so the client **pushes a heartbeat**.



⚙️ Mechanics

- This is the *Framework of the Opposites* in action: pull vs push. The server can't proactively talk to the client unless a persistent connection exists, so the client sends a periodic pulse.
- `POST /heartbeat` : the authenticated user gets marked "alive".
- **When is a user offline?** When we haven't received a heartbeat for "long enough". That threshold is subjective and is business logic, e.g. **30 s**.

- Store **the time of the last heartbeat**, not a boolean:

user_id	last_hb (epoch s)
u1	1000
u2	1050
u3	1060

On heartbeat: `UPDATE pulse SET last_hb = NOW() WHERE user_id = ?. Status = now - last_hb < 30s.`

2.3 Estimate the scale

 Anchor: 2 ints = **8 bytes/row**. 1B users $\approx$ **8 GB**. Always do the critical calculation.

Users	Rows	Storage (8 B/row)
1 M	1 M	8 MB
1 B	1 B	8 GB

- Each entry is `user_id` (4 B int) + `last_hb` (4 B int) = 8 B.
- **Always critically challenge your system.** That doesn't mean you must change it. It means that when there are `n` paths, you evaluate all of them.

2.4 Opposite: don't store every user (sparse instead of dense)

 Anchor: **Absence == offline**. Store only active users and let entries expire.

Mechanics

- All we care about is online/offline. So if a user has **no row**, return offline.
- Expire each entry 30 s after its last heartbeat (delete it). Then *total rows* = *active users*.
- With 1B total users but only **100k active**: $100k \times 8 \text{ B} = \mathbf{800 \text{ KB}}$, instead of 8 GB.
- How to auto-delete? Opposites again:
 1. Write a **CRON job** that deletes expired rows (you own it).
 2. **Offload it to the datastore**: use a KV store with built-in TTL. 

2.5 Which KV store with TTL: Redis or DynamoDB?

 Anchor: Every heartbeat **pushes the TTL forward** (`SET key ... EX 30`). Pick the store by **context**, not by features.

	Redis (OSS)	DynamoDB
In-memory, KV, TTL	 (in-memory, fast)	 KV + TTL
Durability	weaker	 durable
Cost	in-memory DBs are expensive	NoSQL on disk is cheaper

	Redis (OSS)	DynamoDB
Vendor	none (self-host)	AWS lock-in
Ops	you maintain it; needs more people	managed; easy for a lean team; multi-region

- On every heartbeat: update the entry in Redis/DynamoDB with `ttl = 30s`.
- The context that decides it: product stage, engineering team size, business constraints, and the team's familiarity with the vendor.
- (Scaling Redis is covered in week 8.)

2.6 What real systems do: WebSockets

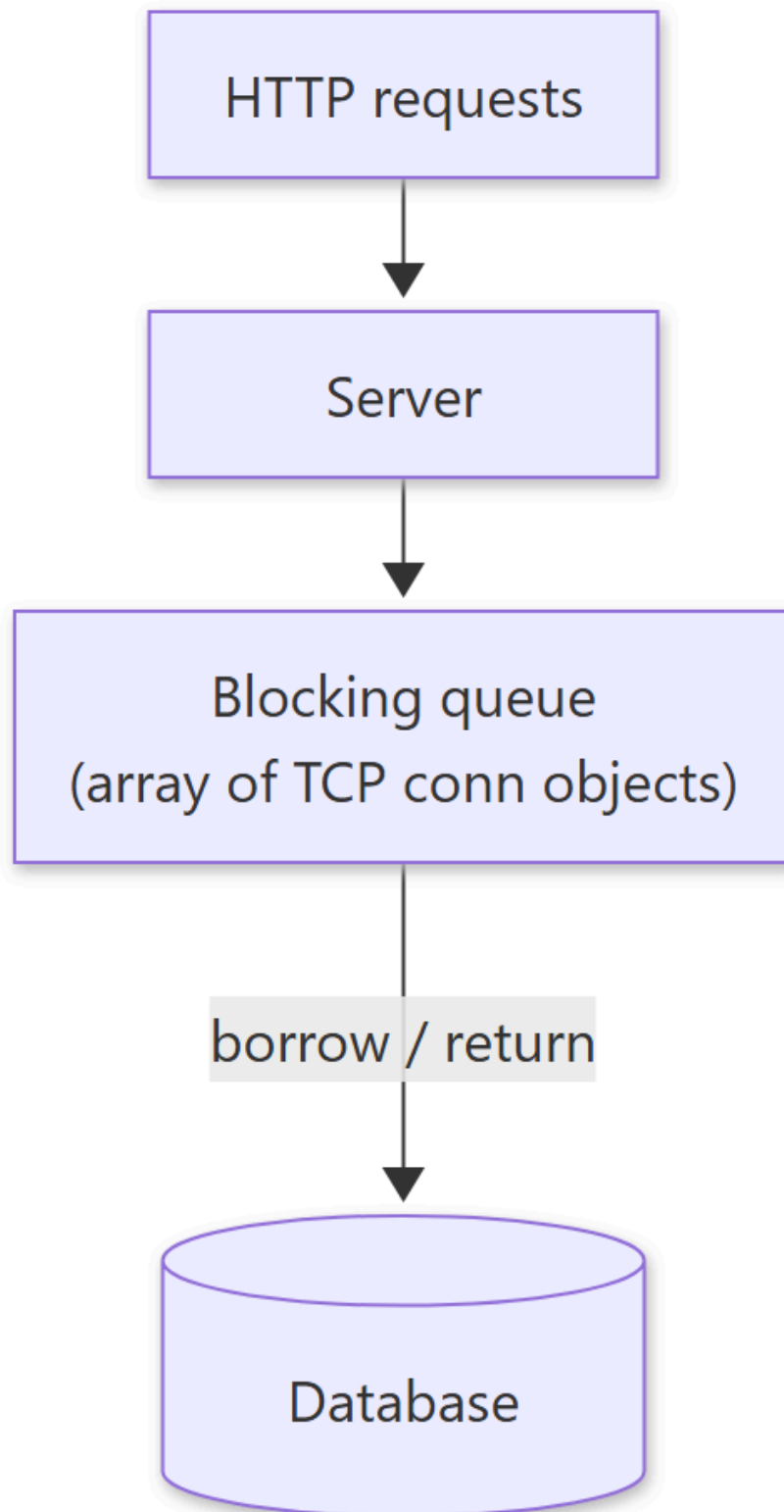
- In practice, presence systems use **WebSockets** (a persistent connection). Day 1 keeps it simple with HTTP heartbeats.
- If you use Socket.IO, it layers its own protocol on top of WebSocket (it is not the raw RFC 6455 spec), with a built-in heartbeat (`pingInterval / pingTimeout`) plus automatic connection monitoring and reconnection. With any other library, you can implement your own heartbeat.
- Scaling WebSockets is hard; that comes in week 2.

2.7 How is the DB doing? Connection pooling

 Anchor: **10 s heartbeat × 1M active users = 6M DB writes/min**. The hidden cost is opening a new connection every time.

Mechanics

- 1 user sends 6 heartbeats/min. With 1M active users that's **6M requests/min**, and each heartbeat is one DB call.
- These are micro reads/updates, so the expensive part is the **network**: each fresh TCP connection costs a 3-way handshake (plus a 2-way teardown).
- Fix: a **connection pool**, i.e. pre-established connections between the server and the DB that get reused.
- The numbers make the case: if the query itself takes ~1 ms but setting up a connection adds a lot more on top, reusing connections saves the majority of the time per call.



How a pool is built: a **blocking queue** (often a circular array) whose elements are TCP connection objects. A request takes one connection out. If none is free, it **waits until one is returned**.

 **So what:** The same "heartbeat + timeout" idea is how **failure detection in distributed systems** works (simple version).

Exercise: implement a thread-safe connection pool using a bounded blocking queue.

Recall: Why does storing only active users with a TTL beat storing all users?

Absence means offline, so storage scales with *active* users ($100k \times 8 \text{ B} = 800 \text{ KB}$) instead of *total* users ($1B \times 8 \text{ B} = 8 \text{ GB}$), and the datastore deletes expired rows for you. No CRON job needed.

Recall: Redis or DynamoDB for presence? What decides it?

Context. Redis is in-memory and fast but costly and self-maintained. DynamoDB is durable, managed, multi-region and cheaper on disk, but brings AWS lock-in. A lean team favours managed; a team that already runs Redis may self-host.

Recall: Why does a connection pool help a heartbeat service so much?

Each heartbeat is a tiny DB update, so per-request TCP setup/teardown dominates the cost. Reusing pre-established connections removes that overhead from 6M calls a minute.

3. Case study: Multi-user blogging platform (Medium)

The foundation topics this course keeps returning to: **Database & Caching · Scaling · Delegation · Concurrency · Communication.**

Scope: one user writes many blogs; many users. We look only at the key design decisions.

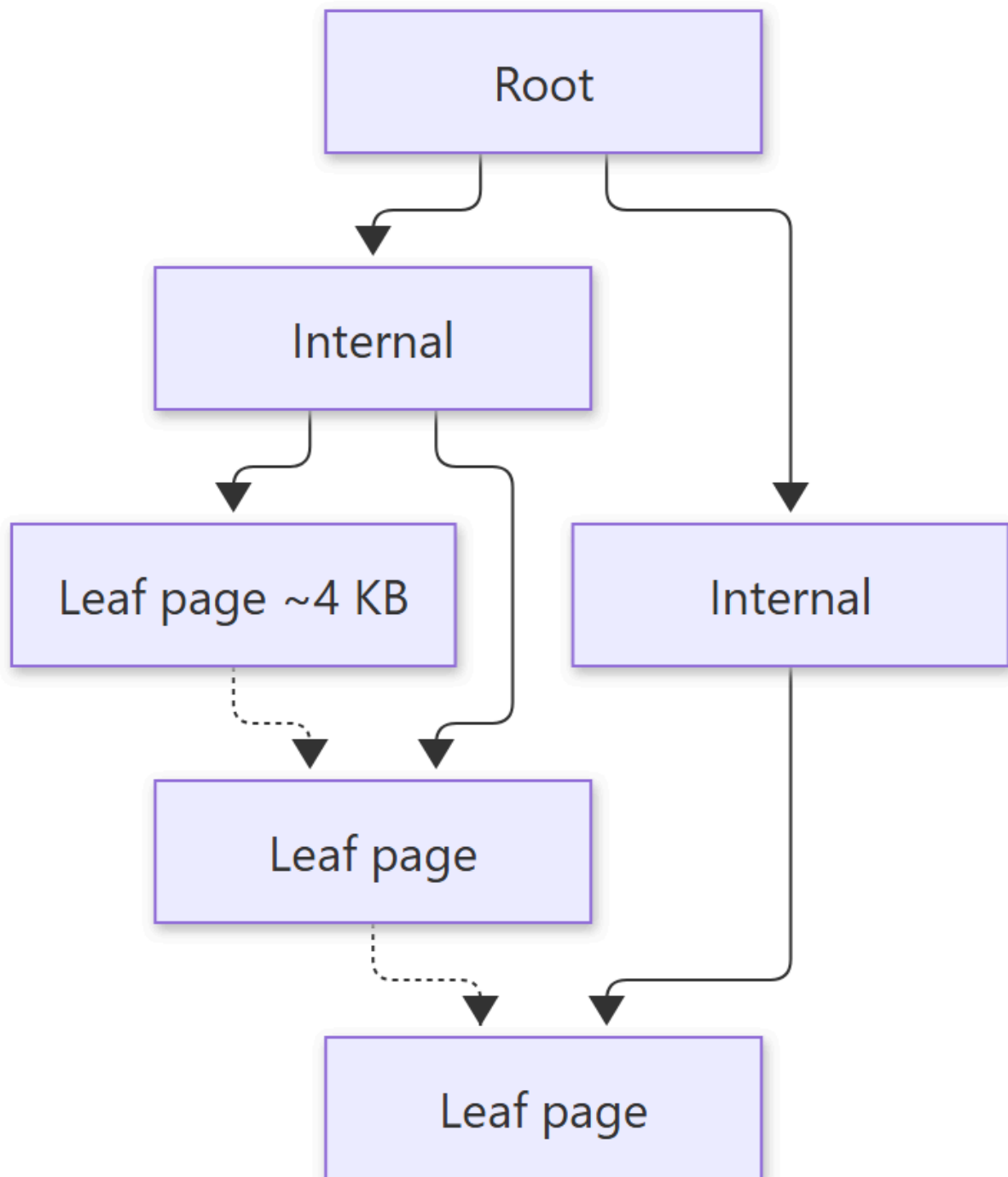
3.1 Schema

users	blogs
id	id
name	author_id
bio	title
	body
	is_deleted
	published_at

3.2 Soft delete (`is_deleted`)

 Anchor: **Delete = UPDATE** `is_deleted = true`, not DELETE.

- Reasons: **recoverability, archival, audit.**
- It's also easier on the database engine: a real delete removes a key from the B+ tree index, which can trigger **tree rebalancing** (nodes merge). An update doesn't change the tree's shape.

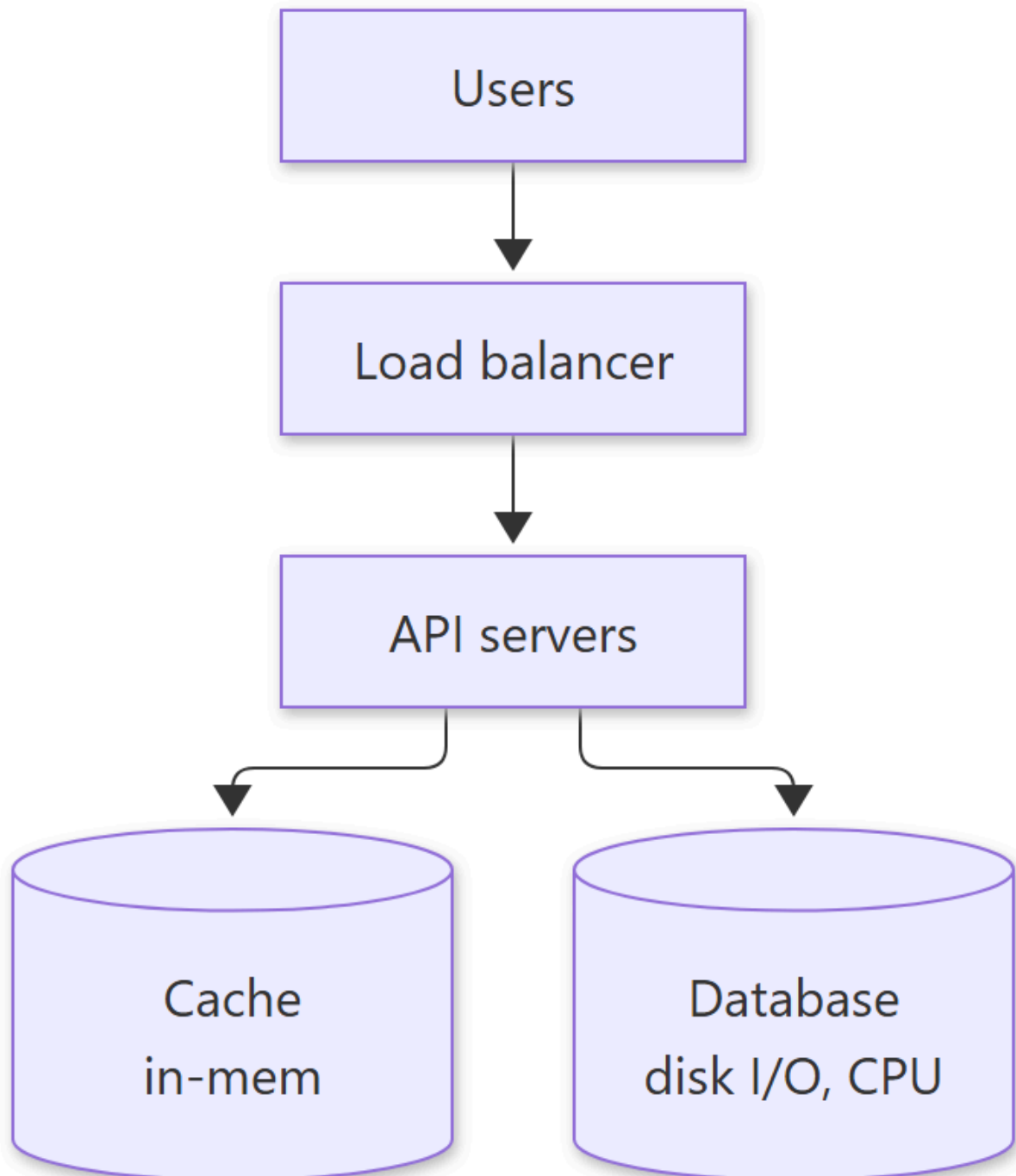


- Side note: `SELECT A, B, C FROM table` still reads the **entire row** from disk; only the projected columns are returned. Wide rows cost I/O even if you select a few columns. This also affects indexes.
- (Post-reads cover `datetime` vs `int`, and `TEXT` vs `VARCHAR`.)

3.3 Caching

💡 Anchor: **Caches are just glorified hash tables.** Use one to avoid repeating any expensive work.

- A cache reduces response time by saving heavy computation or I/O. Typical uses: avoid disk I/O, network I/O, or compute.
- Caches are not only RAM-based. A **CDN** is a cache too.
- Analogy: copying the topper's answers instead of solving the hard assignment yourself is caching. You skip redundant work.
- Concerns that come with a cache: **invalidation**, **co-location**, **fallback on a cache miss**, and **surges in traffic**.



3.4 The problem: many concurrent misses for the same key

 Anchor: On a miss, let **one** request go to the DB and make the others **wait** for its result.

When a popular blog isn't in cache, many API calls miss at the same moment and all hit the DB together. That's a **cache stampede** (thundering herd).

The session's fix: a semaphore per key, so the first request fetches and the rest wait.

```
sem_map = {} # concurrent map: key -> semaphore
res_map = {} # key -> fetched value

get_blog(k):
    v = cache.get(k); if v: return v
    s = sem_map.get(k)
    if s: # someone is already fetching k
        s.wait(); return res_map.get(k)
    else: # I am the first
        sem_map[k] = new Semaphore(); sem_map[k].block()
        v = db.get(k); cache.put(k, v); res_map[k] = v
        sem_map[k].signal(); sem_map.remove(k)
    return v
```

- The slides name this **debouncing**: many requests for the same key reach the cache at once, one goes through to the DB, and the others wait. CDNs do this too.

 Recall: Name three reasons for soft delete, plus the DB-engine reason.

Recoverability, archival, audit. And an UPDATE doesn't remove an index key, so the B+ tree doesn't have to rebalance.

 Recall: What is a cache stampede and how does the per-key semaphore stop it?

Many concurrent misses on the same key all hit the DB at once. The first request takes a per-key lock and fetches the value; the others wait on that lock and then read the stored result. One DB call instead of N.

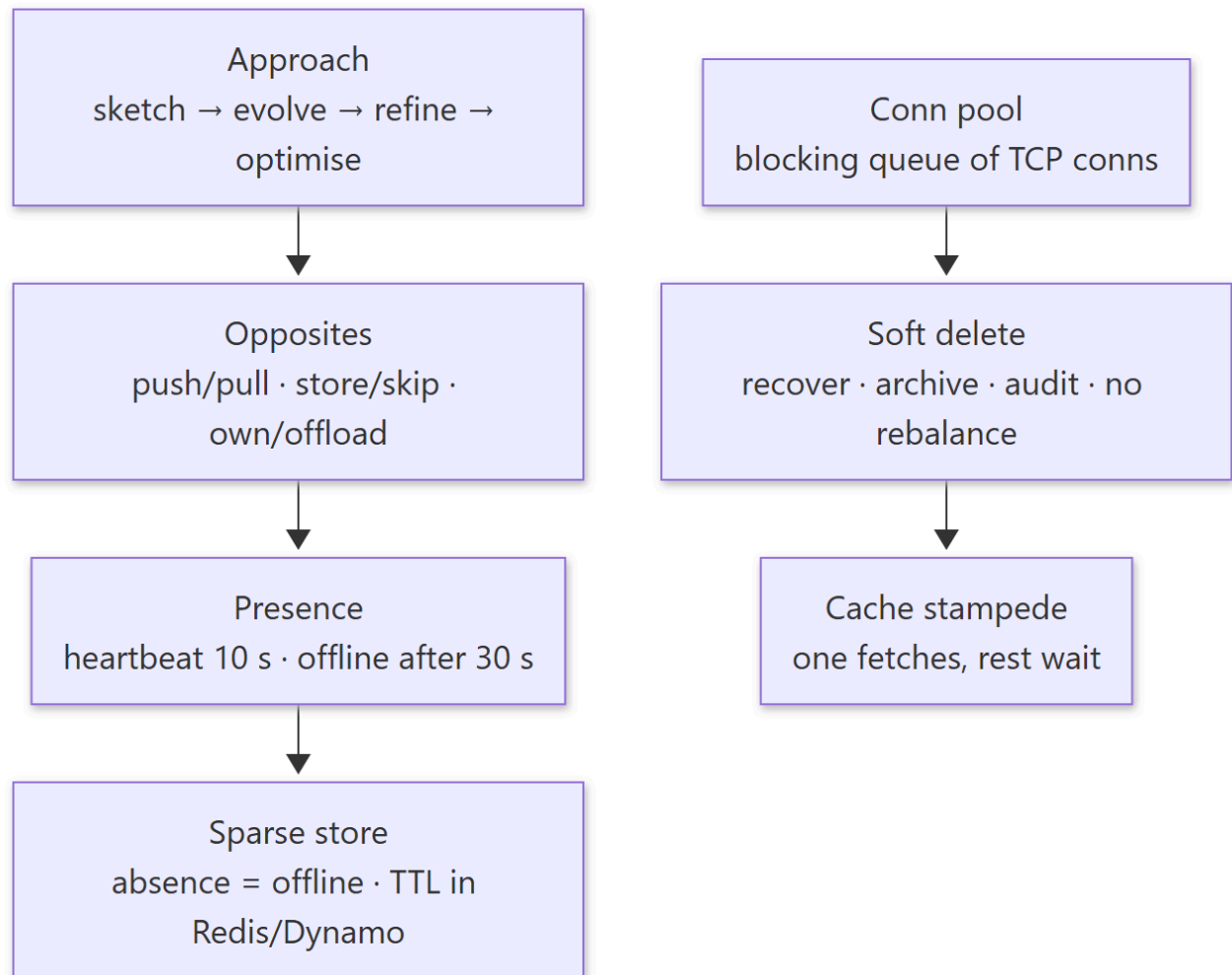
What these notes add beyond the slides

- **Name check: "debouncing" / "request hedging"**. The slides note this pattern is "also called request hedging". In common usage, **request hedging** is something else: sending a *duplicate* request to a second replica when the first is slow, to cut tail latency. The pattern on the slides is usually called **request coalescing** or **single-flight** (Go's `singleflight`), or cache-stampede protection. Use those names in interviews.
- **Race in the pseudocode**. `sem_map.get(k)` followed by `sem_map[k] = ...` is check-then-act. Two threads can both see "no semaphore" and both go to the DB. Use an atomic `putIfAbsent` / `computeIfAbsent` so exactly one thread wins. Also, a waiter can read `res_map` after the winner

removes the semaphore, so `res_map` entries need their own cleanup (or just re-read the cache).

- **Arithmetic checked:** $1\text{B} \times 8\text{B} = 8\text{GB}$ ✓, $100\text{k} \times 8\text{B} = 800\text{KB}$ ✓, $10\text{ s heartbeat} \rightarrow 6/\text{min} \rightarrow 6\text{M}/\text{min}$ for 1M users ✓.
- **Heartbeat interval vs offline threshold:** a 30 s threshold with a 10 s heartbeat tolerates two lost pulses before marking someone offline. A good rule of thumb is $\text{threshold} \approx 2\text{--}3 \times \text{the interval}$.

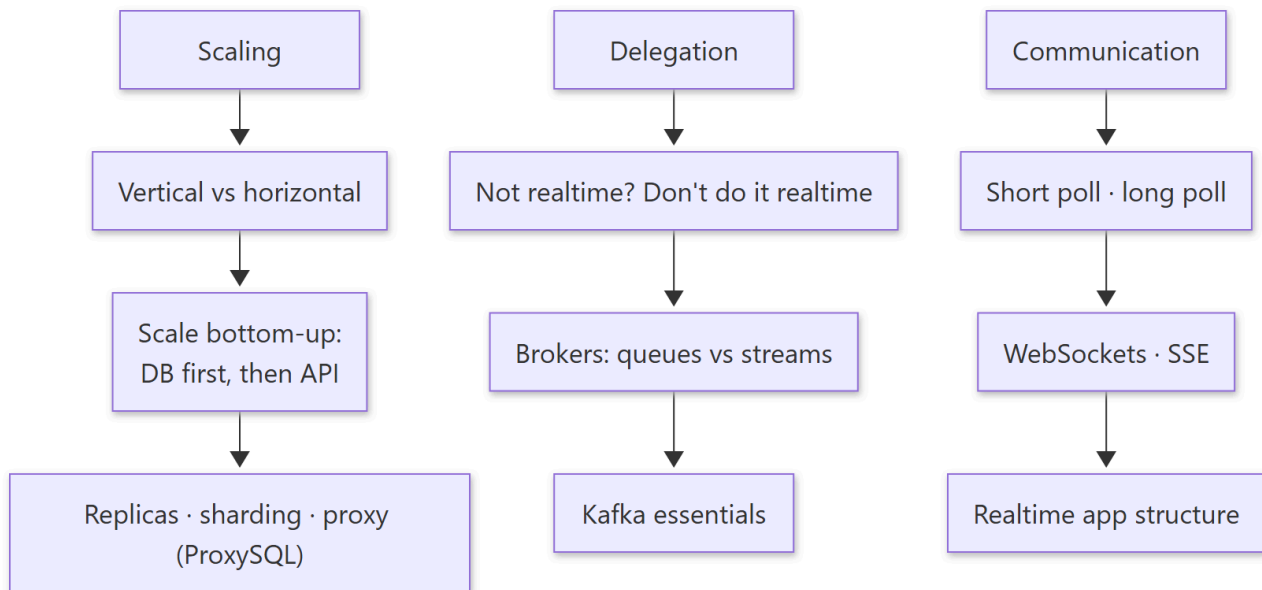
One-page memory card



01-02 · Arpit Bhayani's session notes: Foundational topics (Scaling, Delegation, Communication)

Source: 01-02.pdf (System Design Masterclass, Google Drive, view-only). Study notes in my own words, not a copy. Pages covered: 1–14 of 14. Unreadable pages: none.

Map of the session



Agenda: **Scaling + ProxySQL primer · Delegation + Kafka essentials · Communication** (short poll / long poll / WebSockets / SSE).

1. Scaling

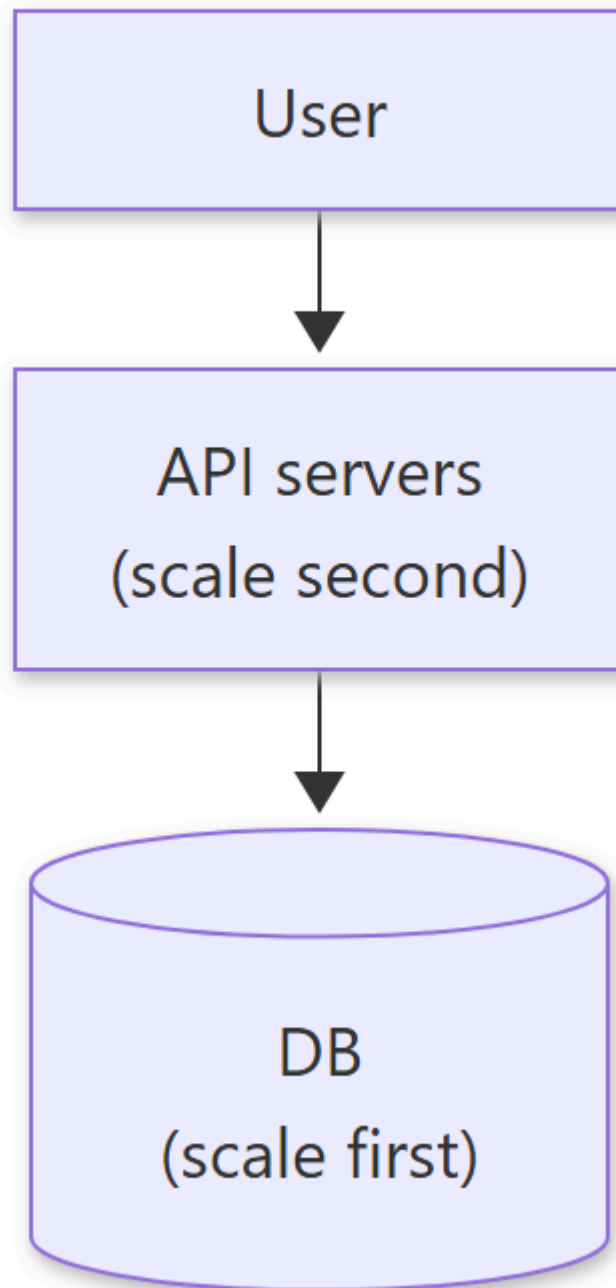
🔗 Anchor: Scaling = handling a large number of **concurrent** requests. Horizontal $\approx$ "infinite" scale, **but there's a catch.**

	Vertical scaling	Horizontal scaling
How	Make the box bulkier ("Hulk"): more CPU, RAM, disk	Add more boxes
Gives you	Simple; no code change	Near-linear amplification + fault tolerance
Limit	Hardware ceiling	Your stateful components must keep up

- Decide using **unit tech economics**: load-test one unit to find what it handles, then work out how many you need.

1.1 The catch: scale bottom-up

🧠 Anchor: Adding API servers is pointless if the **DB and cache** behind them can't take the extra concurrent load. **Scale the DB first, then the API.**



- Scaling one layer isn't enough when the layer it depends on can't keep up.

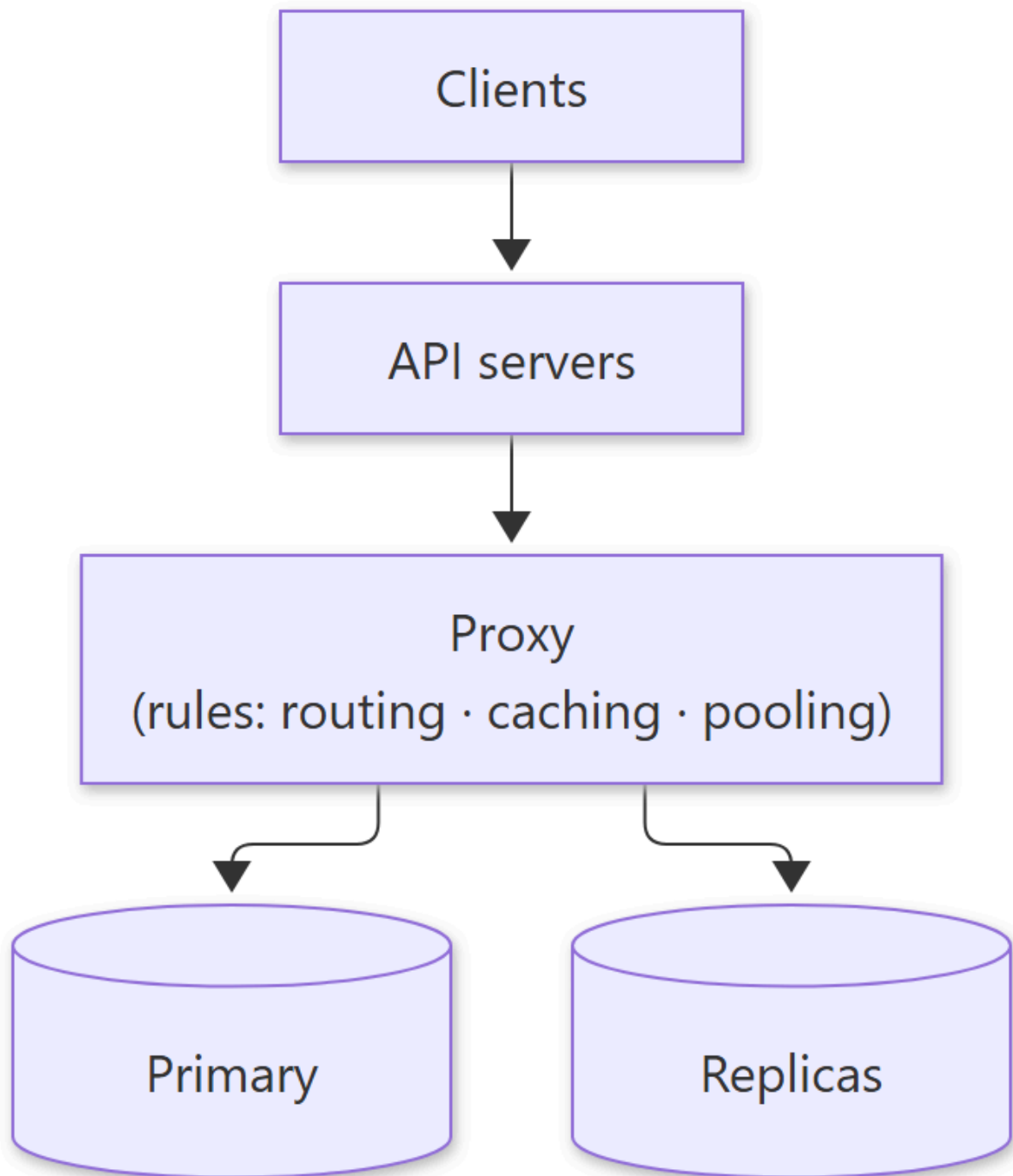
1.2 Ways to scale the DB

Vertical
bigger DB box

Read replicas
API writes to primary,
reads from replicas

Sharding
split keys: a-j · k-t · u-z
each shard independent

- **Who knows the topology?** Either every API server knows which DB/shard/replica to talk to, **or** you put a **proxy** in between that knows the topology and routes for you (e.g. RDS Proxy, ProxySQL).
- Proxy rules can cover **routing, caching, connection pooling**, and more.



1.3 ProxySQL primer

⚙️ **Mechanics** (all configured with SQL against ProxySQL's admin tables, then `LOAD ... TO RUNTIME;` `SAVE ... TO DISK`)

1. **Hostgroups** represent different clusters. For example, hostgroup 10 = primary, hostgroup 20 = replicas.
2. Put **multiple replicas in the same hostgroup**. ProxySQL spreads load across them, e.g. **least-connection routing**.
3. **Query rules** route by pattern: send `^SELECT` to the replica hostgroup (20), everything else to the primary (10).

4. Rules can also route by **schema**: e.g. `analytics_db` → replicas, `transactions_db` → primary.

Scaling ProxySQL itself: vertically, or horizontally by running several ProxySQL nodes behind an **L4 load balancer**, each connected to all DB groups.

 Recall: Why must you scale bottom-up?

The stateful layers (DB, cache) take the concurrent load from every API server. Adding API servers just pushes more load onto a DB that can't take it, so scale the DB first.

 Recall: Two ways for an API to "know" DB topology?

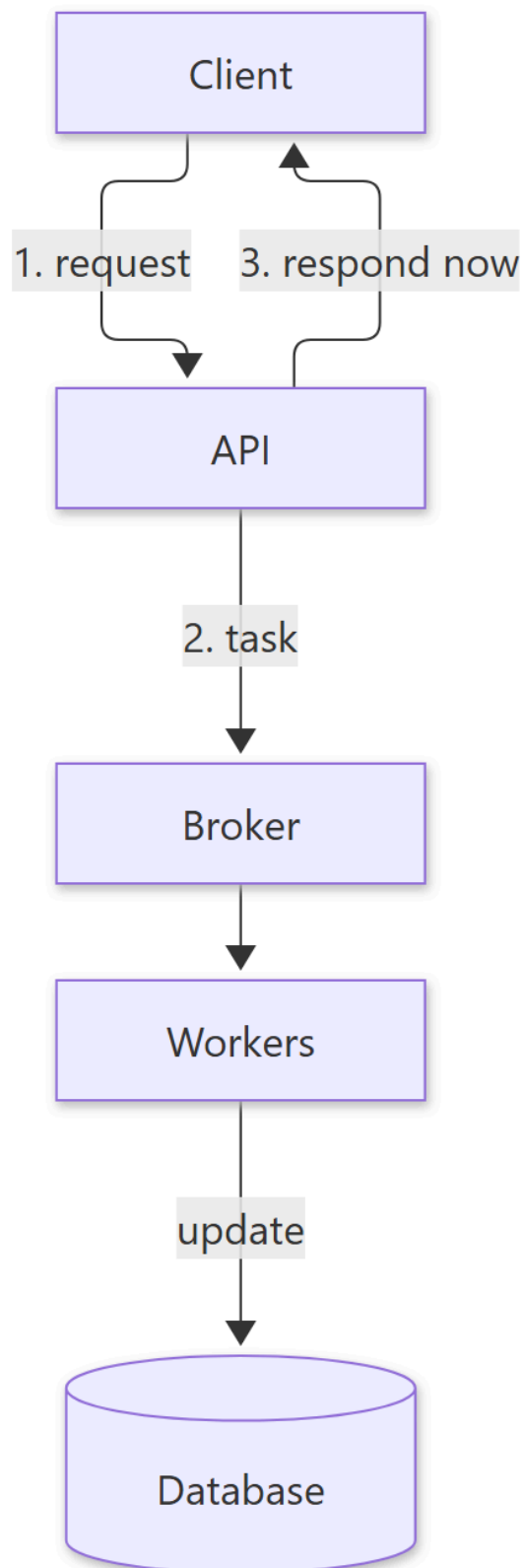
Bake the topology into every API server, or put a topology-aware proxy (ProxySQL, RDS Proxy) in between that handles routing, pooling and caching.

2. Delegation

 Anchor: **"What does not need to be done in realtime should not be done in realtime."** This is the mantra for performance.

Example: show "74 essays" (number of blogs) on an author's profile.

2.1 Delegate and respond



The API puts a task on a broker and responds immediately. Workers pick tasks up and do the slow part.

What to delegate:

- Long-running tasks that would outlast the request timeout (spinning up an EC2 instance, video encoding)
- Heavy computation or queries
- Batch writes
- Anything that can be **eventually** consistent

2.2 Brokers: queues vs streams

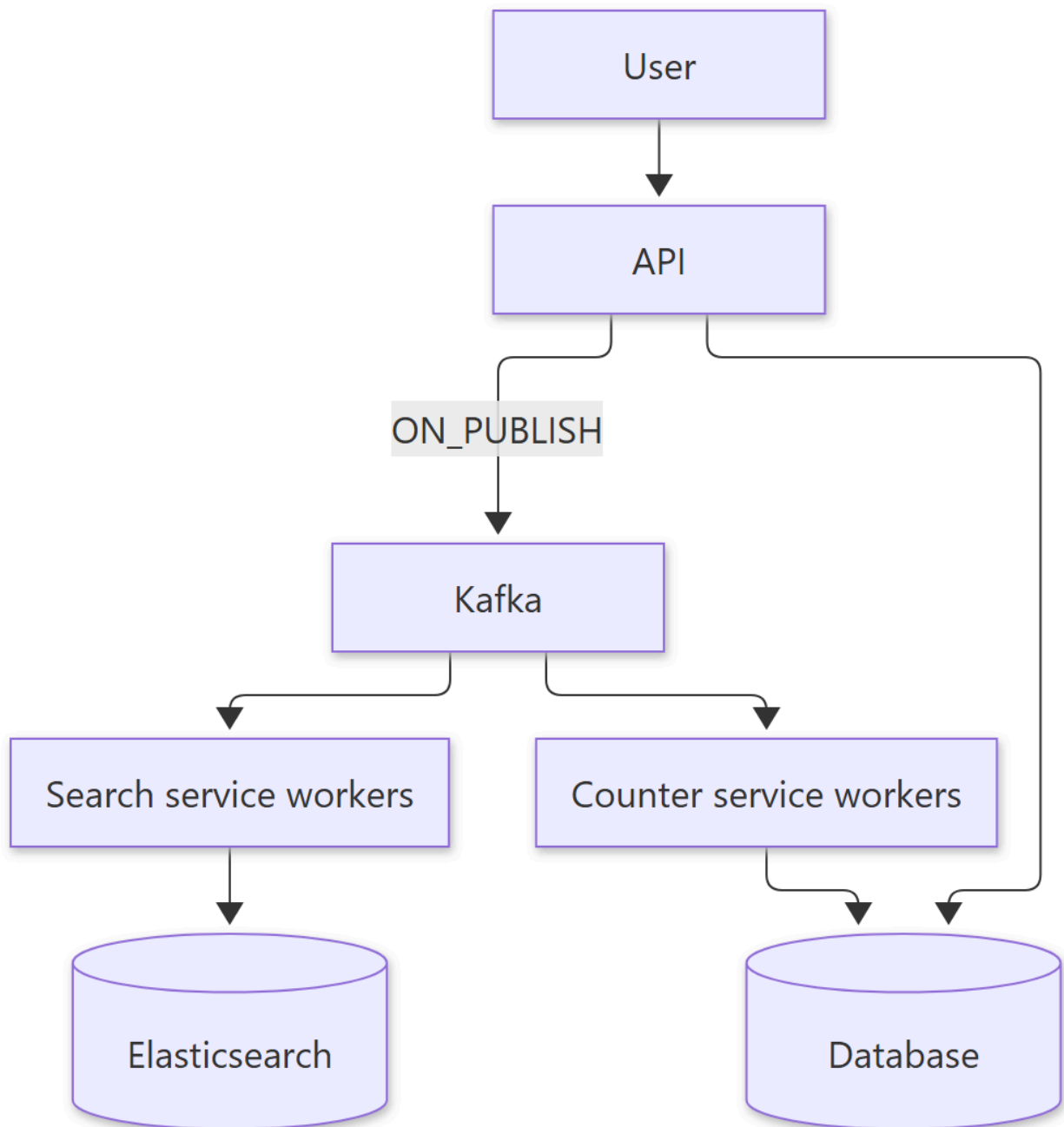
 Anchor: **Queue** = each message goes to **one** consumer. **Stream** = each message can be read by **many** consumer groups.

	Message queues	Message streams
Examples	SQS, RabbitMQ	Kafka, Kinesis
Delivery	Each message is consumed by one of the consumers	Messages are retained; every consumer group reads all of them
Good for	Work distribution	Fan-out of the same event to several independent systems (search, analytics)

2.3 Example: `total_blogs` counter

- Store `total_blogs` **pre-computed** on `users`, to avoid joins and runtime counting.
- On publish, the API emits `ON_PUBLISH` to SQS; a worker does `total_blogs++`. Similarly, `ON_DELETE` → `total_blogs--`.
- Want to index blogs in Elasticsearch? You *could* add that logic to the same worker...

...but that's a bad design: no separation of concerns. Counting and search indexing belong to two different teams. Fix: use **Kafka**, so the same event is consumed by **two kinds of consumers**.



Exercise: set up RabbitMQ and Kafka locally, write simple producer/consumer code, and note the difference in semantics.

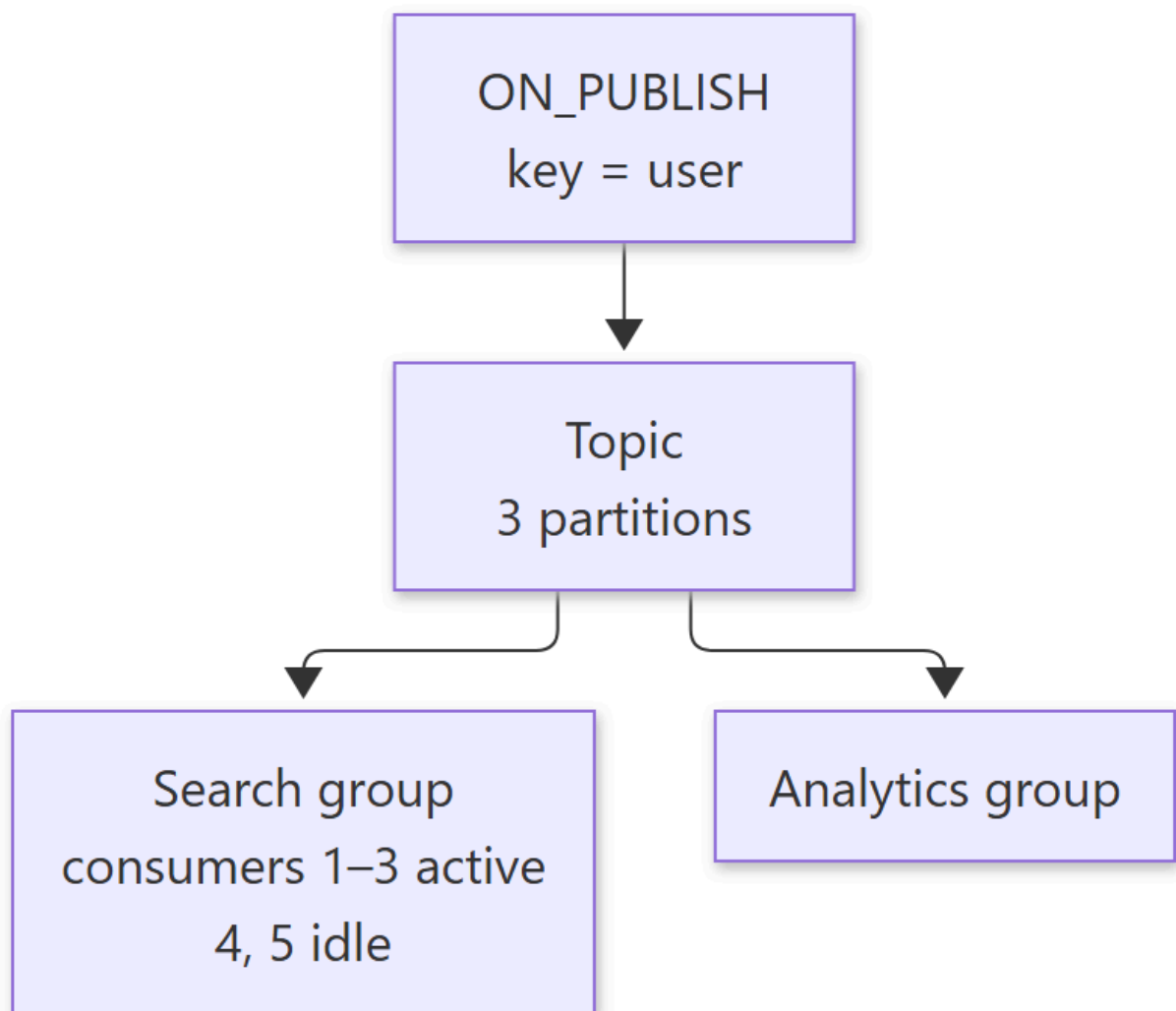
2.4 Kafka essentials

🔗 Anchor: Topic → **n partitions**. Hash key picks the partition. **Order only within a partition**. Consumers per group ≤ partitions.

⚙️ Mechanics

- Kafka is a message stream that holds messages. Messages go to a **topic**, and every topic has **n partitions**.

- The configured **hash key** decides which partition a message lands in (e.g. key by user, so all of `u1`'s events go to one partition).
- **Ordering is guaranteed within a partition only**, not across partitions.
- **Limitations:**
 - You **cannot reduce** the number of partitions after creating a topic.
 - Within one consumer group, **#consumers ≤ #partitions** is what's useful: with 3 partitions, a 4th or 5th search consumer gets no messages.
- A different consumer group (e.g. analytics) reads the same topic independently.



 Recall: Why move from SQS to Kafka for the blog example?

Separation of concerns. The counter and the search index are different teams' jobs. A stream lets each team run its own consumer group on the same `ON_PUBLISH` event, whereas a queue hands each message to just one consumer.

 Recall: What happens with 5 consumers in a group on a 3-partition topic?

Only 3 get partitions; 2 sit idle. Parallelism is capped by partition count, and partitions can't be reduced later.

3. Communication

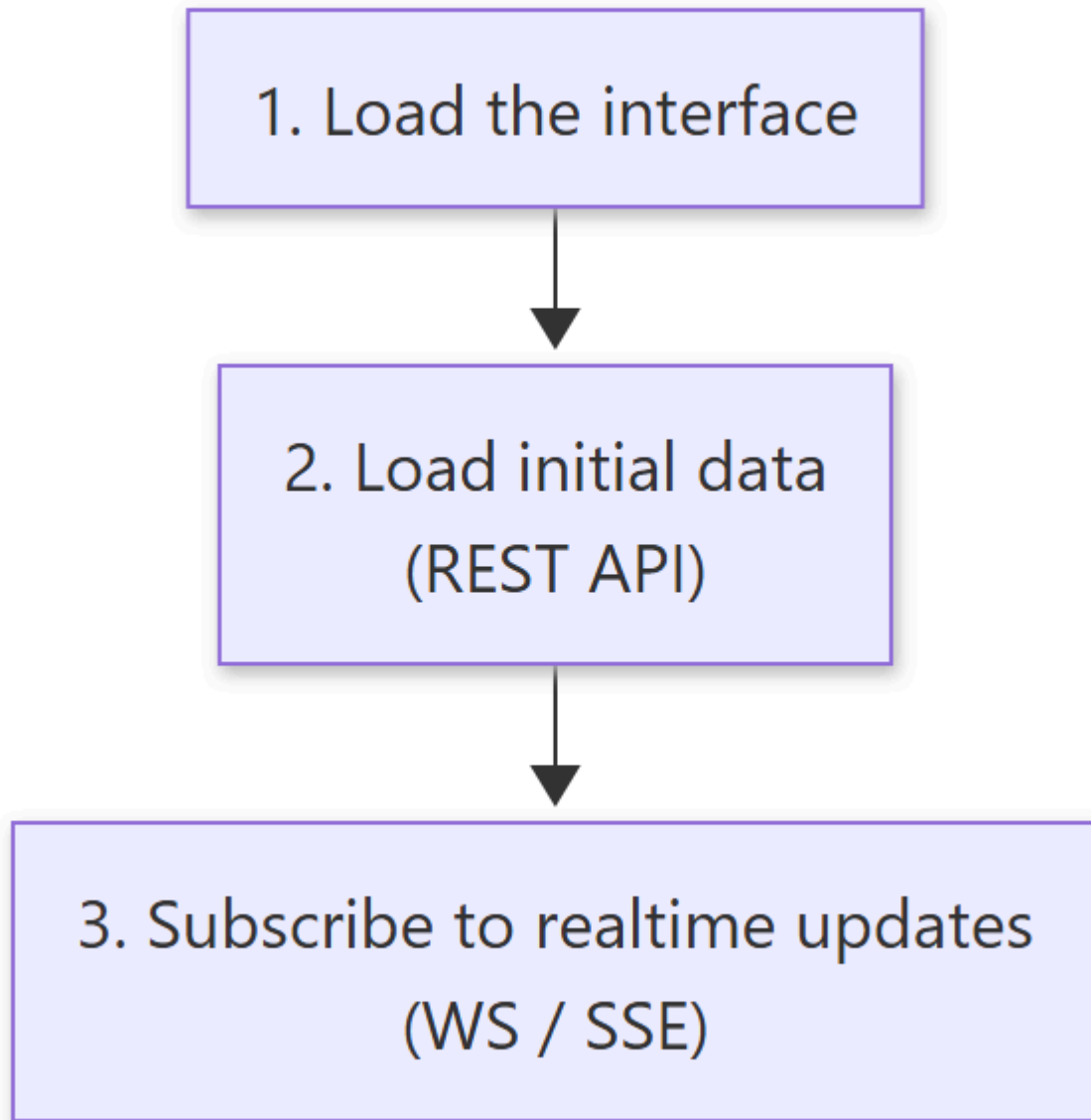
 Anchor: **Who initiates, and how long is the connection held?** That one question separates the four styles.

Style	How it works	Example	Cost
Normal request/response	Client asks, server does the heavy lifting, connection open until the response (with a timeout)	Any API	—
Short polling	Client asks again every few seconds; server answers right away	Refreshing a cricket score; "is the server ready?"	HTTP overhead; lots of empty request/response pairs
Long polling	Client asks; server holds the connection and responds only when data is available (or on timeout, then the client re-asks)	Respond only when the ball is bowled	Held connections on the server
WebSockets (ws/wss)	Bidirectional channel kept open; the server can push	Realtime chat, stock ticker, live experiences, multiplayer games	Stateful connections to scale
Server-Sent Events	Unidirectional: server → client stream over HTTP	Stock ticker, deployment-log streaming, ads, feed updates	One direction only

- EC2 provisioning example: short polling checks the status every few seconds; long polling gets a single response when the machine is ready.
- WebSocket advantages: **realtime** data transfer and **low communication overhead** (no per-message HTTP request).

3.1 How realtime apps are usually structured

 Anchor: **Load the UI → load the initial state via REST → subscribe for live updates.**



- Stock table: initial prices come from REST, then ticks stream in.
- Deployment logs: the existing log loads through a regular API, and new lines are streamed.

 Recall: Short vs long polling in one line each?

Short polling: server replies immediately, client repeats every few seconds. Long polling: server holds the request and replies only when there's data (or at timeout).

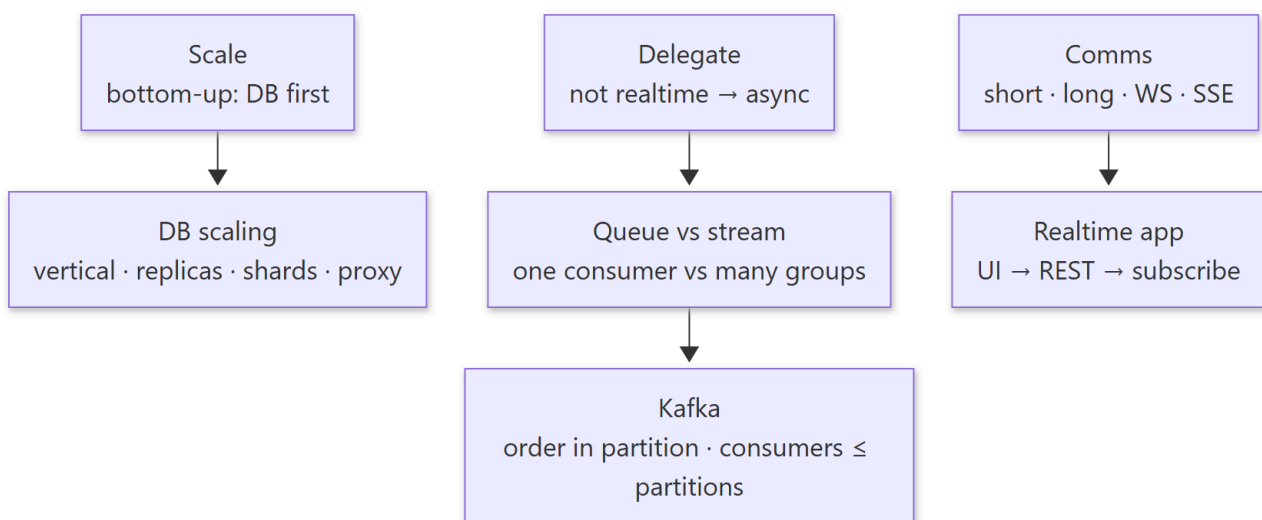
 Recall: When pick SSE over WebSockets?

When only the server needs to push (logs, tickers, feed updates). SSE is simpler, plain HTTP and one-way. Use WebSockets when the client also sends frequent messages (chat, games).

What these notes add beyond the slides

- **Bottom-up scaling extends to caches and queues:** any stateful dependency (cache cluster, broker, third-party API with rate limits) caps how far stateless servers can scale.
- **"#consumers ≤ #partitions" is per consumer group.** Extra consumers act as hot standbys that take over a partition on rebalance, so they're not entirely useless.
- **Counters via events are eventually consistent and need idempotency.** If a worker retries `total_blogs++` after a crash, the count drifts. Use an idempotency key per event, or periodically recompute.
- **Long polling vs SSE:** both hold a connection, but long polling closes after each response and re-opens. SSE keeps one connection for many events and has built-in reconnection with `Last-Event-ID`.

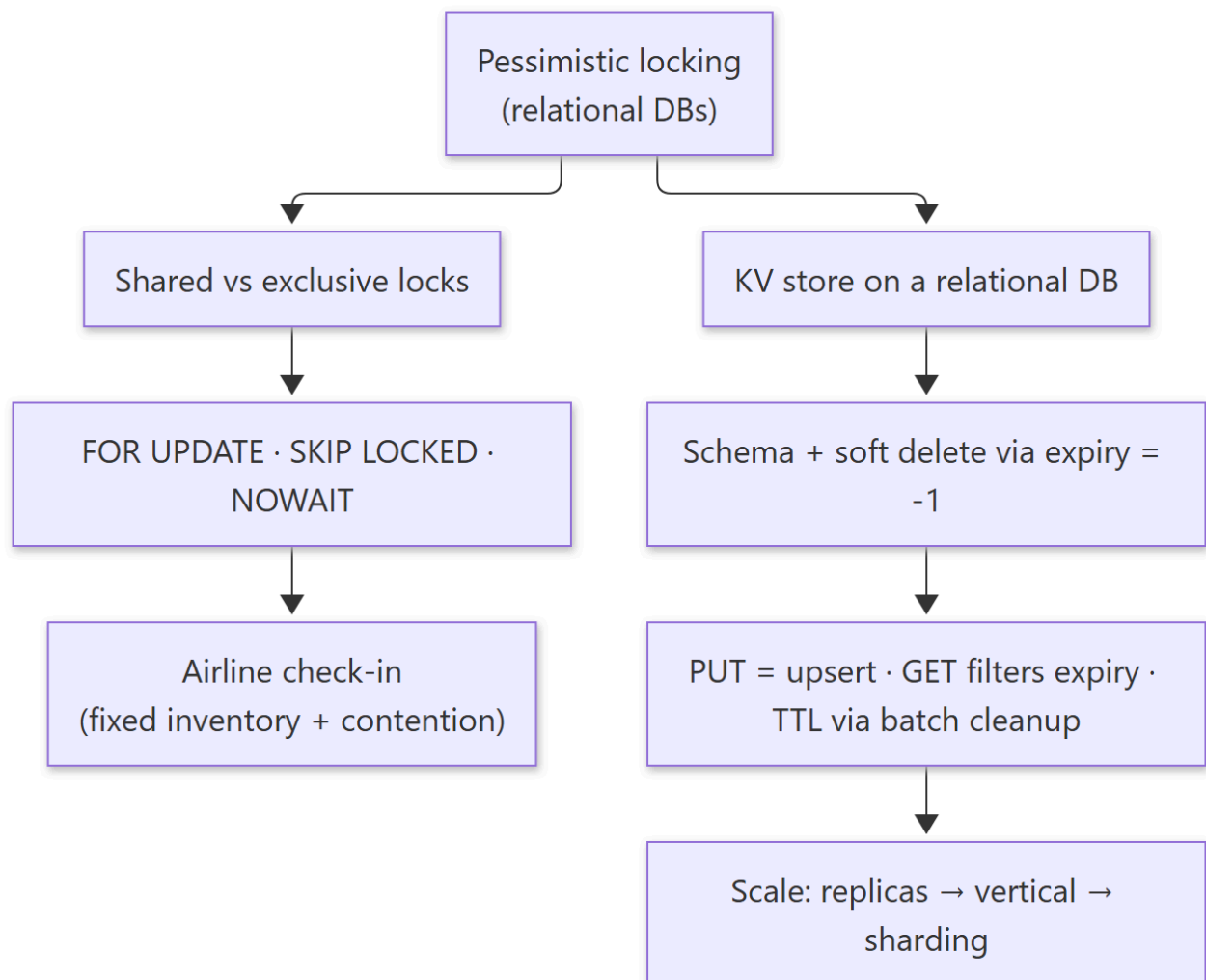
One-page memory card



02-01 · Arpit Bhayani's session notes: Databases (Pessimistic locking, Airline check-in, KV store on SQL)

Source: 02-01.pdf (System Design Masterclass, Google Drive, view-only). Study notes in my own words, not a copy. Pages covered: 1–14 of 14. Unreadable pages: none.

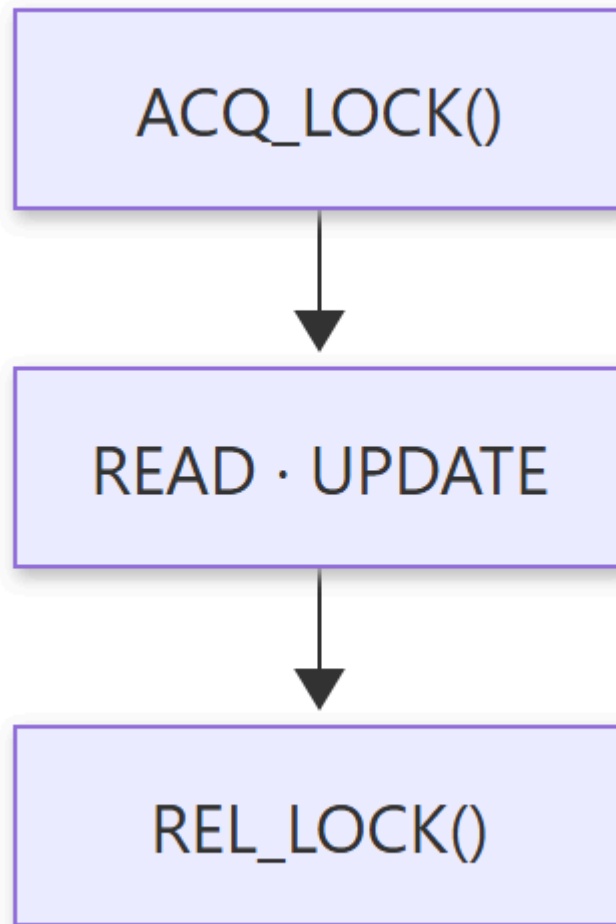
Map of the session



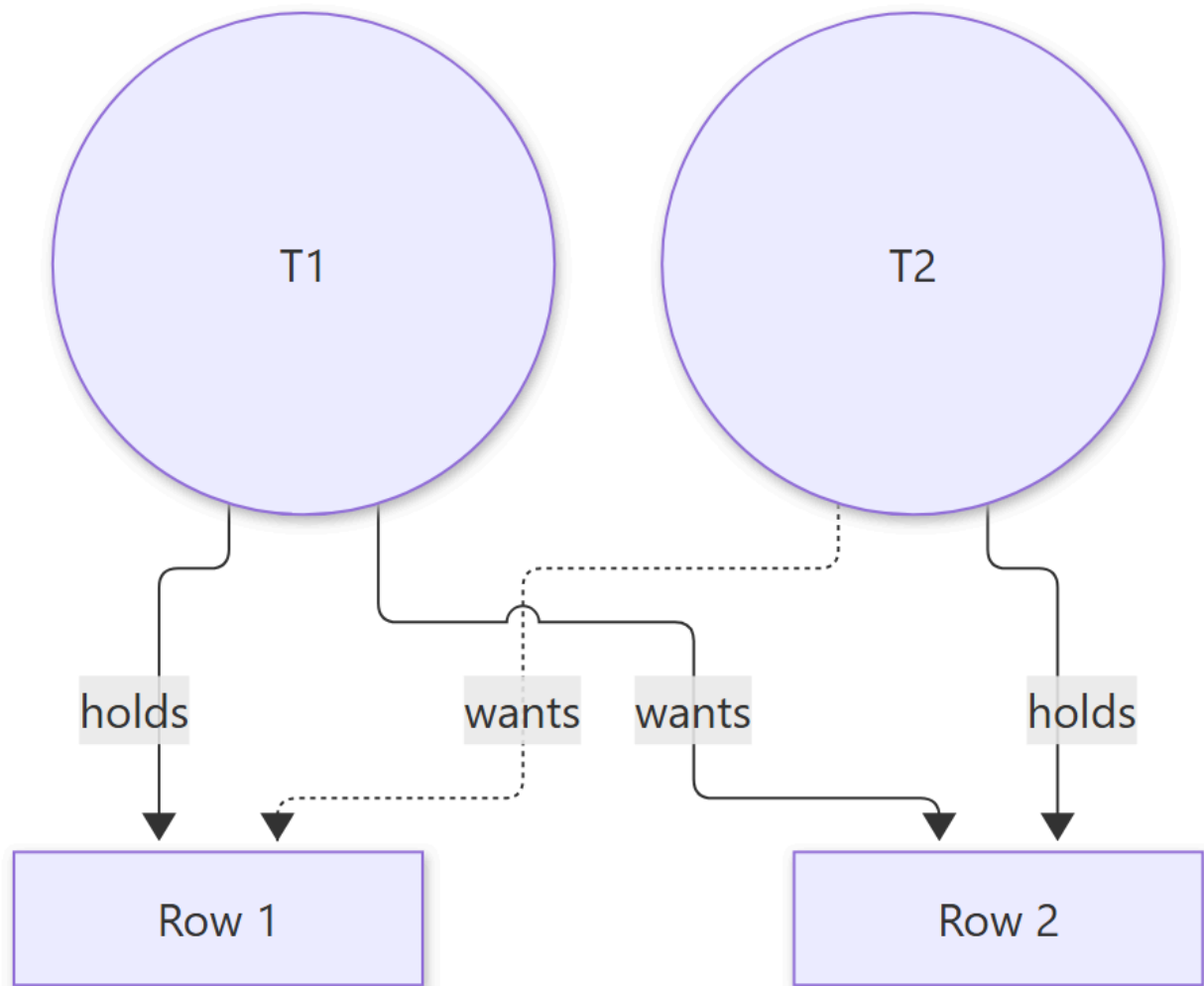
Agenda: **Pessimistic locking in relational DBs** · **Designing an airline check-in system** · **Designing a KV store on a relational DB.**

1. Pessimistic locking

Anchor: **Lock first, then read and update, then release.** This protects data sanity (consistency + integrity) against concurrent updates.



- Locks exist to protect the **sanity** of the data, meaning its consistency and integrity, when updates run concurrently.
- Two strategies: **shared locks** and **exclusive locks**. Locks are released when the transaction **commits or rolls back**.
- **Risk: deadlock.** T1 holds row 1 and wants row 2; T2 holds row 2 and wants row 1. The database detects the cycle and **kills one transaction** so the other can proceed.



1.1 Exclusive locks: `SELECT ... FOR UPDATE`

🔗 Anchor: `FOR UPDATE` = "these rows are **mine to write**." Anyone else who wants them **waits**.

- Reserved for **writing** by the current transaction. Other transactions can't take a lock on, or modify, those rows until it finishes.
- Example: T1 runs `SELECT ... WHERE id IN (1,2,6) FOR UPDATE`. T2 runs `SELECT ... WHERE id IN (3,4,6) FOR UPDATE`. Rows 3 and 4 are free, but **row 6 overlaps**, so T2 **waits** for T1.

1.2 Variations: `SKIP LOCKED` and `NOWAIT`

Clause	Behaviour if the row is already locked	Use it for
<code>FOR UPDATE</code>	Wait for the lock	Correctness when you need <i>that</i> row
<code>FOR UPDATE SKIP LOCKED</code>	Drop locked rows from the result set and carry on	"Give me <i>any</i> free item": seats, job queues, coupons
<code>FOR UPDATE NOWAIT</code>	Fail immediately with an error (MySQL: <code>ERROR 3572 ... NOWAIT is set</code>)	Fail fast instead of piling up waiters

Exercise: read about shared locks and code them up for different use cases.

 Recall: What do FOR UPDATE, SKIP LOCKED and NOWAIT each do when the row is locked?

FOR UPDATE waits. SKIP LOCKED leaves the locked row out of the result and moves on. NOWAIT errors out immediately.

 Recall: How does a relational DB resolve a deadlock?

It detects the wait-for cycle and rolls back one of the transactions (the victim) so the other can proceed. Your app must retry the victim.

2. Airline check-in system

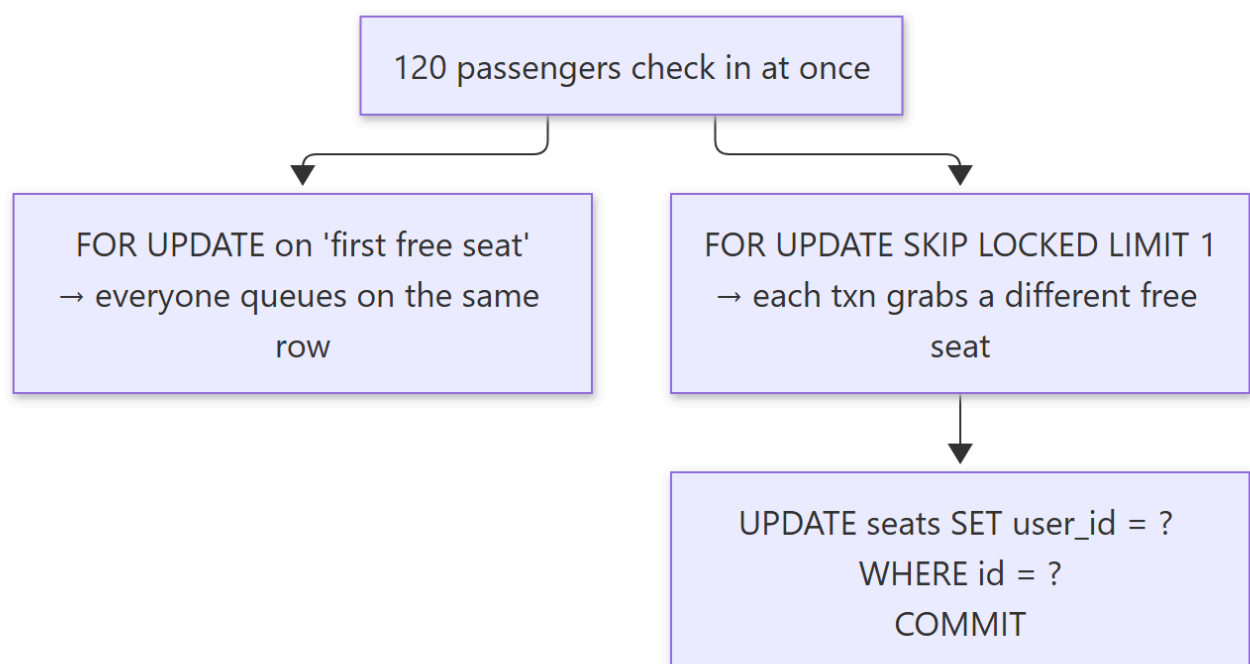
 Anchor: **Fixed inventory + contention.** 120 people pick seats at the same moment, and no seat may be given twice.

Requirements:

- Multiple airlines; each airline has multiple planes (flights); each flight has **120 seats**; each flight has multiple **trips**.
- A user books a seat on one trip of a flight.
- **Handle many people trying to pick seats on the same plane at the same time.**

Similar systems (same shape: fixed inventory + contention): CoWIN vaccine slots, IRCTC train tickets, BookMyShow, flash sales.

The session demonstrates it with locking. Here's how the clauses above map onto it:



- With plain `FOR UPDATE`, every transaction tries to lock the **same** "first free seat" row, so they serialise and run slowly.
- With `SKIP LOCKED`, each transaction skips seats others are holding and takes the next free one, so all 120 proceed in parallel and no seat is double-booked.

Exercise: implement the full example, plus shared locks for different use cases.

 **Recall: Why is SKIP LOCKED the right tool for seat allocation?**

Users don't need a *specific* seat. Concurrent transactions skip rows others have locked and each grabs a different free seat, so there's no serial queue and no double allocation.

3. Designing a KV store on a relational database

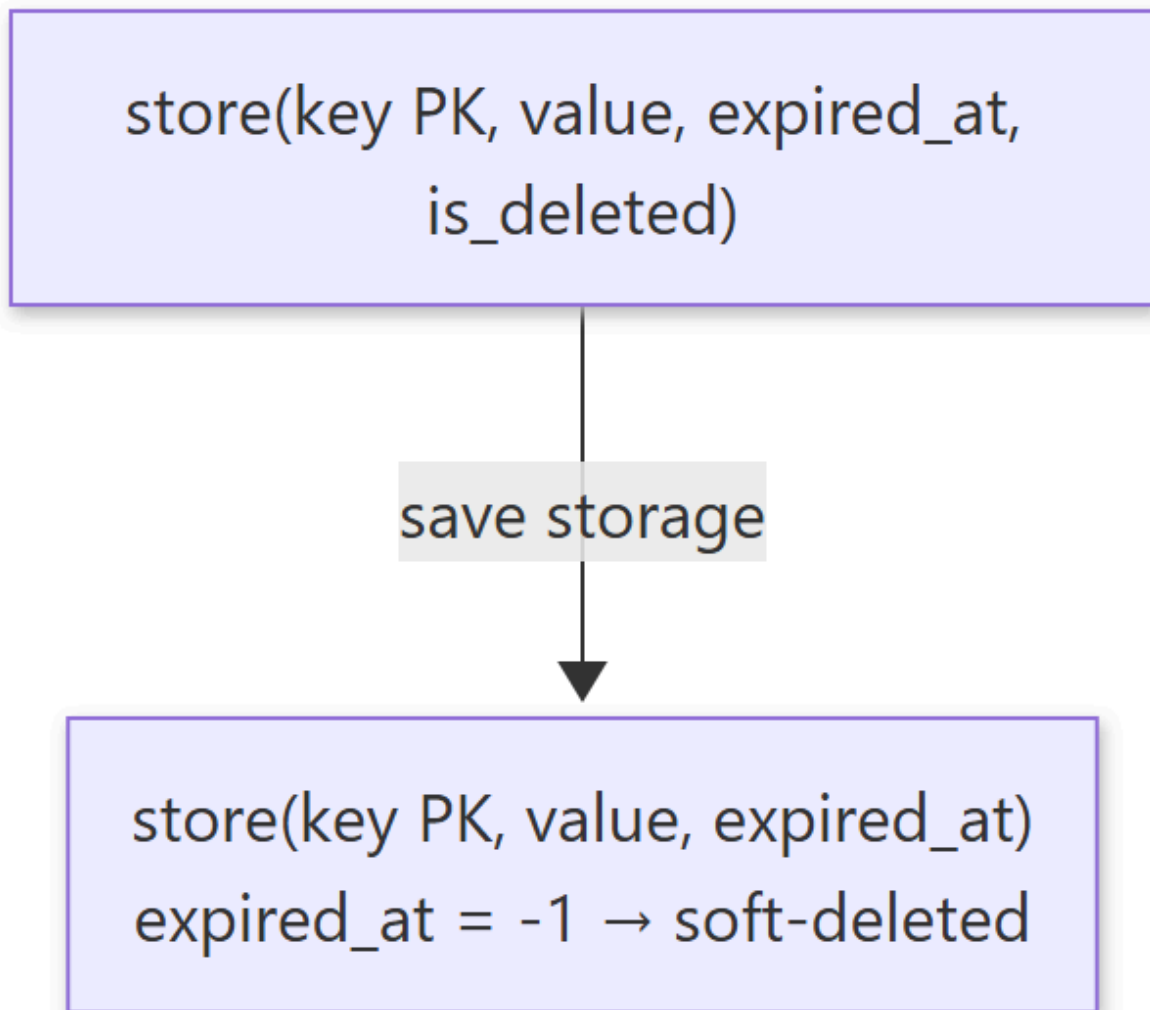
 **Anchor:** An HTTP-based, distributed **KV store backed by MySQL**: storage/compute separation, like DocumentDB on Aurora.

Requirements: horizontally scalable; `GET`, `PUT`, `DEL`, `TTL`. All operations are **synchronous** (per key).

Brainstorm: storage, optimising storage, insert, update, TTL.

3.1 Schema

 **Anchor:** Fold `is_deleted` into `expired_at`: `expired_at = -1` **means deleted**. One less column on every row.



- Start on a **single MySQL node** and scale as demand grows.
- `expired_at` = an **absolute** expiry time. Index it.
- **Saving storage:** why keep a separate `is_deleted` column? Use a **special value**, `-1`, in `expired_at` to mark deletion.
- **Hard delete:** a separate batch/periodic cleanup job removes soft-deleted and expired rows. This minimises I/O and B-tree rebalancing during normal operation.

3.2 Operations

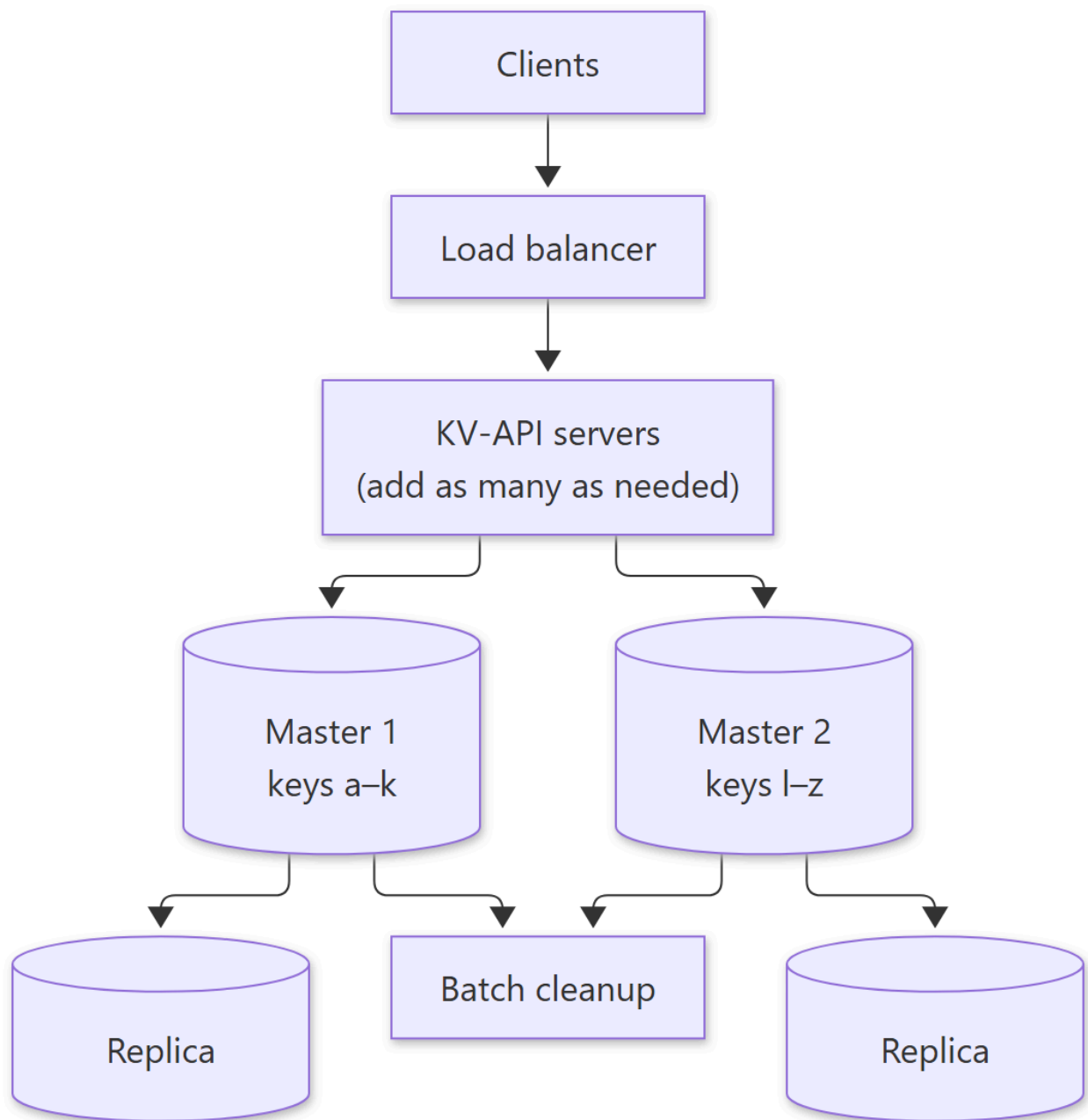
Op	Implementation
PUT(k, v, ttl)	Upsert , not "GET, then INSERT or UPDATE". MySQL: <code>REPLACE INTO store VALUES (k, v, expiry)</code> ; PostgreSQL: <code>INSERT ... ON CONFLICT ... DO UPDATE</code>
Concurrent PUTs	Locks protect integrity: one updates, the others wait (<code>SELECT ... FOR UPDATE [NOWAIT]</code> then <code>UPDATE ... WHERE key = k</code>)
GET(k)	<code>SELECT * FROM store WHERE key = k AND expired_at > NOW()</code> , so expired rows are filtered out on read

Op	Implementation
DEL(k)	Soft delete: <code>UPDATE store SET expired_at = -1 WHERE key = k AND expired_at > NOW()</code> (no hard delete, less disk load)
TTL / GC	CRON every t minutes: <code>DELETE FROM store WHERE expired_at <= NOW() LIMIT 1000</code> . That catches both expired and soft-deleted (-1) rows

- Why the `AND expired_at > NOW()` on DEL? Don't touch rows that are already expired or deleted, which avoids useless writes.
- Why `LIMIT 1000` on the cleanup? Without it, one giant DELETE locks many rows, bloats the undo log, spikes replication lag, and can stall live traffic. Delete in small batches.
- The rejected first attempt was `INSERT` with a `created_at`. It breaks on a second `PUT` for the same key, so use an upsert.

3.3 Scaling the KV store

 Anchor: **Stateless KV-API servers scale freely. The DB scales in steps:** read replicas → vertical → shard.



1. **Add KV-API servers** as load grows, as long as MySQL can handle it.
2. **Read-heavy (90:10)** and one DB can't cope → add **read replicas** (route with a proxy / ProxySQL). Replicas have **lag**.
 - Add replicas only when **reading stale data is OK** and **reads** >> **writes** (e.g. 99:1).
 - Let clients ask for consistency per read: `GET /key?consistent=true` sends that read to the **master** (DynamoDB offers the same option).
3. **Master overloaded with writes** → first **scale vertically**. If that's still not enough, **partition (shard)**.
 - **Range-based** (a-k → m1, l-n → m3, n-z → m2) or **hash-based** partitioning.
 - The routing map lives in a coordinator (e.g. ZooKeeper) that the API servers read. **Each master owns an exclusive fragment of the data.**
 - Each shard runs its own batch cleanup and has its own replicas.

 Recall: How does the KV store implement DEL and TTL without hard deletes on the hot path?

DEL sets `expired_at = -1` (soft delete). GET filters `expired_at > NOW()`. A CRON job hard-deletes rows with `expired_at <= NOW()` in batches (LIMIT 1000), which catches both expired and deleted rows.

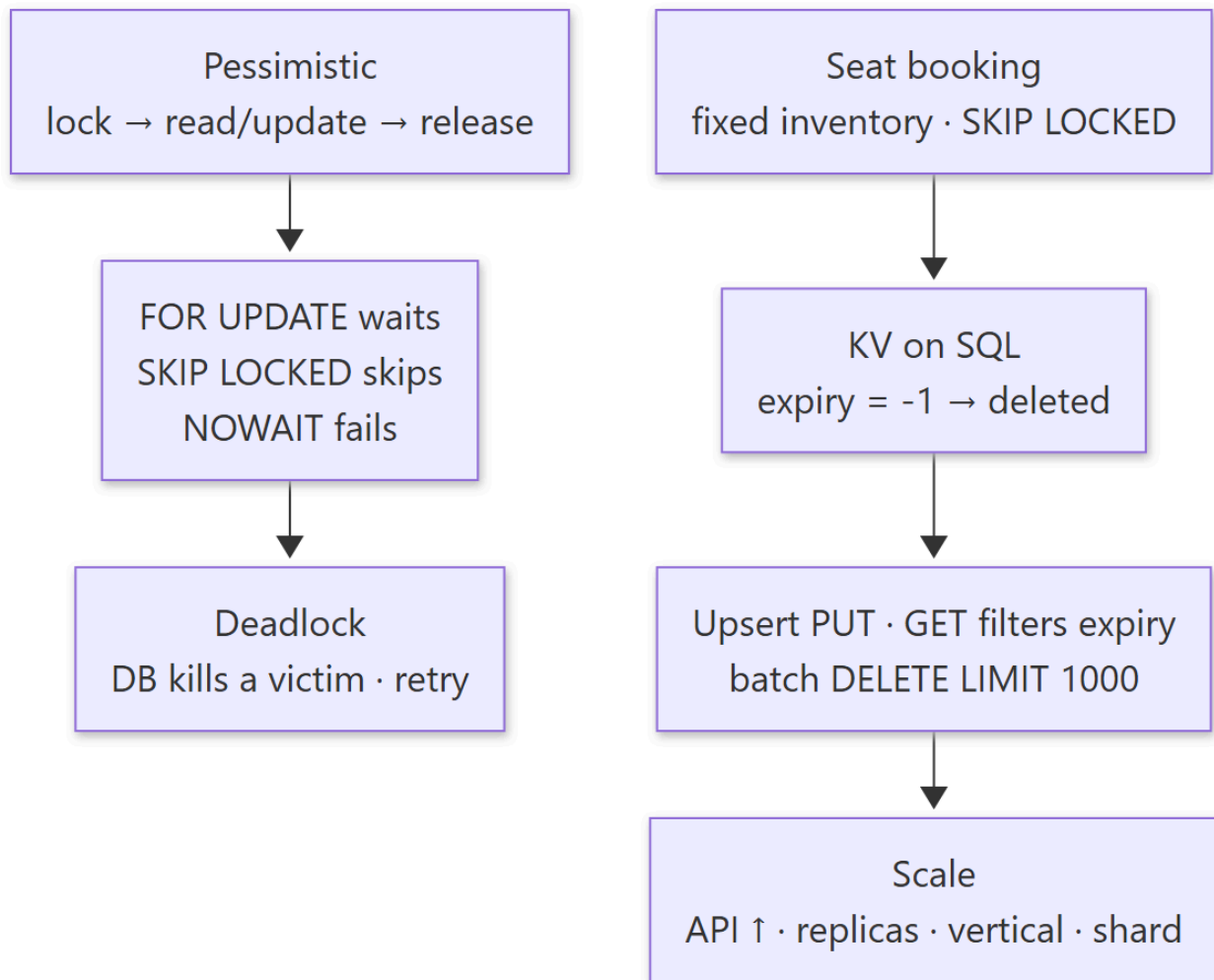
 Recall: When do you add read replicas vs shard?

Replicas: reads dominate and slightly stale reads are acceptable (offer `consistent=true` to read the master). Shard: the master can't keep up with writes even after scaling vertically.

What these notes add beyond the slides

- **Exclusive locks don't block plain reads in MySQL/Postgres.** The slide says other transactions can't *read* rows locked FOR UPDATE. That's true for **locking reads** (FOR UPDATE, FOR SHARE). A plain SELECT uses MVCC and reads the last committed version without waiting. Know this distinction for interviews.
- **Deadlock victim selection:** the slide says the transaction that detects the deadlock kills itself. In InnoDB, the **deadlock detector** picks a victim, usually the transaction that has modified the fewest rows. Either way: catch the error and retry.
- **REPLACE INTO is delete + insert** in MySQL. It fires delete triggers, changes auto-increment IDs and does extra index work. INSERT ... ON DUPLICATE KEY UPDATE is usually the better upsert.
- **SKIP LOCKED also turns a SQL table into a job queue** (workers SELECT ... FOR UPDATE SKIP LOCKED LIMIT n). That's the same pattern as seat allocation.

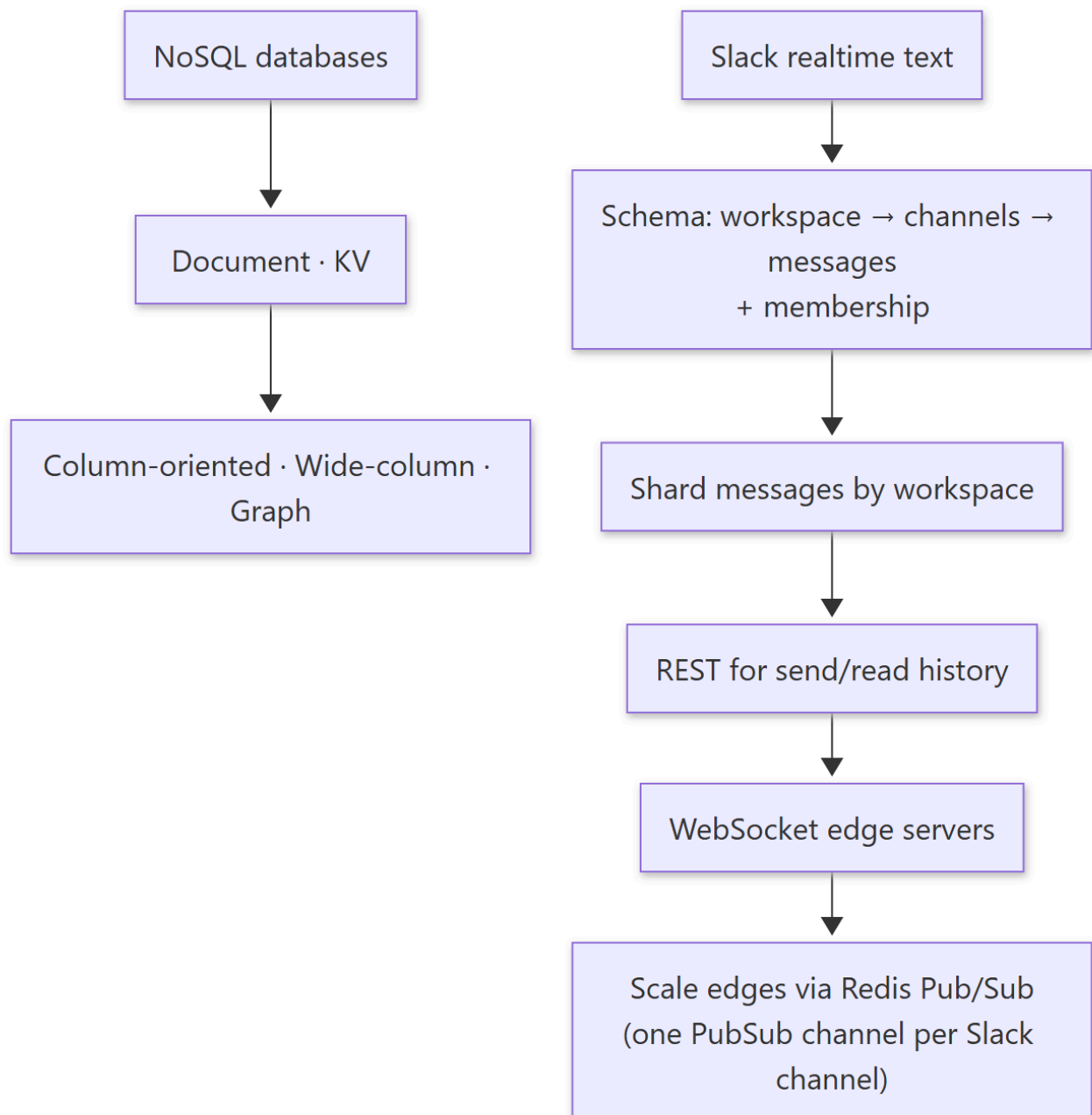
One-page memory card



02-02 · Arpit Bhayani's session notes: Non-relational databases + Slack's realtime text communication

Source: 02-02.pdf (System Design Masterclass, Google Drive, view-only). Study notes in my own words, not a copy. Pages covered: 1–19 of 19. Unreadable pages: none.

Map of the session



Agenda: **Non-relational databases · Designing Slack's realtime (text) communication.**

1. Non-relational (NoSQL) databases

📌 Anchor: NoSQL usually trades **consistency** for **scalability and availability**, so most are **eventually consistent**. But "NoSQL scales" does **not** mean "SQL doesn't".

- Data is structured non-relationally.
- Logic check from the slides: $a \rightarrow b$ does not imply $\neg a \rightarrow \neg b$. **Anything that can be sharded scales**, SQL included.

Type	Shape	Strengths	Examples
Document	Mostly JSON docs	Complex queries; partial updates to a document; closest to relational	MongoDB, Elasticsearch
Key-value	key $\rightarrow$ value	Key-wise access; heavily partitioned; no complex queries	Redis, DynamoDB, Aerospike
Column-oriented	Each column stored contiguously	Analytics: reads only the columns in the query	Redshift (data warehouses)
Wide-column	Column families; a family's columns stay together on disk across rows	Huge write-heavy tables, flexible columns	Cassandra
Graph	Nodes + edges	Social behaviour, recommendations, fraud detection	Neo4j, Neptune, Dgraph, TigerGraph

1.1 Row-oriented (OLTP) vs column-oriented (OLAP)

📌 Anchor: `SELECT avg(price) WHERE ts = ...` on a 100-column table needs **2 columns**. A row store reads all 100 and throws away 98.

Column store (OLAP)

price: R1 R2 R3 ...

ts: R1 R2 R3 ...

Row store (OLTP)

R1: sym·price·name·exch·ts

R2: sym·price·name·exch·ts

- A column store reads only the columns in the query and doesn't even skim the others. That's why it suits **massive analytics and data warehouses** (e.g. Redshift).

Post-reads and exercises:

1. The C-Store paper (*C-Store: A Column-Oriented DBMS*).
2. Load data into Neo4j and paginate 1,000 entries.

3. Read how Redshift stores data (AWS docs).
4. Load sample data into MongoDB and Elasticsearch, then try aggregations and facets.

 Recall: Why is a column store faster for analytics?

Analytic queries touch few columns over many rows. A column store reads only those columns' contiguous blocks (and compresses them well); a row store must read every full row.

 Recall: Name a graph DB use case the slides call "solid".

Fraud detection (also social behaviour modelling and recommendations).

2. Designing Slack's realtime text communication

Requirements: multiple users, multiple channels · users DM, or message in a channel · realtime chat · scroll back through historical messages.

Similar systems: realtime chat, interactions, realtime polls, creator tools.

Brainstorm topics: channels, messages, checkpoints, realtime communication (membership). The hard parts:

1. Message persistence and fan-out
2. Socket membership, both realtime and semi-persistent
3. Realtime messages that fail to deliver
4. Edge servers are exposed directly on the internet

2.1 Data model

 **Anchor (Insight 1): A DM is just a channel with two people.** Model channels well and DMs come free.

workspace	channels	messages	membership
id, name	id, workspace_id, name, type (dm/group)	id, channel_id, user_id, message, ts	channel_id, user_id, checkpoint (last message id read)

- An early draft queried DMs by sender and receiver (`WHERE sender = ? AND receiver = ? OR ...`). Treating a DM as a channel removes that whole special case: `SELECT * FROM messages WHERE channel_id = ? ORDER BY created_at DESC`.
- `membership` can also hold per-user settings: `is_pinned`, `is_muted`, `notification_prefs`, `last_active_at`.

2.2 Where messages live

 **Anchor:** There will be a **lot** of messages, so use a **sharded DB, sharded by workspace**. All of one workspace's messages sit in one shard, so scrolling never needs a cross-shard query.

- Pick any DB you can shard (DynamoDB and Cassandra both fit). Shards are mutually exclusive.
- Access pattern: **range queries** on a channel's history, plus point updates and deletes.

2.3 Persistence comes first: three ways to deliver messages

 Anchor (Insight 2): Slack is an **enterprise** product, so **persisting** a message matters more than delivering it instantly.

#	Strategy	Example
1	Persistence first , then deliver	Slack
2	Realtime delivery first + async persistence	WhatsApp (also persists locally on the device)
3	Realtime, no persistence	Zoom chat

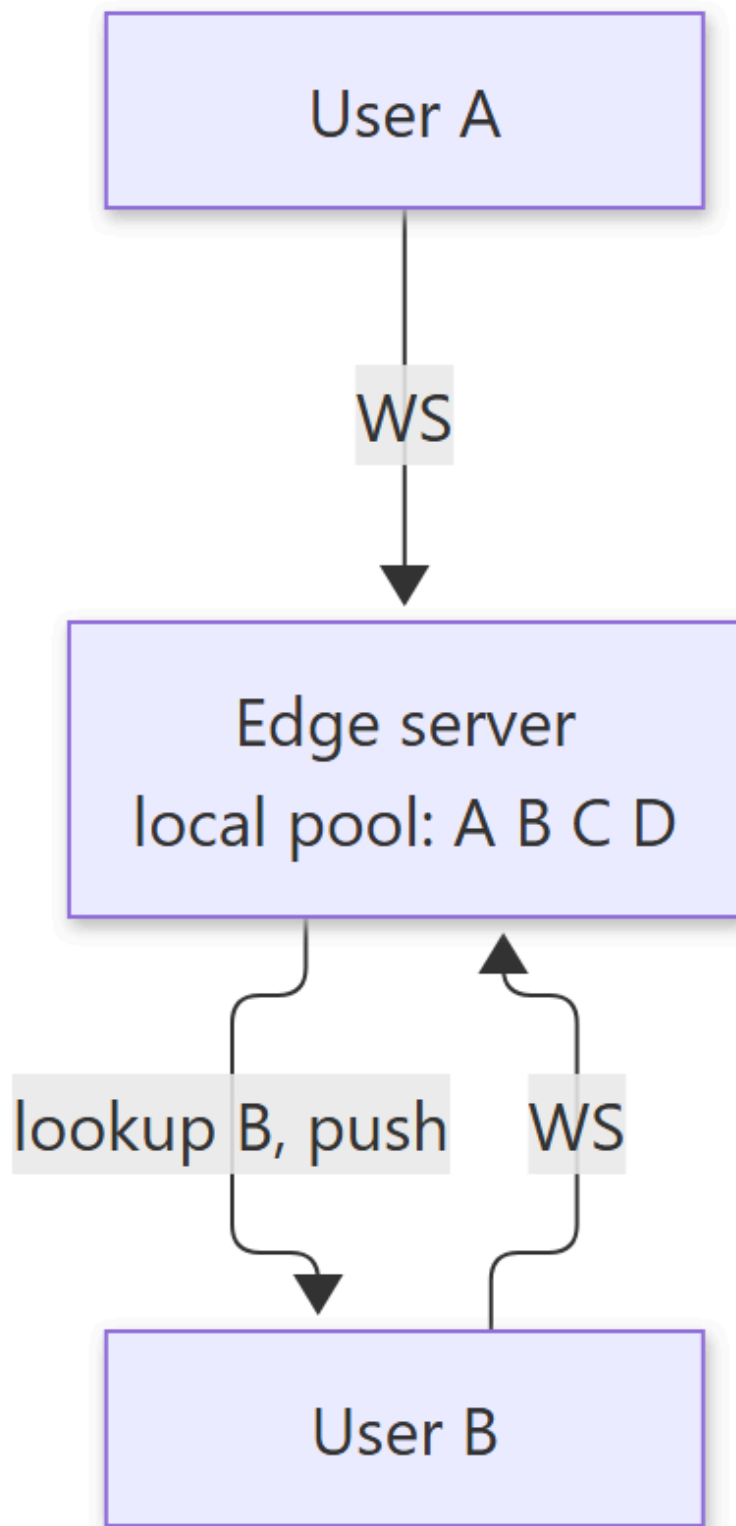
2.4 The APIs (REST)

- `send_message(from_user, channel, text)`: the API server finds the right shard from the channel and stores the message there. Works for DMs and channels alike.
- `read_messages(channel_id)`: return the most recent messages, then page backwards through older ones on that shard.
- But **REST short polling is a poor UX** for something as realtime as messaging, so we need something better.

2.5 Realtime: WebSocket edge servers

 Anchor: **One WebSocket per user** to our infra, multiplexing **all** realtime traffic. A fleet of **edge servers** holds those sockets.

- WebSockets are **persistent TCP** connections and they're **expensive**, and browsers limit them. So multiplex every kind of realtime traffic (chat, notifications, etc.) over **one** connection.
- Any service that needs to push to a user goes **through the edge servers**.
- Each edge server keeps a **local connection pool**: it knows which users are connected to it and how to reach them (Socket.IO manages this). For $A \rightarrow B$ on the same edge server: find B in the local pool and send.
- In Slack's case, the messaging service talks to the edge servers to deliver to recipients.



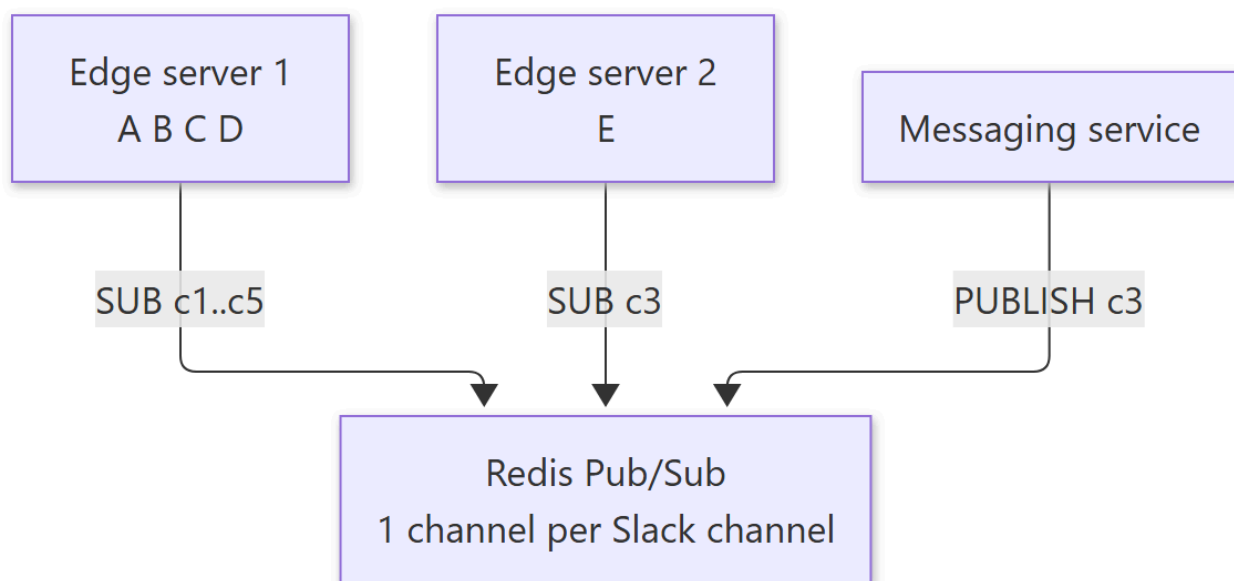
2.6 What if a realtime message is missed?

1. Every so often, the client calls the API to fetch the last ~50 messages and renders them, just in case some were missed.
2. The **checkpoint** (last message id read) won't have advanced, so the client asks for anything newer than it.

2.7 Scaling edge servers: bridge them with Pub/Sub

🔗 Anchor: Don't connect every edge server to every other one. **Each Slack channel gets one Pub/Sub channel**, and an edge server **subscribes** to the channels its users belong to.

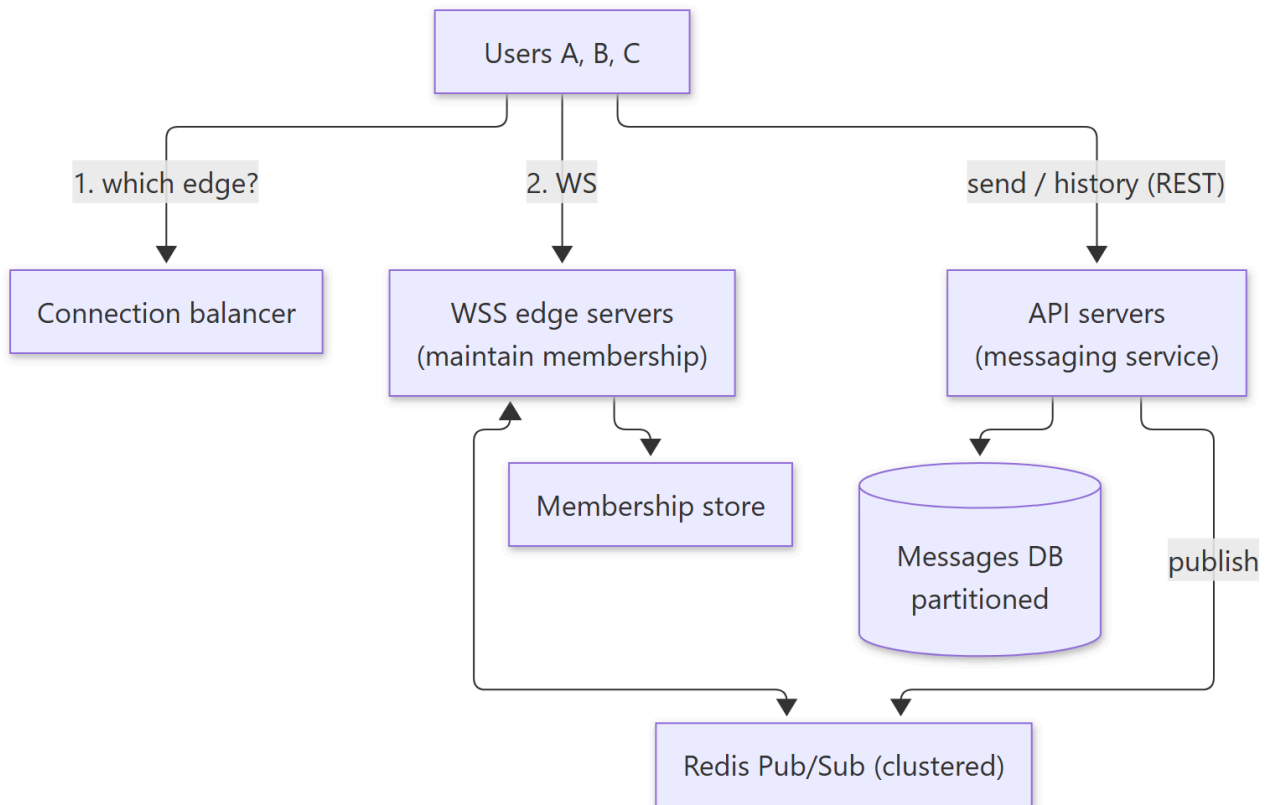
- Say each edge server holds 4 users. The 5th user connects to another server. A → B works through the local pool, but A → E only works if the message reaches edge server 2. We have to bridge the gap.
- Full mesh between servers? With 100 edge servers that's a terrible idea.
- **Realtime Pub/Sub (Redis):**
 - A is in (c1, c2, c3), B in (c2, c4, c5), C in (c1, c3), D in (c4). So edge server 1 subscribes to c1–c5.
 - E joins, is in c3, and lands on edge server 2. Edge server 2 subscribes to c3.
 - A publish to c3 reaches **both** servers, and each pushes to its local members.



2.8 How many TCP connections can a box hold?

- A TCP connection is identified by `<src ip (32b), src port (16b), dst ip (32b), dst port (16b), protocol>`.
- Between one client IP and one server ip:port there can be at most ~65,535 connections, because only the source port varies. Across many clients the tuple differs, so a server isn't capped at 65k. Adding NICs/IPs (**dual-NIC** edge boxes) raises per-pair limits too.

2.9 Overall architecture



- A **connection balancer** tells a client which edge server to connect to. Edge servers are "naked" on the internet, or sit behind an L4 load balancer with stickiness.
- Edge servers maintain membership (user ↔ channel subscriptions).
- The API servers (messaging service) persist first, then publish to Redis Pub/Sub for realtime fan-out.

Recall: Why model DMs as channels?

A DM is a channel with two members. One schema, one query path (`WHERE channel_id = ?`), one fan-out mechanism. No sender/receiver OR queries.

Recall: Why shard messages by workspace?

All of a workspace's channels live on one shard, so history scrolling and channel queries never cross shards. Workspaces are naturally isolated.

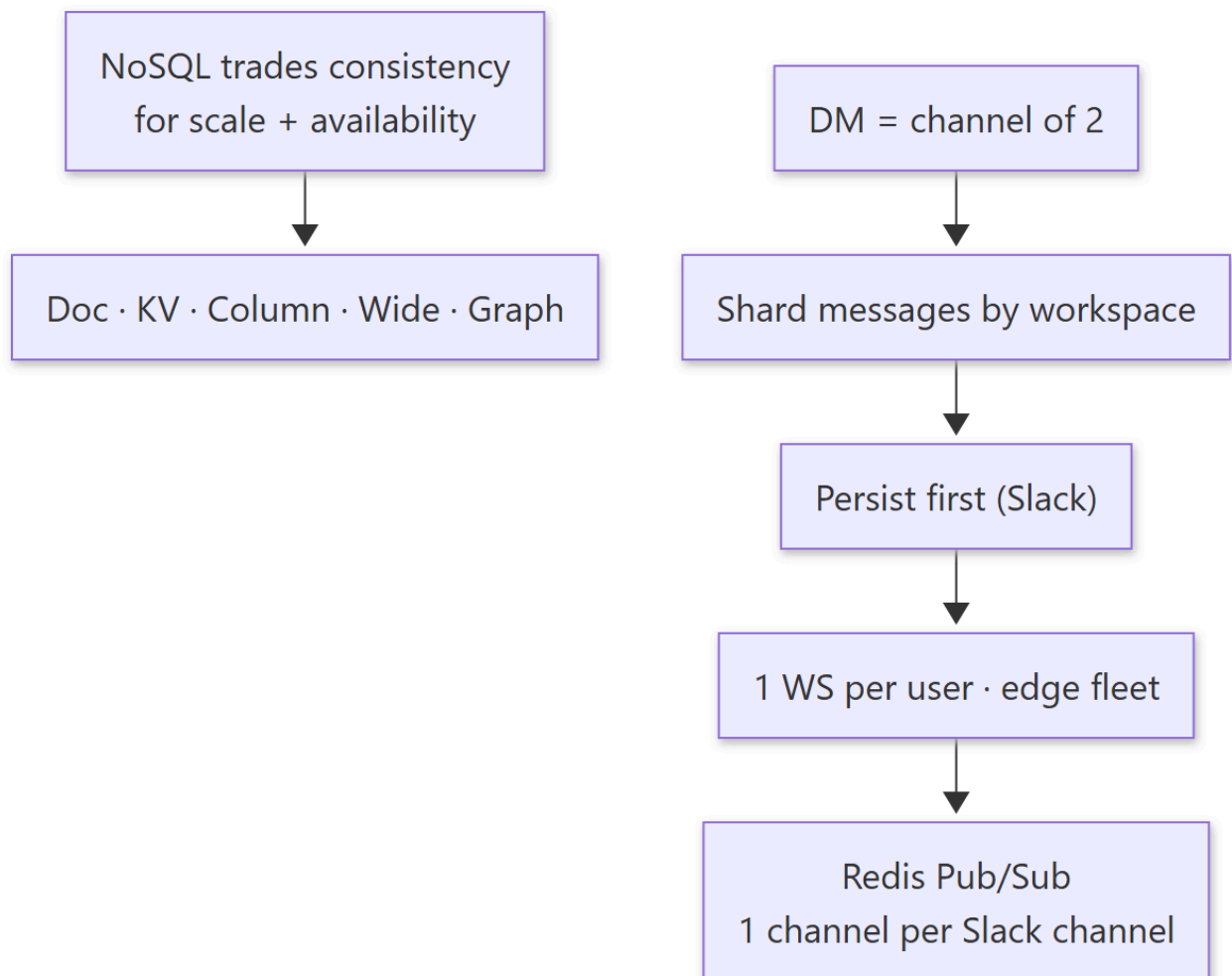
Recall: How does a message from A (edge 1) reach E (edge 2)?

The messaging service publishes to the Redis Pub/Sub channel for that Slack channel. Edge 2 is subscribed because E is a member, so it receives the message and pushes it down E's WebSocket.

What these notes add beyond the slides

- **Redis Pub/Sub is fire-and-forget.** If an edge server is disconnected at publish time, it never sees the message. That's why the slides' fallbacks (periodic fetch, checkpoint) are essential, not optional. For stronger delivery, use Redis Streams or Kafka.
- **Workspace sharding has a hot-spot risk:** one giant enterprise workspace can outgrow a shard. Real systems add sub-sharding (by channel) for the largest tenants.
- **The 65,535 limit is about source ports per (src ip, dst ip:port) pair.** On a server, the practical caps are file descriptors and memory per connection, not 65k.

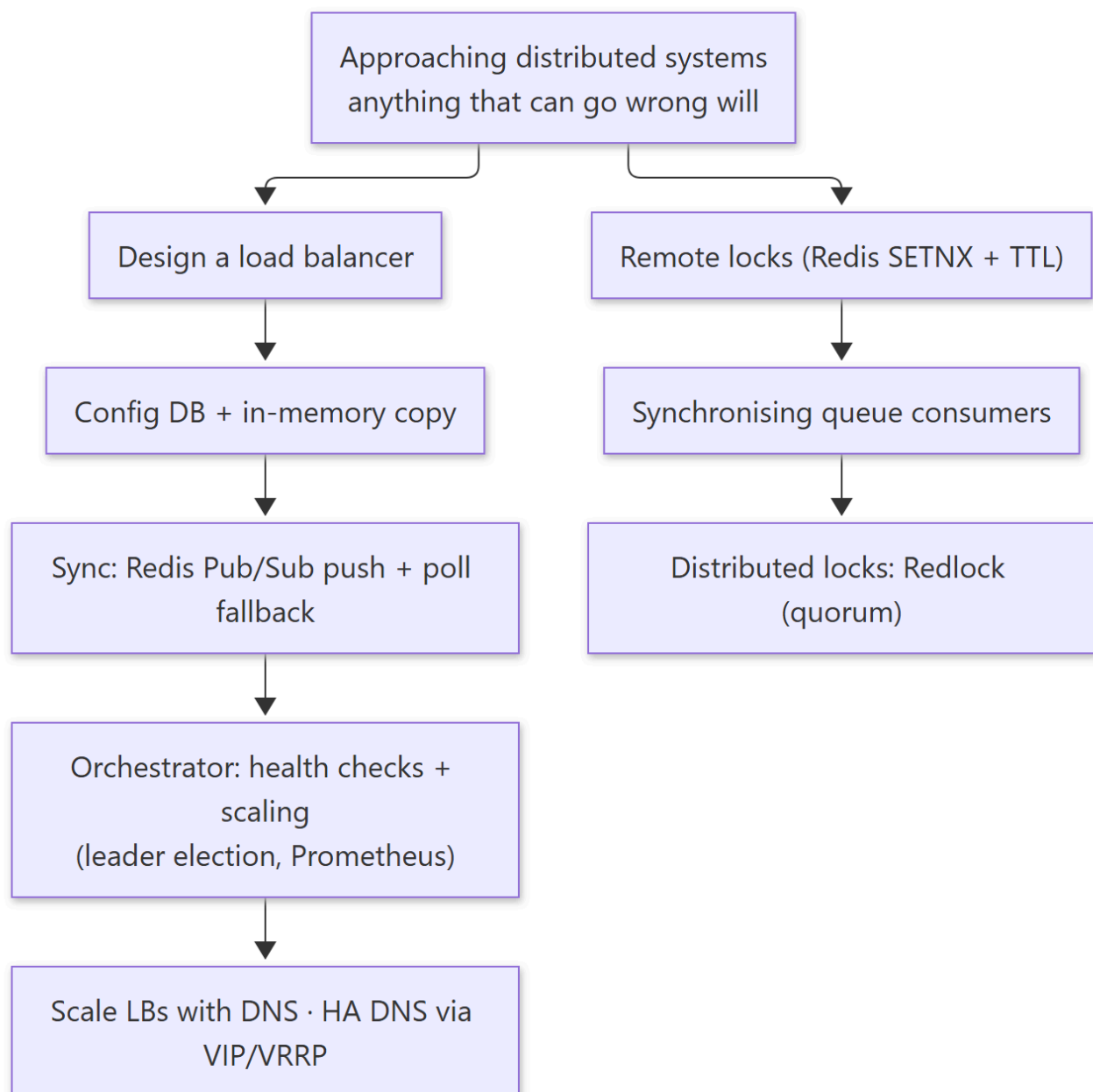
One-page memory card



03-01 · Arpit Bhayani's session notes: Distributed systems, Load balancer design, Remote & distributed locks

Source: 03-01.pdf (System Design Masterclass, Google Drive, view-only). Study notes in my own words, not a copy. Pages covered: 1–21 of 21. Unreadable pages: none. Page 5 is a whiteboard summary of the whole LB design; it's covered piece by piece below.

Map of the session



Agenda: **Approaching distributed systems · Designing a load balancer · Remote and distributed locks · Synchronising consumers.**

1. Approaching distributed systems

 Anchor: **"Anything that can go wrong will go wrong."** This is both the best and the worst thing about distributed systems, and designing *for* it is the key to a good one.

- A distributed system = many components on many machines, working as **one coherent system** to solve a bigger problem.
- For every component, ask: what if it's down, slow, or partitioned? Then design the fallback.

2. Designing a load balancer

 Anchor: We take LBs for granted, but they're among the most important components in any system. **Absorb the patterns used here to achieve high availability**; they recur everywhere.

Why an LB: fault tolerance; no over-loaded server. **Requirements:** balance the load · tunable algorithm · scale beyond one machine. **Terminology:** *LB server* (the balancer) and *backend server* (what it forwards to).

Brainstorm: LB configuration, config sync, admin console, monitoring, scaling, availability.

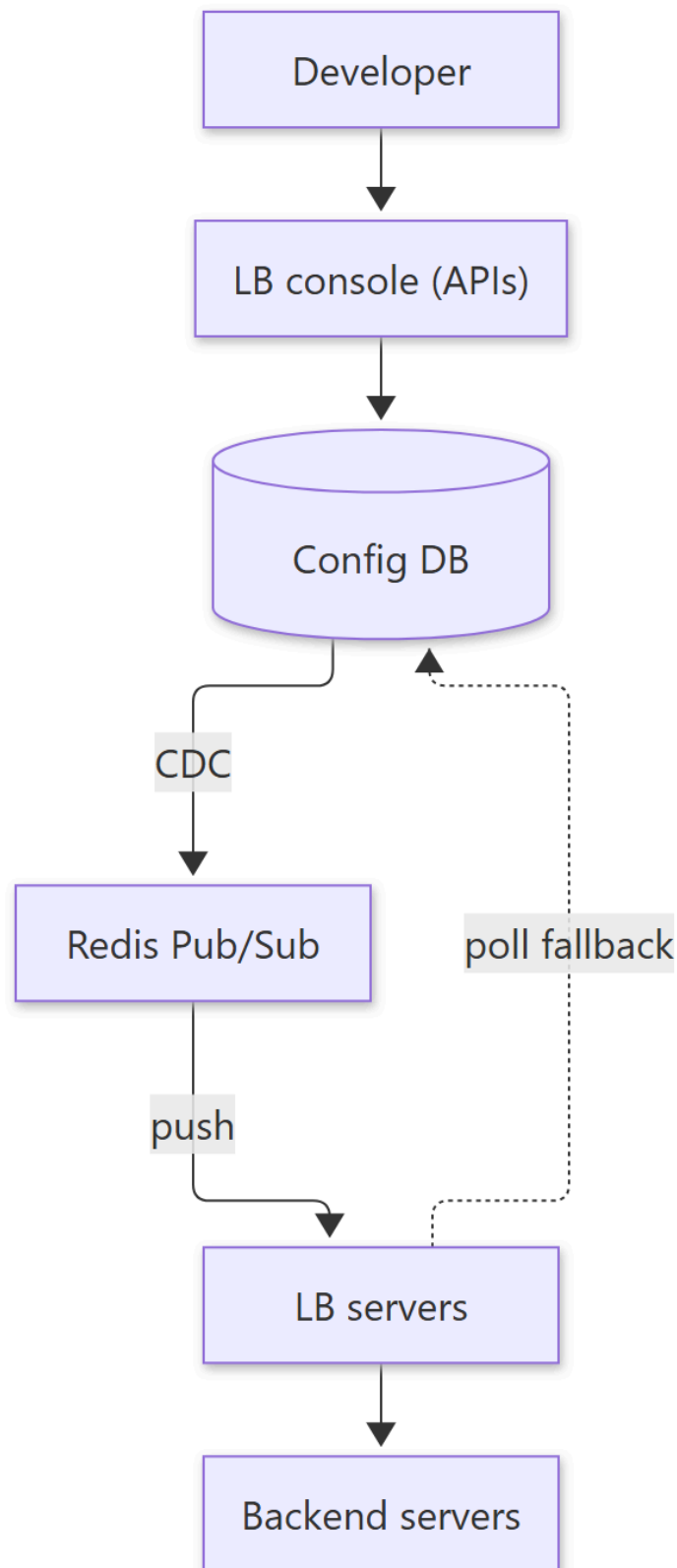
2.1 Configuration

- Each LB needs config: algorithm, backend server list, health endpoint, poll frequency, etc.
- Store **config per LB** in a **configuration DB**. A simple KV works: `lb_id → { b_servers: [...], algo: "..."}.`
- Calling the DB on every request would be catastrophic for latency, so **keep an in-memory copy on each LB server** and keep it in sync.

2.2 Keeping config in sync

 Anchor: **Push for speed, poll as a safety net.**

Approach	How
Pull	A CRON job on each LB server re-reads the config periodically
Push (reactive)	The LB console writes the config DB, then publishes a change on Redis Pub/Sub ; LB servers update immediately



Availability questions:

1. **Redis Pub/Sub down?** Fall back to polling until it's back.
2. **Config DB down?** Use a replica. The config is already cached on each LB anyway, so only *config updates* are affected; reads keep working.
3. **What about the LB servers themselves?** See the orchestrator.

2.3 Orchestrator (focused responsibilities)

Anchor: The orchestrator **watches health** (backends and LB servers) and **scales** LB servers. It writes changes to the DB, and CDC/Pub/Sub carries them out to the LBs.

- Keeps an eye on the **health of backend servers**. If one is unhealthy, it updates the backend list in the DB, and that change reaches the LB servers through Pub/Sub.
- Monitors the **LB servers' health** and **scales them up and down**.
- Health check example: `GET /health` should return 200 OK, polled every ~15 s.
- Read: **φ-accrual failure detection** (gives a suspicion *level* instead of a binary up/down).

Who monitors the orchestrator? It recovers itself through **leader election**:

- An orchestrator **master** monitors the workers and assigns them **mutually exclusive** work.
- **Workers** do the grunt work: heartbeats and monitoring.
- If the master dies, the workers elect a new one (e.g. coordinated through ZooKeeper).

How does it decide to scale LB servers? On CPU, memory and number of TCP connections. Where's that data? A **monitoring component (Prometheus)**. LB servers and backends export CPU/mem/network metrics; the orchestrator queries Prometheus. The same metrics can be shown to customers in the LB console.

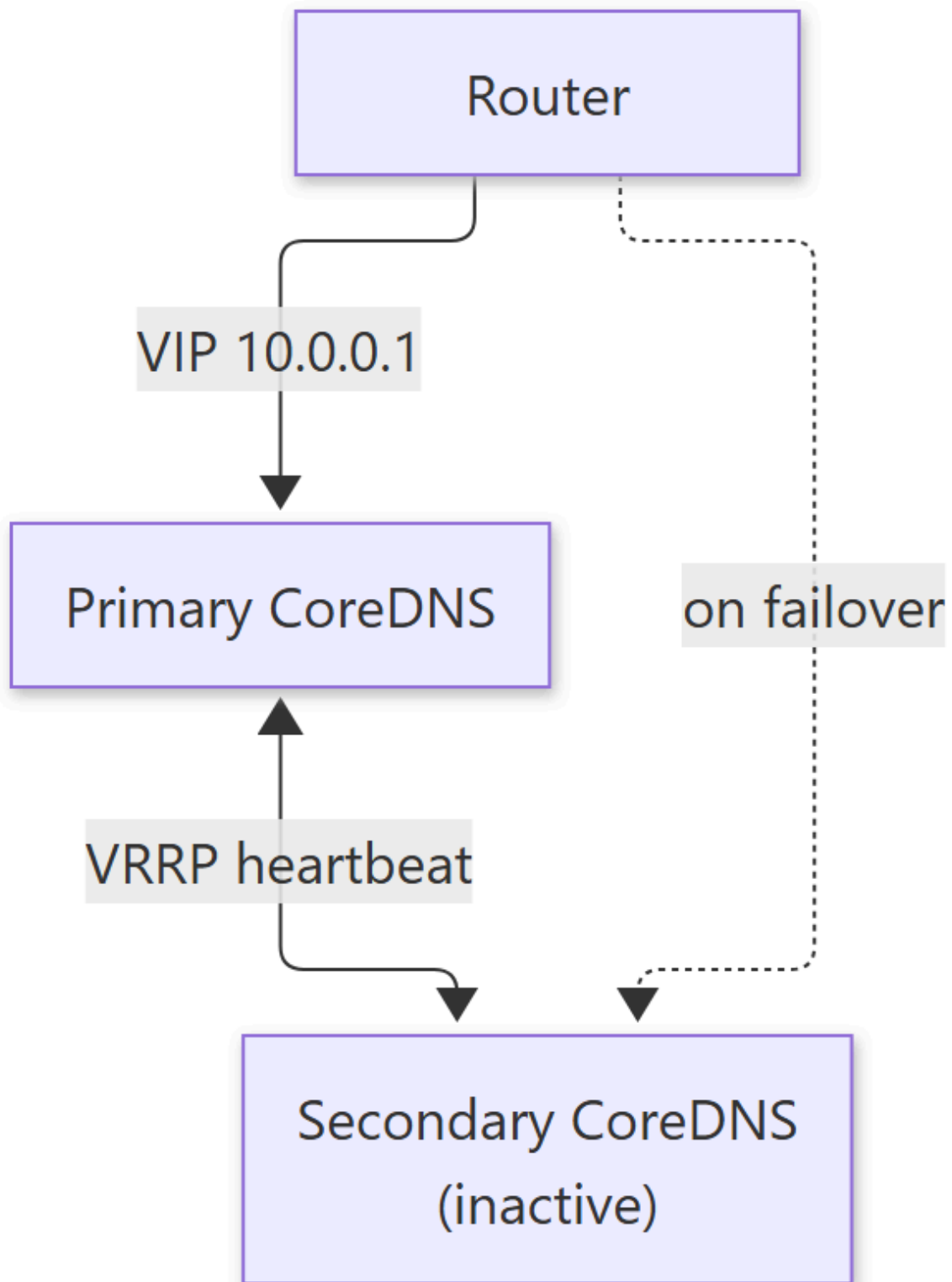
2.4 Scaling the LB tier itself: DNS

Anchor: What is shared and scales well? **DNS**. Put many LB servers behind one domain name.

- `lb.payments.google.com` → DNS returns one of 10.0.0.1 / 10.0.0.2 / 10.0.0.3, **balancing resolutions by weight**.
- Run your own **CoreDNS** in private infra. It just resolves names to IPs, so it's very lightweight (~**32K RPS on a 4 GB machine**), and clients cache the result, which means even fewer requests.
- This is a common pattern across many systems: route to the right geo, or hide multiple replicas behind one name.
- At AWS scale you don't run one LB infra for everything; you **shard** it.

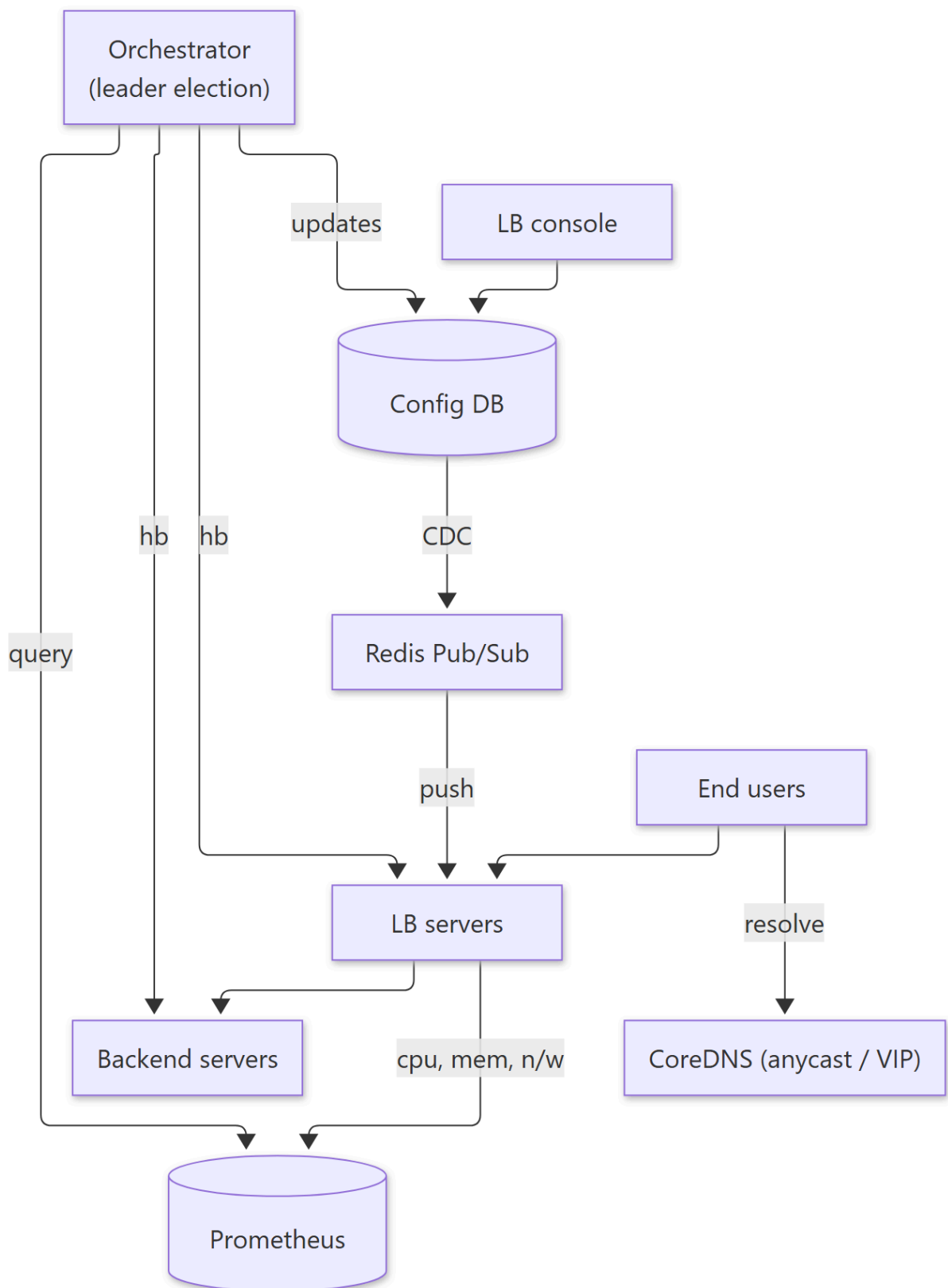
2.5 High availability for DNS: Virtual IP

Anchor: **Two DNS servers share one Virtual IP**. If the primary dies, the secondary takes over the VIP, and **no client config changes**.



- **VRRP** (protocol) + **ARP** (announce the new MAC for the VIP) + **VIP**. The VIP is configured at the instance, router, firewall, etc.
- When the secondary detects the primary is down, it "takes over" the VIP.
- Read: **Anycast** + **BGP** (many machines advertising the same IP; BGP requires authorisation, since only authorised networks can advertise certain prefixes) and VRRP for redundancy and failover.

2.6 Final LB architecture



Reads: *Maglev: A Fast and Reliable Software Network Load Balancer* (Google) · Anycast as a load-balancing feature · ϕ -accrual failure detection.

Recall: How do LB servers get config fast and safely?

Each LB keeps an in-memory copy. Changes are pushed via config DB → CDC → Redis Pub/Sub. If Pub/Sub is down, LBs poll. If the config DB is down, the cached copy keeps traffic flowing and only updates pause.

Recall: How do you scale the LB tier and make DNS highly available?

Weighted DNS (CoreDNS) spreads clients across many LB servers. DNS itself runs as primary + secondary sharing a Virtual IP via VRRP; on failure the secondary claims the VIP (ARP), so clients change nothing.

Recall: Who watches the orchestrator?

It recovers itself: a master assigns mutually exclusive work to workers, and leader election picks a new master if it fails.

3. Remote locks

 Anchor: Threads sync with **mutex/semaphore**, processes with **disk** (lock files), machines with a **remote lock** held by a central **lock manager**.

- Example of a process-level lock: `apt-get upgrade` can't run twice at the same time (it holds a lock file).

3.1 Synchronising consumers over an unprotected remote queue

- A message broker queue with **no protection**: we want **only one consumer** to read from it at a time.
- Consumer pseudocode: `ACQ_LOCK()` → `READ_MSG()` (process and delete) → `REL_LOCK()`. Everyone else waits on `ACQ_LOCK`.

What we need from the lock manager:

- **Atomic operations**, so two machines can't both acquire.
- **Automatic expiration**, so a crashed holder can't lock everyone out forever.

So: Redis or DynamoDB. Redis is the popular choice because it's in-memory and fast. Store `queue_id` → `consumer_id` with an expiry, e.g. `q7: consumer2 [EX 300]`, meaning consumer 2 holds the lock for at most 5 min.

```
acquire_lock(q):
    me = get_my_id()
    loop:
        if redis.set(q, me, nx=True, ex=300): return    # SET if Not eXists + TTL
        # else retry
```



```
release_lock(q):                # must be ATOMIC: run as one Lua script via EVAL
    if redis.get(q) == me: redis.delete(q)
```

- Why check-then-delete in Lua? If the lock expired and someone else took it, a plain `DEL` would delete **their** lock. The `get == me` check and the delete must happen atomically.
- Where else? **MongoDB transactions** use locks on the rows they touch.

 Recall: Two properties a lock manager must have?

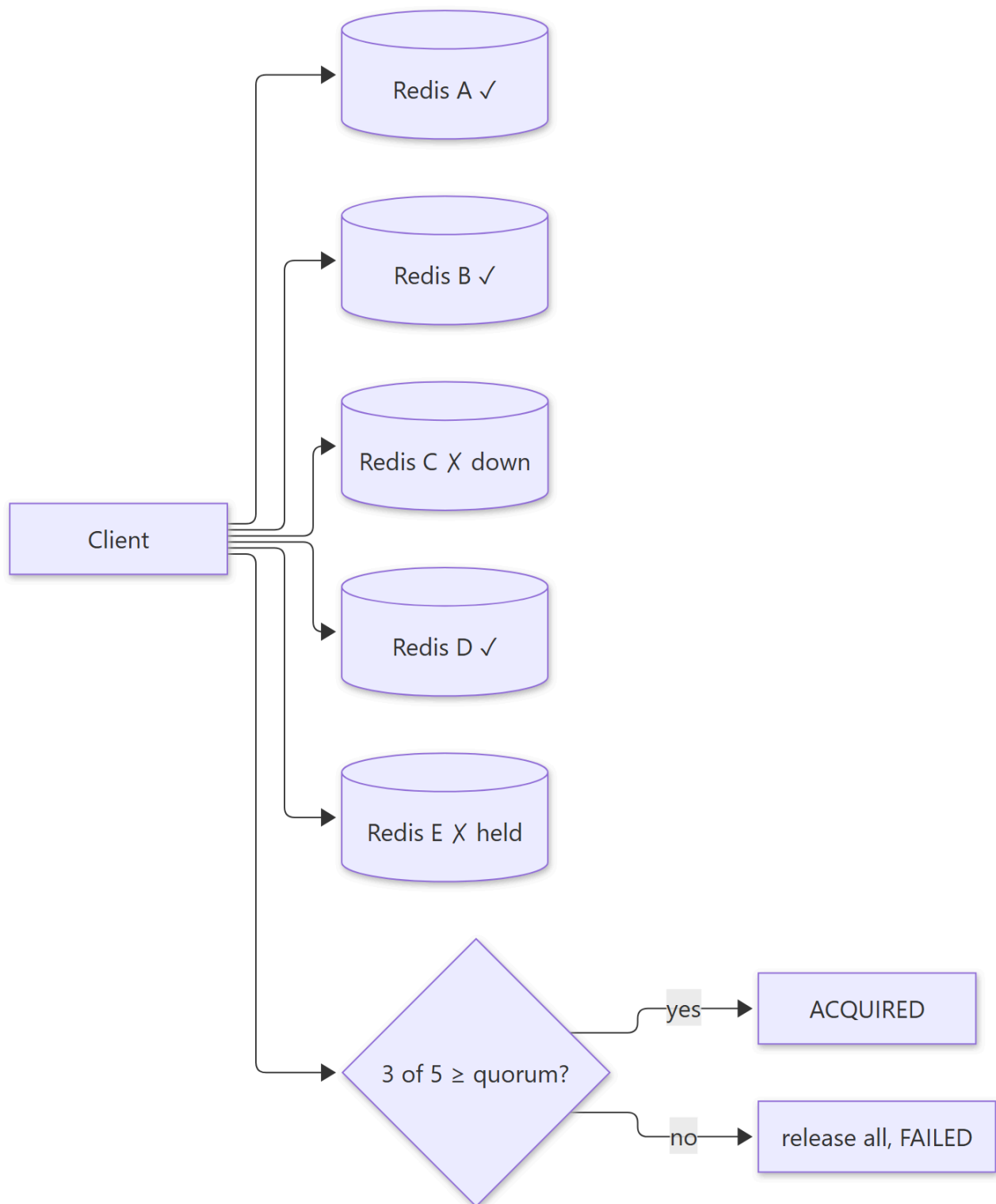
Atomic acquire (SETNX-style, so only one wins) and automatic expiry (TTL, so a crashed holder doesn't deadlock everyone).

 Recall: Why must release be a Lua script?

"Read the owner, then delete" must be atomic. Otherwise you can delete a lock that expired and was re-acquired by someone else.

4. Distributed locks: Redlock

 **Anchor:** Take the remote lock and **distribute it: 5 independent Redis masters**, and you hold the lock only if you win a **majority (> 50%)** of them.



- 5 Redis master nodes, **no replication, all independent**.
- Acquire: try `SETNX` (with a timeout) on all 5. If acquired on **> 50%**, the lock is **acquired**. Otherwise release the ones you got and return **failed**.

```
quorum = ceil(len(servers) / 2)      # 3 of 5
acquired = sum(r.set(q, me, nx=True, ex=300) for r in servers)
if acquired >= quorum: return OK
for r in servers: r.eval("if get(q) == me then del(q)") # release
```

Why bother? There's no single point of failure. The trade-off:

Setup	Throughput	Correctness	Availability
Single Redis	↑	↑	↓ (SPOF)
Master + async replica	↑	↓ (a lock can be lost on failover)	↑
Redlock (N independent)	↓	↑	↑

- **Consensus always slows things down** (think of picking a restaurant with your friends). Lock-acquire time grows with the number of Redis nodes and with the number of competing clients.

Reads: *Chubby: lock service for loosely coupled distributed systems* · Martin Kleppmann's critique of Redlock and the reference implementation · distributed transactions / 2PC · JunoDB, DragonflyDB. **Exercises:** simulate distributed locks with threads; build a load balancer.

 Recall: How does Redlock decide a lock is held?

The client tries SETNX + TTL on N independent Redis masters. If it wins a strict majority (e.g. 3 of 5) within the timeout, it holds the lock; otherwise it releases what it got and fails.

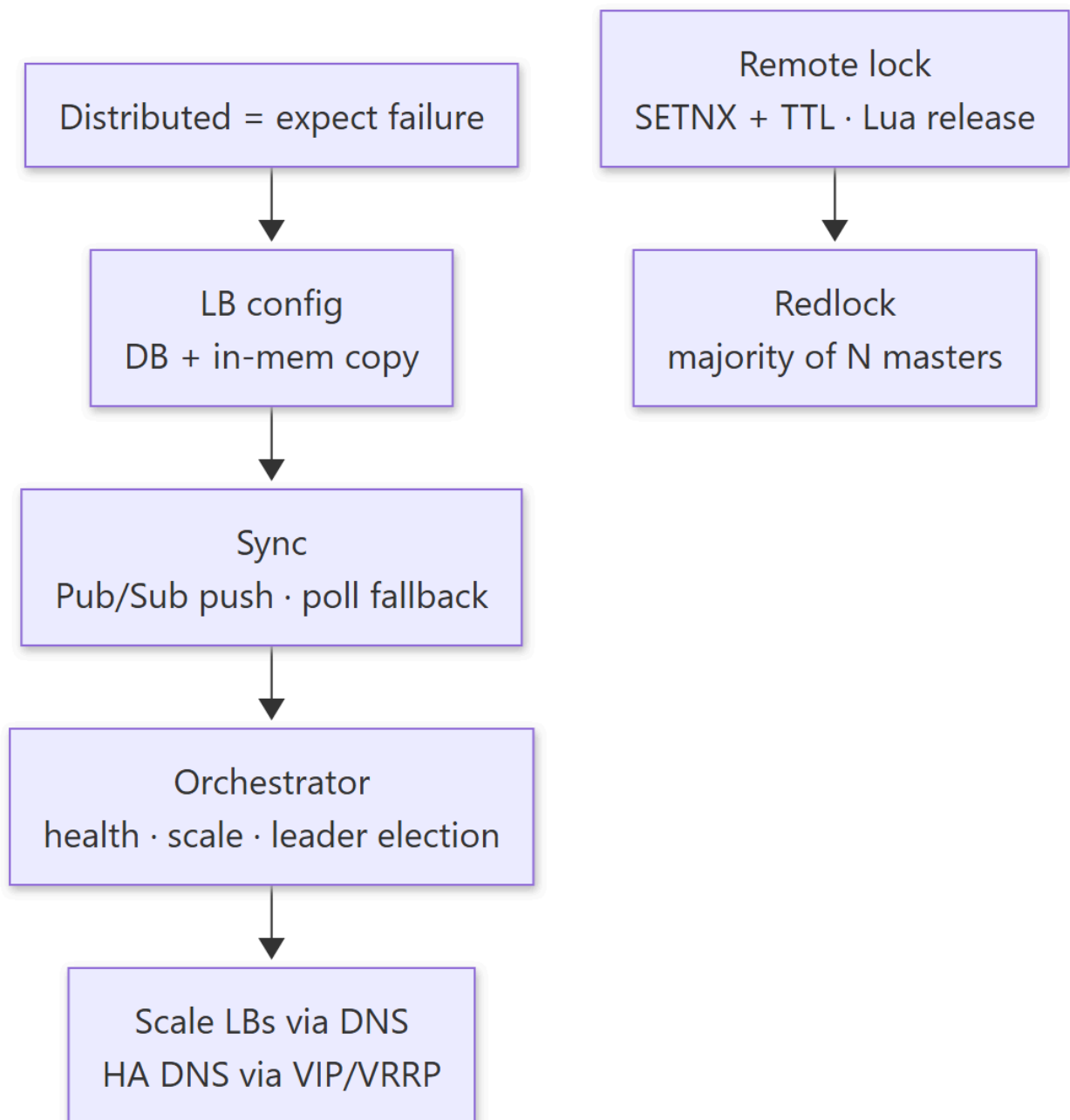
 Recall: Why not just use one Redis master + replica for locks?

Replication is async. The master can grant a lock, crash before replicating, and the promoted replica grants the same lock to someone else, which loses correctness.

What these notes add beyond the slides

- **TTL locks need fencing tokens.** If a consumer pauses (GC, slow I/O) longer than the 300 s TTL, its lock expires, another consumer acquires it, and **both** act. Kleppmann's fix: the lock service hands out a monotonically increasing **fencing token**, and the protected resource rejects writes carrying an older token. Redlock alone doesn't provide this, and that's the core of his critique.
- **Redlock depends on bounded clock drift** across nodes, since TTLs are wall-clock based. Chubby/ZooKeeper/etcd use consensus plus sessions instead.
- **Busy-wait in `acquire_lock`**: the loop spins on Redis. Add backoff with jitter (or BLPOP-style waiting) to avoid hammering the lock manager.
- **Arithmetic checked:** $\text{quorum} = \text{ceil}(5/2) = 3$ ✓ (> 50% of 5).

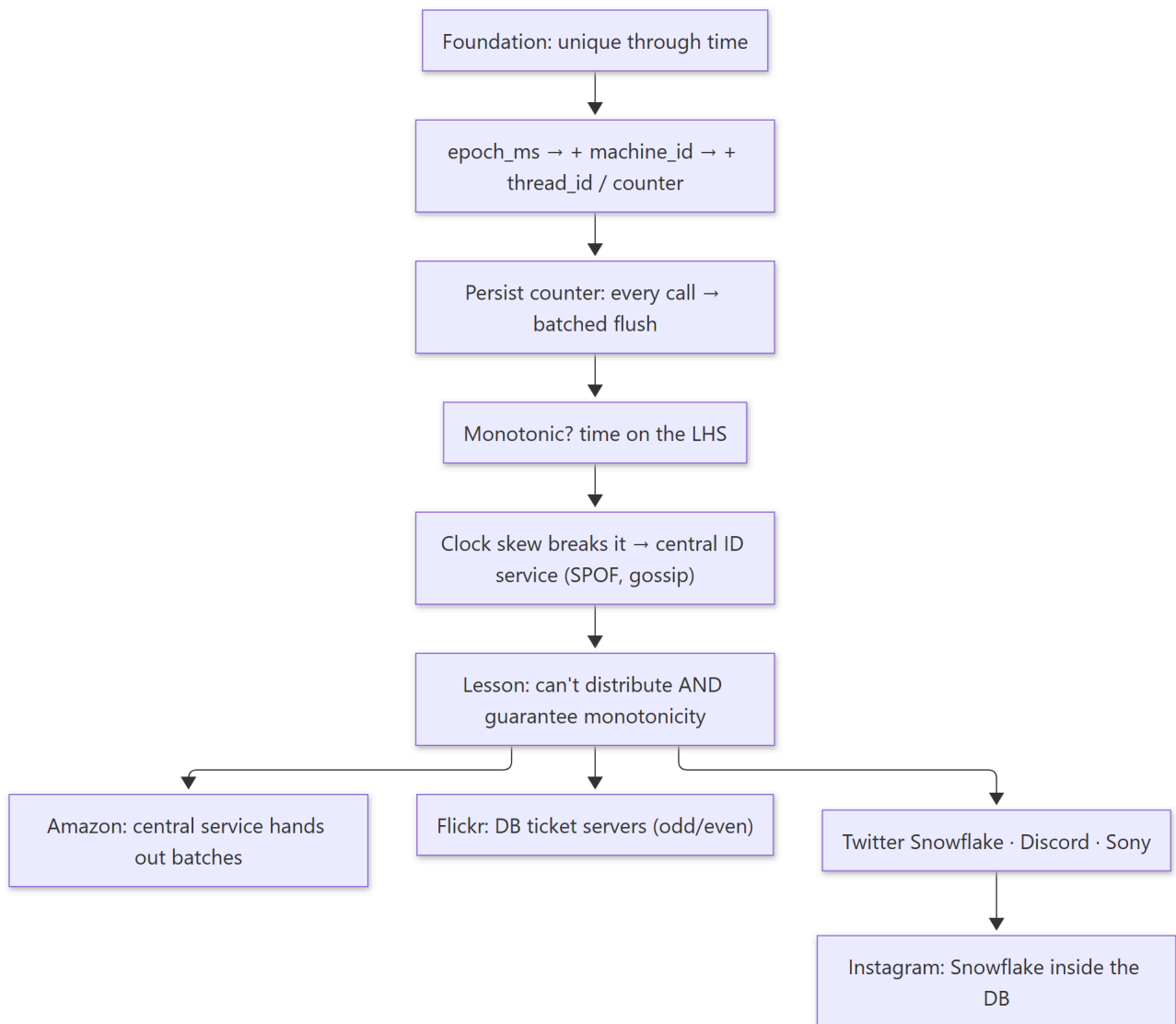
One-page memory card



03-02 · Arpit Bhayani's session notes: ID generation (from a function to Snowflake)

Source: 03-02.pdf (System Design Masterclass, Google Drive, view-only). Study notes in my own words, not a copy. Pages covered: 1–23 of 23. Unreadable pages: none.

Map of the session



Agenda: **Foundation for ID generation · Monotonically increasing IDs · Central ID generation service · Database ticket servers · Twitter's "insane" idea · Instagram bettering it.**

1. The problem

 Anchor: **Write a function that returns something unique every time it's called**, as part of the *application logic*, not a central service.

- An ID gives uniqueness to an object, row, document or event.
- **Why do we need it?** In a sharded database with independent shards (no replication), each shard's **auto-increment will collide** with the others. So instead of relying on the DB, the application **provides the ID at insert time**.

2. Building the function step by step

 Anchor: Each fix answers "what collides now?": same ms → same machine → same thread → restart.

Version	ID =	Breaks when
1	<code>epoch_ms()</code> (time always moves forward)	Two machines, or two calls within the same ms
2	<code>machine_id + epoch_ms</code>	Two threads on one machine in the same ms
3	<code>machine_id + thread_id + epoch_ms</code>	(alternative: a counter that resets every few minutes)
4	<code>machine_id + epoch_ms + counter++</code> (atomic)	Time is now redundant...
5	<code>machine_id + counter++</code>	Counter is in memory: a restart regenerates the same IDs
6	Save the counter to disk on every call	Correct, but a disk I/O per ID
7	Flush every N calls (batching) and skip ahead by N on restart	✅ Safe and fast

2.1 Batched flush + safe recovery

 Anchor: On restart, **start from** `last_flushed + N`. You may skip some IDs, but you'll never repeat one.

- `counter = load() + 1000`. Under a mutex: `counter++`; if `counter % 1000 == 0`, `save_counter()`.
- Worked example with flush every 3: IDs 1, 2, **3 (flush)**, 4, 5, **6 (flush)**, 7, 8, **crash**. Disk says 6. Restart at **6 + 3 + 1 = 10**, which is safe. 7 and 8 were used, and 9 is skipped harmlessly.
- Use `O_SYNC` (or `fsync`) so the flushed value really is on disk.

 Recall: Why restart at `last_flushed + N + 1`?

Up to N IDs may have been issued after the last flush and before the crash. Jumping past all of them guarantees no reuse; the cost is a few skipped IDs.

3. Monotonically increasing IDs

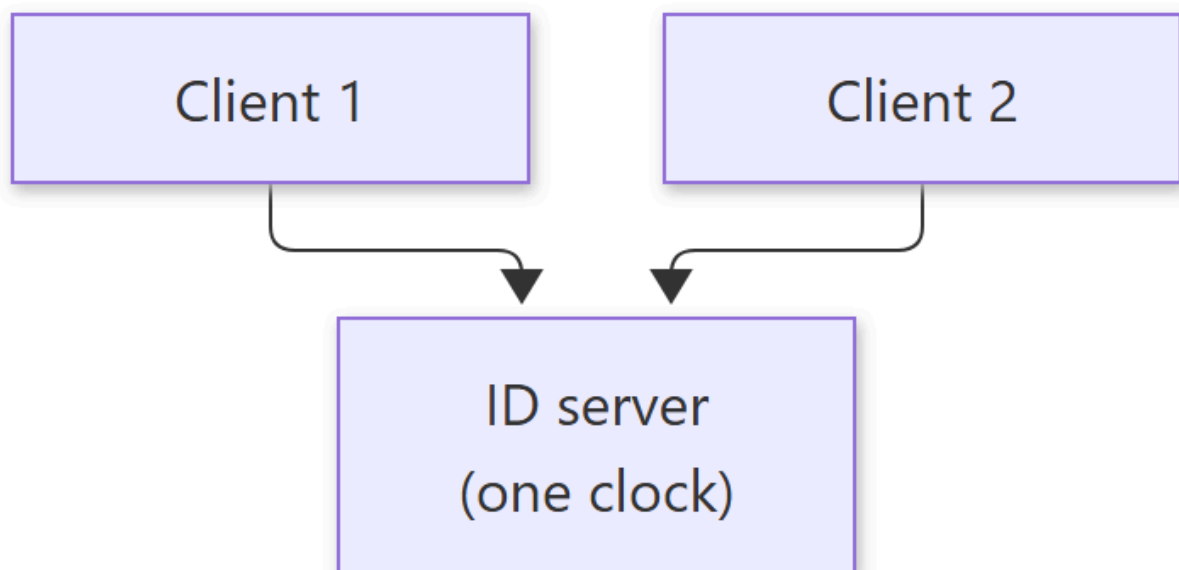
Anchor: **Put time on the left (most significant bits)**. Then sorting by ID = sorting by time ("sortability"); the right side only breaks ties.

- Why monotonic? **Conflict resolution**: who came first? If T1 sets a = 20 and T2 sets a = 30, which one wins?
- Swap `machine + time` for `time + machine`. The LHS decides the order; the RHS (counter or machine id) breaks ties: 1729 0002, 1729 0001, 1730 0002...
- **All ID generation algorithms put time on the LHS.**

3.1 Clock skew breaks it

- Clocks across machines drift. At the same instant, m2 says 23, m4 says 24, m7 and m9 say 23. The IDs come out 232, 244, 237, 239: **not monotonic**.
- So now we're really talking about **clock synchronisation**, not ID generation.

3.2 Central ID service: fixes skew, adds new problems



- One server runs the same `get_id`, so there's one clock and the IDs are monotonic. **But it's a SPOF**: the server or its disk can fail.
- Several ID servers behind an LB? Now they must **gossip to agree on a value**. The whole thing gets super complicated.

3.3 The lesson

Anchor: **There's no way to distribute ID generation and still guarantee monotonicity** (at high throughput). Real systems **relax a constraint**: non-integer IDs, or no strict monotonicity.

 Recall: Why put the timestamp in the most significant bits?

Numeric order then matches creation order (sortable, roughly monotonic). Lower bits only break ties within the same millisecond.

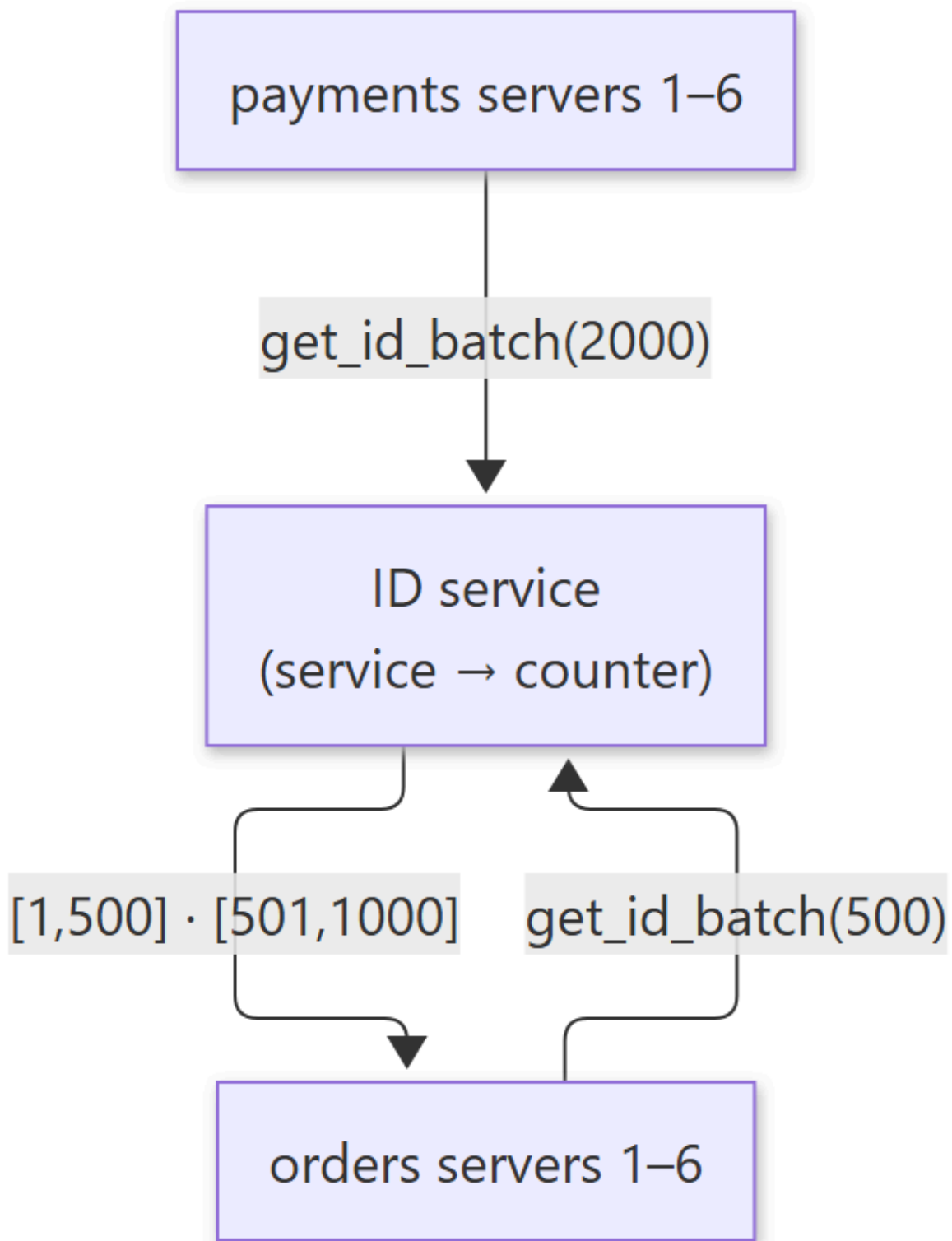
4. Real-world approaches

4.1 How long do 64 bits last?

- At 500 IDs/s $\times$ 100 services: $500 \times 100 \times 86,400 = \mathbf{4.32 \text{ billion IDs/day}}$.
- $2^{64} \div 4.32 \times 10^9 \approx \mathbf{4.3 \text{ billion days}}$. That's effectively forever.

4.2 Amazon's way: a central service that hands out batches

 Anchor: Don't call the ID service per ID. **Lease a range** ($[1, 500]$) and hand IDs out locally until it runs out.



- The service keeps a counter per *service* (orders: 0 → 500 → 1000...). Each app server asks for a batch (500, 1000, 2000...) and asks again when it's exhausted.

4.3 Why not just UUIDs?

Anchor: **If your indexes don't fit in memory, your DB can't be fast.**

- UUIDs are **128-bit** (16 B) vs a 4 B int: bigger indexes, more disk I/O on lookups.
- They're **random**, so inserts land all over the B+ tree. That's bad for index locality, though randomness is good for security (not guessable).
- **MongoDB ObjectId**: 12 bytes = 4 B epoch seconds + 5 B random + 3 B counter. Time is on the LHS, but it's still 96 bits, so it doesn't index as well as a 64-bit int.

4.4 Flickr: database ticket servers

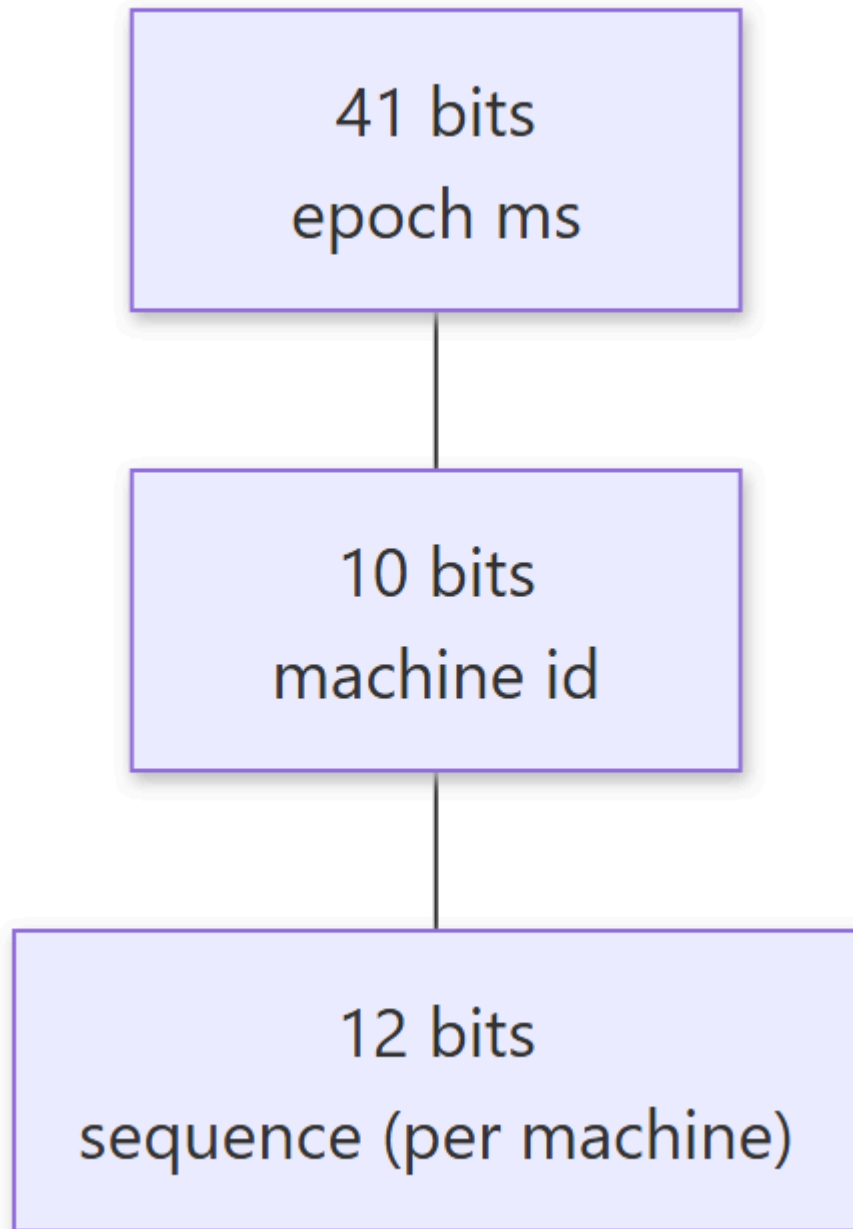
- Flickr's DB was sharded, and they needed globally unique IDs without UUIDs (which index badly).
- A **central ticket server** on MySQL: whoever wants an ID runs a query against it.

```
CREATE TABLE tickets (
  id    BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  stub CHAR(1) NOT NULL DEFAULT '',
  PRIMARY KEY (id), UNIQUE KEY (stub)
);
-- one row ('a'); bump it to get the next auto-increment value:
INSERT INTO tickets (stub) VALUES ('a')
ON DUPLICATE KEY UPDATE id = id + 1;
```

- The idea: deleting and re-inserting the single row makes MySQL generate the next auto-increment `id`. `DELETE + INSERT` in one transaction works but is expensive, so use an **upsert**.
- **Upserts in MySQL**: `INSERT ... ON DUPLICATE KEY UPDATE` (updates the row in place) vs `REPLACE INTO` (deletes the old row and inserts a new one; the session cites it as **~32x slower**). Both are atomic.
- **SPOF fix**: two ticket servers, one issuing **odd** and one **even** IDs, round-robin between them. If one goes down, the other keeps serving (e.g. 2, 4, 6, ... 200, 202).

4.5 Twitter Snowflake

 Anchor: **41 bits ms-time | 10 bits machine | 12 bits per-machine counter**, generated **inside each app server** as a library call.



- Used for tweet IDs; also adopted by Discord and Instagram.
- **Not a central service.** It runs in the API servers as a native function: the ID is computed while the tweet is stored.
- **Roughly sortable:** the first 41 bits are ms, so IDs move forward every ms. You can fetch objects before or after a time, or before or after an ID.
- **Pagination:** Twitter doesn't use `LIMIT/OFFSET`. It uses `since_id` ("I've read up to this tweet, give me the next set"). That's extremely efficient (it seeks straight into the index), and sharded DBs use the same approach.
- **Discord:** same logic, but the epoch is the first second of 2015, which gives a longer usable time range.
- **Sony:** *Sonyflake*, an open-source Go implementation.
- Why it scaled Twitter: **decentralised**. No moving parts, no service, just a function call, and no extra load on the DB.

4.6 Instagram: Snowflake inside the database

🔗 Anchor: **41 bits ms since 2011 | 13 bits logical shard id | 10 bits per-shard sequence**, computed by a **PL/pgSQL function** as the column default at INSERT.

- Requirements: sortable by time (pagination, filtering, batch processing), ~64 bits to stay efficient on indexes, **no new service**.
- Instagram runs **thousands of logical shards** on a **handful (10–15) of physical DB servers**. A logical shard is just a separate database/schema on a server (the slide shows `CREATE DATABASE` on MySQL; Instagram itself used Postgres schemas). Moving a logical shard to a new server is a dump and restore.

```
id BIGINT NOT NULL DEFAULT insta5.next_id()

next_id():
  seq_id    := nextval('insta5.table_id_seq') % 1024      -- 10 bits
  result    := (now_ms - custom_epoch) << 23             -- 41 bits time (shifted past 13+10)
  result    |= shard_id << 10                             -- 13 bits shard (5 here)
  result    |= seq_id                                     -- 10 bits sequence
```

Exercises:

1. Implement Snowflake both in the API and in the DB (as a stored procedure).
2. Create shards `insta_1 ... insta_5` on one DB server, create a `posts` table, run `ALTER TABLE` across every shard, then stand up a second DB server and move one shard to it with a dump and restore.

🔗 Recall: Snowflake vs Instagram bit layout?

Snowflake: 41 ms | 10 machine | 12 sequence (in the app). Instagram: 41 ms | 13 logical shard | 10 sequence (in a Postgres function at INSERT).

🔗 Recall: How do Flickr's ticket servers avoid a SPOF?

Two MySQL ticket servers, one generating odd and one even IDs (auto-increment offset/step), with clients round-robinning. Either can serve alone if the other fails.

🔗 Recall: Why does `since_id` pagination beat LIMIT/OFFSET?

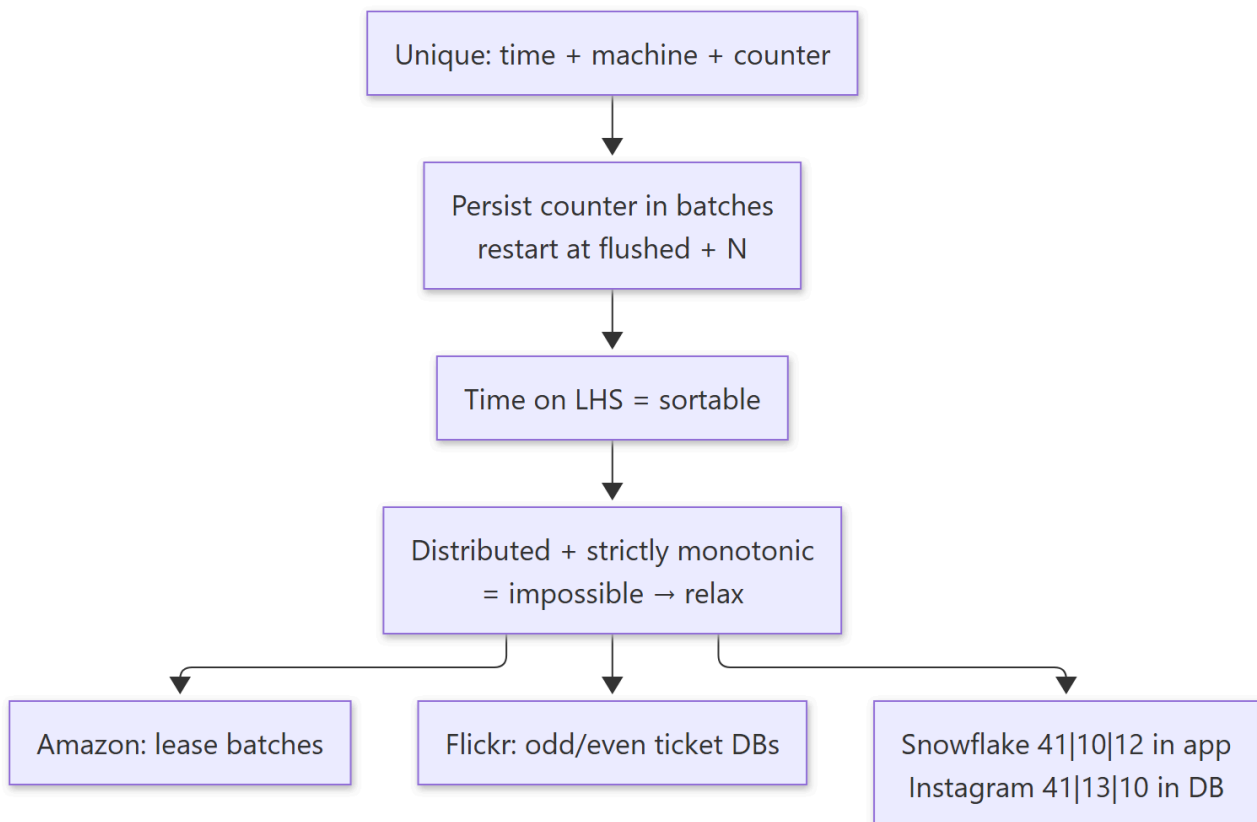
OFFSET makes the DB walk past and discard N rows. `WHERE id > since_id LIMIT k` seeks directly in the index, so the cost stays constant however deep you page.

What these notes add beyond the slides

- **Arithmetic corrections:**

- The slide divides 2^{64} by 4.32×10^9 and gets "4,611,686,018 days". $2^{64} \div 4.32 \times 10^9 \approx \mathbf{4.27 \times 10^9 \text{ days}}$ ($\approx 4.6 \times 10^9$ is $2^{62} \div 10^9$). Either way it's millions of years, so the conclusion stands.
- UUID vs int: 16 B vs 4 B is **4x** larger keys, not 3x (vs an 8 B BIGINT it's 2x).
- **Snowflake is 1 + 41 + 10 + 12 = 64 bits**. The top sign bit is kept 0 so IDs stay positive in signed int64. 41 bits of ms ≈ 69 years from the custom epoch, and 12 bits = 4,096 IDs per ms per machine.
- **Clock going backwards** (NTP correction) can produce duplicate Snowflakes. Real implementations refuse to generate, or wait, until the clock passes the last-seen timestamp.
- **Instagram's sequence is per table/shard**, so two shards can share a sequence value. Uniqueness comes from the shard bits.

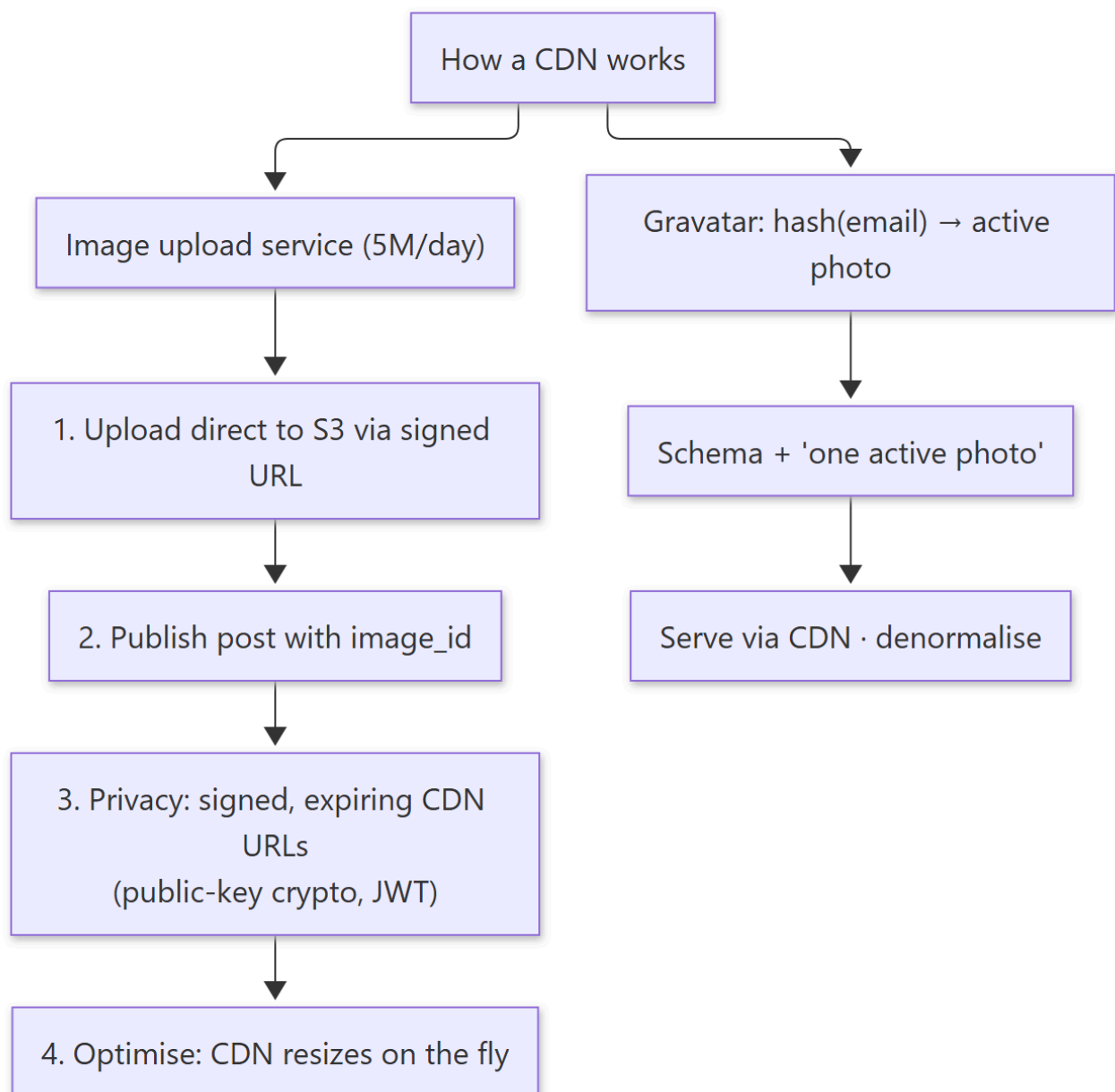
One-page memory card



04-01 · Arpit Bhayani's session notes: CDN, Photo uploads at scale, Private photos, Gravatar

Source: 04-01.pdf (System Design Masterclass, Google Drive, view-only). Study notes in my own words, not a copy. Pages covered: 1–19 of 19. Unreadable pages: none. The agenda also lists "Dynamic OG images"; the slides only mention it as a demo (GitHub, Reddit, Dev.to) on the last page.

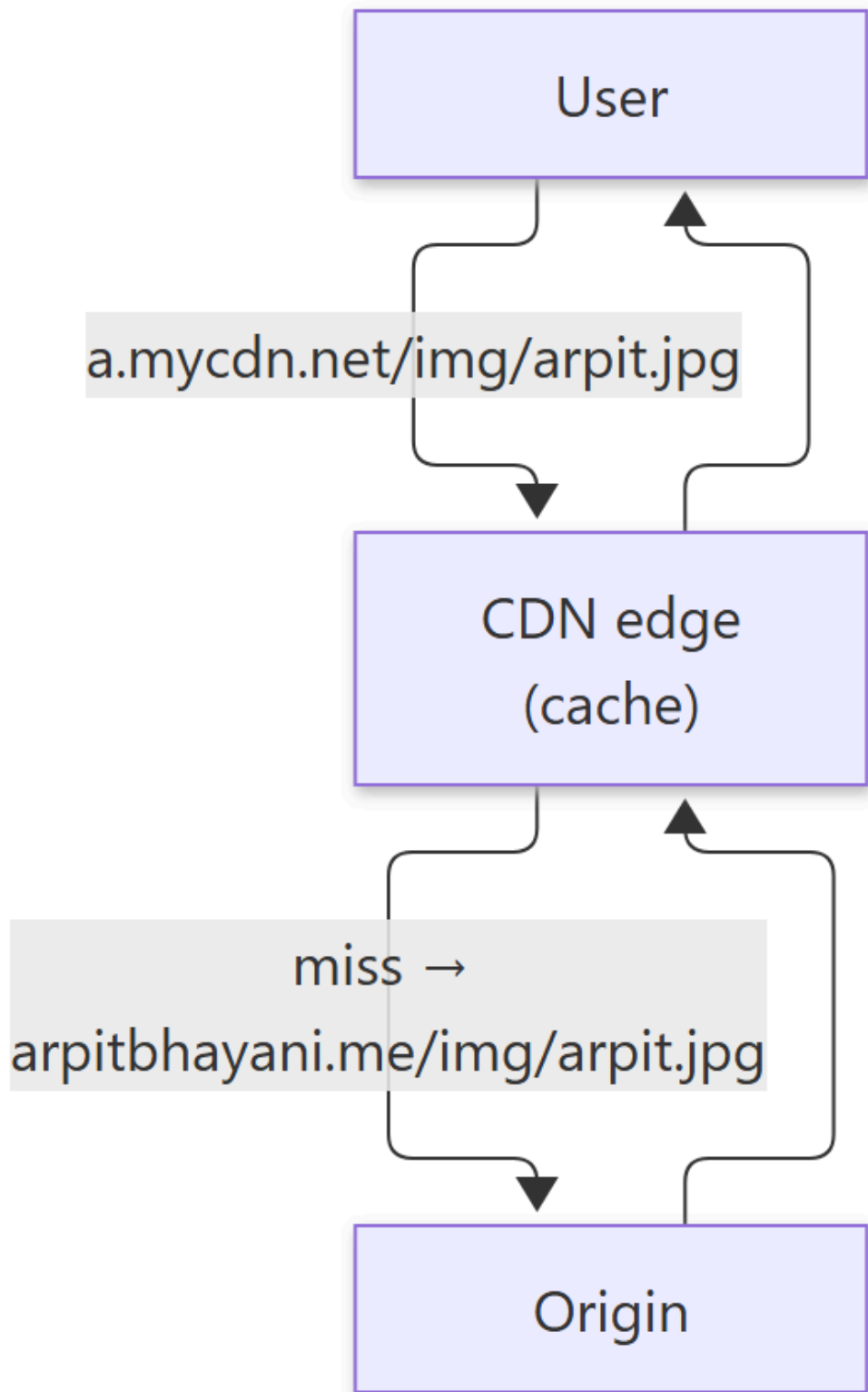
Map of the session



Agenda: **How a CDN works** · **Uploading photos at scale** · **Private photos and public-key cryptography** · **Gravatar** · **Dynamic OG images**.

1. How a CDN works

📌 Anchor: A CDN **sits transparently between users and your origin**. It has its own domain, forwards a miss to the origin, **caches** the response, and returns it.



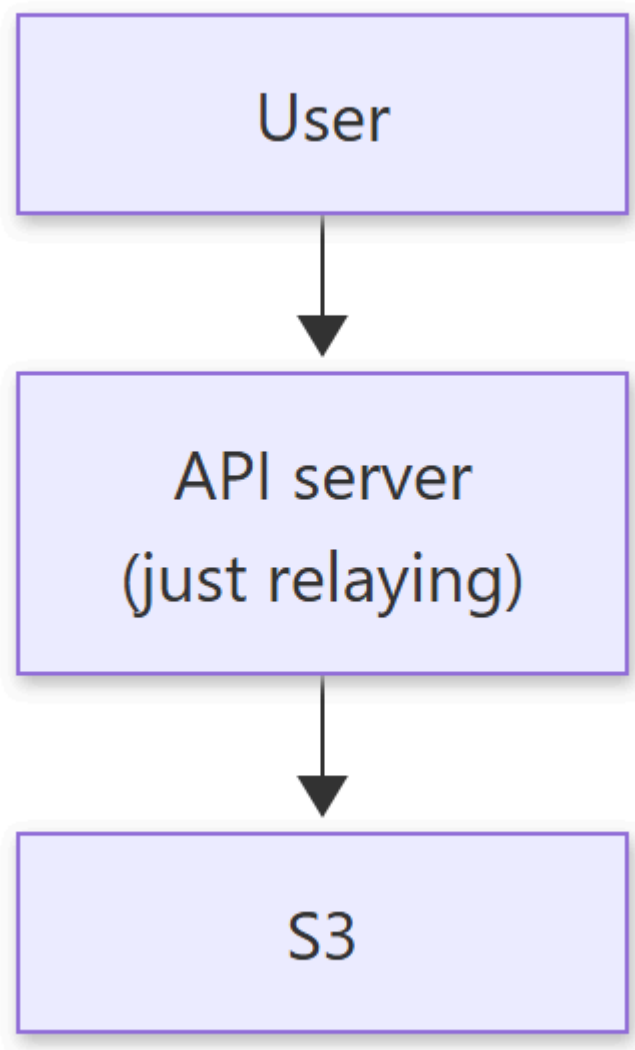
- Origin: `https://arpitbhayani.me`, serving anything (JSON, bytes, images, HTML, video).
- The CDN's own domain, e.g. `a.mycdn.net`, is configured to map to that origin.
- The user hits `a.mycdn.net/img/arpit.jpg`. The CDN finds the origin for `a.mycdn.net`, forwards the request to `arpitbhayani.me/img/arpit.jpg`, **caches it** and returns it. The next user is served from cache.
- The session builds a toy CDN to make this concrete. **Exercise:** sign up for Cloudflare, configure a CDN, and explore its features.

2. Image upload service

Requirement: 5M photo uploads per day. **Brainstorm:** storage, data flow, separation of concerns, privacy, extensibility, optimisation. **Key question: should the upload go through your API server? No.**

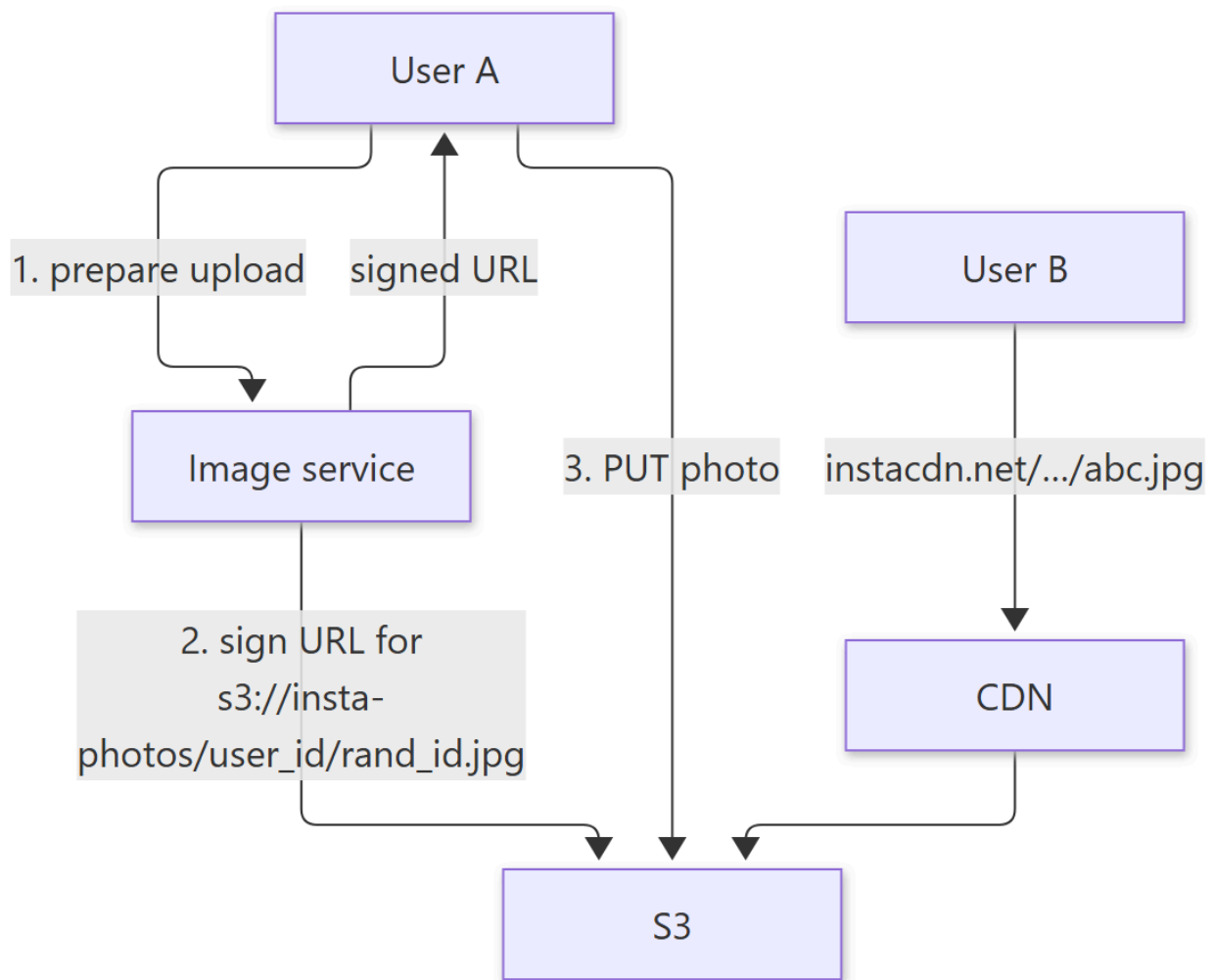
2.1 Why not proxy uploads through the API

 Anchor: user → API → S3 **doubles the bandwidth**, and the API server just relays bytes, burning CPU, memory, network and disk for nothing.



2.2 Step 1: upload straight to S3 with a signed URL

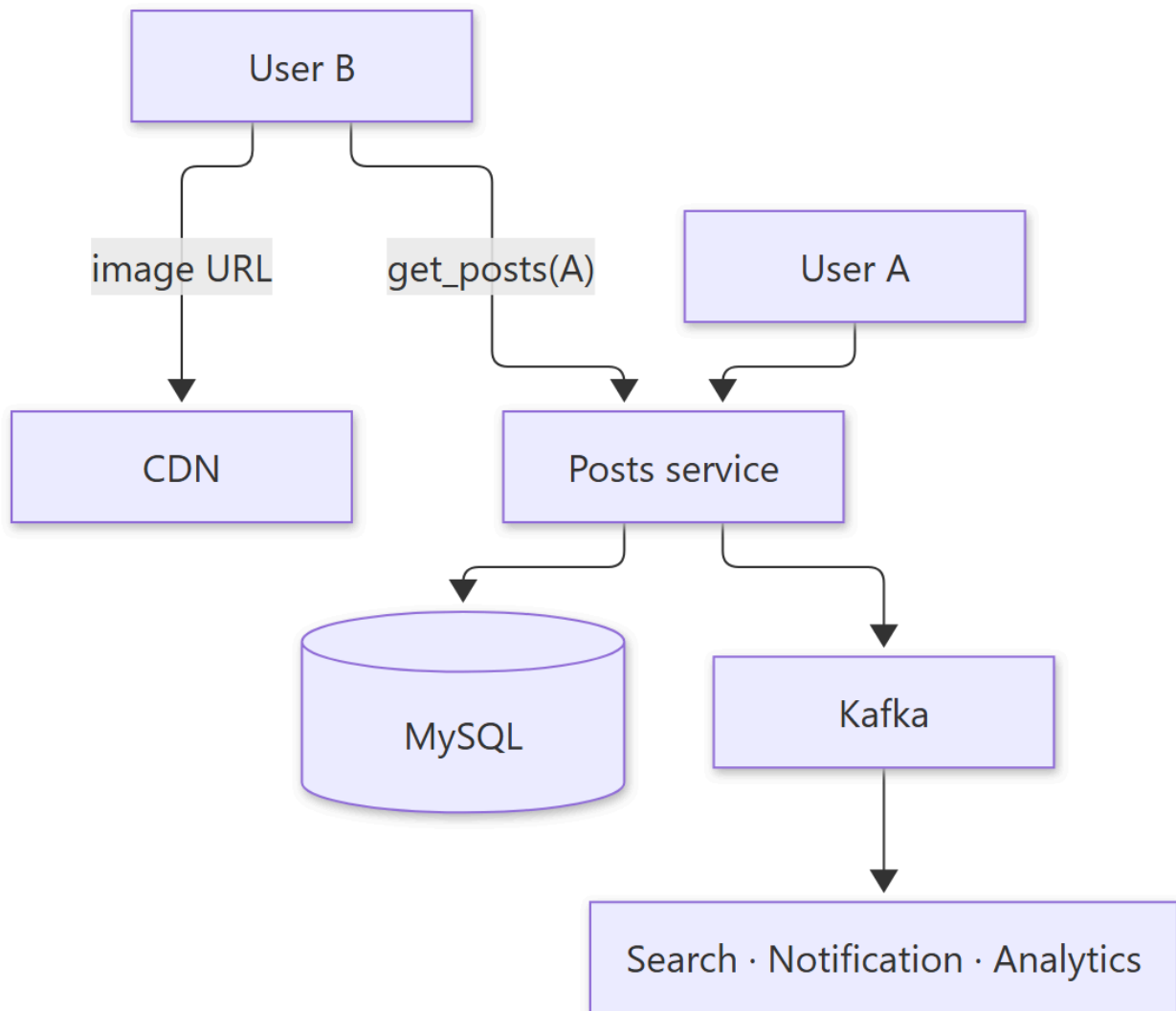
🔗 Anchor: **Ask for permission, then upload directly.** The image service hands out a **pre-signed S3 URL** for one path; the client PUTs the bytes to S3.



1. User A tells the **image service** it wants to upload.
2. The image service generates a **random image id** and asks S3 for a **signed URL** that lets *anyone holding it* upload to that exact path, e.g. `s3://insta-photos/<user_id>/<random_photo_id>.jpg`.
3. User A uploads **directly** to S3, and the file lands at that path.
4. Other users view it through the CDN (`instacdn.net/...`), which fetches from S3.

2.3 Step 2: publish the post

- The client remembers the **random image id** it got and creates the post: `posts(id, user_id, image_id, caption)`. (LinkedIn and CDNs work the same way.)
- The photo's S3 path can be computed: `s3://insta-images/<user_id>/<image_id>`. No `img_url` column is needed, which saves space.
- The **posts service can validate** that the `image_id` belongs to the same user.
- The image service and posts service could even be the same process. On publish, the posts service emits to **Kafka**, and search, notifications and analytics consume it.



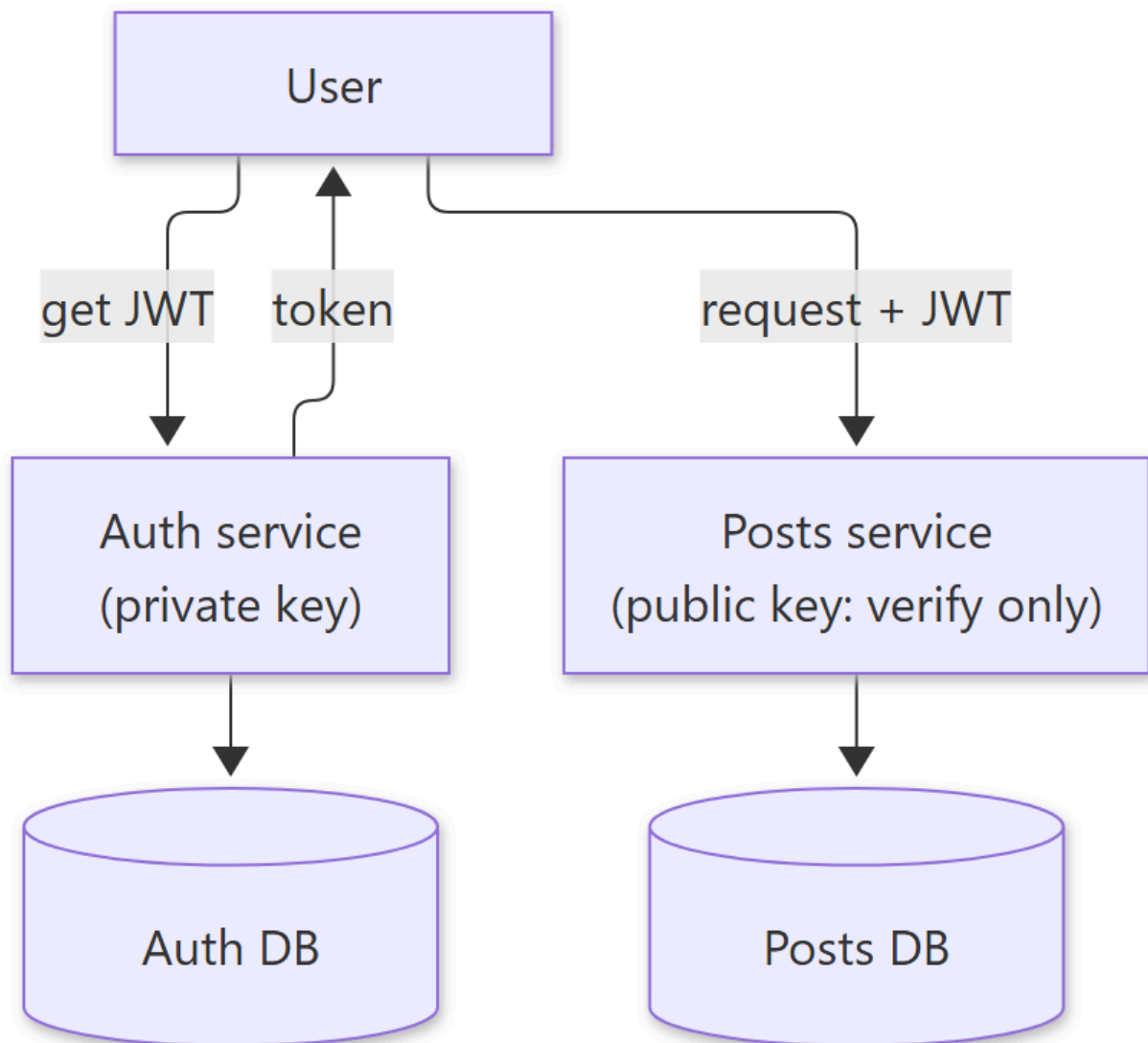
2.4 Step 3: privacy (are Instagram's private photos really private?)

Anchor: **No, but nearly.** Every photo URL carries a **timed signature**. The CDN checks it; if it's invalid or **expired**, the URL stops working.

- URL shape: `https://instacdn.net/.../1729_4275.jpg?ig_cache_key=...&oh=...&oe=...&nc_sid=...`
- When an `<img>` tag requests it, the **CDN validates** the attached keys. Valid and not expired: return the image. Otherwise: bad URL.
- URLs are **short-lived** because the certificates/signatures behind them are short-lived. A leaked URL soon dies.

2.5 Public-key cryptography and JWT

Anchor: **Sign with a private key in one place; verify with the public key anywhere.** No call back to the issuer is needed.



- A JWT is **header** (alg, typ) + **payload** (sub, name, exp, ...) + **signature** over header-payload.
- The posts service **doesn't need to ask the auth service** whether a token is valid. Verifying the signature is all it needs.
- General rule: **when system A can't (or shouldn't) call system B but still needs to trust B's word, think public-key cryptography.**

2.6 Step 4: image optimisations

📌 Anchor: Don't send a celebrity's 5 MB photo to every follower. Let the **CDN transform on the fly**:
.../1729_4275.jpg?w=360.

- Users differ in geography, bandwidth, devices and processing power, so serve a resolution that suits their stats.
- Instead of building your own image optimiser, use the CDN's feature: it loads the original, resizes to 360 px, **caches the variant**, and serves it.
- **Exercise:** try image transformations on Cloudflare (or similar).

Recall: Why upload via signed URL rather than through your API?

Proxying doubles bandwidth and wastes API CPU, memory and network on relaying bytes. A signed URL lets the client upload straight to S3 for one exact path, with time-limited permission.

Recall: How are "private" Instagram photos protected?

The CDN URL carries a short-lived signature. The CDN verifies it on each request and rejects invalid or expired ones, so a shared link stops working soon after.

Recall: Why can a service verify a JWT without calling the auth service?

The auth service signs with its private key; anyone with the public key can verify the signature and expiry locally.

3. Gravatar

Anchor: **One embeddable URL for your profile picture everywhere:**

`gravatar.com/avatar/{hash(email)}.`

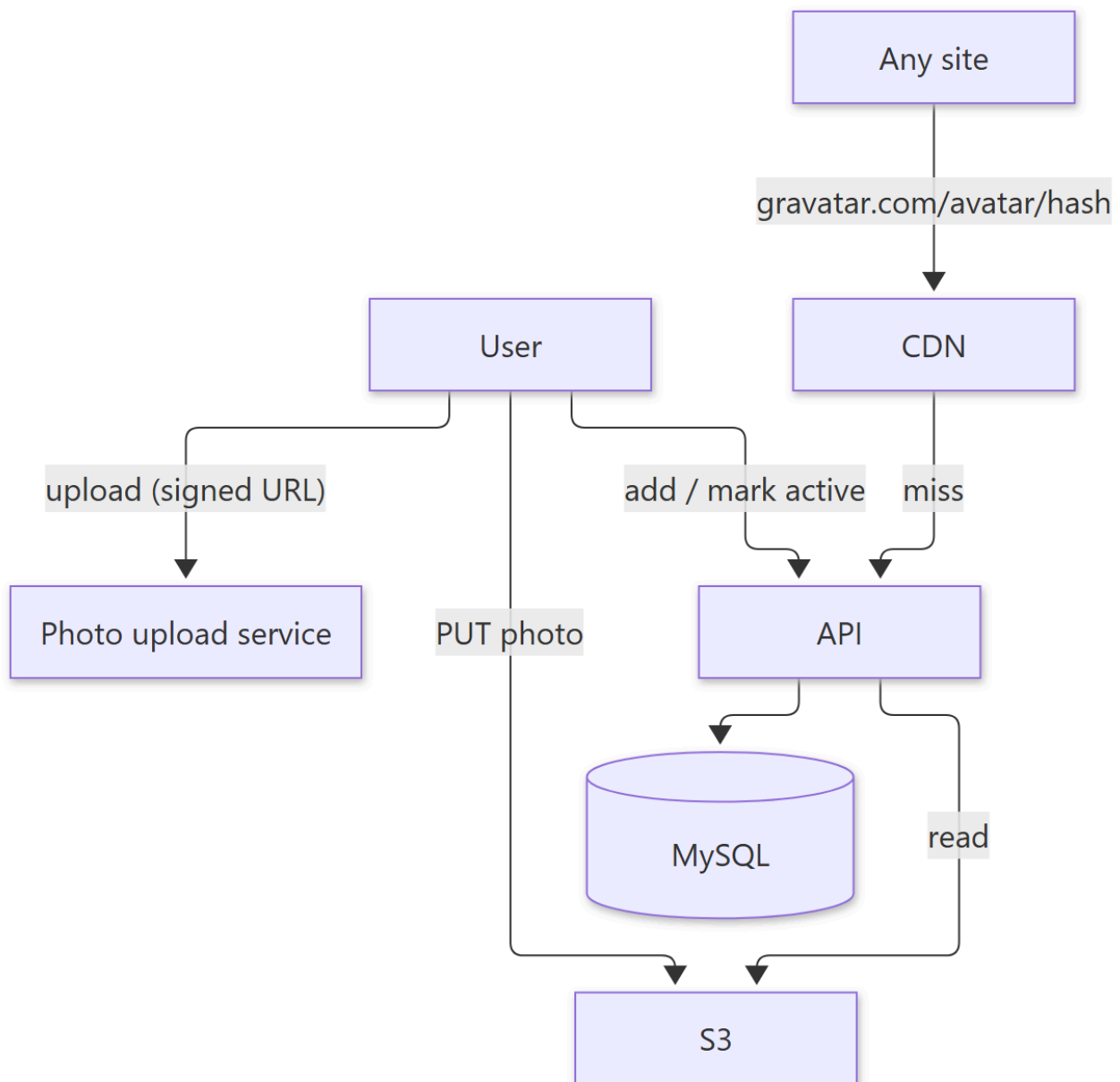
- `hash("arpit@gmail.com") = 0eafd172` → `` renders the **current** profile picture.
- **Requirements:** a user can upload multiple pictures and mark one as active; the active one is returned.

3.1 Schema

users	photos
id, email, hash	id, user_id, is_active

- **Partial index** (PostgreSQL) on `(user_id, is_active) WHERE is_active = true`. It's tiny, because only active rows are indexed.

3.2 Flows



1. **Upload:** same pattern as before. Ask the photo upload service for a random id + signed URL for `s3://gravatar-images/{user_id}/{random_photo_id}`, then upload to S3.
2. **Register:** POST to add the photo to the `photos` table.
3. **Mark active** (in one transaction): `UPDATE photos SET is_active = false WHERE user_id = ? AND is_active = true; UPDATE photos SET is_active = true WHERE user_id = ? AND id = ?;`
4. **Render:** `api.gravatar.com/photos/{hash}` looks up the active photo (`photos JOIN users WHERE users.hash = ? AND is_active`), reads it from S3 and returns it.
5. **Serve via CDN:** `map avatar.gravatar.com/{hash} → api.gravatar.com/photos/{hash}`. You get the CDN's speed and scale, plus a clean URL for users.

3.3 Optimisation: denormalise

- Put `active_photo_id` (or even the active S3 key) on `users`. Rendering becomes **one lookup by hash**, **no join**, and "mark active" becomes a single-row update.

Also in this session: how GitHub, Reddit and Dev.to generate **dynamic OG images** (link-preview cards).

Watch: "How Instagram counts hashtags".

 Recall: How does Gravatar find the right picture from a URL?

The URL holds hash(email). Look up the user by hash, find their active photo (join or denormalised `active_photo_id`), read it from S3. The CDN caches the response.

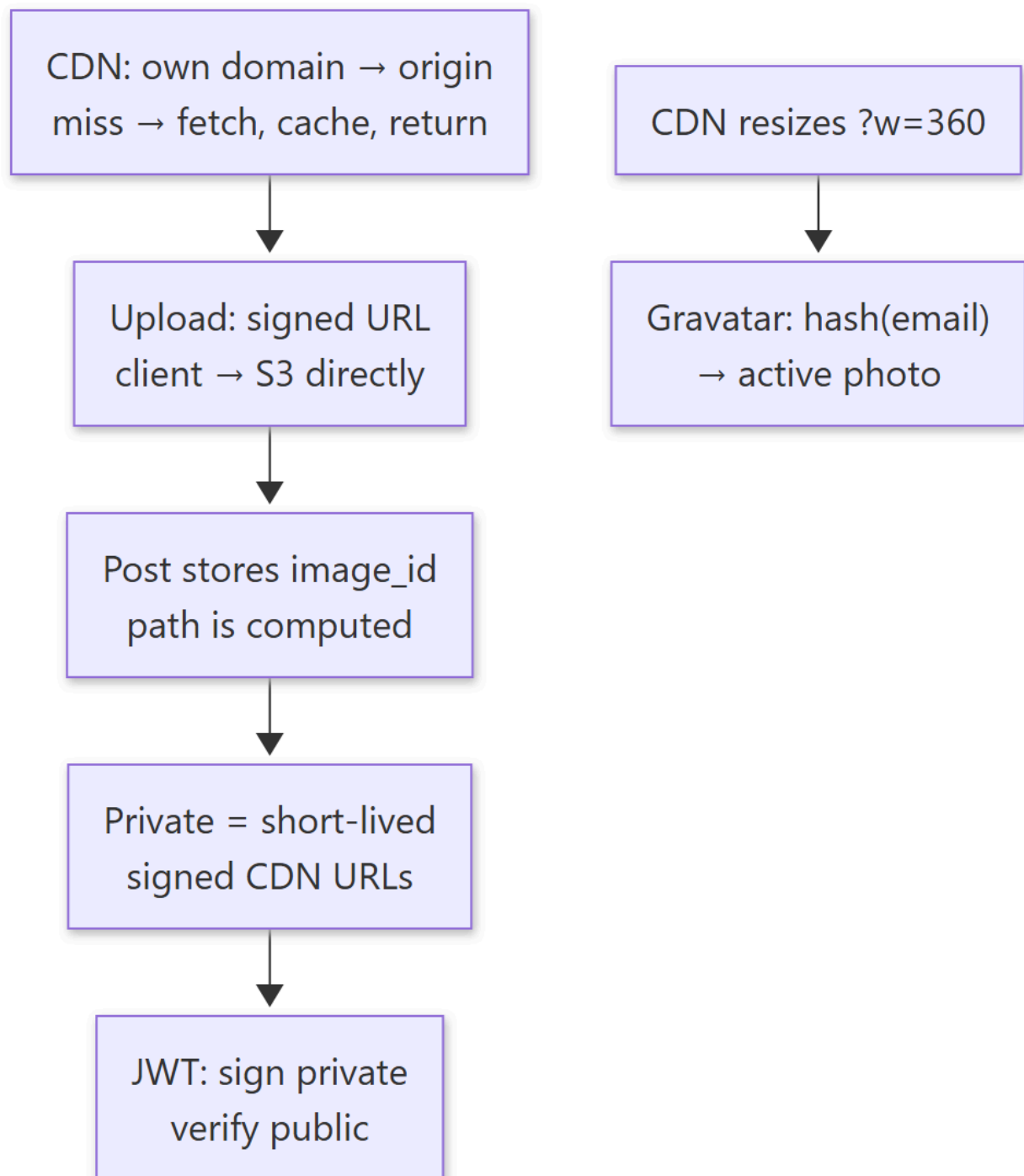
 Recall: Two ways to keep exactly one active photo per user?

Flip flags in one transaction (old → false, new → true), optionally enforced by a unique partial index. Or store `active_photo_id` on the users row, so there's nothing to keep in sync.

What these notes add beyond the slides

- **HS256 vs RS256:** the slide's JWT header says `HS256` (HMAC, a **shared secret**) but the flow uses a private key to sign and a public key to verify. That's **RS256/ES256** (asymmetric). With HS256 every verifier holds the signing secret, so the "posts service can only verify" property is lost.
- **Make "one active photo" enforceable:** a **unique** partial index `UNIQUE (user_id) WHERE is_active` makes the DB reject a second active row, so concurrent "mark active" requests can't leave two.
- **CDN cache vs "current picture":** if `avatar/{hash}` is cached for hours, changing the active photo won't show up. Use short TTLs, or purge that path on change.
- **Signed upload URLs should be tight:** short expiry, fixed content-type and size limits, so a leaked URL can't be used to dump arbitrary data.

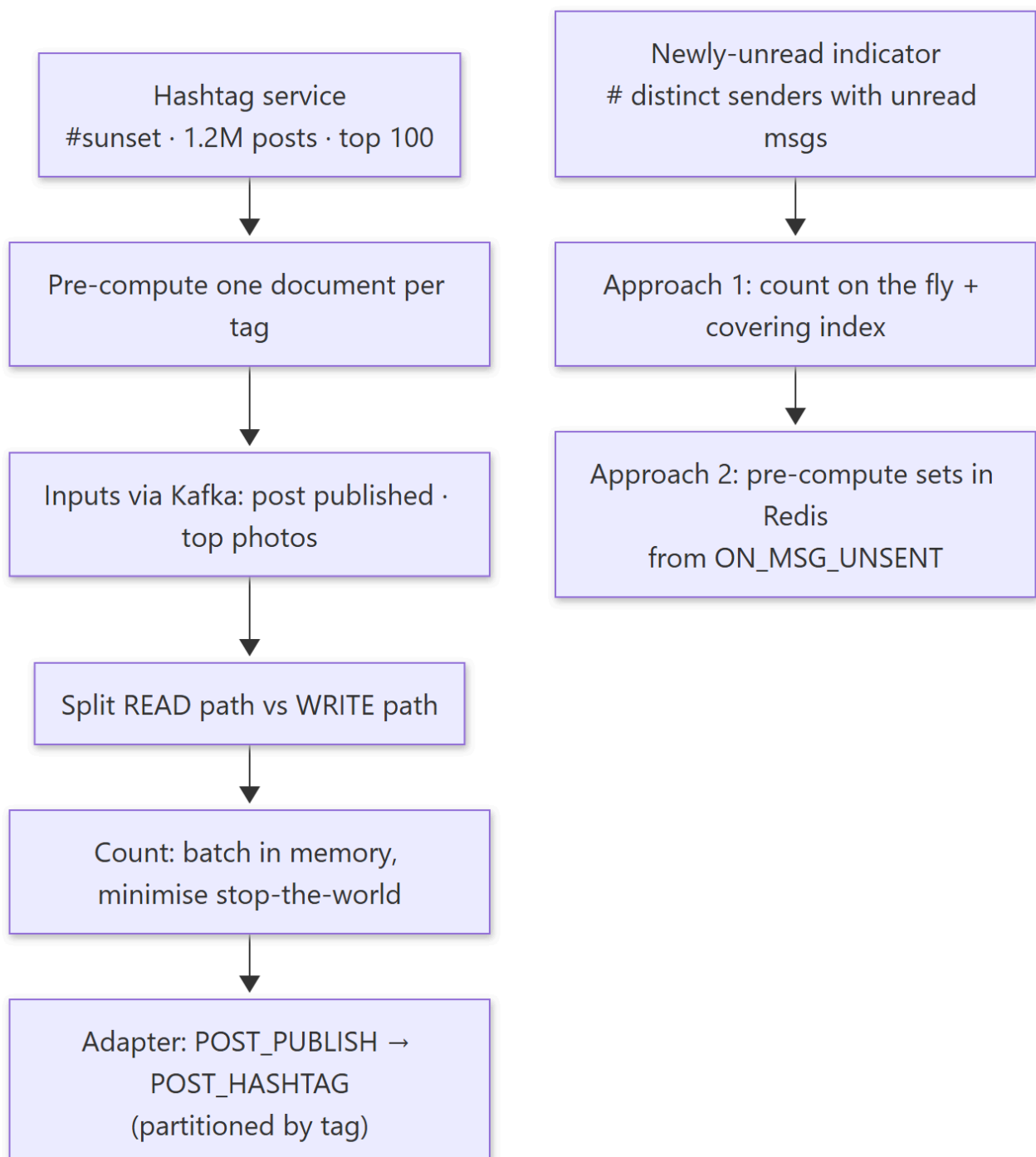
One-page memory card



04-02 · Arpit Bhayani's session notes: Hashtag service + Newly-unread message indicator

Source: 04-02.pdf (System Design Masterclass, Google Drive, view-only). Study notes in my own words, not a copy. Pages covered: 1–19 of 19. Unreadable pages: none. The notification system is covered in a separate YouTube video (plus Razorpay's notification system), not in these slides.

Map of the session



1. Hashtag service

🧠 Anchor: The hashtag page (**#sunset · 1.2M posts · top 100 photos**) must load **super fast**, so **pre-compute everything into one document per hashtag**.

The aim is the best user experience while being clear about the trade-offs.

Assumptions: millions of hashtags; some service already decides the "top" photos for a hashtag and tells us. **Brainstorm:** storage, counting at large volume, inter-service communication, partial updates (posts service → hashtag service), super-fast response times, avoiding N+1 queries, minimising "stop the world".

- "Pagination is over-optimisation" here: absolute correctness isn't necessary. Lazy-load more as the user scrolls (GET /tags/<name>?page=1).

1.1 Storage: what goes in the hashtag document?

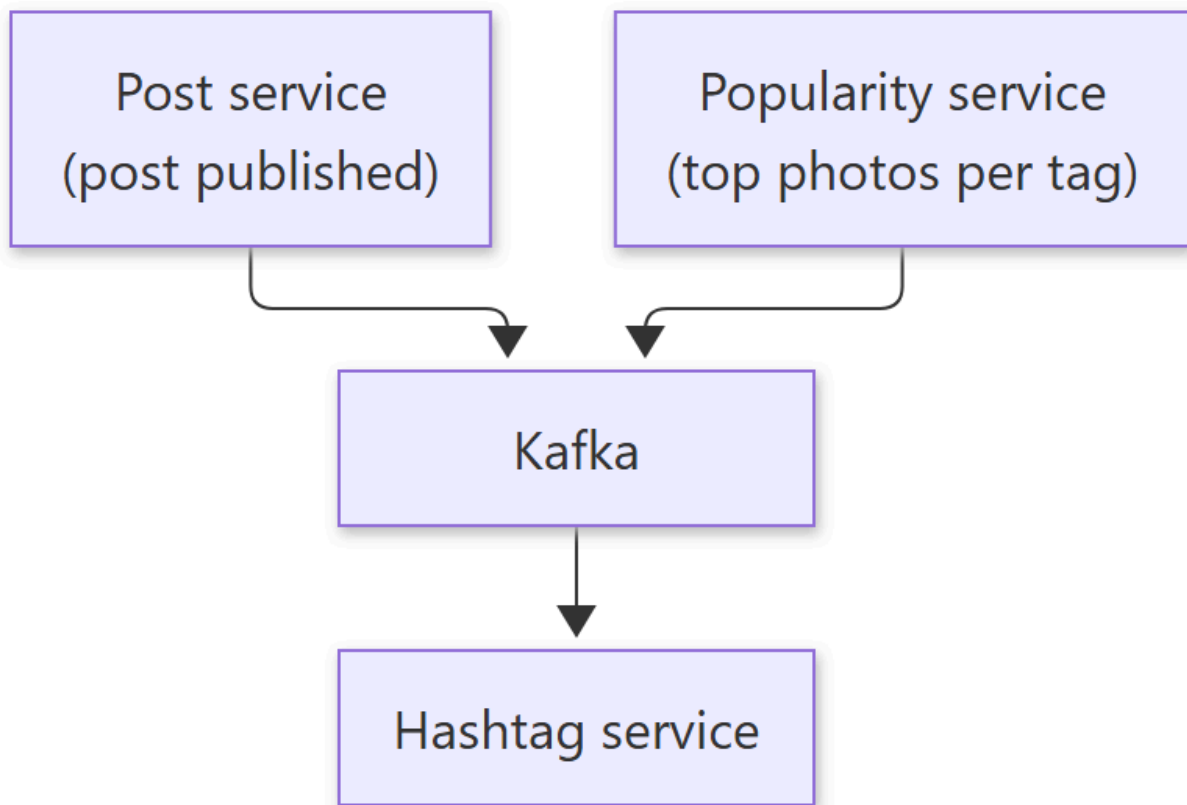
```
{ id: "sunset", total_posts: 1200000, top_100: [ ... ] } # partitioning key: tag
```

Store in top_100	Pros	Cons
1. List of post ids	Less data/storage	Extra enrichment calls to the posts DB per request (N+1)
2. Entire post details (~2.5 KB each: id, user, img_url, caption, ...)	Fast: no enrichment, one read	More data; copies can go inconsistent , and the update time and memory are high

The session picks **pre-computed, fully enriched** documents. That's what "super-fast" means.

1.2 Inputs

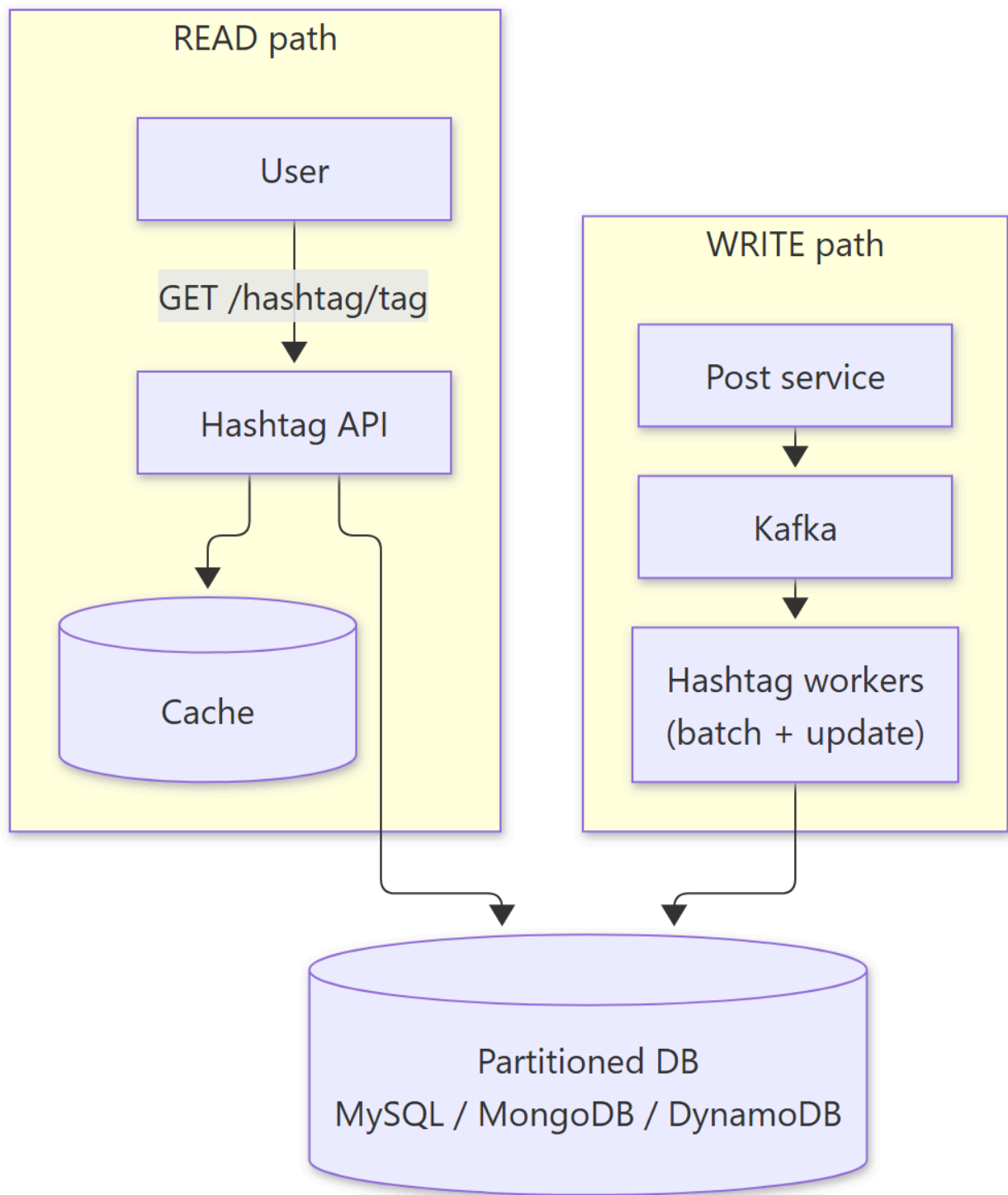
🧠 Anchor: **Kafka is the glue**. The hashtag service just consumes events.



- Event 1: a post was published ($\rightarrow$ count++).
- Event 2: the top photos for a hashtag were re-evaluated ($\rightarrow$ replace `top_100`).
- Key requirement: one request, `/hashtag/<tag>`, returns `{tag, total_photos, top_photos[...]}`.
Everything is returned in a single response, **pre-computed and pre-evaluated**. This is the core of the system.

1.3 Separate the READ path and the WRITE path

🧠 Anchor: **Identify the read path and the write path, and optimise them independently.**



Read path optimisations	Write path optimisations	Storage requirements
Central cache · API does the bare minimum · DB handles key-value reads (tag → details)	Ingest into Kafka · read from Kafka · quick in-memory counting · batch writes to the DB	Partial updates · atomicity · key-based access

Main hashtag API servers don't count. Worker nodes batch and update.

1.4 Counting at high volume: three worker designs

#	Design	Writes to DB
1	Post service writes the hashtag DB directly	One per post per tag (heavy coupling)
2	Naive Kafka consumer: for each tag in the post, <code>db.update(tag, count++)</code>	One per tag per event
3	Batch: accumulate <code>m[tag] += 1</code> in memory, flush every N events or 1 min with <code>db.incr(tag, count)</code> , then commit the Kafka offset	One per tag per batch ✓

```
loop:
  msg = kafka.get_msg()
  for tag in extract_tags(msg.caption): m[tag] += 1
  if processed == N or timer_expired(1 min):
    flush(m); kafka.commit()
```

1.5 Minimising "stop the world" during a flush

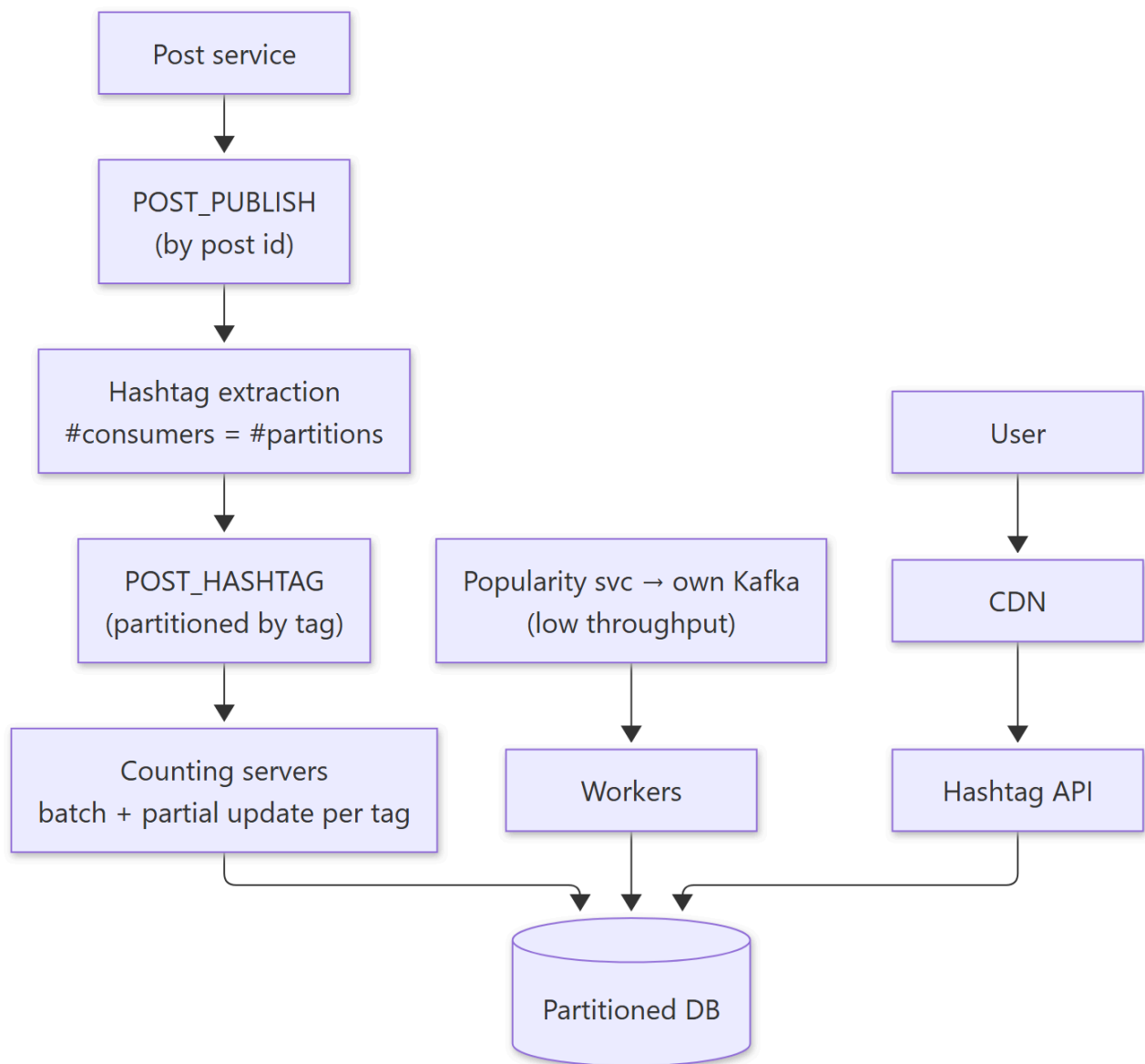
🔗 Anchor: Hold the lock **only to swap maps**, then write the old map to the DB **outside** the lock.

Strategy	Inside the lock	Cost
Stop the world	Loop over the map writing to the DB, clear it, reset the count	Consumers blocked for the whole DB write
Deep copy	Copy the map, clear the original; write the copy async	Copying a big map is slow
Minimal stop the world	Swap pointers (<code>ma, mp = mp, ma</code>), unlock; go <code>writeToDB(mp)</code>	✓ Lock held for O(1)

1.6 The partitioning problem → an adapter

🔗 Anchor: `POST_PUBLISH` is partitioned by **post/user**, but counting batches well only if each **hashtag** goes to one consumer. So **re-partition**.

- A post with 8 hashtags needs **one event per hashtag** so each tag can be counted and batched efficiently.
- Write an **adapter** (hashtag extraction): read `POST_PUBLISH` → extract the hashtags → emit *n* events to a `POST_HASHTAG` topic **partitioned by hashtag**.
- Now each counting server owns a set of tags, so its in-memory batches are complete and there's no cross-server contention on the same tag.



Two options inside the extraction step:

Option	How	Trade-off
1. Kafka adapter	Re-publish per-tag events to POST_HASHTAG; counting servers aggregate	Less simple, more moving parts, slower, but more efficient
2. Threads	The extraction consumer counts per tag in-process and writes directly	Simpler and faster, but less efficient (each consumer sees every tag, so batches are smaller)

- The popularity service's per-hashtag "top photos" updates are **not high-throughput**, so simple workers write them straight to the DB.
- HA of brokers → **dead-letter queues** for messages that keep failing.
- Reads go through a **CDN** in front of the hashtag API.

Key takeaways: Kafka as glue · the adapter pattern · effective batching and counting · read- and write-path optimisations.

Recall: Why re-partition from POST_PUBLISH to POST_HASHTAG?

Counting batches well only if all events for a tag land on one consumer. POST_PUBLISH is keyed by post/user, so an adapter emits one event per hashtag into a topic keyed by hashtag.

Recall: How do you flush in-memory counts without blocking consumers?

Under the lock, swap the active map with an empty one ($O(1)$), release the lock, then write the old map to the DB asynchronously. Commit Kafka offsets after the write succeeds.

Recall: Enriched post details vs post ids in the hashtag doc?

Ids are small but need N enrichment lookups per read. Full details make the read one fetch, but cost more storage and can go stale. The session chooses pre-enriched for speed.

2. Newly-unread message indicator

 Anchor: The badge shows **how many different people** sent you messages you haven't read, **not** how many messages. 100s of unread messages from 3 people → ③.

Requirements: near-realtime; update when new messages arrive. `# new unreads = # distinct senders with at least one unread message`.

2.1 Approach 1: count on the fly (MySQL)

user	messages	user_activity
id, ...	id, msg, from, to, timestamp	user_id (PK), last_read_at

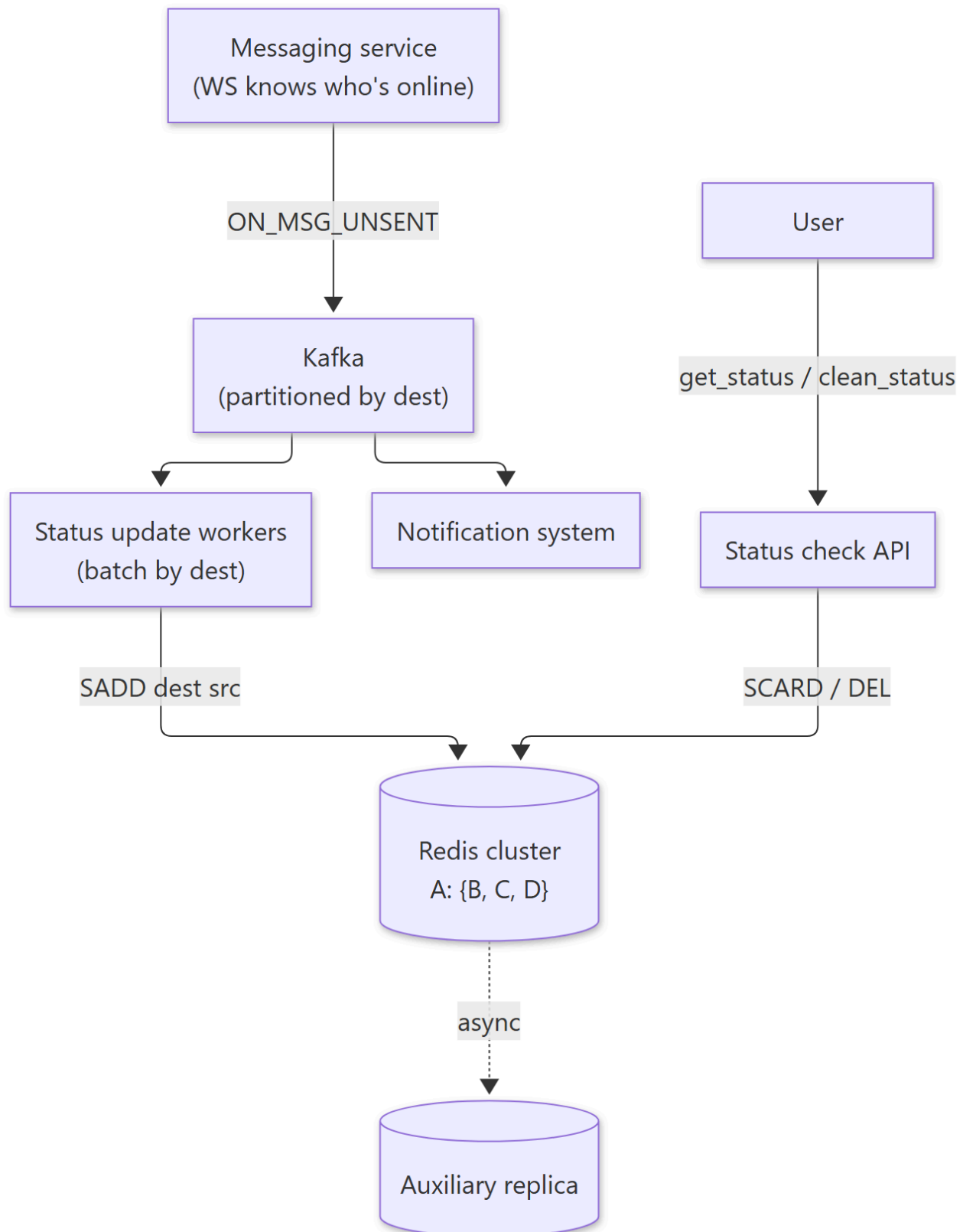
```
SELECT COUNT(DISTINCT from_user) FROM messages
WHERE to_user = ? AND timestamp > :last_read_at;
```

- A pretty decent approach, but it **may not scale**.
- **Index choice matters** (the session walks through each):
 - `idx(to)`: finds the user's rows, but must then fetch each to check the time and sender.
 - `idx(timestamp)`: scans everyone's recent messages.
 - `idx(ts, to)`: the range on `ts` comes first, so `to` can't narrow it well.
 - `idx(to, ts, from)`: **a covering index**. Equality on `to`, range on `ts`, and `from` is read straight from the index with no table lookup. 

2.2 Approach 2: pre-computed (Day N)

 Anchor: **Count at write time, not read time**. Undelivered messages feed a **per-user Redis set of senders**; the badge is just the set's size.

- **Input:** when is a message "unread"? When it **wasn't delivered**. WebSockets know whether the user is connected (or an online/offline service does), so the messaging service emits `ON_MSG_UNSENT {src, dest, msg}` to Kafka.



- Workers do `SADD(dest, src)`. Sets dedupe for free: 10 messages from A, C and B leave `user_id → {A, B, C}`.
- **Read:** `get_status` = size of the set. **Clean** when the user reads (`DEL / SREM`).

- Batch by `dest` in workers, since Kafka is partitioned by destination.
- **Size estimate:** $1\text{M users} \times (4\text{ B} + 100\text{ senders} \times 4\text{ B}) \approx \mathbf{404\text{ MB}}$, which fits in Redis comfortably.
- The same `ON_MSG_UNSENT` stream can also feed the **notification system**.
- This is "not your Day-0 solution". Start with Approach 1.

Exercises:

1. Implement all 3 hashtag worker approaches: naive, naive + batch, efficient batch.
2. Populate `on_msg_unsent` (use WebSockets to know who is online).
3. Run Redis in cluster mode, write data and see how it's distributed.
4. Implement the naive unread indicator: without an index, with the mentioned index, and with different index orders; compare response times.

Recall: What does the unread badge count, and what Redis structure fits?

The number of distinct senders with unread messages. A Redis SET per recipient (SADD sender) dedupes automatically; the badge is SCARD.

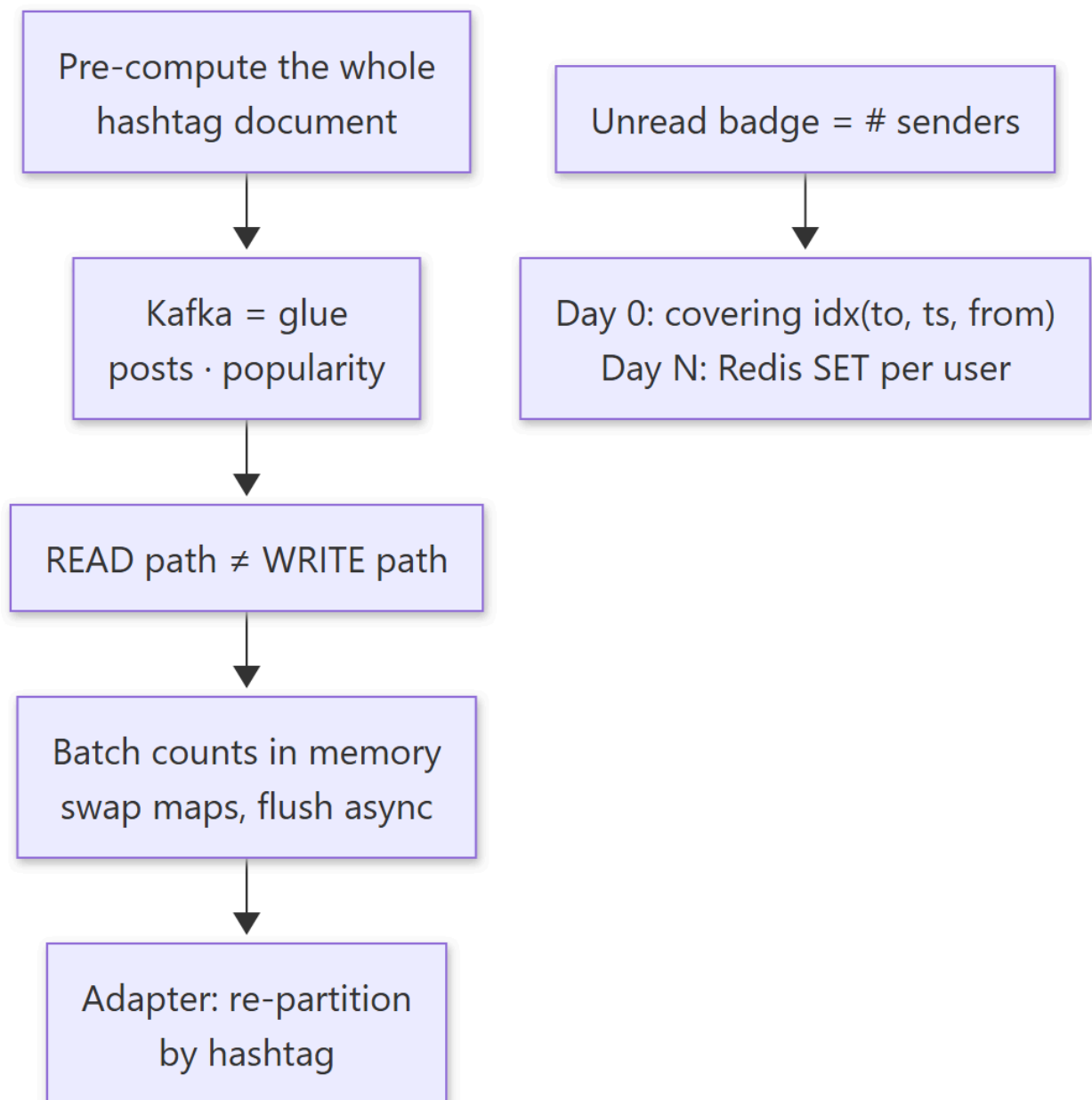
Recall: Best index for COUNT(DISTINCT from) WHERE to = ? AND ts > ?

A covering composite index (to, ts, from): equality column first, then the range column, then the selected column, so the query never touches the table.

What these notes add beyond the slides

- **Index column order rule:** equality columns first, then range, then columns you only read. After a range column, later columns can't be used to *seek*, only to cover.
- **At-least-once + batching:** commit Kafka offsets *after* the batched DB write, or a crash loses counts. If a batch is retried, increments can double. Accept approximate counts (fine for "1.2M posts"), or store per-partition offsets alongside the counts.
- **Hot hashtags** (#love, #instagood) can overload one `POST_HASHTAG` partition. Pre-aggregating in the extraction step (the "threads" option) or salting hot keys spreads that load.
- **Arithmetic checked:** $1\text{M} \times (4 + 400)\text{ B} = 404\text{ MB} \checkmark$.

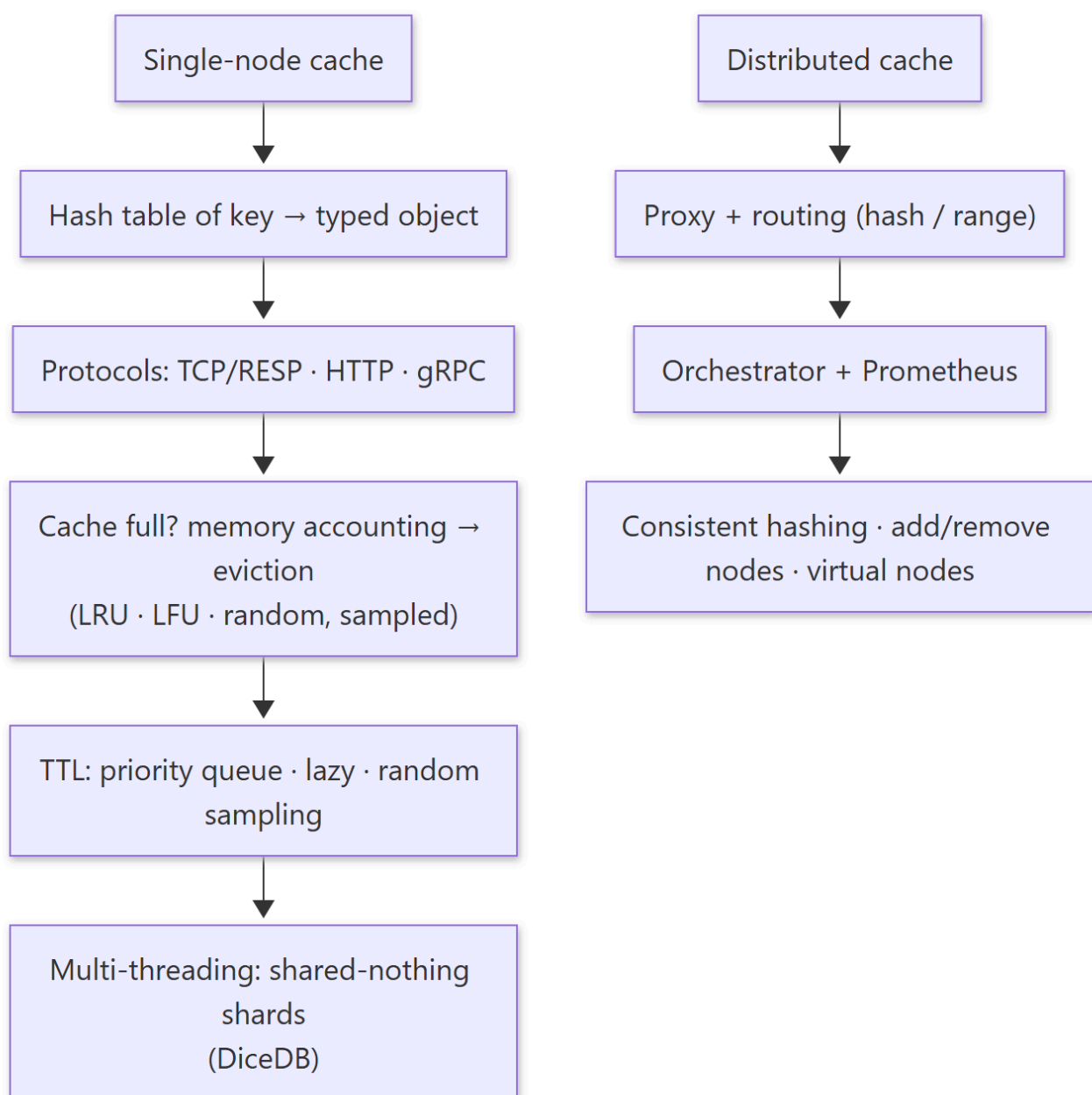
One-page memory card



05-01 · Arpit Bhayani's session notes: Designing a distributed cache (Redis / DiceDB internals, consistent hashing)

Source: 05-01.pdf (System Design Masterclass, Google Drive, view-only). Study notes in my own words, not a copy. Pages covered: 1–21 of 21. Unreadable pages: none. Pages 3–5 are a whiteboard overview; pages 6–13 go through the same ideas cleanly, and the notes follow those.

Map of the session



Agenda: **Designing a distributed cache: single node (Redis / DiceDB internals) → distributed (consistent hashing + operational complexities)**. Guiding principle: spot the overheads and the trade-offs.

1. Requirements and brainstorm

🧠 Anchor: **High throughput, low latency, distributed**. GET, PUT, DEL, TTL, plus a few data structures (set, list, geo, HLL, Bloom filter). **Scale is the default**.

Brainstorm: communication, storage, cache full, eviction, TTL.

2. Single-node cache

2.1 Storage

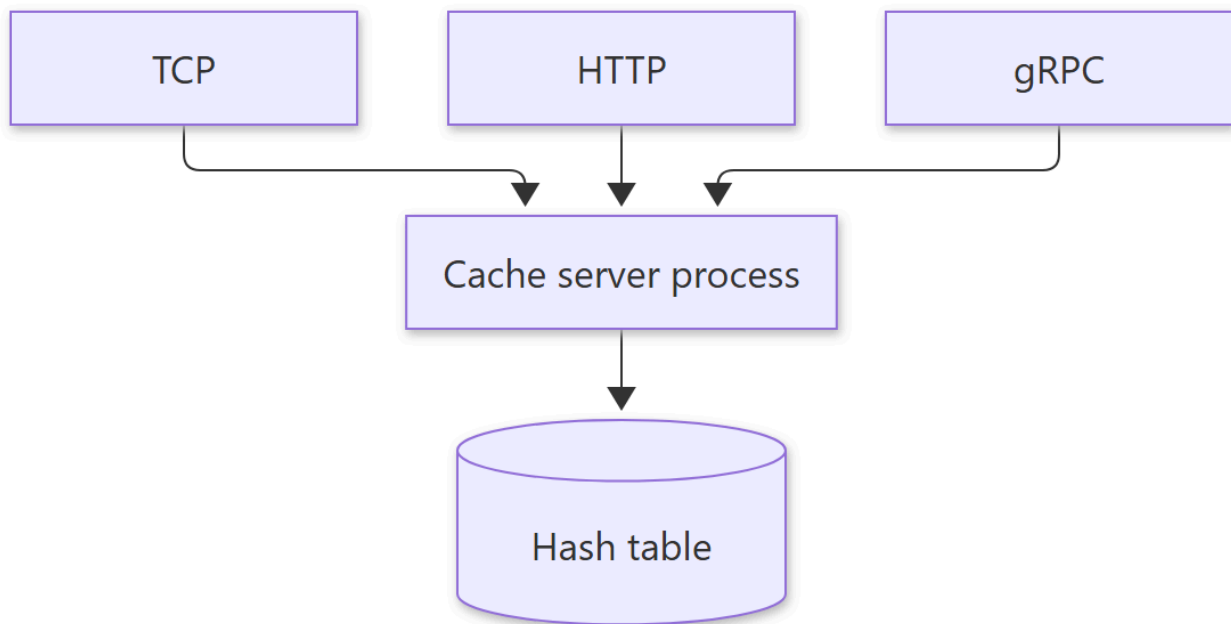
🧠 Anchor: A cache is a **hash table**: `key → object { type, data }`. The **type** decides which operations are allowed.

- Bare minimum: GET / PUT / DEL. Extend to complex structures (sets, lists...) by giving each key a typed object.
- `SET k1 "arpit"`, `SET k2 22`, `INCR k1` (type error), `SADD k3 "x" → k3 → {x}`.
- Each object carries overhead: value pointer + type + `expires_at` + `last_accessed_at` ($\approx 8 + 8 + 8 + 1 \approx 25$ B on the slide). With millions of keys, that overhead matters.

2.2 Communication

🧠 Anchor: Support **raw TCP, HTTP and gRPC** by abstracting the protocol layer away from the hash table.

Protocol	Pros	Cons
HTTP	Simple; great tooling (curl, requests)	Huge per-request overhead (headers) for tiny payloads
Raw TCP + a wire protocol (Redis RESP : <code>SET k v</code>)	Minimal overhead, fast	Needs custom clients
gRPC	Typed, HTTP/2 multiplexing	Heavier stack



2.3 Cache full → eviction

🧠 Anchor: You can trigger eviction only **once you know the cache is full**, so **account for memory** on every allocation.

How do you know it's full?

- Cap the **number of keys**, or
- Cap the **cumulative size of values**: wrap allocation (`zmalloc`) to keep `total_usage += size` and `zfree` to subtract. If `total_usage + a > max_mem`, evict first. (Could the OS tell you? Not cheaply or precisely enough.)

Eviction algorithms:

Algorithm	Evicts	Good for
LRU	Least recently used	Recency-heavy traffic (CDN, Google News)
LFU	Least frequently used	Stable popularity (Wikipedia)
Random	Any key	Cheap, surprisingly OK

- Read about **O(1) LRU and LFU** implementations.
- **Exact LRU costs memory**: a doubly linked list adds $\sim 3 \times 8 \text{ B}$ pointers per key, which adds up to many MB at scale. That's a space-vs-time-vs-correctness trade-off.
- **Redis approximates LRU with sampling**: sample a few random keys, keep the best candidates in an **eviction pool** (~ 16 slots, sorted by idle time) and evict the least recent. No list, and no CPU/page thrashing from maintaining one.

2.4 TTL handling

🧠 Anchor: Expiry = **absolute time per key**. Delete expired keys **lazily on access** *and* **actively by random sampling**.

Approach	How	Trade-off
1. Priority queue	Min-heap ordered by absolute expiry; free from the top	Simple and consistent, but an extra data structure
2. Lazy (passive) deletion	On GET: if expired → delete, return nil	Keys never fetched are never deleted
3. Random sampling (Redis, active)	Sample 20 keys with TTLs, delete the expired ones; if > 25% were expired, repeat	Bounded CPU per cycle; keeps expired keys to < ~25%

- The idea behind #3: if the sample has < 25% expired keys, the whole population probably does too. This is **not suitable for disk-backed DBs**, where random sampling means random I/O.
- In practice, combine lazy (which catches most, ~90%) with active (the remaining ~10%).
- **But Redis is single-threaded, so how does background work run?** Through its event loop: `activeExpireCycle` (`expire.c`), `databasesCron` / `serverCron` (`server.c`), `aeCreateTimeEvent` / `processTimeEvents` (`ae.c`, which fires events whose `when < now`), `aeProcessEvents` and `aeMain` (`ae.c`). Timers are interleaved with I/O on one thread.

🔄 Recall: How does Redis expire keys without scanning everything?

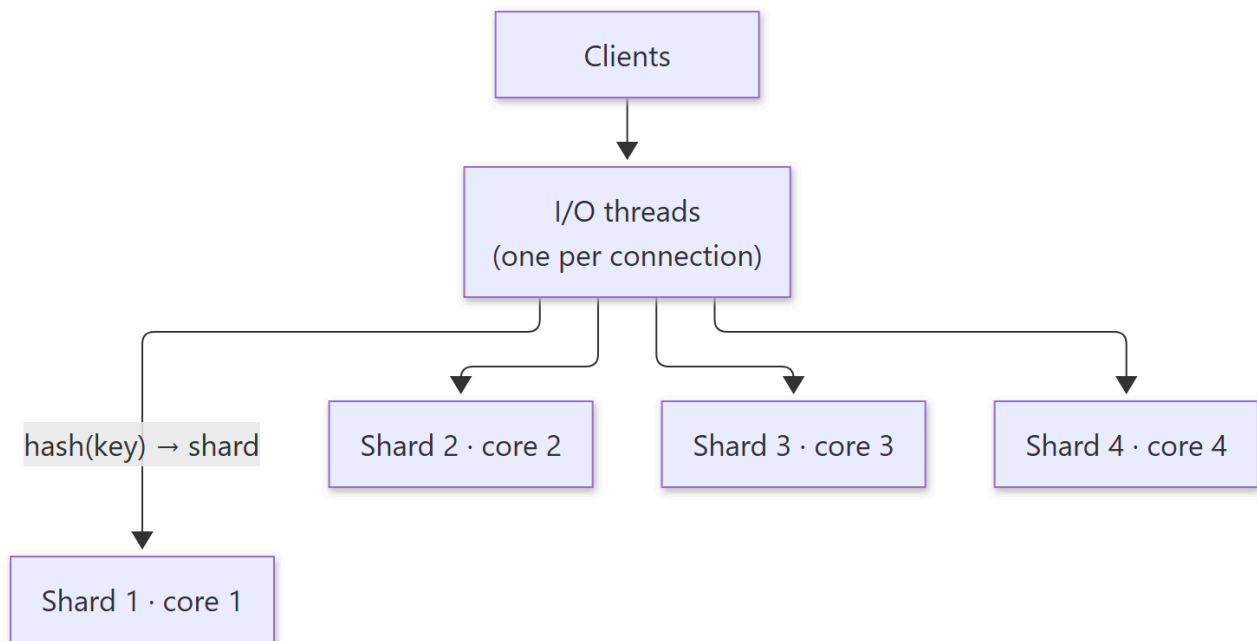
Lazily on access, plus an active cycle that samples ~20 keys with TTLs, deletes the expired ones, and repeats while more than 25% of the sample was expired.

🔄 Recall: Why does Redis approximate LRU instead of keeping a linked list?

A DLL costs extra pointers per key (memory) and pointer-chasing on every access (CPU, cache misses). Sampling plus a small eviction pool gets close to LRU cheaply.

2.5 Going multi-threaded (DiceDB)

🧠 Anchor: To maximise throughput, **minimise contention**, which leads to **shared-nothing**: one hash-table shard per core, owned by one thread.

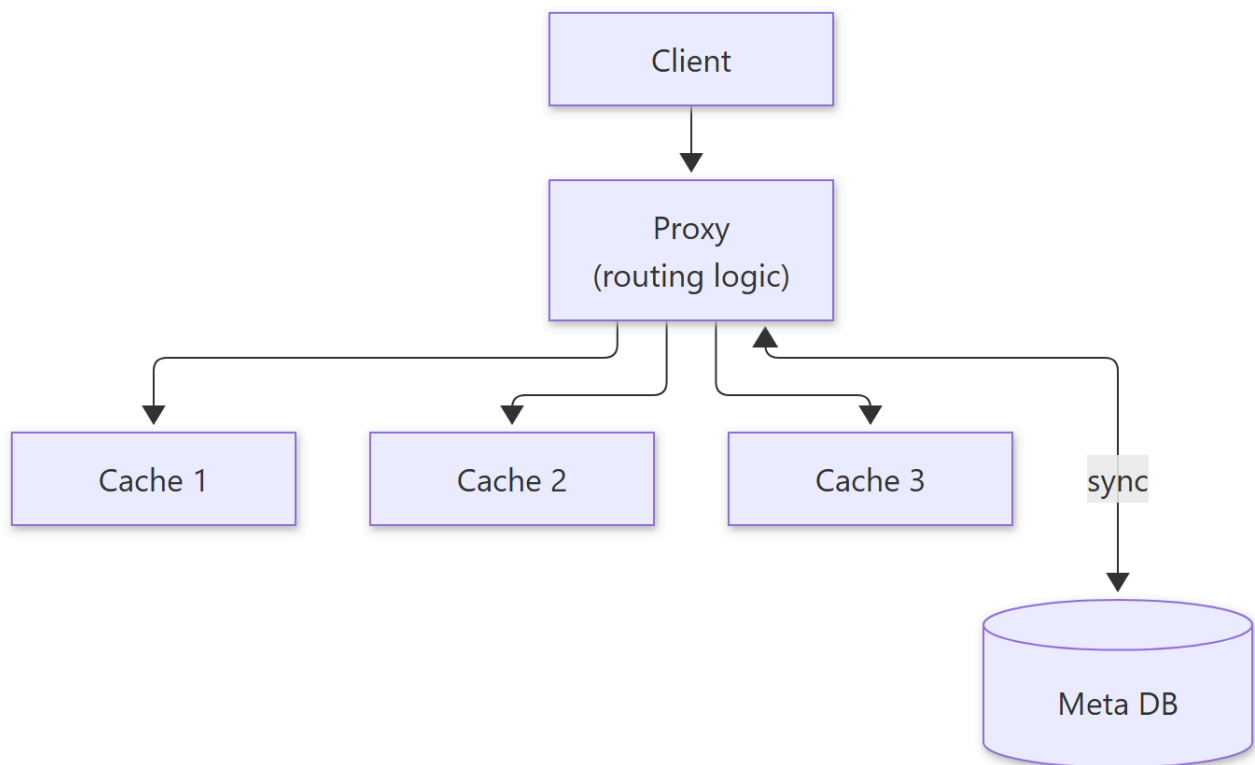


- A single hash table + many threads → **contention** (mutexes slow everything down). A single-threaded server leaves the other cores idle.
- **Split into N hash tables (shards), N = #cores.** 12 threads contending on one lock becomes 3 threads per lock: better throughput, all cores used. On a 4-core box handling 12 threads per core, that's **48 threads**.
- Two remaining concerns: (1) any thread touching any of the 4 maps still causes contention, page thrashing and poor CPU locality; (2) there's still contention within one table.
- Fix: **affinity**. Pin a **data thread** to each core, owning its local hash table. **I/O threads** (one per client connection) find which shard owns key k (hash) and hand the operation to it.
- DiceDB claims $\sim 10\times$ Redis's throughput this way.

3. Distributed cache

🧠 Anchor: Many single-node caches working as **one coherent cache**. The core problem is **data ownership: which node owns which key?**

3.1 Routing



- **Hash-based:** $i = \text{hash}(\text{key}) \% 3$. **Range-based:** cache 1: a–j, cache 2: j–p, cache 3: q–z.
- The routing logic can live in a **proxy** (syncd with a meta DB), or in the **client** (as with Redis Cluster, where the client knows the slots), or nodes can **forward** among themselves (a DHT such as Kademlia/Chord, as in IPFS).
- **Exercise:** watch the Kademlia DHT video and read the Kademlia and Chord papers.

3.2 Orchestrator

- Monitors the uptime of cache servers (heartbeats) and handles **failovers**. Metrics come from **Prometheus**. State lives in the meta DB.
- Goal: a **truly elastic** cache that scales up and down dynamically. Doing that efficiently comes down to data ownership.

3.3 Why $\text{hash} \% N$ breaks

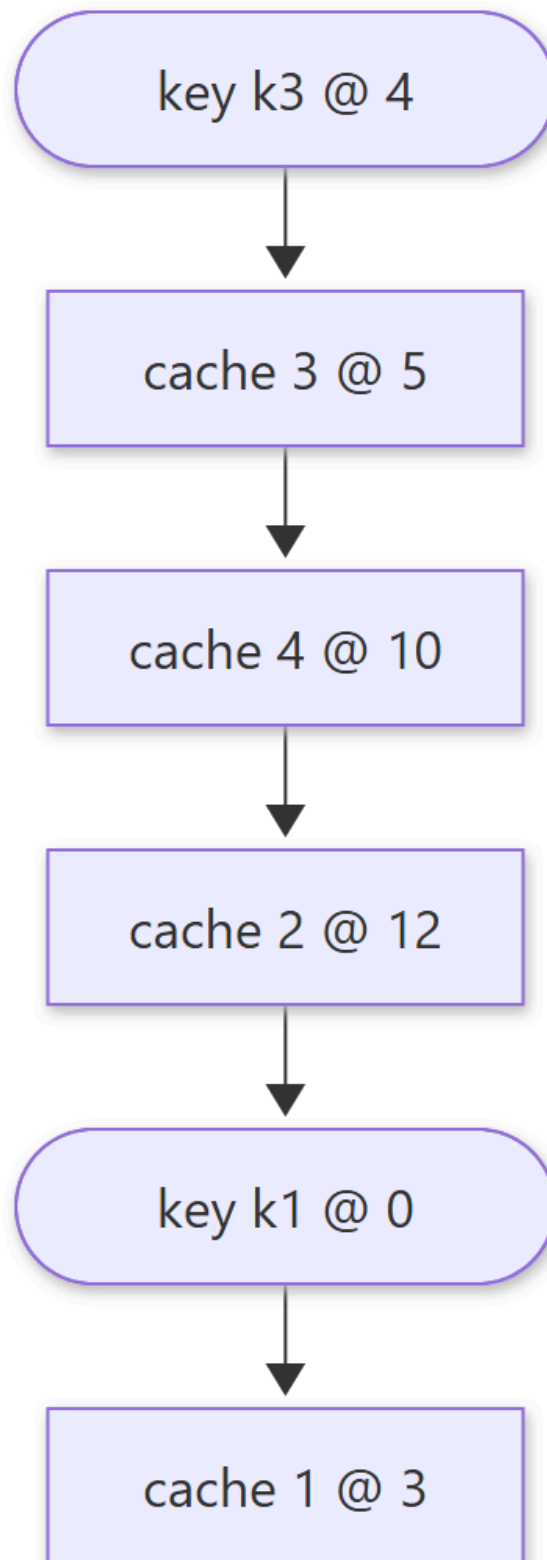
🔗 Anchor: Change N and **most keys move**. Going from 2 to 3 nodes, the slide's 6-key example moves 3 of 6 (50%).

Key	hash % 2	hash % 3
K1	cache 0	cache 0
K2	cache 0	cache 2
K3, K4	cache 1	cache 1
K5	cache 1	cache 0

Key	hash % 2	hash % 3
K6	cache 0	cache 2

3.4 Consistent hashing

 Anchor: Put **nodes and keys on the same ring**; a key belongs to the **next node clockwise**. Adding or removing a node only moves the keys of **one neighbour's arc**.



- Implementation: a **sorted array** of node positions; binary-search the key's hash for the next position (wrapping around).

Adding cache 5 between cache 2 and cache 1 (minimal data transfer):

1. Create a fresh node.
2. Find its place on the ring (it takes part of cache 1's range).

3. Take a snapshot of cache 1 (could take ~10 min).
4. Load the snapshot on cache 5.
5. Set up replication cache 1 → cache 5.
6. Wait for replication to catch up (watch the replication log).
7. When the lag is ≈ 0 : briefly **stop the world** for cache 1 (≈ 1 s), wait until the lag is exactly 0, add cache 5 to the ring, **resume**.
8. Delete the now-unowned data from cache 1 (and the extra copy on cache 5).

Removing a server: graceful shutdown (hand its arc to the next node) vs abrupt outage (the next node starts cold for that arc).

Operational goals: no-downtime movement; minimal network reads and transfers (elasticity).

3.5 Skew and virtual nodes

 Anchor: One hash per node can leave **uneven arcs**. Give each node **several positions** (virtual nodes) using multiple hash functions, so load evens out.

- What's the chance that *both* hash functions are skewed? Low. The ring becomes c2(0), c1'(5), c3(10), c2'(15), c3'(21), c1(30), ...
- The slide's caution: virtual nodes work great for **stateless** routing (load balancers), but for **stateful** data ownership they make moving data more complex (one node's data is scattered across many arcs).

Exercises:

1. Implement consistent hashing with load-balancer routing.
2. Implement virtual nodes to solve skew.

Recall: Why consistent hashing over hash % N?

With hash % N, changing N remaps most keys. On a ring, adding or removing a node only moves the keys in one arc, roughly $1/N$ of the data.

Recall: Steps to add a node with near-zero downtime?

Snapshot the neighbour that owns the arc, load it on the new node, replicate until the lag is ≈ 0 , briefly pause writes until the lag is 0, update the ring, resume, then clean up the data that moved.

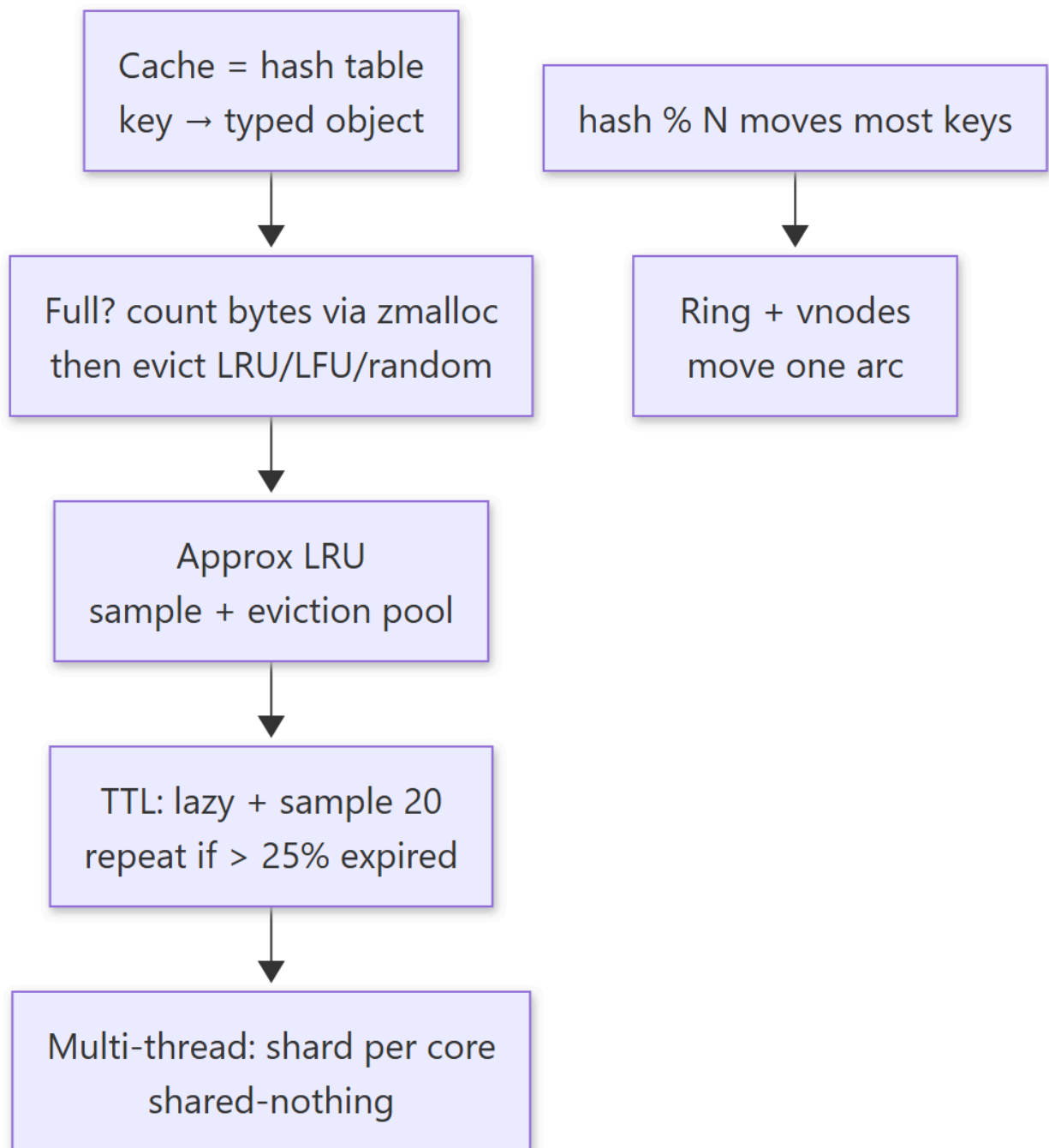
Recall: What do virtual nodes fix?

Uneven arc sizes (skew). Each physical node gets many ring positions, so keys and load spread evenly, and a departing node's load spreads across many nodes.

What these notes add beyond the slides

- **Expected movement with $\% N$:** going from N to $N+1$ nodes moves about $N/(N+1)$ of the keys ($2 \rightarrow 3 \approx 67\%$). The slide's 50% is just its 6-key sample. Consistent hashing moves about $1/(N+1)$.
- **Virtual nodes and stateful systems:** Cassandra and Dynamo *do* use vnodes for stateful data. The benefit is that rebuilding a failed node streams from many peers in parallel. The cost is more ranges to track and move, which is the slide's point. Both views are fair.
- **Redis Cluster doesn't use a ring:** it uses **16,384 hash slots** ($\text{CRC16}(\text{key}) \% 16384$) assigned to nodes. Moving slots is the unit of rebalancing.
- **Eviction pool size:** Redis's `EVPPOOL_SIZE` is 16, and `maxmemory-samples` defaults to 5.

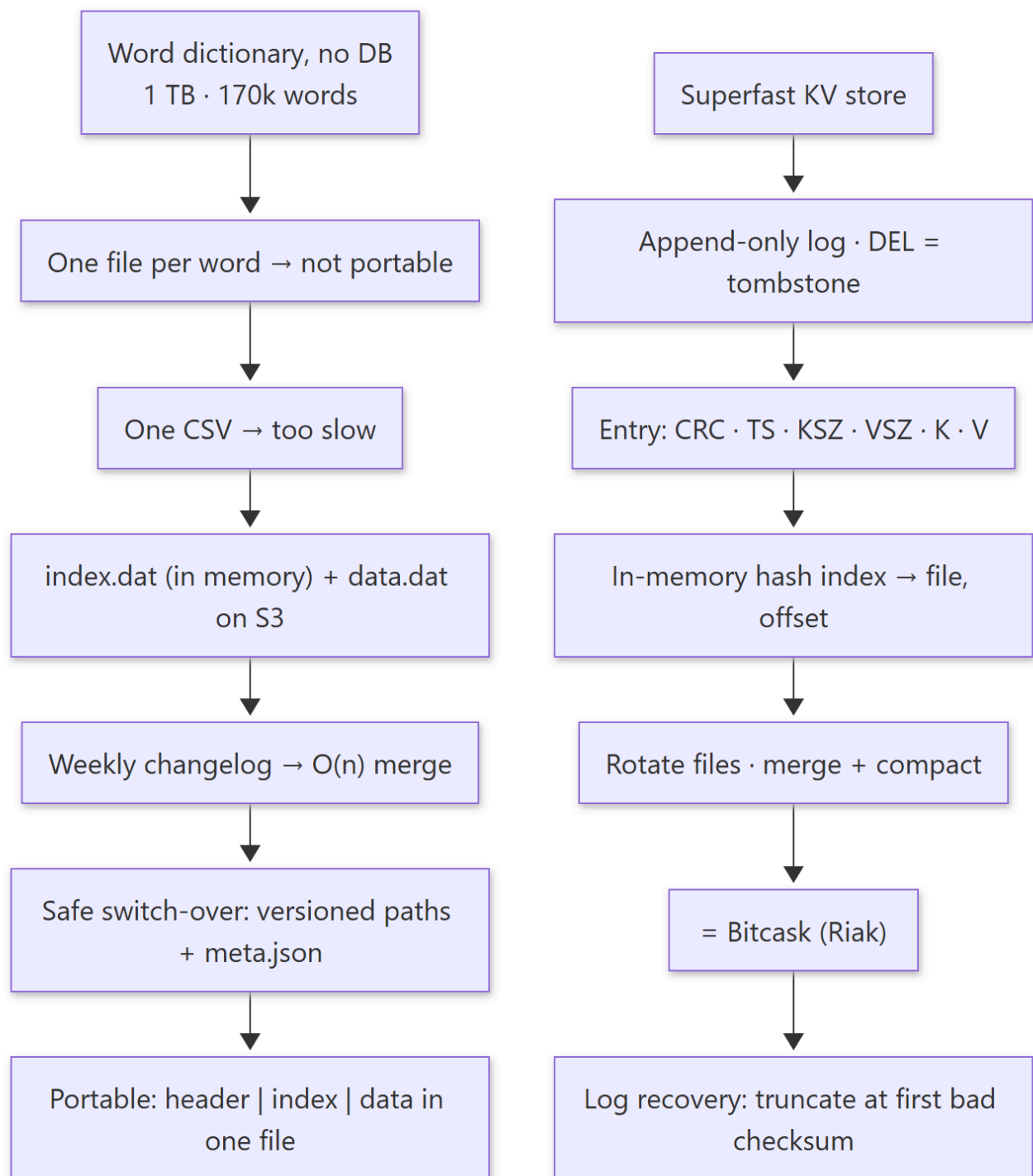
One-page memory card



05-02 · Arpit Bhayani's session notes: Word dictionary without a DB, a superfast KV store (Bitcask), WAL recovery

Source: 05-02.pdf (System Design Masterclass, Google Drive, view-only). Study notes in my own words, not a copy. Pages covered: 1–20 of 20. Unreadable pages: none. Pages 3–4 and 14 are whiteboard overviews; the clean pages that follow each cover the same ideas.

Map of the session



Agenda: **Word dictionary without using any DB · Superfast KV store · WAL recovery.** The theme: storage/compute separation and file layouts.

1. Word dictionary without any database

Requirements:

- **No traditional database:** be creative.

- Words and meanings are updated **weekly** (through a changelog).
- A lookup is always a single word.
- The dictionary is **1 TB** with **170,000 words**; no repeated entries.
- **Portable** (ideally one file), scalable (storage and API servers), "easily" persistent. Response time *can* be high.
- This is the foundation of **data lakes** and **multi-tiered storage**.

1.1 Approach 1: one file per word on S3

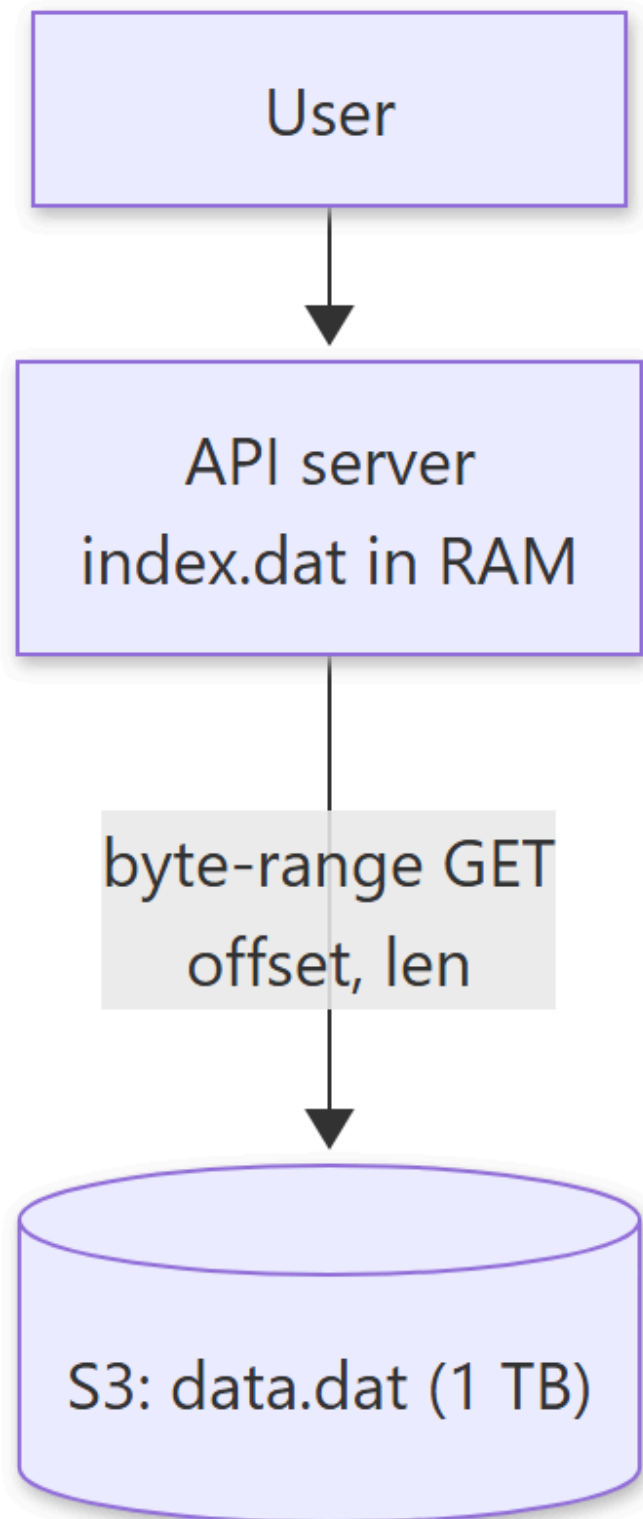
- `s3://word-dictionary/a/apple.txt, .../a/america.txt, ..., .../z/zoo.txt` (a folder per starting letter).
- `get_word(w)` → build the S3 path → read the file → return the meaning. API servers sit behind an LB.
- **✗ Breaks portability** (170k files).

1.2 Approach 2: everything in one file (CSV)

- `word, meaning` per line, sorted. Simplest format, but the file is **1 TB**.
- A lookup would traverse the whole dictionary. **✗ Too slow and expensive.**

1.3 Make lookups fast: indexing

 Anchor: The universal strategy to make lookups faster is **an index**. Keep a small `word → (offset, length)` file **in memory**; read just that byte range from the big data file.



- `index.dat`: `a: 0:127`, `abandon: 127:130`, `ability: 257:100`, ..., `zoo: 1023895:196`. It's a separate file on S3.
- **Size of `index.dat`:** entry $\approx$ avg word length (4.7) + separators/newline (3) + offset and length (2×4 B) = **15.7 B**. Times 171,476 words $\approx$ **2.69 MB**. It fits in memory trivially.
- **Flow:** on boot, the API server loads the index into memory $\rightarrow$ a request arrives $\rightarrow$ look up the offset in memory $\rightarrow$ read that range from S3 $\rightarrow$ return the meaning.

1.4 Weekly updates: merge a changelog

Anchor: Dictionary and changelog are both **sorted**, so updating is **merging two sorted lists**: $O(n)$, like the merge step of merge sort.

Procedure: spin up a new server → download the dictionary and changelog locally → merge in $O(n)$ to create the new dict + index → upload both to S3.

Dictionary	Changelog	New dictionary
a – a', b – b', c – c', d – d', f – f', g – g', h – h'	c – c'', e – e', f – f''	a – a', b – b', c – c'' , d – d', e – e' , f – f'' , g – g', h – h'

- Store records in **binary encoding** (e.g. [3][5]abc defgh... length prefixes) rather than delimited text.

1.5 Switching to the new version safely

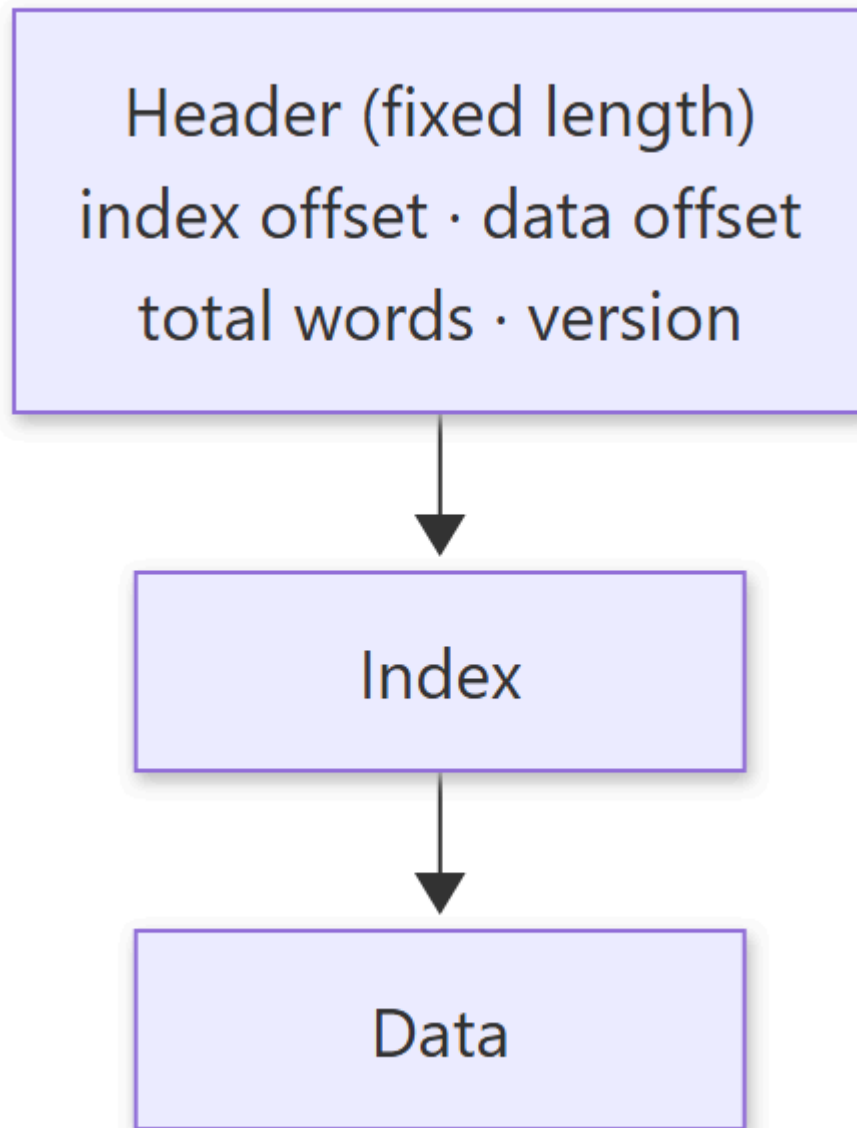
Anchor: Overwriting `data.dat` in place makes servers with the **old index** read **garbage** from the new file. **Write new versions to new paths, then flip a pointer.**

- If `apple` → (100, 1024) in an old index points into a new `data.dat`, users see random text.

Transition	How	Issue
1. Periodic refresh	Servers poll and reload the index	A window of garbage responses; needs graceful reload
2. Reactive Pub/Sub	A standalone job uploads, then publishes on Redis; API servers reload	Still a race unless the files are versioned
3. Parallel setup ✓	Upload to <code>s3://word-dictionary/001/{index,data}.dat</code> , <code>/002/...</code> ; update <code>meta.json</code> → <code>{index: "002/index.dat", data: "002/data.dat"}</code>	Old servers keep serving the old files; new servers load the new meta

1.6 Portability: one file with a header

Anchor: To put index and data in **one file**, don't use a separator. Prefix a **fixed-length header** that stores where the index and data start, plus metadata.



- New flow: the API server reads the fixed-width header → loads the index → starts serving.
- **Real world: multi-tiered storage**, with recent orders in MySQL and historical orders on S3, still queryable (Athena, data lakes). More on that next session.

Exercises:

1. Build the word dictionary locally as one file (header + index + data).
2. Write a simple API/CLI that returns a meaning using only local disk reads, no network.
3. Write `merge(dict, changelog) → new dict`.

 **Recall:** How do you serve a 1 TB dictionary with no DB and fast lookups?

Keep a ~2.7 MB index (word → offset, length) in each API server's memory; fetch only that byte range from the big data file on S3.

 Recall: Why version the files instead of overwriting data.dat?

Servers holding the old index would read wrong offsets in the new file and return garbage. Versioned paths + a meta.json pointer let old servers keep the old pair while new servers load the new pair.

2. A superfast KV store

Requirements: superfast reads, writes and deletes, plus **full persistence** (even on HDD).

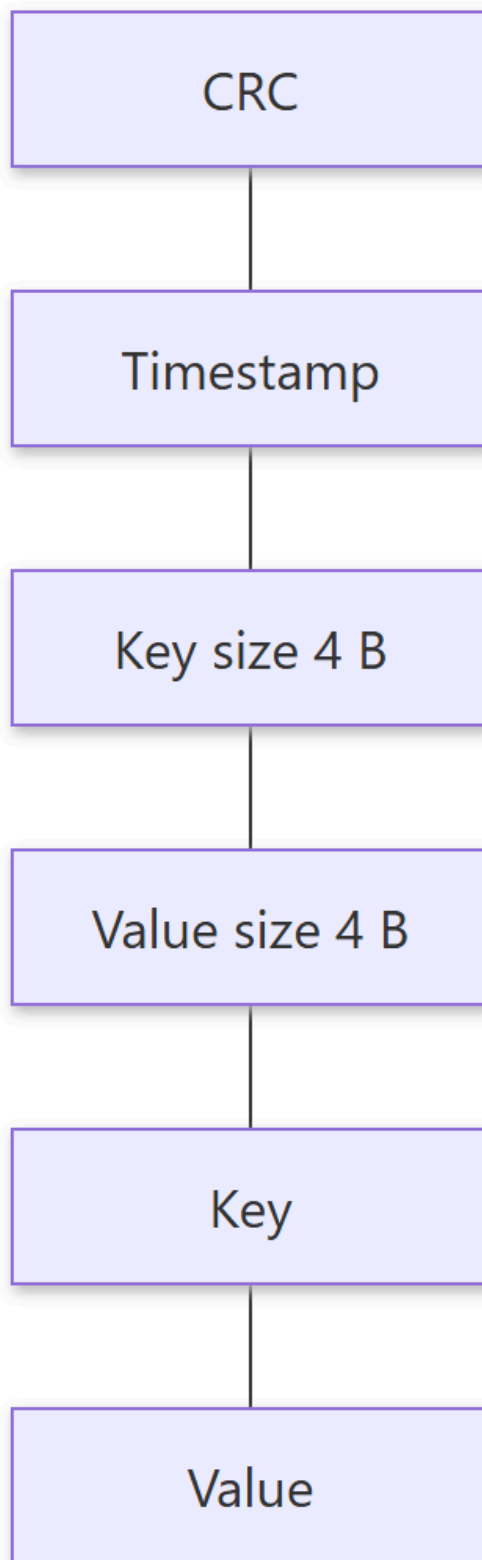
2.1 Log-structured storage

 Anchor: **Append-only, sequential writes, no random updates.** No disk seeks during writes means high write throughput, even on HDD.

- Sequential vs random I/O on HDD is a huge difference (the slide cites $\sim 5000\times$). SSDs gain less from it.
- Simplest design: a single file of KV pairs. `PUT(k, v)` = append, which is lightning fast.
- `DEL(k) = PUT(k, -1)`: a **tombstone** append, also lightning fast.

2.2 What one entry looks like

 Anchor: CRC | TS | KSZ | VSZ | K | V. Sizes tell you **how much to read**; the CRC tells you whether it's **intact**.



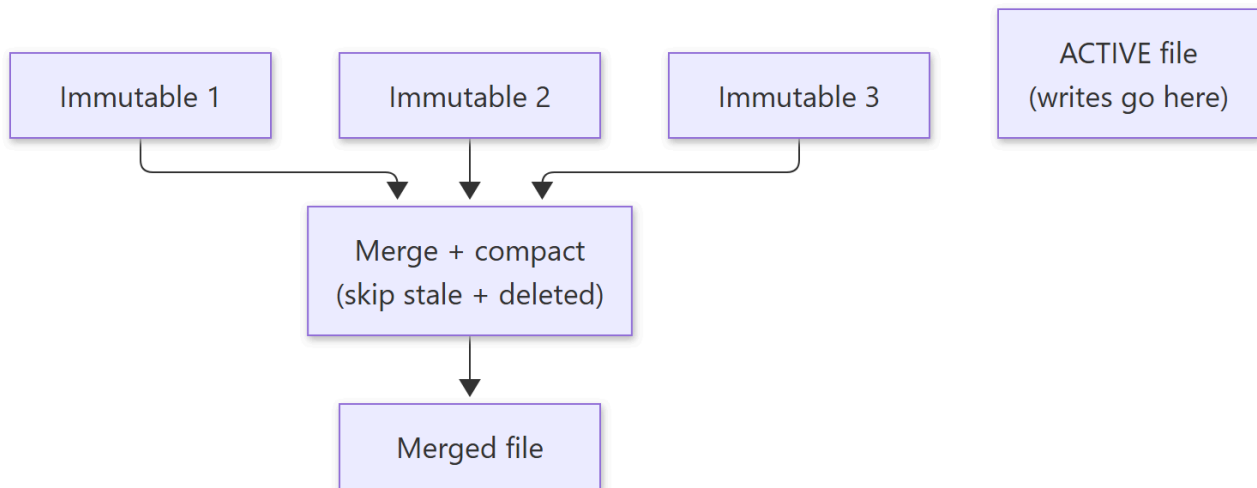
- Keys and values are variable length, so "read until newline" doesn't work. Read K SZ and V SZ (4 B each), then read exactly that many bytes.
- **Integrity:** if the machine crashes while writing the value, the entry is partial. A **CRC** detects corrupt entries.
- **Timestamp** resolves conflicts (the latest wins).

2.3 Fast GET: an in-memory hash index

Anchor: key $\rightarrow$ (file_id, value_size, value_pos, ts) in memory. GET = **one hash lookup + one disk seek + one read**, which is $O(1)$.

- **Limitation:** every key must fit in memory (values don't).

2.4 When the file grows: rotate, then merge and compact



- **Rotate** every t bytes: the old file becomes **immutable**, and a new **active** file takes writes. Reads may hit any file.
- **Merge and compact** the immutable files: keep only the latest value per key, and drop stale and deleted entries. Disk usage becomes a sawtooth.
- Offsets change after a merge, so **update the in-memory index atomically**.

2.5 What we built is Bitcask

- Strengths: fast reads, writes and deletes · high throughput, low latency · saturates disk I/O · easy backups (just copy the immutable files).
- Limitation: keys must fit in memory.
- Bitcask is the storage engine of **Riak** (each Riak node runs a Bitcask instance behind a proxy). Also see **MyRocks** (RocksDB as a MySQL engine) for the LSM variant.

 **Recall: How does Bitcask handle PUT, DEL and GET?**

PUT appends an entry and updates the in-memory index (key $\rightarrow$ file, offset). DEL appends a tombstone. GET looks up the index and does one disk read.

Recall: Why rotate files, and what does compaction do?

Rotation keeps the active file small and makes older files immutable (safe to read, back up, merge). Compaction rewrites the immutable files keeping only each key's latest live value, dropping stale and deleted entries, then atomically swaps the index pointers.

3. Log (WAL) recovery

 Anchor: A crash can only corrupt the **last** entry of an append-only log. On restart, scan, verify each CRC, and **truncate at the first bad one**.

1. Start from the top of the file.
 2. Read entry by entry (CRC | KSZ | VSZ | K | V).
 3. Validate each with its checksum.
 4. On a mismatch, **truncate** the file at that offset.
 5. Start the database.
- Almost every database uses this process to repair a corrupt log.

Exercises:

1. Benchmark sequential vs random I/O.
2. Implement Bitcask, especially the merge and compaction phase.
3. Simulate integrity failures (partial writes) and recover.

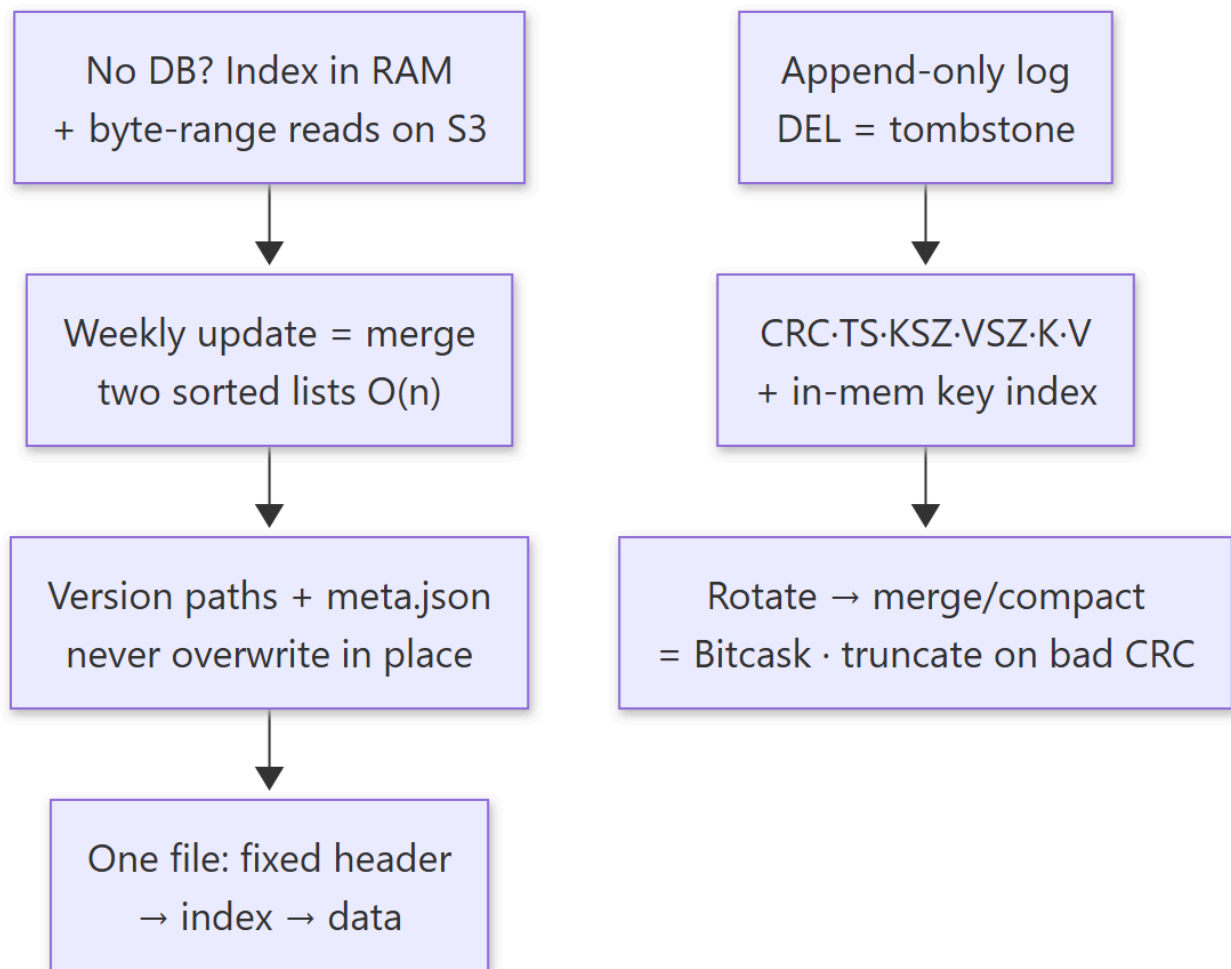
Recall: How does an append-only store recover after a crash?

Scan entries from the start, checking each CRC. The first failing entry (always the torn last write) and everything after it is truncated; then the index is rebuilt.

What these notes add beyond the slides

- **4-byte offsets can't address a 1 TB file.** $2^{32} \text{ B} = 4 \text{ GB}$. The size estimate uses $2 \times 4 \text{ B}$, but a 1 TB `data.dat` needs **8-byte offsets** (the brainstorm page hints at this). The index becomes $\approx 171,476 \times 23.7 \text{ B} \approx 4 \text{ MB}$, still tiny.
- **Bitcask boots faster with hint files:** during merge it writes a small "hint" file (key $\rightarrow$ position) next to each data file, so on restart the index is rebuilt without scanning all the values.
- **Bitcask vs LSM:** Bitcask keeps *all keys* in RAM (fast, limited key count). LSM trees (LevelDB/RocksDB) keep sorted runs on disk with sparse indexes plus Bloom filters, so the key count isn't bounded by RAM and range scans work.
- **Arithmetic checked:** $15.7 \text{ B} \times 171,476 = 2,692,173 \text{ B} \approx 2.6 \text{ MB} \checkmark$.

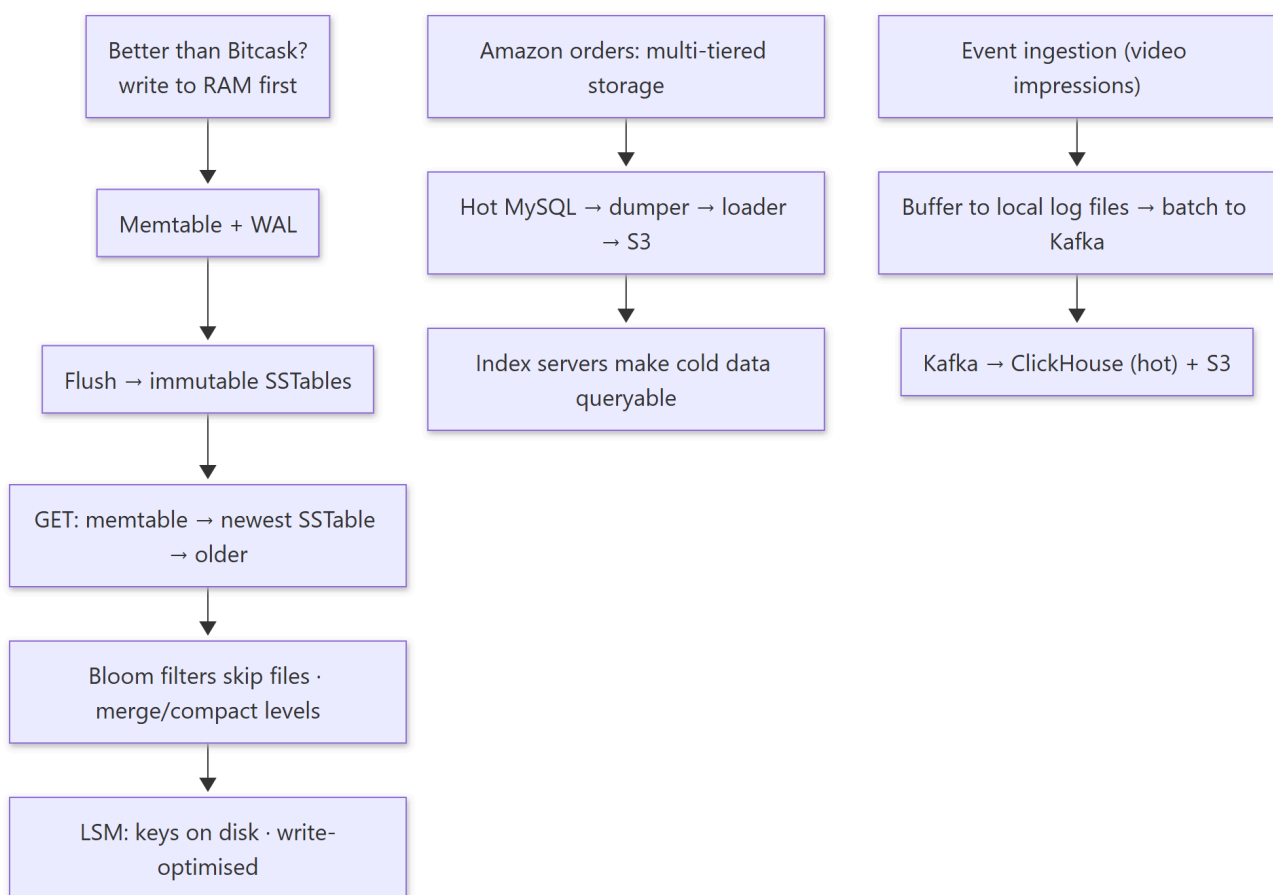
One-page memory card



06-01 · Arpit Bhayani's session notes: LSM trees, Multi-tiered order storage (Amazon), Event ingestion

Source: 06-01.pdf (System Design Masterclass, Google Drive, view-only). Study notes in my own words, not a copy. Pages covered: 1–16 of 16. Unreadable pages: none. Pre-reads named for the next session: "How video works" and the Google File System video (needed before the S3 discussion).

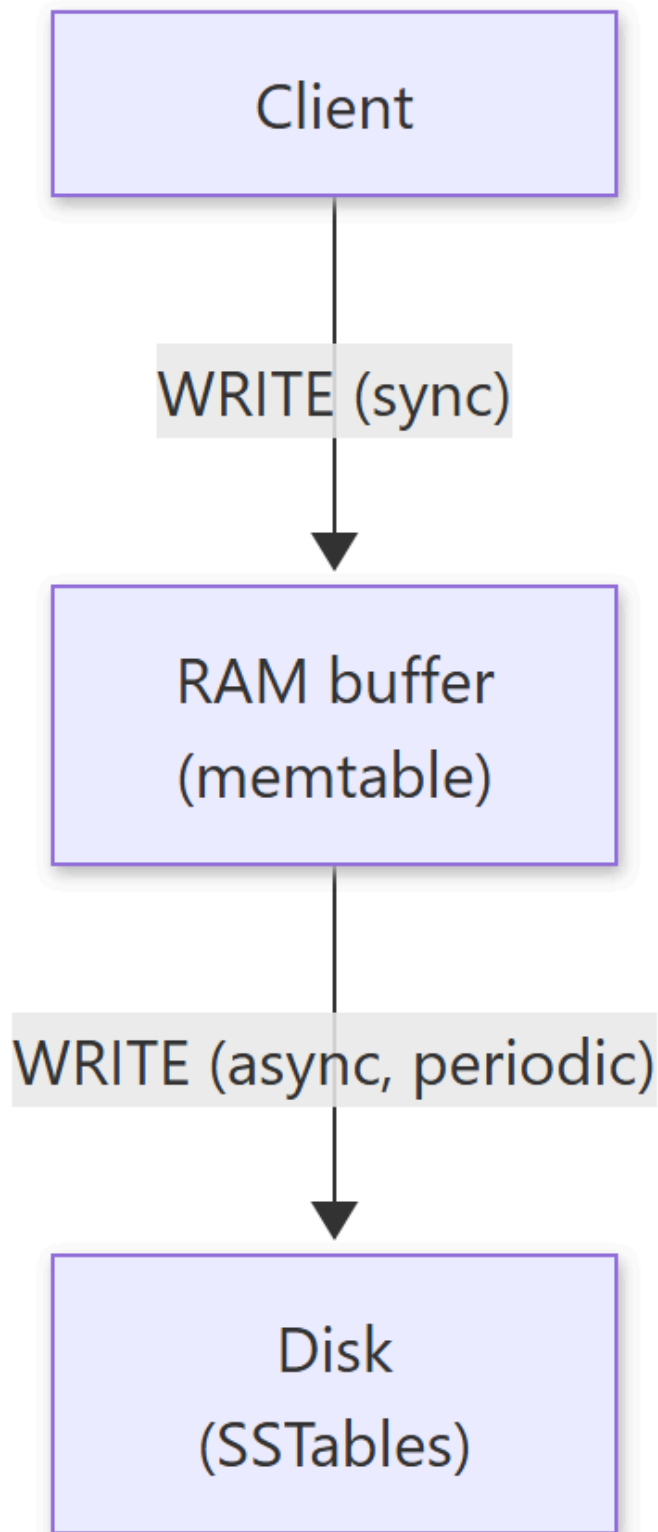
Map of the session



Agenda: **LSM trees: the intuition** · **Multi-tiered, cost-efficient orders at Amazon** · **Event ingestion**.

1. LSM trees: attempting to beat Bitcask

🗨️ Anchor: Bitcask = append-only files + an in-memory index of **all keys**. Core idea: **write to RAM first** (sync), flush to disk later (async, periodic). Writes get even faster, and keys no longer need to fit in RAM.



Brainstorm: in-memory data structure · operations pseudocode · on-disk layout · optimisations · durability · tunability and observability.

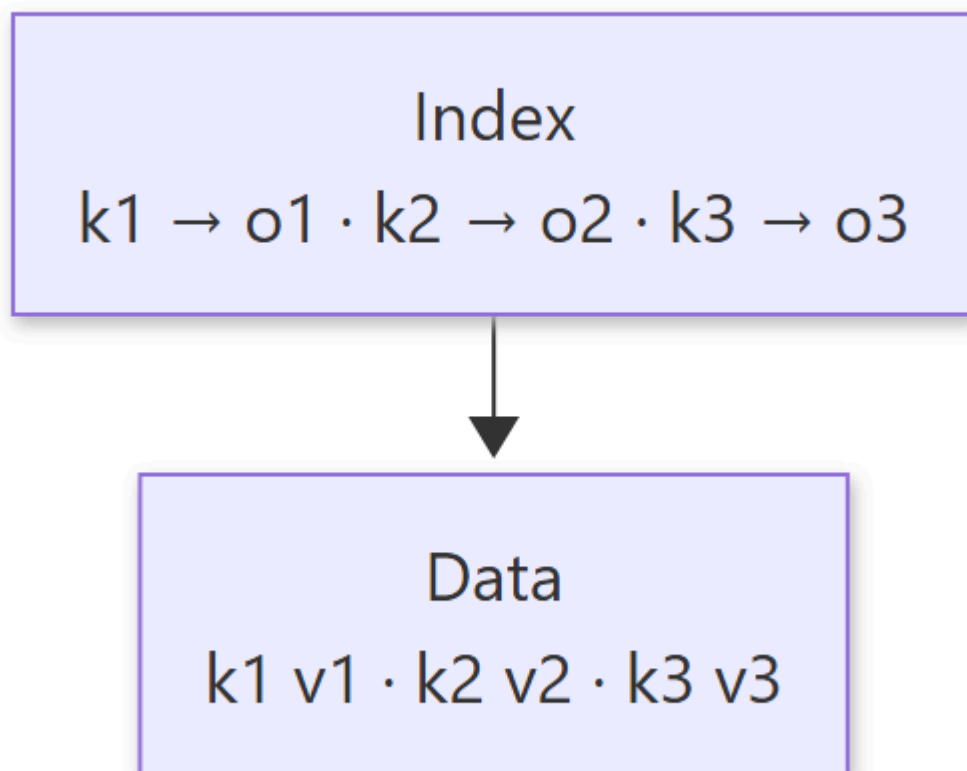
1.1 Operations

```
put(k, v): wal.write(k, v); map.put(k, v)
           if map.is_full(): map.dump(); wal.truncate()
del(k):    map.soft_del(k)                    # tombstone
get(k):    v = map.get(k);      if v: return v  # newest first
           v = disk.get(k);    if v: return v
           return None
on_load(): map = wal.load()                  # replay after a crash
```

- **Reads:** check RAM first. If the key is in memory, it's the **most recent value**. Otherwise check the disk; if it's not there either → NotFound.
- **Periodic flush:** every t minutes (or when the buffer is full) the memtable is flushed to disk in one go. Memory use is a sawtooth.
- **Where to flush?** Appending to one existing file means a long file and a long flush. **A new file on every flush** is faster and more efficient (flush everything in one shot).

1.2 SSTables

🧠 Anchor: Each flush writes a **Sorted String Table**: keys **sorted**, plus an **index** of key → offset. Files are immutable: 001.sst, 002.sst, ... 005.sst.



- `GET(k)`: look in the memtable; if absent, go from the **newest** SSTable to the oldest. In each file, check its **index** rather than scanning. Not found anywhere → 404.

- **Merge and compaction:** many immutable files pile up, so merge them. Since each file is sorted, merging is $O(n)$, the same "merge n sorted lists" as last week's dictionary + changelog.

1.3 Bloom filters: skip files that can't have the key

 Anchor: **"No" is certain, "yes" is maybe.** A per-file bit array lets a GET skip files that definitely don't contain the key.

- Worst case: a key that doesn't exist forces you to check k files. A set per file would be exact but too big at scale.
- **Bloom filter:** hash the key to bit positions. On insert, set those bits. On lookup, if any bit is 0 → **definitely not present**; if all are 1 → **maybe present** (go and check).
- Slide example: apple → 3, banana → 2, cat → 3 (bits 2 and 3 set). Is "dog" present? hash → 6, bit 6 = 0 → **No**. Is "elephant"? hash → 2, bit 2 = 1 → **Maybe**.
- Reference: github.com/arpitbbhayani/abloom (a sketch data structure).

1.4 Durability: the WAL

- Append every update and delete to a **WAL file** first. It's truncated after every successful flush. Recovery replays it (log recovery and corruption handling were covered last week).
- Durability is tunable (how often to fsync the WAL).

1.5 Levels and why LSM

- SSTables are organised into **levels** (L1, L2, L3, L4...). Compaction merges files down into larger, older levels, and each level can carry a Bloom filter.
- **If we write to disk on every write, how is this faster than Bitcask? It isn't.** For **zero data loss** you still need a WAL write per request.
- **How is it better than Bitcask? Keys are disk-bound, not memory-bound**, with comparable write amplification. (Why not Redis? Redis is memory-bound.)
- Spectrum of speed vs capacity: Redis (in-memory) → Bitcask → LSM → B+ tree DBs.
- **B+ trees are great for read-heavy workloads; LSMs are optimised for write-heavy ones.**
- Who uses LSM? **RocksDB, LevelDB, BadgerDB** (and Cassandra, ScyllaDB).

 Recall: Walk through a GET in an LSM tree.

Memtable first (the freshest value). Then SSTables from newest to oldest: consult each file's Bloom filter (skip on "no"), then its index for the offset, and read. Return the first hit; a tombstone means deleted.

 Recall: Bitcask vs LSM in one line?

Bitcask keeps every key in RAM (fast but bounded by memory). LSM keeps sorted runs on disk with indexes and Bloom filters, so the key count is bounded by disk.

 Recall: What does a Bloom filter answer, and with what errors?

"Could this key be in this set?" No = definitely absent. Yes = possibly present (false positives happen, false negatives never do).

2. Designing a cost-efficient orders system for Amazon

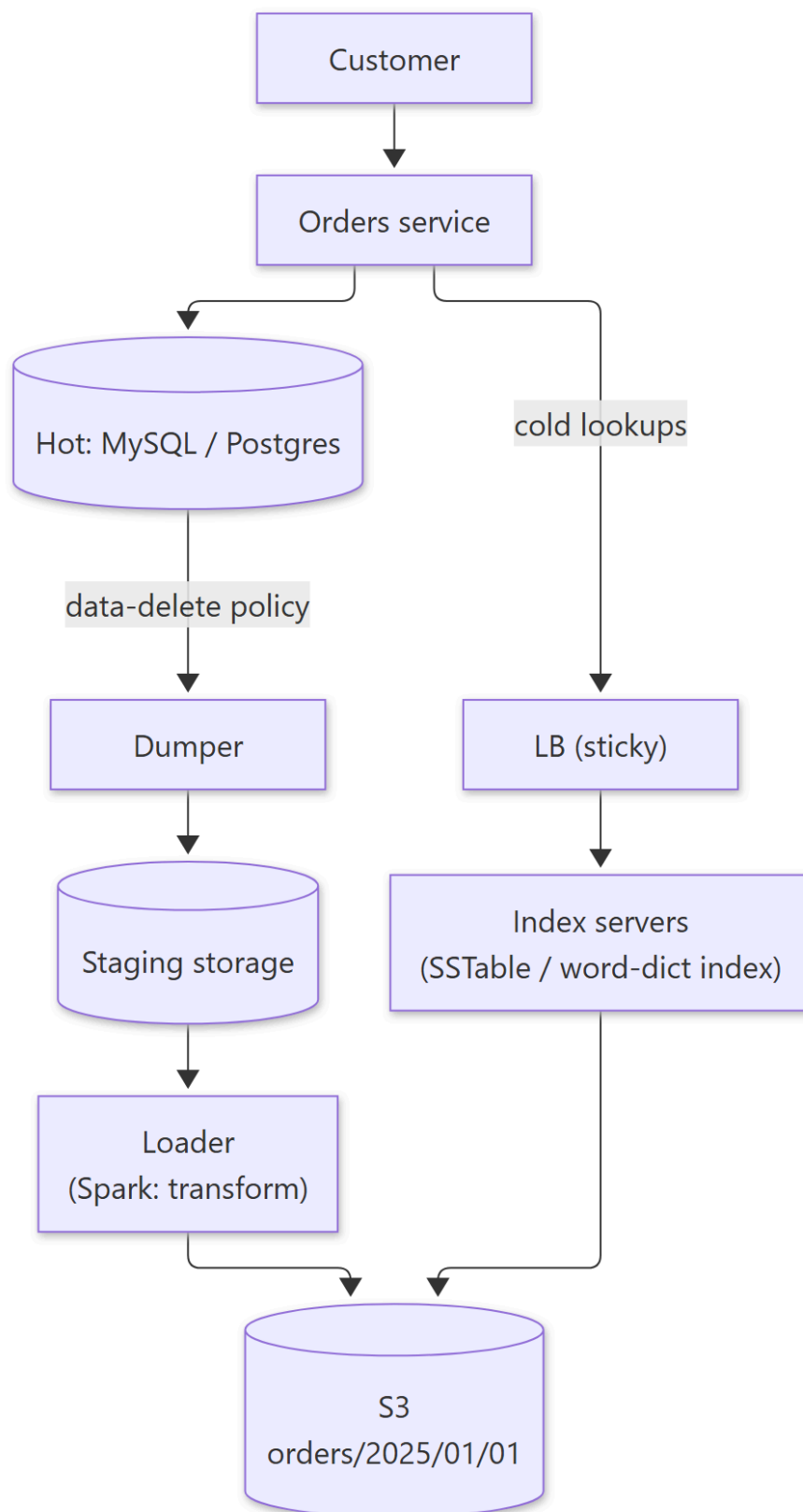
 Anchor: **Move orders between stores by age** to take load off the transactional DB, **without losing the ability to query them**.

Brainstorm: extraction, transformation, loading, file structure, querying, and how to route a query across multiple tiers.

2.1 Access pattern

- The orders service (with payments, logistics and customer support around it) writes to a **transactional MySQL**. Scale this.
- An order's life cycle: lots of reads and writes early on (hot, plus replicas), then infrequent reads, then very rare reads for **compliance** (cold).

2.2 Architecture



- A **dumper** pulls aged orders out of the hot DB (respecting a data-deletion policy) into **staging**. A **loader** (Spark) transforms them and writes to S3, partitioned like `s3://orders/2025/01/01`, and similarly for payments.

- Files on S3 use the **SSTable / word-dictionary layout** (index + data), so **index servers** behind a sticky LB keep the index in memory and do byte-range reads. Hive/Athena-style query engines can sit on the same files.
- **Overhead calculation:** hot ≈ 5 ms, cold ≈ 100 ms. Naive tiering: a hit costs 5 ms, a miss costs 5 + 100 ms (check hot, then go cold). That's fine because misses are rare old orders.

 **Recall:** How do you keep old orders queryable after moving them off MySQL?

Dump them by age, transform them into indexed files on S3 (index + data, partitioned by date), and serve lookups from index servers holding the index in memory, or query engines over S3. Hot misses fall through to the cold tier.

3. Event ingestion (e.g. YouTube video impressions)

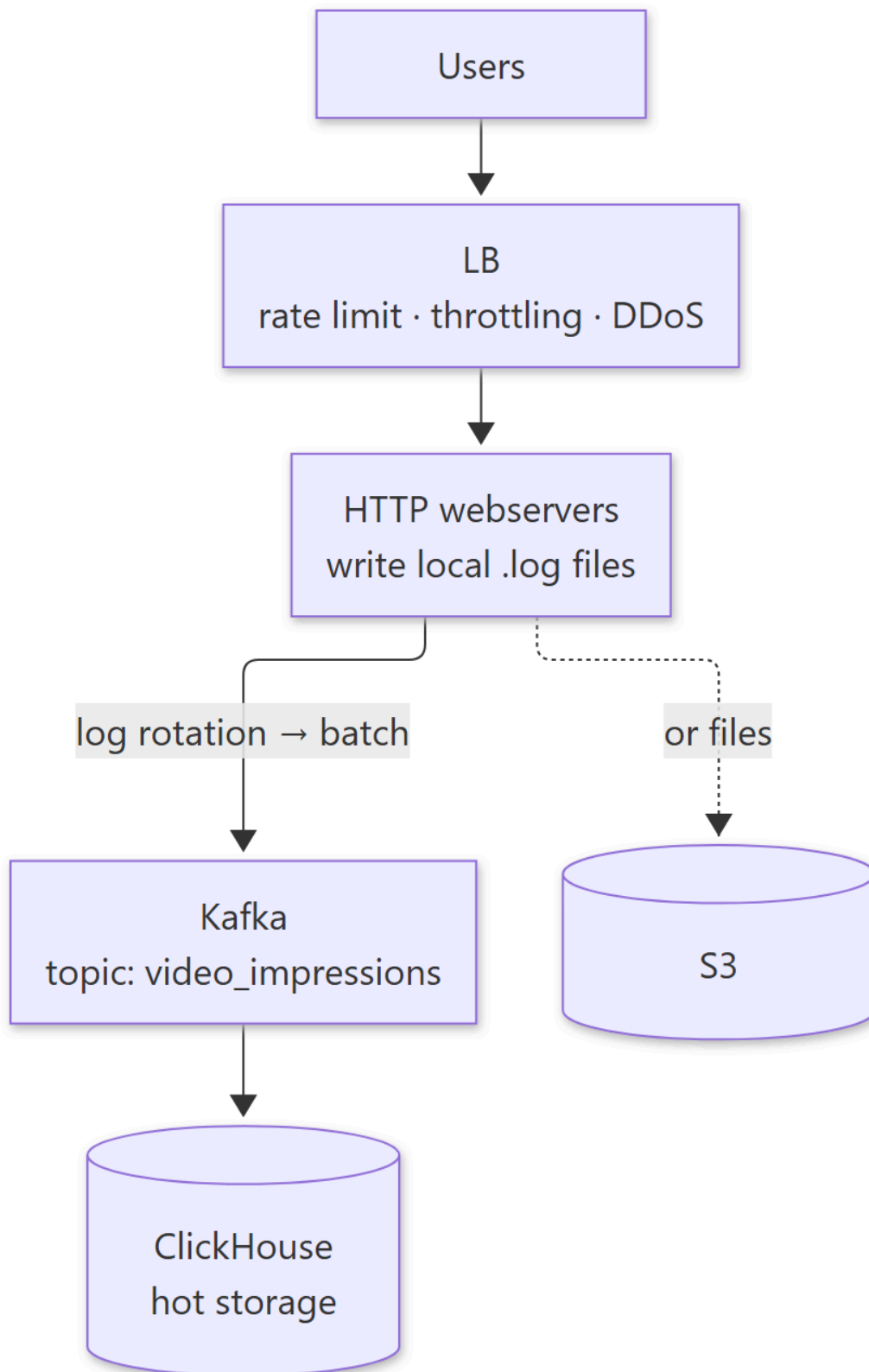
 Anchor: **Goal: high write throughput for semi-structured data.** Don't write each event synchronously. **Buffer locally, batch, ship.**

3.1 What's wrong with the naive version?

1. A **synchronous write to the DB** per impression. Make it async.
2. **Kafka may not keep up** with one message per event (back-pressure on ingestion).

3.2 Buffer to local log files, then batch

```
bw = 0
fp = open(<ts>.log)
process_event(e):
    data = e.json(); fp.write(data); bw += len(data)
    if bw >= max_log_size:          # e.g. 1 MB
        fp.close(); fp = open(<new ts>.log); bw = 0
```



- Webservers write events into **rotating local log files**. A **log-file reader/batcher/emitter** ships each closed file as a batch, so far fewer, bigger messages reach Kafka. (Should the batcher flush every N messages or every ~10 s? Either; it's a tunable.)
- Kafka topic `video_impressions` → consumers load into **ClickHouse** (hot analytical storage), with S3 for the raw files.

- **Partition key:** *not* `video_id`. A viral video would make one partition hot. Spread the load (round-robin or random).
- Stickiness at the LB isn't required. Rate limiting and throttling at the edge protect against floods and DDoS.

 Recall: Why write impressions to local log files before Kafka?

Each event becomes a cheap local append. Rotated files are shipped as large batches, so Kafka sees far fewer requests and the web servers never block on a remote write per event.

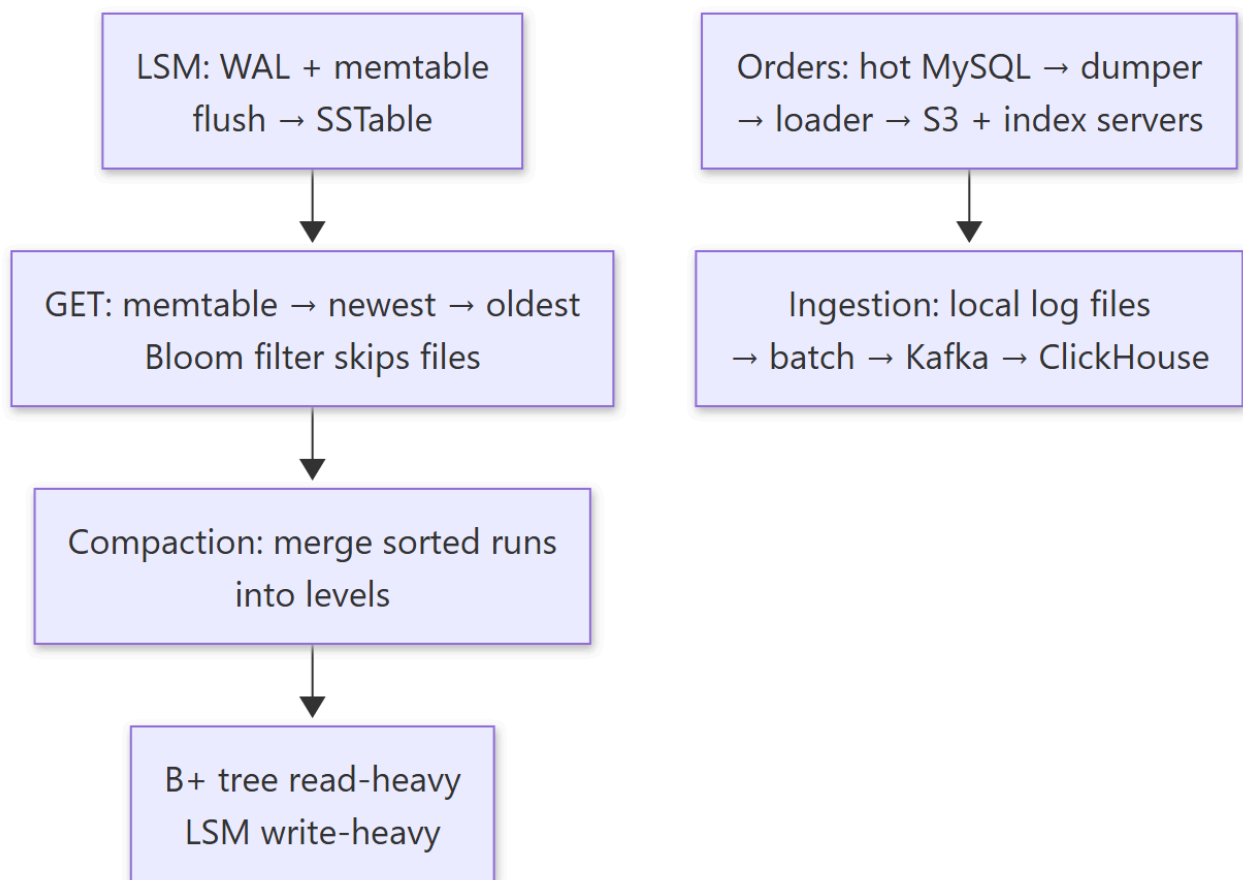
 Recall: Why not partition impressions by video_id?

Popularity is skewed: a viral video would overload a single partition. Spread events evenly instead.

What these notes add beyond the slides

- **Write amplification vs read amplification:** LSMs trade extra compaction rewrites (write amplification) and multi-file lookups (read amplification, reduced by Bloom filters) for cheap sequential writes. B+ trees update in place: fewer rewrites, but random I/O.
- **Bloom filter sizing:** about **10 bits per key gives ~1% false positives** (with ~7 hash functions). That's why per-SSTable filters are cheap enough to keep in memory.
- **Local log buffering trades durability for throughput.** Events in an unshipped file are lost if the host dies. That's fine for impressions (approximate counts are OK); not fine for payments.

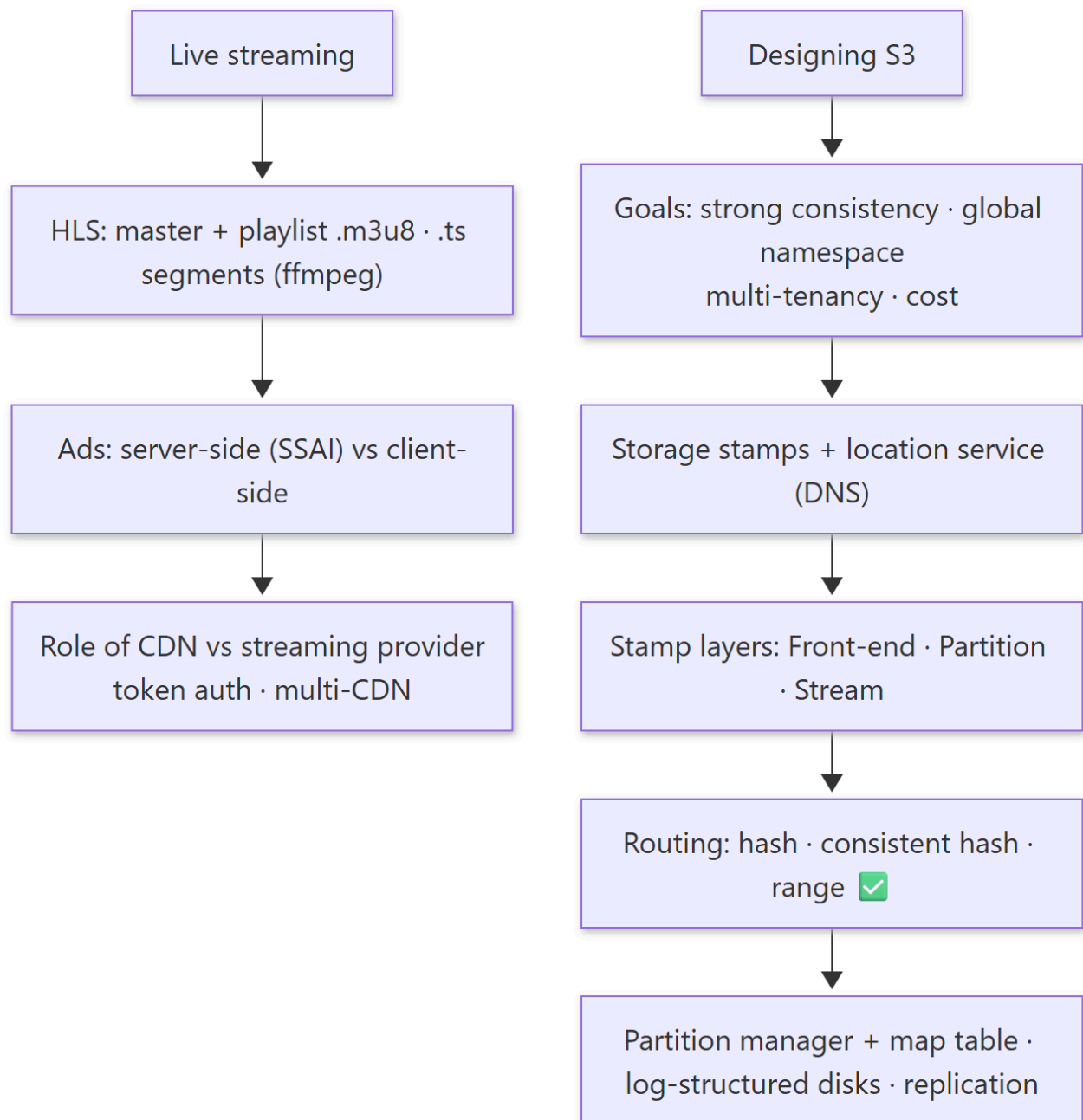
One-page memory card



06-02 · Arpit Bhayani's session notes: High-throughput systems: Live streaming (CDN + ads) and Designing S3

Source: 06-02.pdf (System Design Masterclass, Google Drive, view-only). Study notes in my own words, not a copy. Pages covered: 1–22 of 22. Unreadable pages: none.

Map of the session



Agenda: **Live streaming: the role of the CDN + ads serving · Designing S3.**

1. Live streaming

🗨️ Anchor: Video is cut into **small segments** (e.g. 10 s .ts files) listed in an **.m3u8 playlist**. Players keep re-fetching the playlist; live = new lines are **appended**.

- To understand the operational side of running live streaming at scale, the session points to a podcast with JioCinema's engineering team.
- To understand transcoding: **ffmpeg, m3u8, HLS**.

1.1 HLS files

File	Holds
Master .m3u8	One line per rendition: bandwidth (e.g. 2 Mbps), resolution (640×360, 800×480, 1280×720) → the playlist file for that rendition
Playlist .m3u8	#EXT-X-TARGETDURATION, #EXT-X-PLAYLIST-TYPE (VOD or EVENT/live), then #EXTINF:5.0, segment_0.ts, segment_1.ts, ...

- The **master doesn't change**. A **playlist can only grow**: new lines are appended as segments are created and processed. Media players reload it periodically.
- Making the chunks and playlist (the gist of the ffmpeg command): input video → H.264 video codec + AAC audio codec → hls_time 10 (10 s per chunk) → VOD playlist type → segment filenames segment%06d.ts, numbered from 0 → index.m3u8.

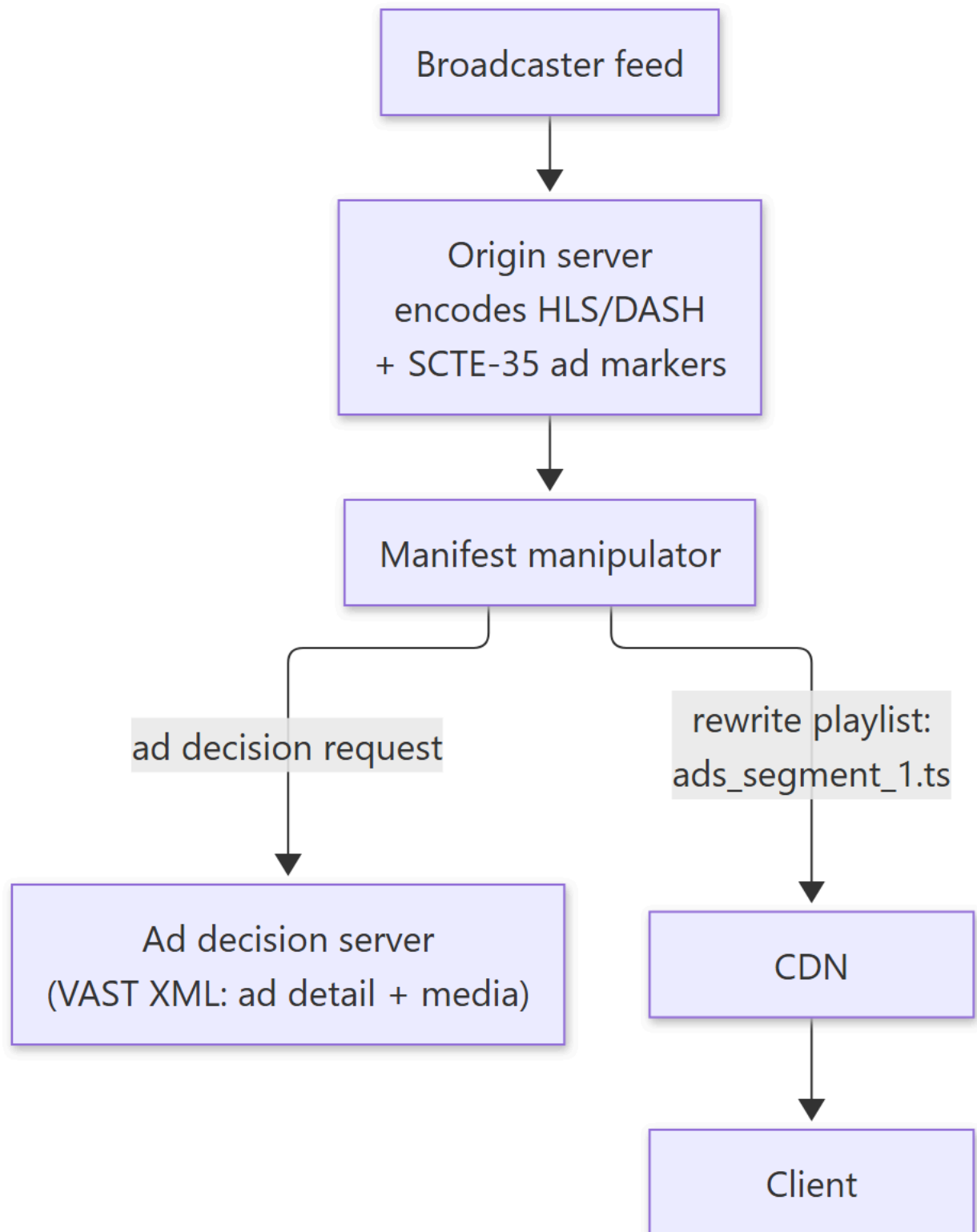
1.2 Ads in live streaming

🗨️ Anchor: **SSAI** stitches ads into the stream **on the server**: seamless and hard to block. **CSAI** inserts them **in the player**: simpler, but you see a glitch or spinner and ad-blockers can catch it.

	Server-side ad insertion (SSAI)	Client-side ad insertion (CSAI)
Where	Ads stitched into the stream at the server	Player inserts ads on "cue" events (JavaScript on player events)
UX	Seamless playback, no buffering between content and ads	Typical loading spinner or glitch on switch
Blockers	Hard to detect	Easier to block (think of ad markers)

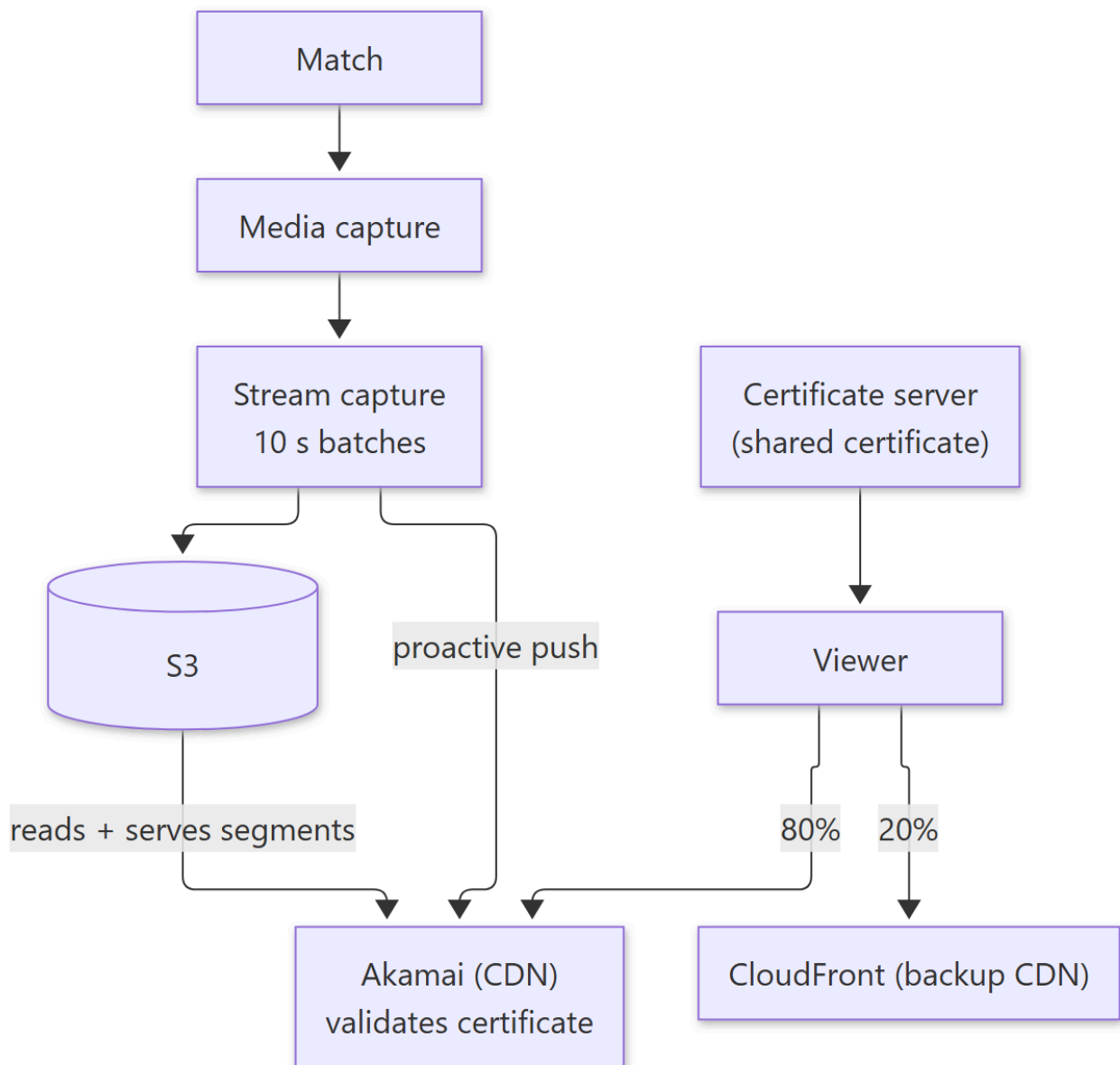
Other formats: **banner ads** (on cue → fetch the ad, render it) and **bumper ads** (short, non-skippable, ~6 s).

How SSAI works (in live streaming the client periodically re-requests the .m3u8 for new segments, which keeps the stream continuous):



- The ad decision server (which ad to show) is a complex system in its own right, and out of scope here.
- The manifest manipulator downloads the ad, splits it into segments and **inserts the ad segment lines** into the playlist. The client just plays what it's told. (YouTube's SSAI works this way.)

1.3 If the CDN does everything, what does the streaming provider do?



- **Private streams stay private** with **token auth at the CDN**: a certificate server issues signed tokens (a certificate shared with Akamai), and the CDN validates them per request (e.g. `match_id/364/...`).
- **Multi-CDN**: most traffic on one CDN, a share on a backup CDN. **Keep testing performance** and **switch in real time** if one degrades.
- The provider runs capture, packaging, auth, ads and the CDN strategy. The CDN does the delivery.

 **Recall: What's in a master vs a playlist m3u8, and how does "live" work?**

Master: renditions (bandwidth/resolution → playlist URL); it doesn't change. Playlist: an ordered list of short .ts segments with durations; for live, new segments are appended and players re-fetch the playlist periodically.

Recall: SSAI vs CSAI?

SSAI rewrites the playlist on the server so ad segments are part of the stream: seamless and hard to block. CSAI has the player fetch and play ads on cue events: simpler, but glitchy and blockable.

2. Designing S3

 Anchor: S3 is "just a KV store": `path` → `blob`. Requirements: **blob storage**, a network file system, **scalable and cheap**.

- Brainstorm: storage, routing, hot partitions, access, load balancing. The **biggest challenge is storage**.
- **Most important design decision: directories are logical and virtual**. Everything is path-based. (Read-heavy or write-heavy? Both.)

2.1 Design goals

Goal	Meaning
1. Strong consistency	<code>WRITE(f, "ball")</code> → <code>READ(f)</code> returns "ball"
2. Global namespace	<code>insta-images / bucket1/a.txt</code> is reachable from all regions under the same name; no regionalised exclusion
3. Multi-tenancy	Load from one customer must not affect others
4. Cost efficiency	Separate infra per customer isn't an option; shared hardware reduces storage and compute cost

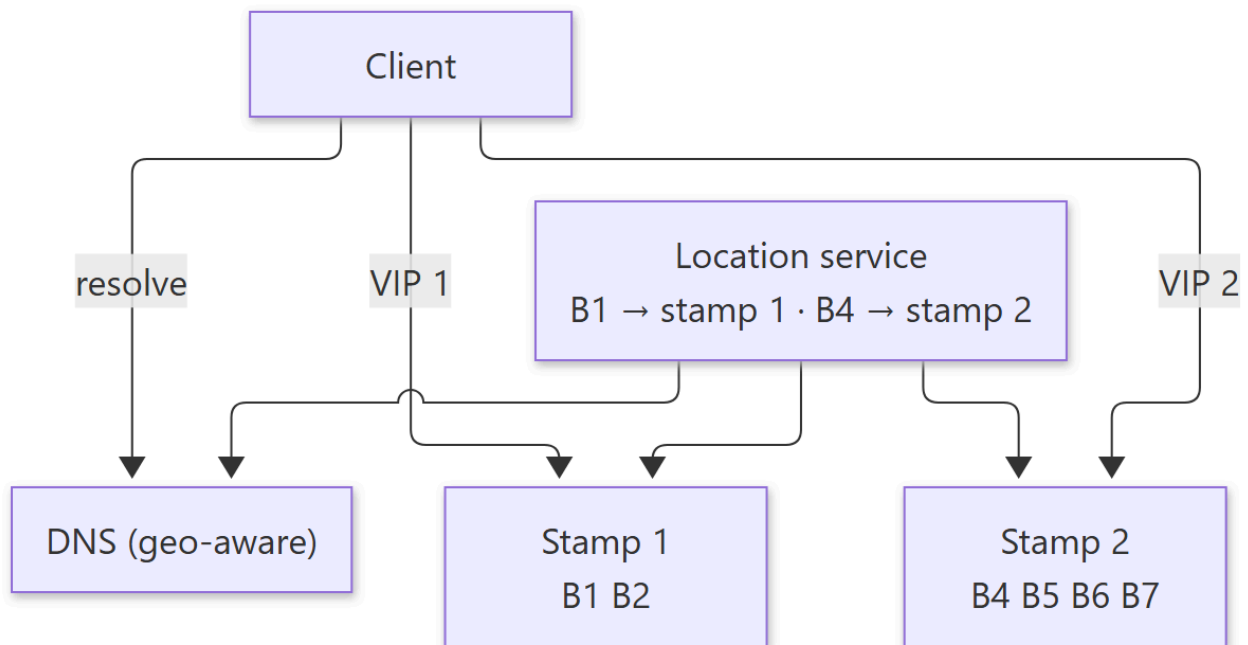
2.2 Global partitioned namespace

- `s3://instaimages/user_123/photo_1.jpg` = **bucket** + **key**. It's globally addressable; region doesn't matter; there's no explicit disjoint partitioning.

2.3 Architecture: storage stamps + location service

 Anchor: A **storage stamp** is a cluster of N racks (10–20 racks × ~18 storage nodes ≈ **30–50 PB**). A **location service** maps each account/namespace to a stamp and replicates across stamps.

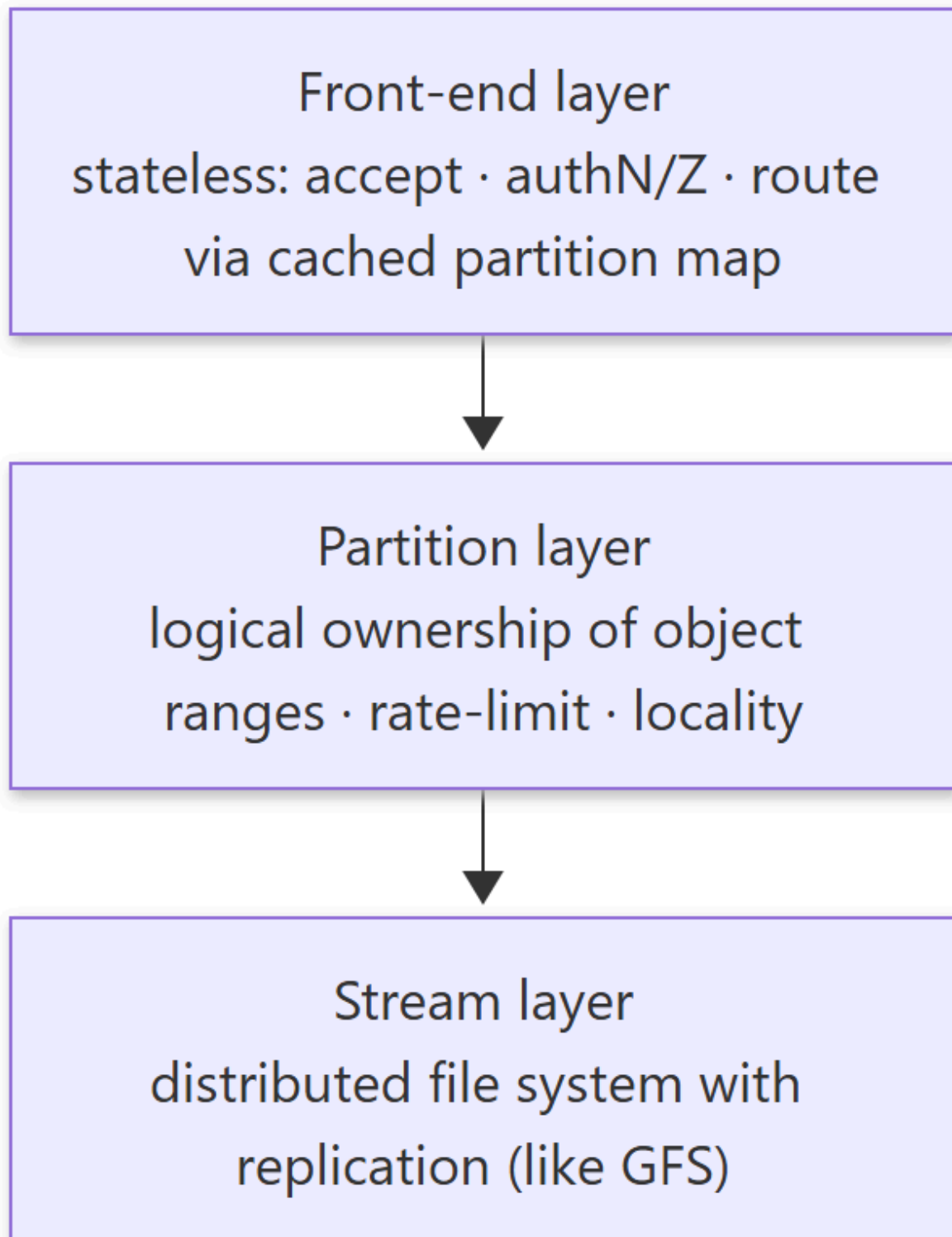
- Each rack has redundant networking and power. Stamps scale horizontally.
- Keep each stamp at ≤ **70%** utilisation (storage and network); the remaining **30% is reserved** to absorb rack failures within the stamp.
- **Location service**: maps namespace → storage stamp; manages account replication across stamps (redundancy, disaster recovery). It's deployed multi-region and is aware of everything.



- High availability means **redundancy**. The location service can add new stamps easily (horizontal scalability).
- **When a stamp fills up**: create a new stamp → assign new accounts to it → if one account grows huge, pick one: move smaller accounts to another stamp, or **split the namespace** and let the location service balance it (rare).
- Post-read: how Google balances load with **Anycast**.

2.4 Inside a storage stamp: three layers

🔗 Anchor: **Front-end** → **Partition** → **Stream**. These aren't microservices; each is a group of servers solving one problem.



Stream layer (files): a distributed file system with replication. It knows how to store and replicate files, e.g. 3 files with $RF = 2$ across regular VMs (storage nodes), with a coordinator. Read any distributed-FS paper (e.g. the Google File System) for the gist.

Front-end layer:

- A set of stateless servers in the stamp: accept requests (GET s3://bucket1/file.txt from the LB), **authenticate and authorise**, and **route** to the right partition server using a **cached partition map**.
- **Large objects (1 GB)** can't go in "one" request, so front-ends **stream** data to and from the stream/partition layer over persistent connections, sending as soon as data is available. Transfer-encoding chunked and HTTP streaming give a massive efficiency boost.
- Exercise: a TCP server that reads a 1 GB file and streams it to a client that receives it as a stream.

Partition layer:

- Should front-ends talk to the coordinator directly? Throttling and isolation become hard, and so does ownership. So the partition layer gives **logical ownership** of a partition (and locality), along with optimisation and mutual exclusion for multi-tenancy.
- The partition server process is **co-located** with the stream layer on each storage node.
- A path like s3://insta-images/user1/photo1.jpg tells the location service which stamp. **Within the stamp**, where does the file go?

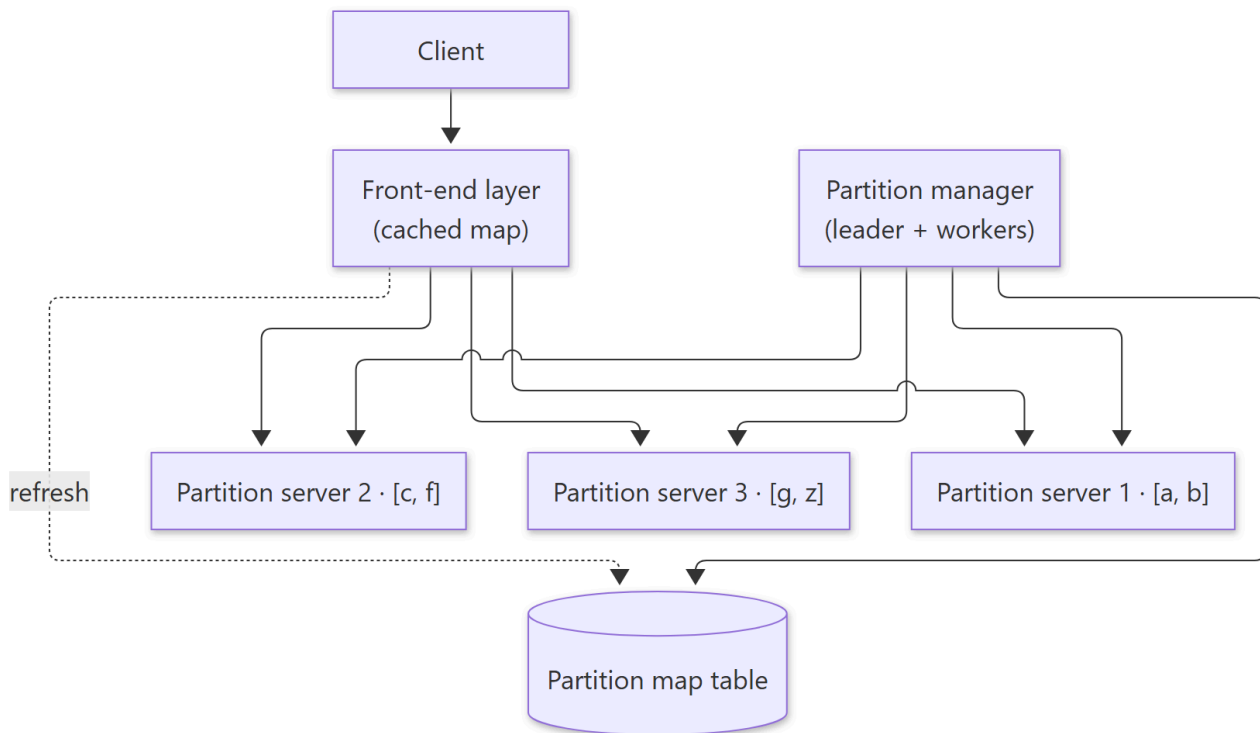
2.5 Routing: which partition server owns what?

 Anchor: **Range-based partitioning** wins for S3: it keeps an object's locality, gives control and isolation, and lets you **split a hot range in two**.

Strategy	How	Pros	Cons
1. Hash	<code>f(key) % n</code>	Near-random, uniform; no explicit config	Changing $n \rightarrow$ rebalancing and re-indexing; one bucket's files spread across nodes; no tenant isolation
2. Consistent hashing	Partitions and keys on a ring; the next node owns the data	Minimal transfer when nodes change; near-uniform	Still no tenant isolation; one tenant's workload affects others
3. Range 	<code>[a-e], [f-h], [i-k], ...</code>	Easier performance isolation; locality ; control over placement	Needs a map (below)

- Hash-based approaches **lose the locality of objects**. We want more control.
- **Hot node?** Scale out by **splitting** its range in two (e.g. `[a-z] → [a-m] + [n-z]`, mutually exclusive). Also **throttle** requests per account beyond a limit.

2.6 Partition map table + partition manager



- The **partition map table** records which partition lives on which node. The **partition manager** manages partition movement and assignment.
- **One partition is owned by exactly one partition server**, but one server can own **many** partitions.
- The partition layer adds **rate limits/throttling** and **hot-partition handling**. Compute load is isolated while data stays on the same storage nodes (logical vs physical).
- **No SPOF in the manager**: a leader + workers. Workers do the heavy lifting; the leader watches them. If the leader dies → re-election.

2.7 Storage layer: what kind of disk, and why?

- The storage spectrum runs cache → RAM → SSD → disk → tape. **Spinning HDDs are cheap**. Getting write performance out of them means **no disk seeks**, so use **log-structured storage**.
- When writes hit the stream layer, it **always writes sequentially**. No random writes.

2.8 Data durability

🔗 Anchor: The only way to achieve durability is **duplicating data**: **sync** replicas inside the stamp, **async** copies across DCs and regions.

- Storage node 1 → **SYNC** → storage node 2 (same stamp, different rack). **ASYNC** → storage node 3 (replicated across DCs). Async also feeds cross-region S3 replication. (Business continuity plan.)

Papers to read: *Windows Azure Storage: A highly available cloud storage service with strong consistency* (this design closely follows it) · building a database on S3 (**SlateDB**, **Rockset**) · GFS, HDFS, Facebook's SCUBA.

 Recall: What are S3's four design goals?

Strong consistency, a global namespace, multi-tenancy (isolation), cost efficiency (shared hardware).

 Recall: Name the three layers in a storage stamp and their jobs.

Front-end: stateless; auth and routing via a cached partition map; streams large objects. Partition: logical ownership of key ranges, throttling, hot-range splitting. Stream: replicated, append-only distributed file system on the disks.

 Recall: Why range partitioning over hashing for S3?

It keeps a bucket's objects together (locality), allows per-tenant isolation and control, and lets you split a hot range. Hashing scatters tenants and needs rebalancing when n changes.

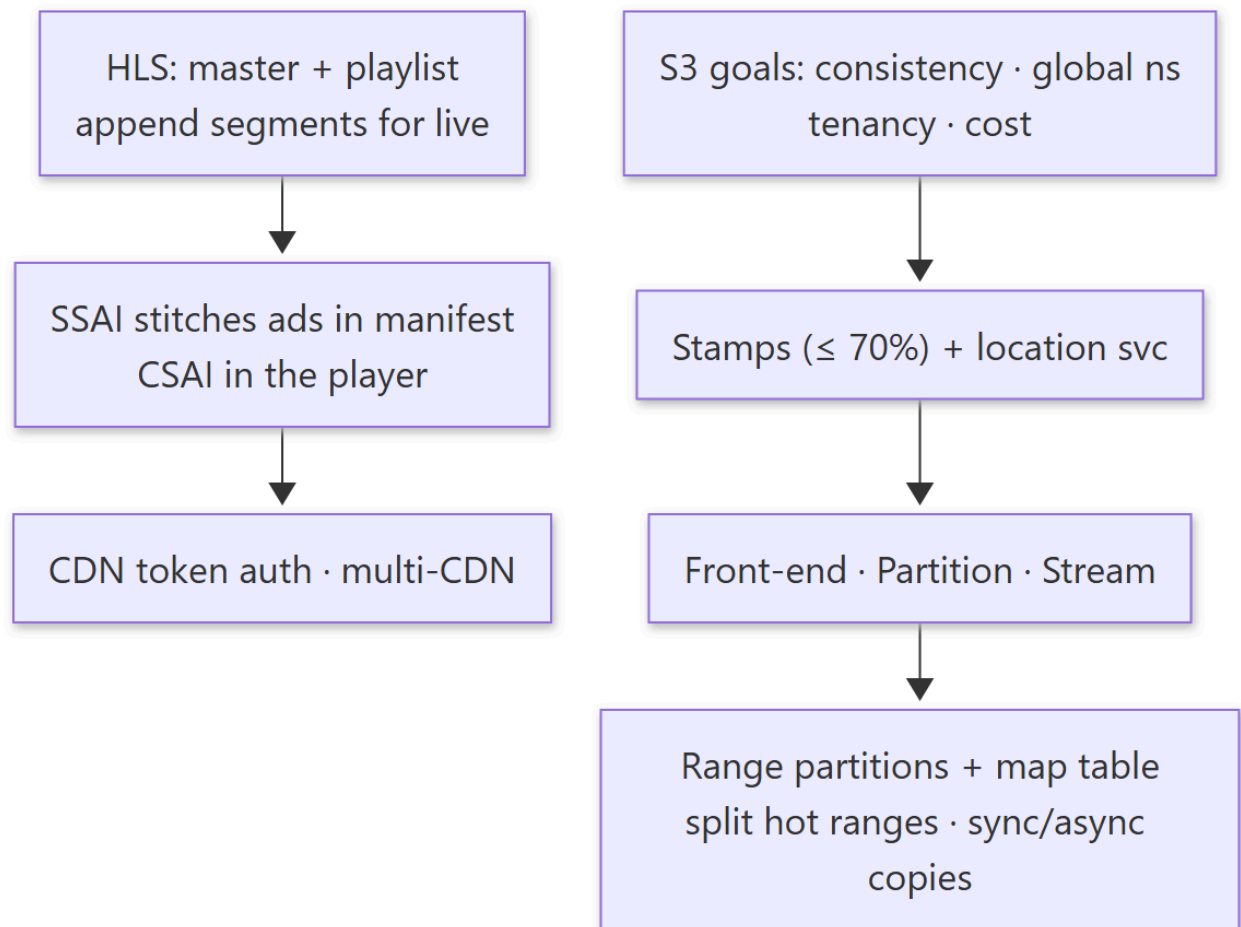
 Recall: Why keep stamps at 70% utilisation?

The 30% headroom absorbs rack failures (re-replicating the lost data) without overflowing the stamp.

What these notes add beyond the slides

- **Name check:** the "storage stamp / location service / front-end · partition · stream" design is **Windows Azure Storage** (SOSP 2011). Amazon S3's internals differ in detail, but this is the canonical public design to describe in interviews, and the slides recommend that paper.
- **Partition map consistency:** front-ends cache the map. When a partition moves, a stale front-end hits the wrong server, which must reject the request so the front-end refreshes. Ownership needs a lease or epoch so two servers never both serve a range.
- **The 70/30 rule** is capacity planning for failure: the slack must be at least the largest failure domain you plan to survive (a whole rack $\approx 1/10 - 1/20$ of a stamp).

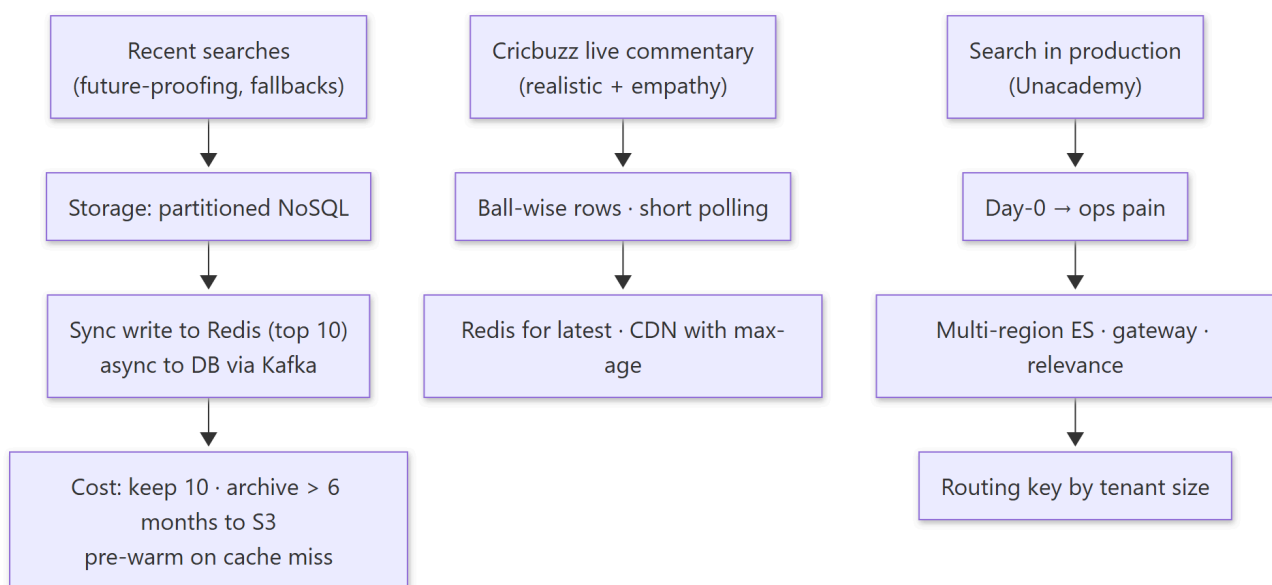
One-page memory card



07-01 · Arpit Bhayani's session notes: Information retrieval: Recent searches, Cricbuzz live commentary, Elasticsearch in production

Source: 07-01.pdf (System Design Masterclass, Google Drive, view-only). Study notes in my own words, not a copy. Pages covered: 1–19 of 19. Unreadable pages: none. Pages 3–4 and 10 are whiteboard overviews; the clean pages after them cover the same ideas.

Map of the session



Agenda: **Designing recent searches · Designing Cricbuzz's text commentary · Running Elasticsearch in production.**

1. Designing "recent searches"

🗨️ Anchor: **50% of users tap the search bar within the first 5 s, and 30% of all searches happen through recent searches.** So recent searches must be **ready before the user asks.**

Focus area: future-proofing and fallbacks.

Key questions: bounded or unbounded data? Is stale data OK? Cross-device consistency? How to store it? Data guarantees? Cost efficiency? Order of operations? A better UX and fallbacks?

1.1 Storage

- We store users' **search queries**: a large volume, **high write throughput**, no relations, and access is always **per user**. So use a **partitioned NoSQL DB** (MongoDB, Elasticsearch).

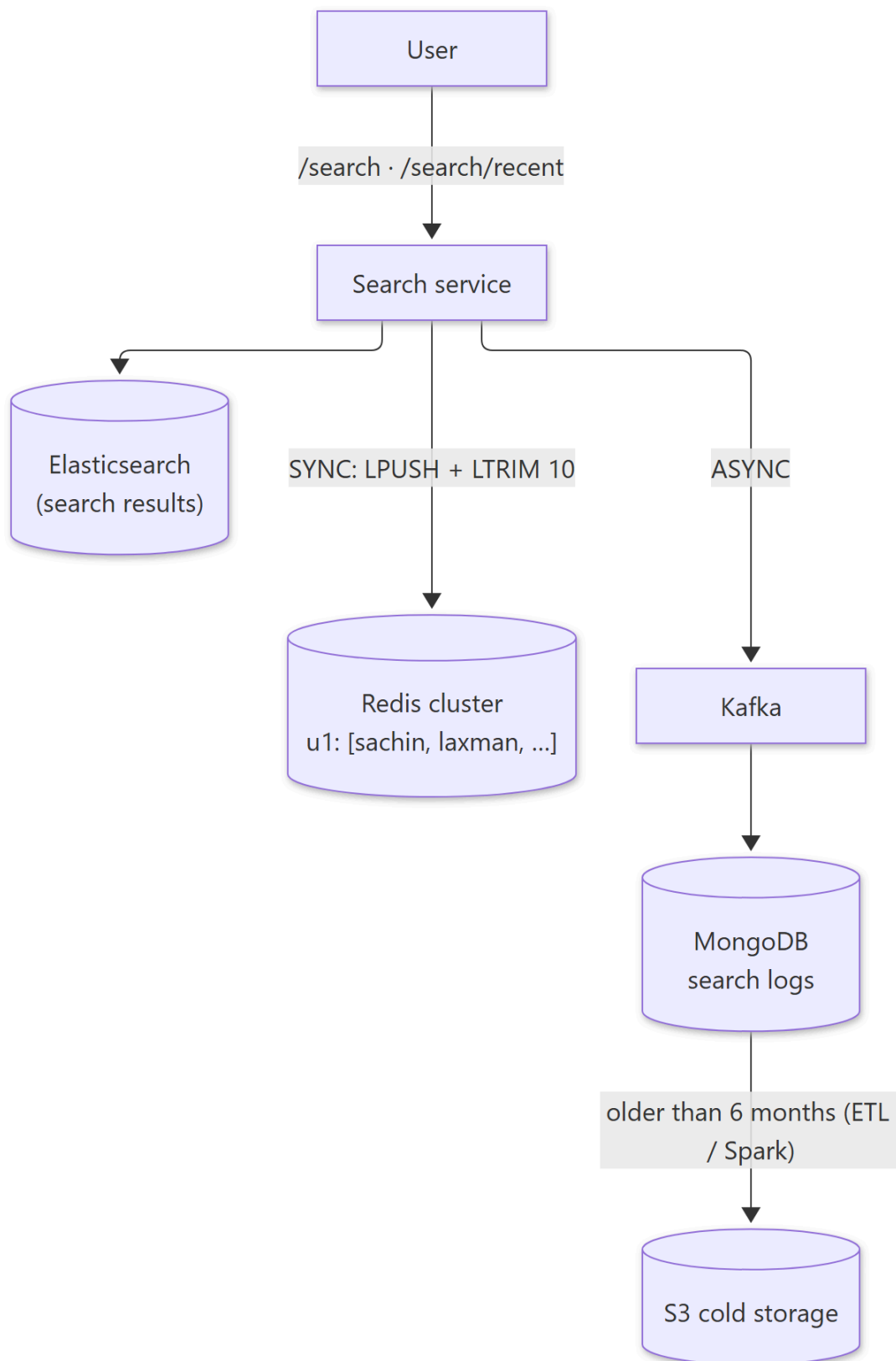
- Decisions: one record per query per user, or one record per user? Should the write from the API to the DB be **sync or async**?
 - Async → via SQS or Kafka. Sync → can the DB take it? (Search is one of the most-hit APIs.) There's also a delay before the record shows up in the DB.

1.2 Let user behaviour drive the design

Behaviour	Implication
50% tap the search bar within 5 s	High read requests; recent searches must update in near real time and be pre-loaded
30% of searches go through recent searches	High write ingestion

- The delay between a search and it being persisted should be **zero**: (1) do a **synchronous write**; (2) because it's both read- and write-heavy, use an **in-memory store** holding the **pre-computed** result. Quick lookups, no disk, no separate index needed.
- Why not read straight from the partitioned DB? A user's records are physically scattered (`u1, sachin` · `u2, sehwag` · `u1, laxman` ...), so a LIST per user is expensive at huge read volume.

1.3 Final design



- **SYNC write to the in-memory store, ASYNC to the partitioned DB.** Async helps extensibility (other consumers can use the stream), and losing a Kafka write occasionally is OK at this volume.
- Redis holds `u1 → ["sachin", "laxman", ...]`: the **pre-computed** recent 10. Extremely fast reads with no processing, and the same on every device.

- The search log has many other uses: imagine you're at a marketplace. Aggregate `query → count` (trending), sort per user by time, do analysis (fuzzy matching, related queries), keyed by user, query, region and timestamp. Writes are heavy, so it's a **cheap, persistent** log.
- **DLQ** if the Redis update fails: `try: redis.update(u, q) except: dlq.push(u, q)`.

1.4 Cost optimisation

1. Redis stores **only the recent 10** per user, not every query.
 2. Historical queries may never be accessed again, so **archive** anything **older than 6 months** to S3 via ETL jobs (Spark). That saves a lot of money and compute. Whether old history is shown at all, or with higher latency, is a **product call**.
- **Storage estimate:** 10 queries × 16 B × 1M users = 160 MB.

1.5 UX: what if recent searches aren't in Redis?

- Keys get deleted from Redis (DAU ≪ total users). Fetching from the DB on demand takes time, which is poor UX.
- **Pre-warm** instead: as soon as the user opens the app, load their recent searches into Redis, so they're ready by the time the search bar is tapped.

Recall: Why write recent searches synchronously to Redis but asynchronously to the DB?

Users expect to see what they just searched immediately (zero delay) and reads are very hot, so a pre-computed in-memory list per user is written sync. The full history is for analytics and backup, so an async Kafka write is fine.

Recall: Two cost optimisations for recent searches?

Keep only the last 10 per user in Redis; archive logs older than 6 months from the DB to S3.

2. Designing Cricbuzz's live text commentary

 Anchor: **Design realistically, with empathy.** Commentary changes about once a minute, so **short polling + caching** beats WebSockets.

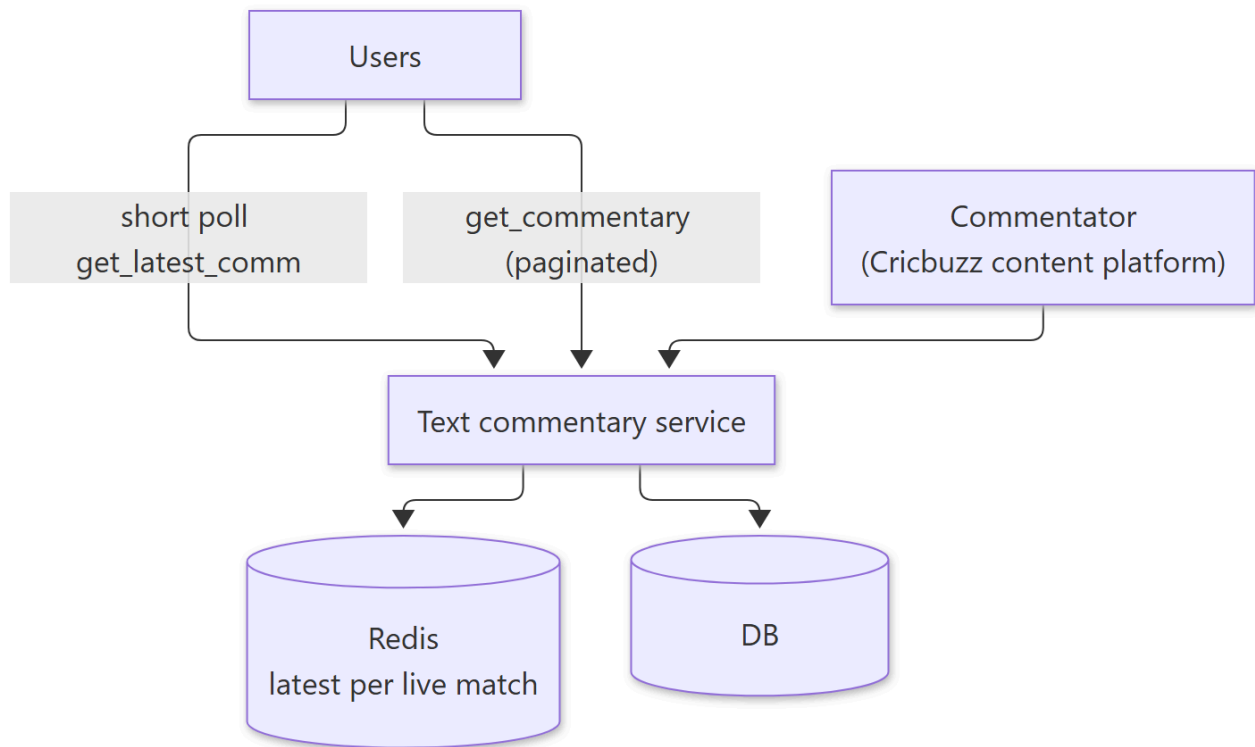
Requirements: users see live text commentary (Cricbuzz is India's 8th most-visited site) · cost-efficient architecture · good user experience. **Brainstorm:** storage, access, cost optimisation, communication; the user interfaces (users and commentators).

2.1 Storage

- Per match, store **ball-wise commentary**: `match_commentary(match_id, innings, ball, text)`.
- About 600 rows per ODI × ~50,000 matches.

2.2 Access

- The API fetches the **latest commentary** (last ~15 balls). **No per-user offset**; just show the latest.
- **Short polling** ✓, **WebSockets** ✗. Sockets would be expensive and underused (about **one event per minute**). This isn't a "realtime" use case.
- Everyone fetches the same latest entries, so **cache them in Redis**. New commentary → write the **cache and the DB** directly (you can't wait for cache invalidation).



- **Cost:** archival + short polling. **Durability:** retry. **Good UX:** direct updates to Redis; reads of the latest commentary come from Redis.
- **Consistency:** Redis is updated by the commentary service when a commentator writes. The ball entry is always **upserted** (overwritten if it already exists).

2.3 Serving via CDN (without Redis)

- `/matches/<mid>/commentary` → the last 20 entries. `/matches/<mid>/commentary?page=1` → subsequent pages.
- Put a **CDN** in front with `Cache-Control: max-age=5` and **cache prefetching at ~90%** of the TTL. The latest commentary comes from the CDN; page 1 onwards and other APIs go to the API.
- Prefreshing works because the TTL is small: the CDN re-fetches *before* expiry, so users never wait on a miss.

Recall: Why short polling, not WebSockets, for cricket commentary?

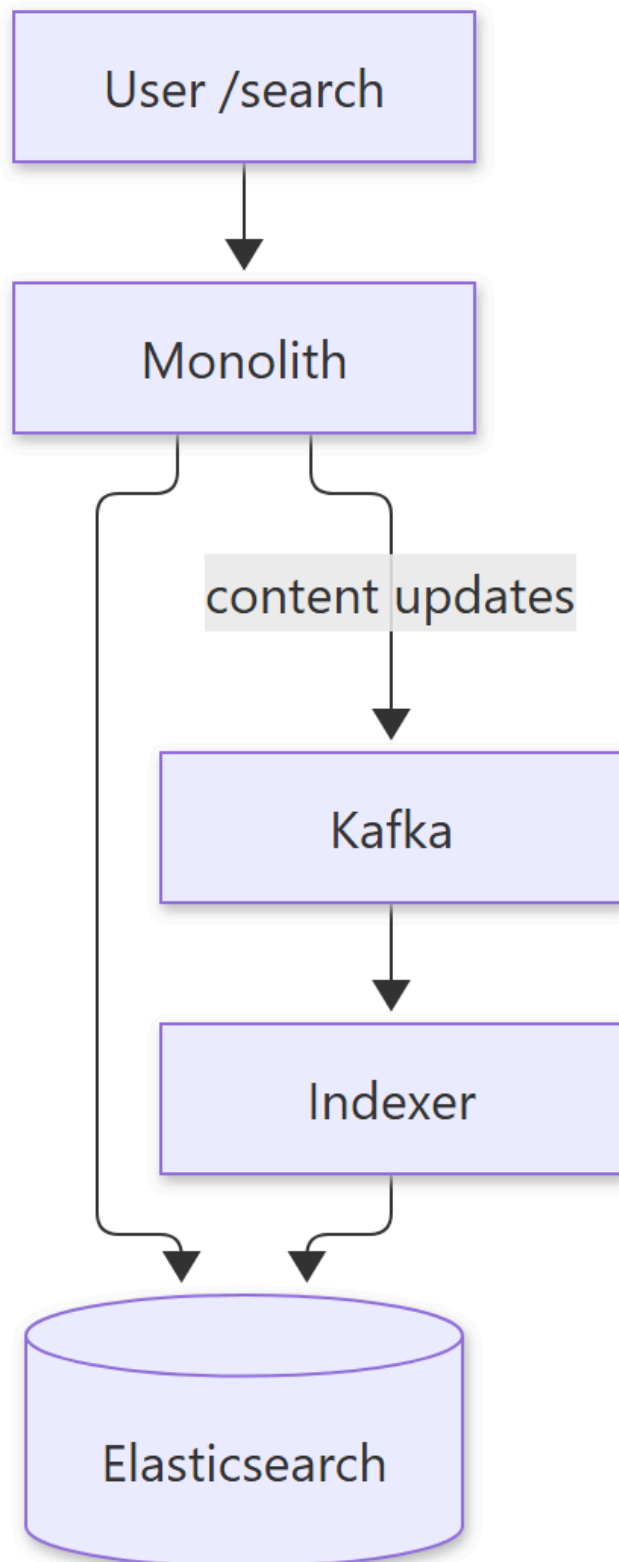
Updates are about one per minute and everyone reads the same data. Persistent sockets for millions would be expensive and mostly idle; cached short polls (Redis or CDN with a few seconds' max-age) are cheaper and good enough.

3. Running Elasticsearch in production (Unacademy)

 Anchor: Search is a **platform**: it's everywhere (live search, recent updates, dashboards, ...), so it needs stability, relevance experiments, and **tenant isolation**.

Requirements: better stability and reliability · improved relevance and search experimentation · large customer growth · a consistent search experience across all microservices and features.

3.1 Day-0 architecture and its pain

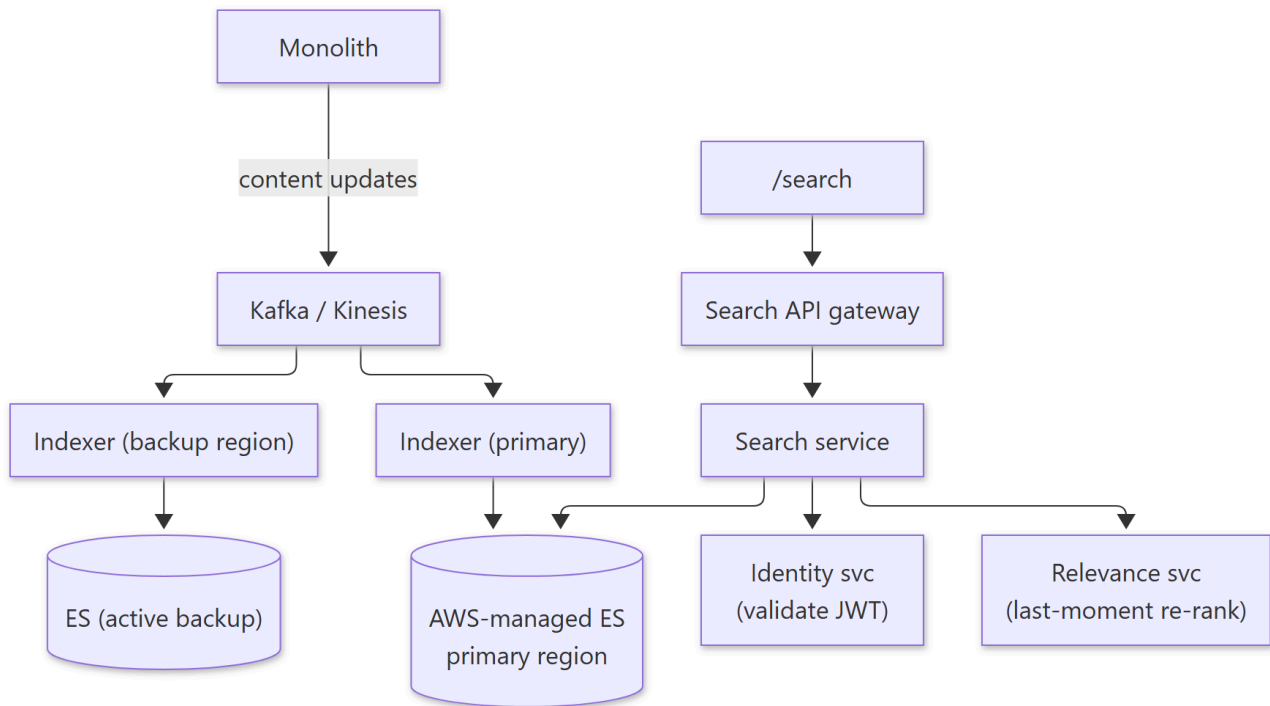


Operational concerns:

1. When ES goes down or is unhealthy, getting back to health is painful: time-consuming and needs expertise.
2. The Day-0 architecture is fine early on but limits performance and experimentation at scale.

3. Debugging ES needs core expertise.
4. No automatic replication or latency-based routing.
5. Large tenants need *many* nodes, which shoots up the cost.
6. Rate limiting and throttling are missing.

3.2 New architecture



- **Write path:** content updates go to Kafka/Kinesis, and separate indexers fill the primary *and* a backup-region ES cluster.
- **Read path:** search API gateway → search service → identity (JWT validation) + relevance (**re-ranks results at the last moment**) → managed ES.
- **Multi-region failover + latency-based routing.** If there's a single region and it has a maintenance window or outage: copy (~2 h), load, sync, fail over, switch back when done. Latency-based routing is a nice by-product (e.g. a user travelling, or switching from office Wi-Fi to mobile hotspot).

3.3 Hot shards → routing keys by tenant size

🔗 Anchor: Some tenants are far bigger than others, so random distribution creates **hot shards** and **under-used shards**. Use a **routing key**, sized by tenant.

- An ES index has multiple shards; the **routing key** decides which shard a document lands on: `PUT /index-airbnb/_doc/1?routing=xyz`. The classic routing key is `tenant_id`.
- **Strategy:**
 - **Small tenant** → all its docs on **one shard**: `shard = tenant_id % NUM_SHARDS` (e.g. `729 % 10 = shard 9`).
 - **Large tenant** → **split across several shards**: `routing = tenant_<id>_<doc_id % N>` (e.g. `tenant_729_2`, `tenant_729_1`).
- Benefits: (1) scalable even for large tenants; (2) handles the **noisy-neighbour** problem; (3) no custom service or config, just pass the routing key; (4) operationally simple.

Recall: How does the routing-key strategy avoid hot shards?

Small tenants are pinned to one shard ($\text{tenant_id} \% \text{shards}$), so their queries hit one shard. Large tenants spread across N shards by adding $\text{doc_id} \% N$ to the routing key, so no single shard carries a giant tenant.

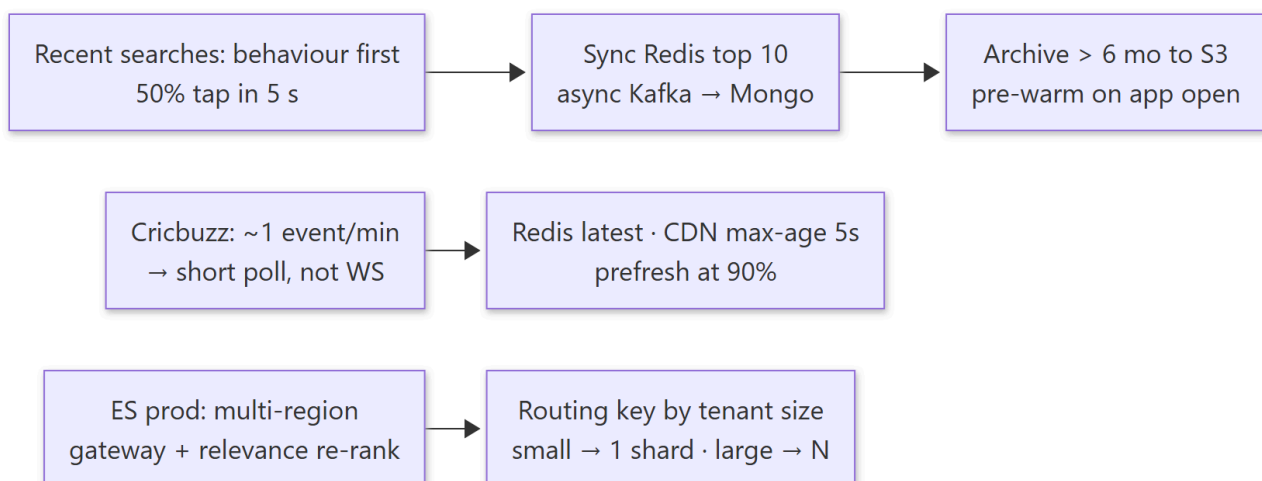
Recall: What did the production search revamp add over Day 0?

Separate indexers writing to a primary and a backup-region ES; a search gateway with identity (JWT) and relevance re-ranking; multi-region failover and latency-based routing; routing keys for tenant isolation.

What these notes add beyond the slides

- **Redis recipe for "recent 10"**: `LREM key 0 q (dedupe) → LPUSH key q → LTRIM key 0 9`, in a `MULTI` or Lua script so concurrent searches don't interleave.
- **Arithmetic checked**: $10 \times 16 \text{ B} \times 1\text{M} = 160 \text{ MB}$ ✓ (that's raw bytes; Redis per-key and list overhead easily doubles it).
- **CDN prefetch** (refresh at ~90% of TTL) is "stale-while-revalidate" in standard HTTP terms: `Cache-Control: max-age=5, stale-while-revalidate=...`.
- **ES routing caveat**: with a custom routing key, queries must pass the same routing to stay on one shard. Otherwise ES fans the query out to all shards and the isolation benefit is lost.

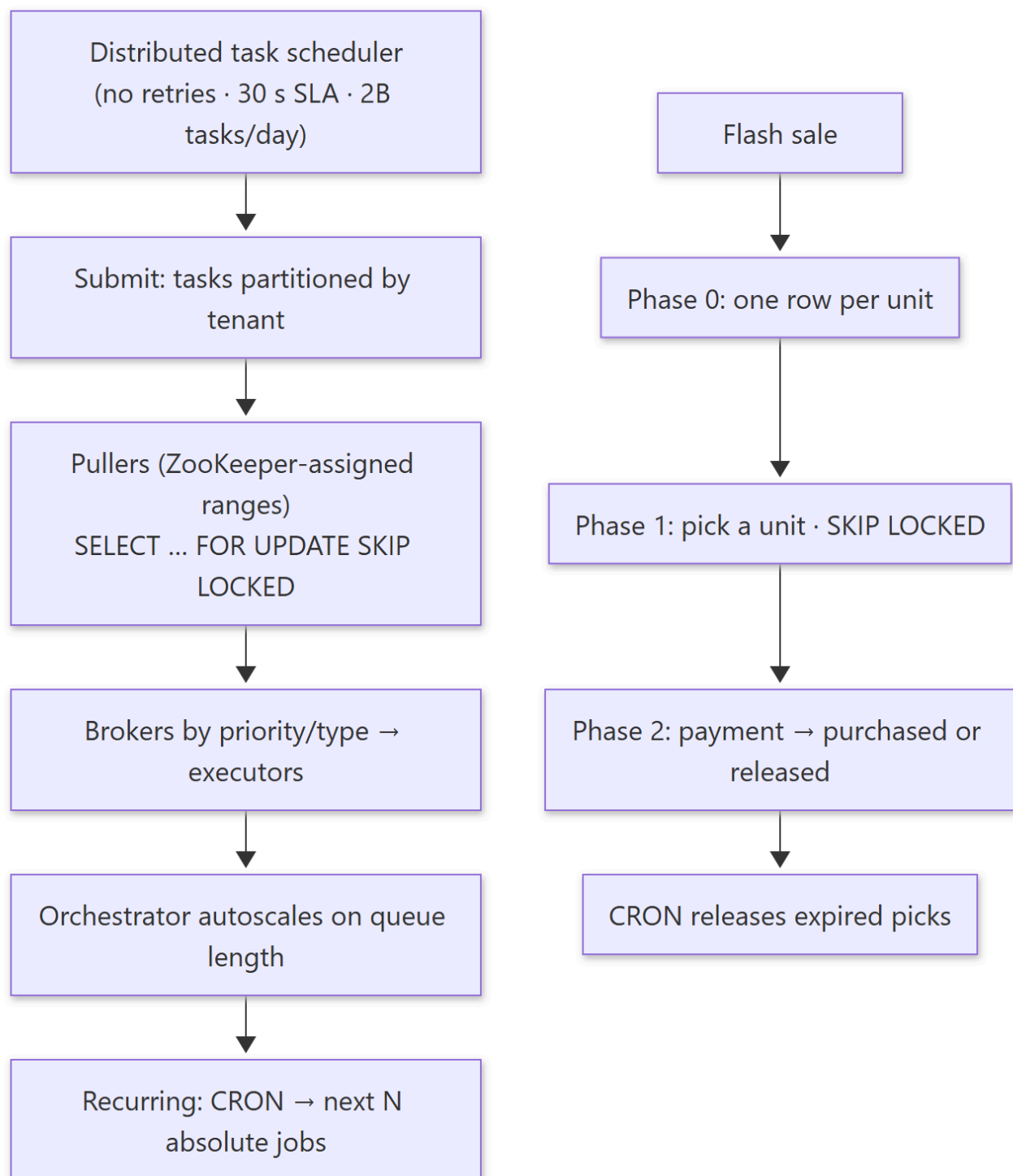
One-page memory card



07-02 · Arpit Bhayani's session notes: Distributed task scheduler + Flash sale

Source: 07-02.pdf (System Design Masterclass, Google Drive, view-only). Study notes in my own words, not a copy. Pages covered: 1–17 of 17. Unreadable pages: none. Pages 4–6 are whiteboard drafts; the clean pages 7–11 cover the same design and the notes follow those.

Map of the session



Agenda: **Distributed task scheduler · Flash sale.**

1. Distributed task scheduler (without retries)

📌 Anchor: Like **Dkron / AWS CloudWatch Events**: run a task **at a fixed time** or **on a CRON**, within a **30-second SLA**, for many tenants (~**2B tasks/day**). Executing a task means **making an HTTP call**; the target service does the real work.

Requirements:

- Schedule a task to run at a certain time (fixed) or as a recurring task (cron).
- **30 s SLA**: a task scheduled at 10:01:00 must start before 10:01:30.
- Minute-level granularity. Multi-tenant (2B tasks/day; each tenant is a team in the org).
- Extreme scale + SLA → multi-tenancy, replicable infra, good unit economics.

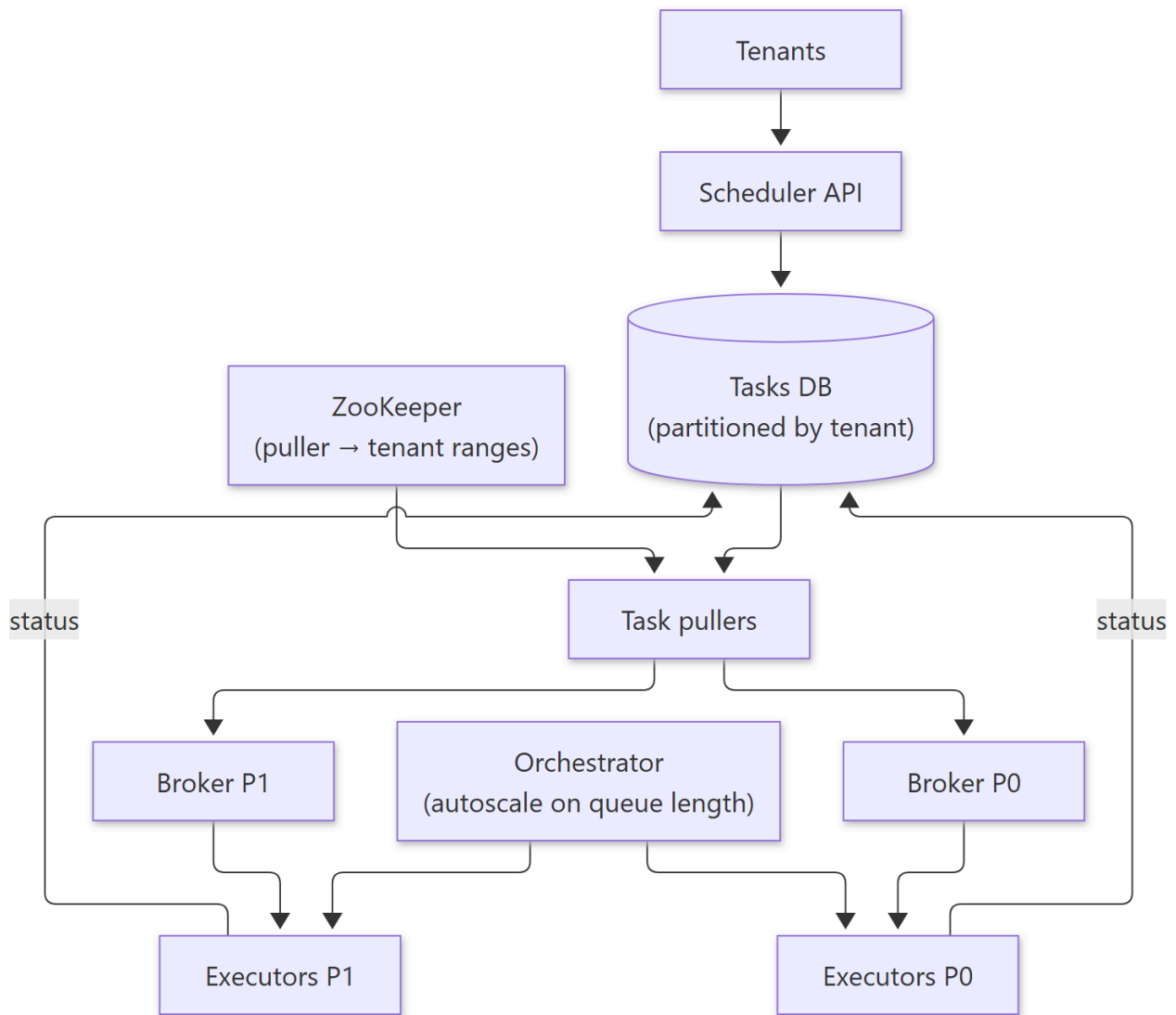
Brainstorm: store, pick, execute, task routing, multi-tenancy. A strict SLA means the system must be **horizontally scalable and fault tolerant**, and every component you add must respect that.

1.1 Submitting a task

- A task is defined by a **command**: a self-contained script or HTTP call that has everything it needs to run.
- Multi-tenant: many microservices and consumers use it. At ~2B tasks/day one DB node isn't enough, so **partition tasks by tenant** (customer/microservice).

1.2 Execution

- A set of **executor nodes** runs tasks, segregated by **priority** (P0 vs P1) and **type** (memory-heavy vs CPU-heavy).
- Executors report **status** back to the DB. An **orchestrator autoscales** executors based on **queue length** at the brokers and on predicted scale.
- From storage to execution: **pick tasks as fast as possible**, then send them for execution as fast as possible.



1.3 Task pullers

🔗 Anchor: Pullers **pull due tasks and enqueue them**. Each owns a **range of tenants** (kept in ZooKeeper), so they never compete, and a dead puller's range gets reassigned.

- They must be **horizontally scalable and fast**.
- On boot, a puller asks ZooKeeper for a range of services/customers, queries the DB for those tenants' due tasks, and enqueues them.
- **Fault tolerant**: if a puller dies, its range is assigned to another puller.

1.4 Pulling fast and safely (plain MySQL)

```
-- jobs(id, job, scheduled_at, picked_at, started_at, ended_at)
SELECT * FROM jobs
WHERE scheduled_at <= NOW() + 5s      -- slight look-ahead
  AND picked_at IS NULL
ORDER BY scheduled_at ASC
LIMIT 10
```

```
FOR UPDATE SKIP LOCKED;  
-- then UPDATE jobs SET picked_at = NOW() WHERE id IN (...) (same txn) and enqueue
```

- `SKIP LOCKED` lets several pullers grab **different** rows concurrently without blocking each other (the same trick as airline seats in 02-01).
- **Measure the SLA in parts:** `picked_at - scheduled_at` (pull lag) and `started_at - picked_at` (queue lag). The total must stay under 30 s.
- Properties: high availability, fault tolerance, true horizontal scalability, single-node efficiency.

1.5 Recurring tasks

 Anchor: **CRON** → **absolute** → **regular flow**. Expand a cron schedule into its **next N concrete runs** and insert them as normal jobs.

tasks	jobs
id, task, schedule (cron), scheduled_at	id, task_id, scheduled_at, picked_at, started_at/ended_at

- When a task is created: compute its **next 10 executions** and add them to `jobs`.
- When a job is picked: compute the next one(s) and add them (e.g. 10:00 → 10:01 ... 10:05).
- Insert with `INSERT ... ON CONFLICT DO NOTHING` so re-computing never creates duplicates.

Exercises (very similar to a task scheduler): build SQS on top of MySQL · build Kafka on top of MySQL · build any primitive broker using Bitcask as its storage engine.

 **Recall: How do many pullers fetch due tasks without stepping on each other?**

Each puller owns a disjoint tenant range from ZooKeeper, and within that it uses `SELECT ... WHERE scheduled_at <= now AND picked_at IS NULL ORDER BY scheduled_at LIMIT n FOR UPDATE SKIP LOCKED`, then marks the rows picked in the same transaction.

 **Recall: How are cron tasks executed in this design?**

The cron expression is expanded into the next N absolute run times, inserted as ordinary jobs (idempotently). Each pick schedules more, so they flow through the normal path.

 **Recall: What scales the executors?**

An orchestrator watching broker queue length (and predicted load), autoscaling executor fleets per priority or type.

2. Flash sale

 Anchor: **Fixed inventory + contention**. Model it like a real store: **one row per physical unit**, and let people "pick it off the shelf" with `SKIP LOCKED`.

Setup: a fixed set of items and units (e.g. **Mi 8 × 10,000 units**); many people come in to buy in a short window. Brainstorm: breakdown and empathy. (Pre-read: Shopify's flash-sale talk.)

2.1 The naive approach

- `product(id, name, qty)` and `UPDATE product SET qty = qty - 1 WHERE id = ?` inside a transaction with `SELECT ... FOR UPDATE`.
- **Every buyer locks the same row**, so everything serialises on one hot row.

2.2 Phase 0: prepare the stock

units: id	item_id	picked_at	picked_by	purchased_by
1 ... 10000	720	NULL	NULL	NULL

Selling 10,000 Mi 8 phones (item 720) → **10,000 rows** in `units`.

2.3 Phase 1: let them pick it (add to cart)

 Anchor: High throughput + contention → **only N should succeed**. `FOR UPDATE SKIP LOCKED LIMIT 1` gives each buyer a *different* free unit, non-blocking.

```
BEGIN;
SELECT * FROM units
WHERE item_id = 720 AND picked_at IS NULL
ORDER BY id LIMIT 1
FOR UPDATE SKIP LOCKED;
UPDATE units SET picked_at = NOW(), picked_by = 1023 WHERE id = ?;
COMMIT; -- then add to cart
```

- Both queries run in **one transaction**. `SKIP LOCKED` ensures non-sequential, non-blocking execution, and `FOR UPDATE` makes the unit unavailable to others.

2.4 Phase 2: payment

- **Success:** set `purchased_by = ?`, `purchased_at = NOW()` (check that `picked_at` is still yours), create the order, etc.
- **Failure:** make the unit available again: `picked_at = NULL`, `picked_by = NULL`.
- ⚠ **You can't have a distributed transaction spanning add-to-cart and payment.** They're separate steps with compensation.

2.5 Abandoned carts

- A **CRON job** releases "expired" picks: `UPDATE units SET picked_at = NULL, picked_by = NULL WHERE picked_at < NOW() - 12 min AND purchased_by IS NULL;`
- Optionally keep some **extra inventory** as a buffer.

Similar systems (fixed inventory + contention): any ticket booking site, BookMyShow, IRCTC, hotel booking.

Recall: Why one row per unit instead of a qty column?

A qty column makes every buyer lock the same row (serialised). Per-unit rows let SKIP LOCKED hand each concurrent buyer a different free row, so contention spreads across 10,000 rows.

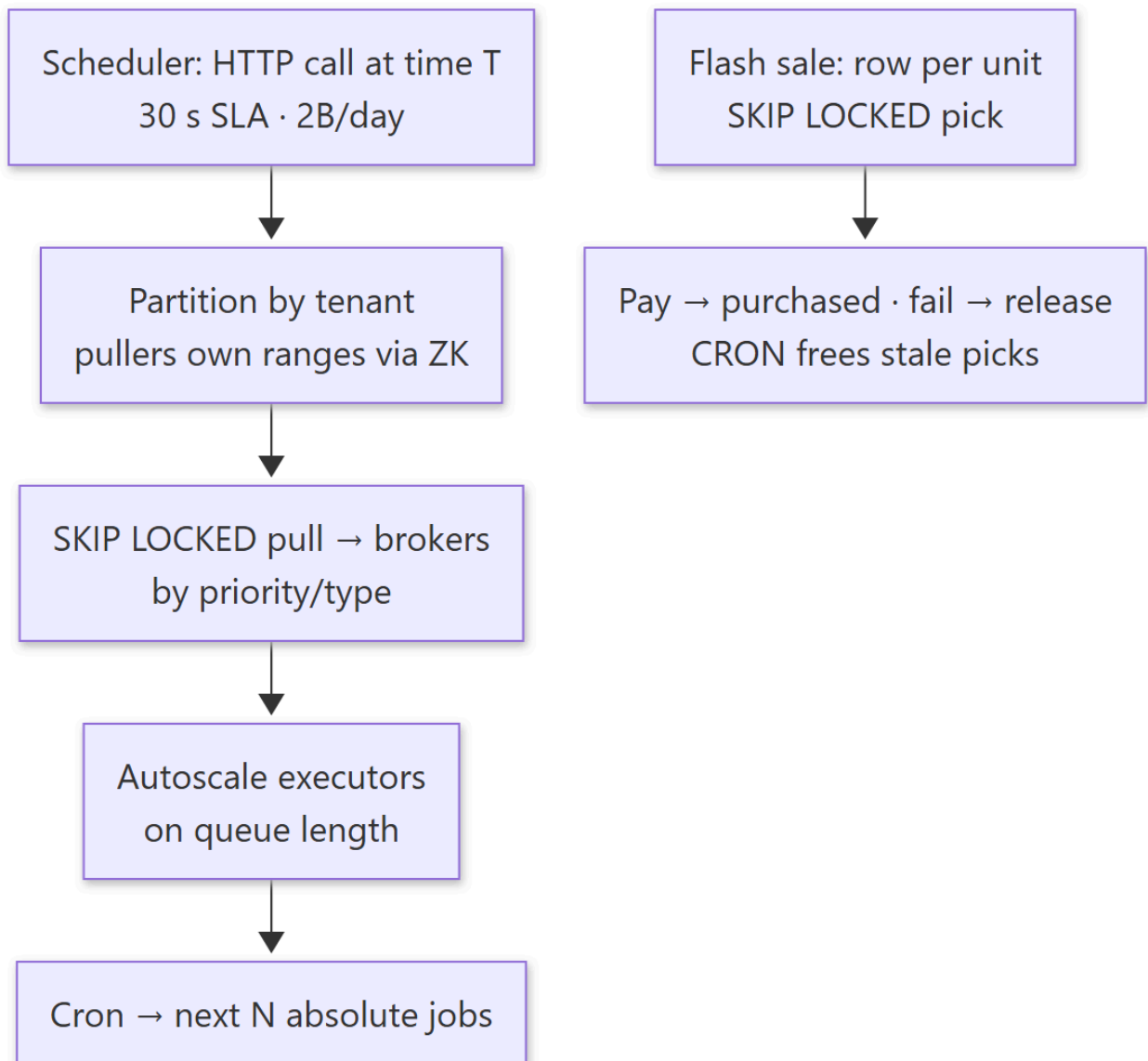
Recall: What happens to units picked but never paid for?

A periodic job clears picked_at and picked_by for units picked more than ~12 minutes ago and not purchased, returning them to stock.

What these notes add beyond the slides

- **"Without retries" is a deliberate scope cut.** Real schedulers need at-least-once execution plus idempotency on the target (the HTTP endpoint gets an idempotency key), retries with backoff, and a DLQ.
- **Look-ahead window:** `scheduled_at <= NOW() + 5s` lets pullers enqueue a few seconds early, absorbing pull lag within the 30 s SLA. Executors can sleep until the exact time if precision matters.
- **2B tasks/day $\approx$ 23k tasks/s on average** (peaks much higher, e.g. on-the-hour crons), which is why one DB node isn't enough.
- **Flash sale at extreme scale:** even per-unit rows can strain one DB. A common next step is a **pre-loaded token queue** (e.g. a Redis list of unit ids: `LPOP = pick`), with the DB updated asynchronously.

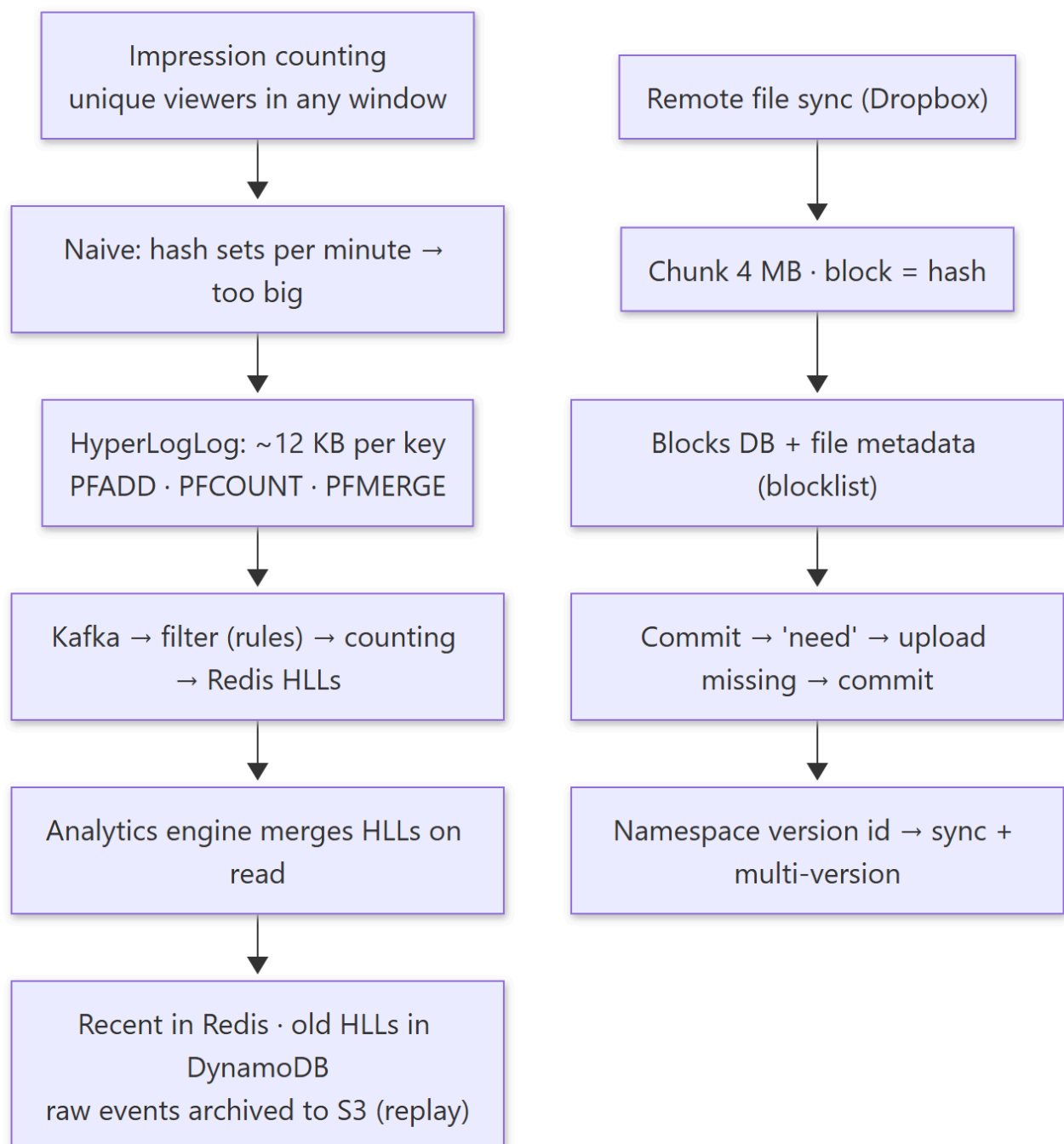
One-page memory card



08-01 · Arpit Bhayani's session notes: Impression counting (HyperLogLog) + Remote file sync (Dropbox)

Source: 08-01.pdf (System Design Masterclass, Google Drive, view-only). Study notes in my own words, not a copy. Pages covered: 1–22 of 22. Unreadable pages: none. Pages 5 and 10 are whiteboard drafts; the clean pages around them cover the same design.

Map of the session



1. Impression counting

🔗 Anchor: "Total **unique** visitors in the last n time units", with n chosen **at runtime**. It powers views on LinkedIn posts, YouTube videos, Google search results, Reddit posts, AdSense ads, Instagram photos, TikTok videos.

- Problem: not everyone reacts, so how do you measure engagement? Ad-tech firms want to render a graph of how a campaign performed.
- **Requirements:** realtime or near-realtime · no fixed aggregates (n is chosen at runtime) · each user counted once per time window · the count can be a close **approximation** · some rules to filter out unwanted events.

1.1 What we must answer

- "Count **distinct** users on a post (ad) from 04/01/2022 to 08/01/2022 10:00" and render the dashboard.
- Example stream: 10:00 A, 10:00 B, 10:01 C, 10:01 B, 10:02 B, 10:02 A → **3** unique (A, B, C) in 10:00–10:02. Then 10:03–10:05 has {A, B, D, E} = **4**. 10:00–10:05 = {A, B, C, D, E} = **5**. Windows **overlap**, so you can't just add the counts.

1.2 Naive: a hash set per time bucket

- `p1:20220401_1200 = {a, b, c, d, e, f}`, `..._1300 = {a, c, z, w, x}`, `..._1400 = {c, d, f, l, z}`. Total unique = **set union** = 10.
- Taxing on CPU and memory, and slow to compute. **Granularity is critical**.
- **At scale:** `user_id` $\approx$ 4 B, and 1M people view an ad in one minute → **4 MB**. Unique visitors in the last hour → $60 \times 4 \text{ MB} = \mathbf{240 \text{ MB}}$ of processing for one ad. Thousands of customers each checking their campaigns → $5000 \times 240 \text{ MB} = \mathbf{1,200 \text{ GB}}$. Not sustainable.

1.3 Approximate it: cardinality estimation with HyperLogLog

🔗 Anchor: Trade a little accuracy for a huge saving in space. An **HLL** estimates the cardinality of a set (and of unions) in about **12 KB**, versus **4 MB** for the exact set ($\sim 0.3\%$).

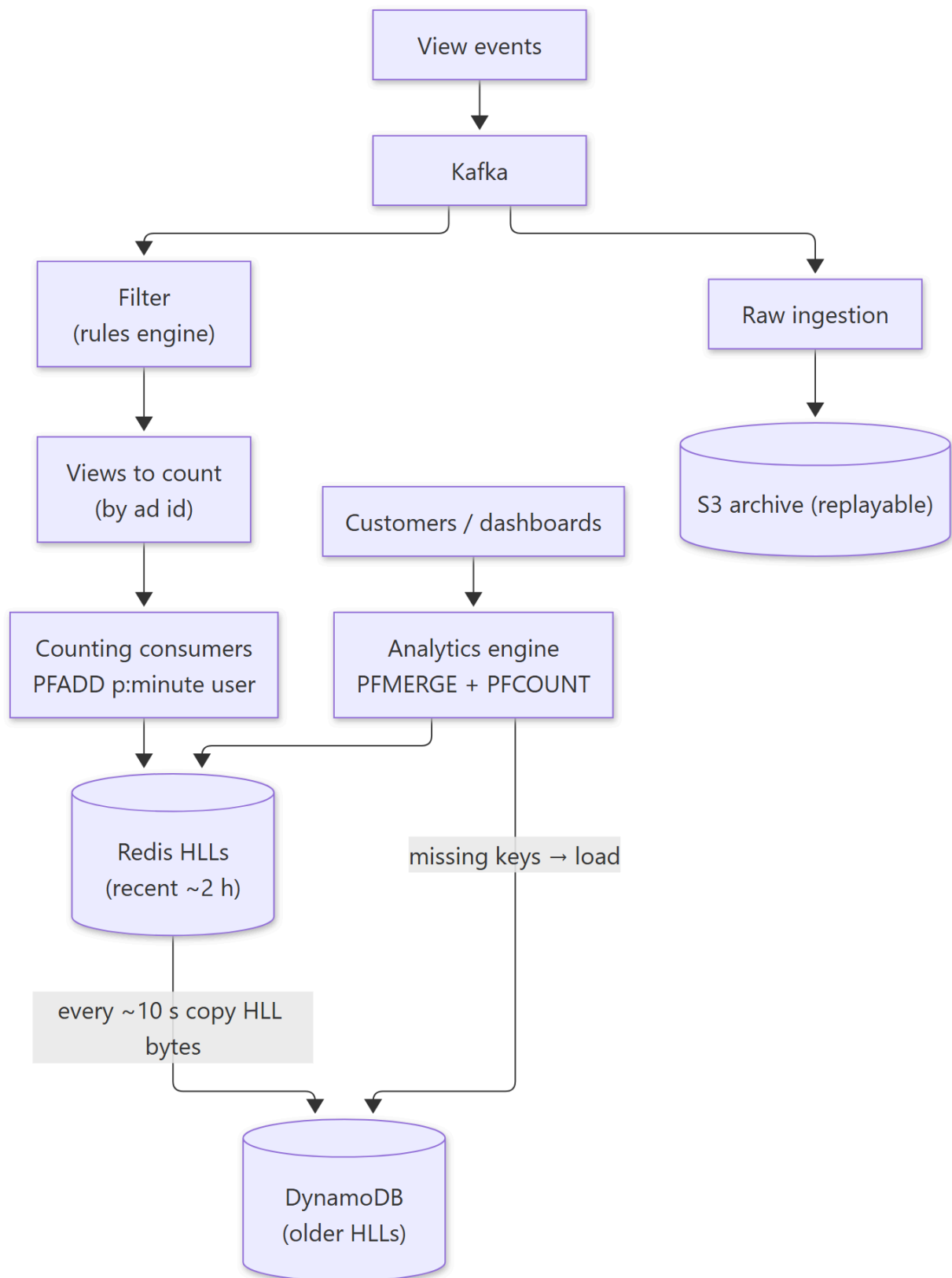
- Space vs time vs accuracy (and money). This is the **cardinality estimation problem**: efficiently approximate the cardinality of the set you get after n unions.
- HyperLogLog builds on the **Flajolet–Martin** algorithm (the session points to Arpit's blog post on it). Here it's used as a black box.
- **Redis gives you HLL out of the box:**

Operation	Redis command	Cost
Add an element	<code>PFADD key elem</code>	$O(1)$
Count (cardinality)	<code>PFCOUNT key</code>	$O(1)$

Operation	Redis command	Cost
Merge N HLLs	PFMERGE dest k1 k2 ...	O(N)
Raw bytes (to persist or move)	GET key / SET key bytes	

- ⚠ **You can't delete from an HLL** (like a Bloom filter, it's lossy).
- Keys like `p1:20220401_1200` = an HLL of the unique visitors to post p1 in that minute.
- **Total unique for p1 over a range:** `PFMERGE temp p1:...1200 p1:...1300 ...` → `PFCOUNT temp`.

1.4 Architecture



- **Filter rules** (realistic): 0 views if a user watches their own video; a minimum watch time (e.g. 5 s); many views from one user within a minute count as 1.
- **Write path:** counting consumers read from Kafka and `PFADD post:minute user_id` into the right HLL. (Events always move forward in time.)

- **Read path:** a separate analytics engine reads the relevant HLLs from Redis, merges them, computes the cardinality (`PFMERGE` + `PFCOUNT`) and responds.

1.5 Does it work at scale?

 Anchor: YouTube has millions of videos and billions of events, so you **can't keep every HLL in Redis**. Keep the **last 30 min to 2 h** in Redis; **periodically copy HLL bytes to a cheap KV store** (DynamoDB) and **load them back on demand**.

1. HLLs are always updated in Redis (only Redis knows HLL semantics).
 2. Every ~10 s, HLLs are copied from Redis to DynamoDB (`GET p1:...1200` → raw bytes).
 3. If a key isn't in Redis, it's brought in from DynamoDB.
 4. For an analytics query, all relevant HLLs are brought into Redis, then the cardinality is computed.
- Data is immutable once a minute has passed, so it's safe to cache and copy. You can also pre-aggregate hour or day HLLs to optimise long-range queries.

1.6 Robustness: replayability and cost

- **What if the rule engine had a bug and you need to replay events?** Archive raw events (ETL) to **S3** (storage-optimised, durable) so they can be replayed later. Filtered events also go to S3.
- A temp database + aggregator handle hot data; a customer console (B2B) reads through the read path.

 Recall: Why can't you sum per-minute unique counts to get the hourly unique count?

The same user appears in several minutes; summing double-counts them. You need a union. HLLs support merge (`PFMERGE`), so you merge the per-minute HLLs, then count.

 Recall: HLL trade-offs?

About 12 KB per key regardless of cardinality, $O(1)$ add and count, mergeable; but approximate (~0.81% standard error in Redis) and no deletes.

 Recall: How does the design keep Redis costs bounded?

Only the recent window lives in Redis. HLL bytes are periodically copied to DynamoDB and reloaded on demand for older ranges; raw events are archived to S3 for replay.

2. Remote file sync (Dropbox)

 Anchor: Files uploaded from one device must be **uploaded efficiently** and **synced efficiently** to the others. Two challenges: **resumable uploads/downloads**, and how clients **learn about changes**.

Brainstorm: resumable uploads and downloads · the core intuition is identifying *what changed* · multi-versioning.

2.1 Chunking makes it better

🧠 Anchor: Split a file into **4 MB blocks**; each block's **hash** is its identifier. A file = its **blocklist** [h1, h2, h3, h4] .

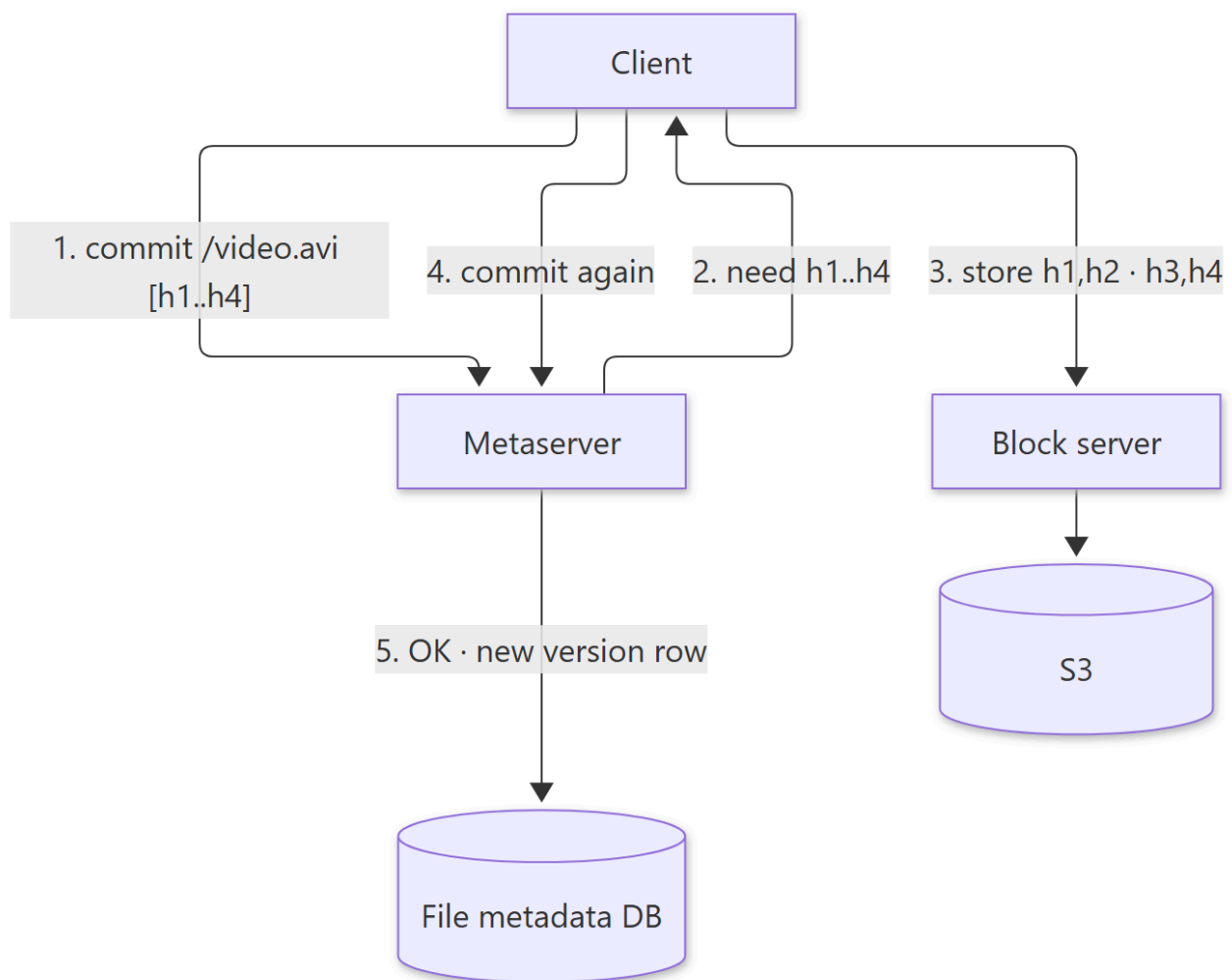
- A 14 MB `video.avi` → blocks of 4 + 4 + 4 + 2 MB → `h1...h4` . That gives **parallel and resumable** uploads.
- How to store a file on S3? (1) each chunk as a new object, or (2) S3 multipart upload. Store blocks content-addressed: `s3://my-dropbox/<account-id>/<block-hash>` . Identical blocks are stored once (dedupe).

2.2 Metadata

Blocks DB	File metadata DB
namespace (account) id · block hash · actual block (S3 path)	namespace id · relative path (<code>/video.avi</code>) · blocklist (<code>h1, h2, h3, h4</code>) · version id (monotonically increasing per account)

- The blocks table answers "does this account already have block h?". The version id turns out to be crucial (see 2.4).

2.3 Upload protocol



1. The client asks the metaserver to commit `/video.avi` with hashes `h1...h4`.
2. The metaserver checks the blocks DB: **none exist**, so it replies "need h1–h4".
3. The client uploads blocks to the **block server**, two at a time, waiting for acks until all are uploaded.
4. The client **re-commits**; this time it succeeds and a new row is added to the file metadata DB.

A file changed? A few bytes changed inside block 3: the new blocklist is `h1, h2, h3', h4`. Commit → the metaserver says "need h3'" → upload **one 4 MB block** → re-commit → version 2.

2.4 How clients learn what changed: the version id

Anchor: **Sequentialise** updates per namespace. Every change increments the namespace's **version id**. A client says "I'm at version 2, what's new?", **just like a Kafka offset**.

version	namespace	path	blocklist
1	arpit	/video.avi	h1 h2 h3 h4
2	arpit	/video.avi	h1 h2 h3' h4
3	arpit	/photo.jpg	h6

version	namespace	path	blocklist
4	arpit	/notes.txt	h7 h8
5	arpit	/photo.jpg	h6'

- It's a unified way of notifying all clients: any update is just the next row in the table. The metaserver forwards everything after the client's cursor.

2.5 Multi-versioning for free

- Old versions are just old rows, so you can reconstruct any version: `v1 of video.avi = h1, h2, h3, h4`. `h3` is still present on the block server, so the file can be rebuilt easily.
- **Similar systems:** S3 versioning; Google Drive and multi-versioning.

Recall: How does Dropbox upload only what changed?

Files are split into 4 MB blocks identified by hash. On commit, the metaserver compares the blocklist with the blocks it already has and asks only for the missing hashes; the client uploads those, then re-commits.

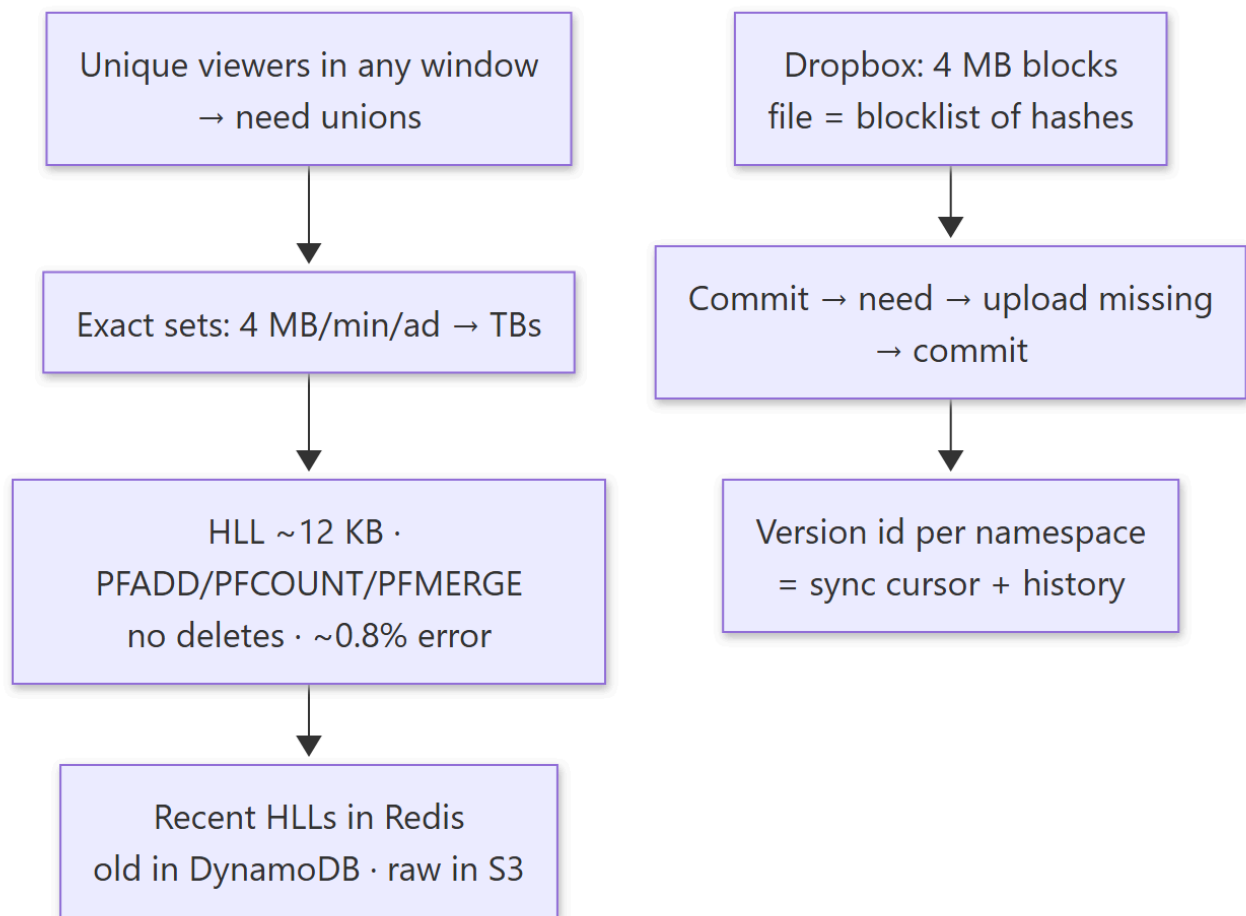
Recall: How do other devices discover changes?

Every commit appends a row with an incrementing per-namespace version id. A device stores the last version it applied and asks for everything after it, like a Kafka consumer offset.

What these notes add beyond the slides

- **Arithmetic check:** $12\text{ KB} / 4\text{ MB} \approx 0.3\%$. The slide's "0.15% of space" doesn't follow from those numbers (0.15% would be ~6 KB). Redis dense HLLs are 12 KB (sparse encoding is smaller for low cardinalities).
- **HLL error:** Redis HLL has a **0.81% standard error**, which is fine for "views", not for billing.
- **Content-defined chunking:** fixed 4 MB blocks mean inserting one byte near the start shifts every block (all hashes change). Rolling-hash chunking (as in rsync/LBFS) keeps boundaries stable.
- **Block hashes per account:** Dropbox scopes dedupe per namespace; global cross-user dedupe can leak information (a hash collision proves someone has the file).

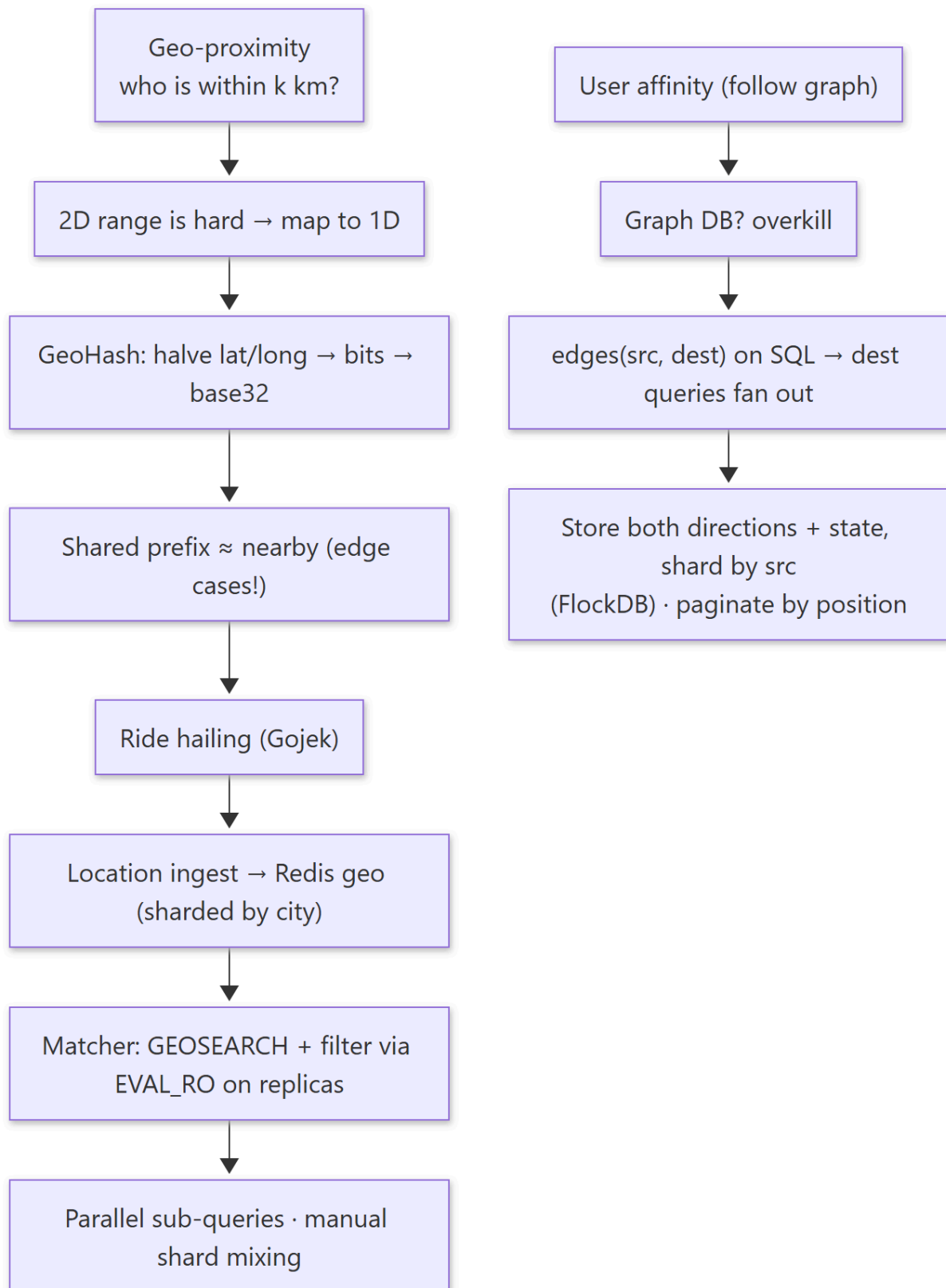
One-page memory card



08-02 · Arpit Bhayani's session notes: Geo-proximity & ride hailing (Uber/Gojek) + User affinity (follow graph)

Source: 08-02.pdf (System Design Masterclass, Google Drive, view-only). Study notes in my own words, not a copy. Pages covered: 1–24 of 24. Unreadable pages: none. Pages 4 and 10 are whiteboard drafts; the clean pages after them cover the same ideas.

Map of the session



Agenda: **Geo-proximity & ride-hailing service (Uber)** · **User affinity service.**

1. Geo-proximity

 Anchor: It's what makes Tinder, Ola, Uber, Swiggy, Zomato, Google Stores, Facebook nearby friends, Yulu and Zoomcar possible: **efficiently finding who or what is near you.**

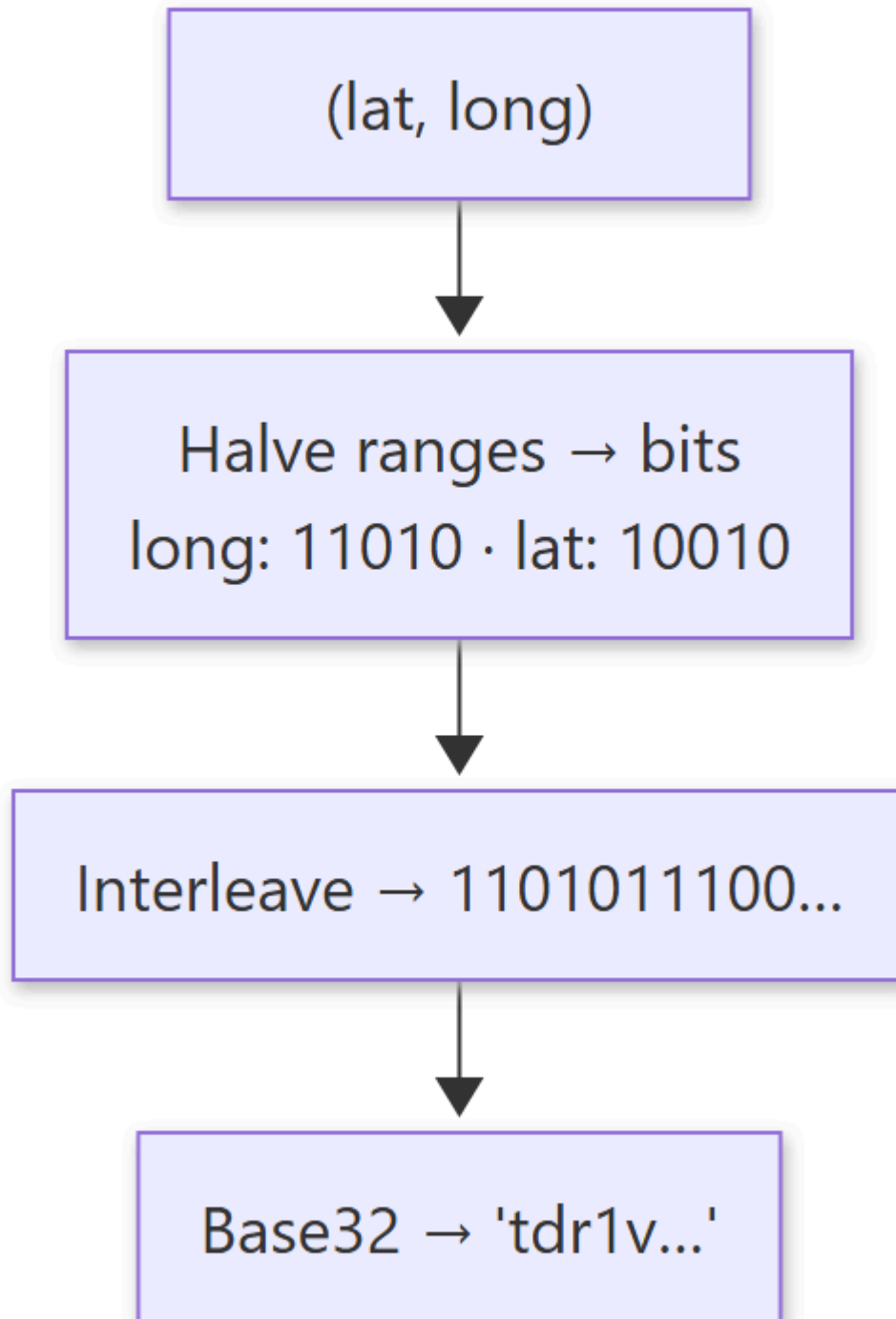
Problem statement: find the people within k km radius of you.

1.1 Why is it so hard?

- Naive: for every point p , check $d(p, a) \leq k$, which is **$O(n)$** per query.
- Bounding box: $x_a - k < x_b < x_a + k$ and $y_a - k < y_b < y_a + k$. With points sorted by x and y you can binary-search each axis, but you're intersecting **two** range results. In-memory and multi-dimensional, that gets complicated.
- **One dimension is easy:** segment trees or a sorted index answer range queries efficiently.
- So **map $(x, y) \rightarrow z$** , a single number, so that nearby points get nearby z . Space-filling curves (**Z-order**, **Hilbert curve**) do this. Interleave the bits: $A = (10011)$, $C = (10010) \rightarrow$ close values.

1.2 GeoHash

 Anchor: **Divide and conquer.** Repeatedly halve the longitude range (left = 0, right = 1) and the latitude range (bottom = 0, top = 1), interleave the bits, and encode in **base 32**.



- Worked example (Bengaluru, lat 12.9783, long 77.5999): long range $[-180, 180]$: mid 0 $\rightarrow 77.6 > 0 \rightarrow 1 \rightarrow [0, 180]$; mid 90 $\rightarrow < \rightarrow 0 \rightarrow [0, 90]$; mid 45 $\rightarrow > \rightarrow 1$; mid 67.5 $\rightarrow 1$; mid 78.75 $\rightarrow < \rightarrow 0 \dots$ Do the same for latitude, interleave, then base32.
- **Zoom in = more bits on the right; zoom out = remove bits from the right.** A longer prefix means a smaller cell.
- 32-bit numbers aren't human-readable, so GeoHash uses **base 32**, splitting each level into **32 blocks** (a 4×8 grid of characters).
- **Closer points → longer common prefix.** `gzzksb` and `gzzksx` are close. The data structure for prefix matching is a **trie**.

- **Edge case:** some points that are physically near can have very different GeoHashes (on either side of a cell boundary). Also query the **8 neighbouring cells**.

The algorithm is so simple it's one SQL query:

```
-- find all people in (roughly) Bangalore
SELECT * FROM people WHERE SUBSTR(geohash, 1, 5) LIKE 'tdr1v%';
```

You don't need to build GeoHash from scratch: geospatial databases (Redis, Elasticsearch, MongoDB) do it for you. Insert or update (lat, long), then ask for the points within 5 km of (lat, long). (Alex Xu has a good video on a proximity service.)

Recall: Why convert 2D coordinates into a 1D value?

1D range queries are cheap (sorted index, B-tree, prefix). Space-filling curves and GeoHash map nearby 2D points to nearby 1D values, so "near me" becomes a prefix or range lookup.

Recall: GeoHash's main gotcha?

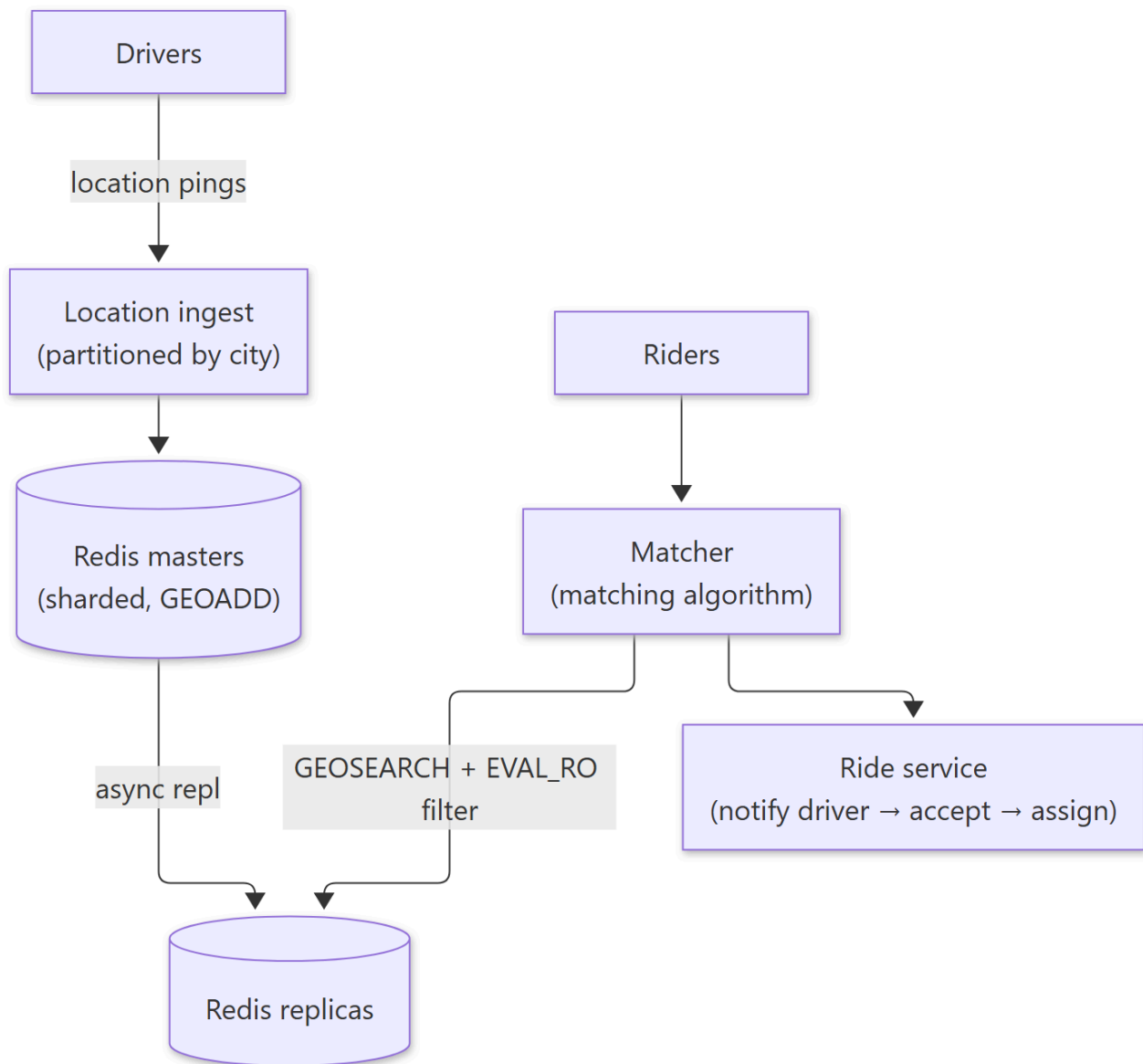
Points just across a cell boundary can have very different prefixes. Always search the 8 neighbouring cells as well as your own.

2. Ride hailing (Gojek / Uber)

 Anchor: **Availability and low latency over consistency.** A driver's location being slightly stale is fine, since locations don't change drastically in a second.

Brainstorm (high-level design): users, riders and drivers · location ingestion (write-heavy) · matching (read-heavy) · the ride service (rating, payment method, area, type of vehicle).

2.1 Architecture



- **Ride flow:** find the closest drivers → notify one for acceptance → if yes, assign.
- **Key design decision: Redis to store and query location.** (1) In-memory, so fast. (2) Supports geo queries (`GEOADD` , `GEOSEARCH`). (3) **Multi-master, multi-replica:** data is sharded and stored on exclusive masters, each with its own replicas.

2.2 Challenge: filtering drivers with EVAL on replicas

- Filtering drivers by constraints (vehicle type, rating, ...) happens through a Lua **EVAL** command. **Redis doesn't run EVAL on replicas**, because a script could contain commands that update data. Fire it on a replica and it replies `MOVED <master ip>`.
- Gojek wanted EVAL on replicas for filtering, so they: (1) raised an issue with the Redis team and proposed an `EVAL_RO` (read-only) command; (2) updated **go-redis** so it can run on replicas. (See the commits, the issues, and their engineering blog.)

2.3 Availability with low latency

- Instead of one big `GEOSEARCH` on a single node or replica, **split the query into smaller queries** (sub-regions) and **fire them in parallel** on multiple nodes:
 - If one node is slow, the others still respond.
 - Parallel execution makes the computation quicker.
 - **Send a partial response if the SLA is breached.**

2.4 Hot shards

- Splitting queries into smaller regions helps with hot shards to some extent. The key step is to **shard the data so that peak-load regions and low-traffic regions are mixed** across masters.
- That allocation was done **manually** (by Gojek's ADS service), because native Redis sharding has no business context.
- **Impact:** a truly horizontally scalable system. To handle more load, just add more nodes. Highly available, low latency.

Recall: Why did Gojek need EVAL_RO?

Driver filtering runs as a Lua script via EVAL, which Redis refuses on replicas (it could write). EVAL_RO marks a script read-only so it can run on replicas and spread the read-heavy matching load.

Recall: How does the matcher stay fast when a node is slow?

It splits the search area into smaller GEOSEARCH queries, runs them in parallel across nodes, and returns partial results if the SLA would be breached.

3. User affinity (follow / following)

 Anchor: Potentially $n \times n$ edges (everyone follows everyone). A graph DB seems obvious, but it's **overkill**: no sophisticated graph algorithms, painful to manage, expensive, and **pagination is tricky**.

Requirements:

- Use a **relational DB**.
- Pagination must be **extremely efficient**: no matter how deep you scroll, the performance should be the same.
- Follower and following counts must be accurate and quick.
- Very quickly answer "does A follow B?"
- Very fast writes, at massive scale.

3.1 First cut: an edges table

src	dest
A	B

src	dest
B	C
C	B
C	A

- A follows B → insert (src=A, dest=B).
- People who follow B: `SELECT * FROM edges WHERE dest = B`. People B follows: `SELECT * FROM edges WHERE src = B`.
- **Challenges:** both queries need indexes on **both columns** (expensive), and how do you grow beyond one DB node?

3.2 Shard by src, and the problem

- 4 shards, `src` is the partition key. "People B follows" → one shard . "Followers of B" (`dest = B`) → **fire the query across all shards, merge, return** .

3.3 Fix: store both directions, keep everything on src

 Anchor: For each follow, write **two rows**: (A, B, FOLLOWS) and (B, A, FOLLOWED_BY). Shard by `src`, and **both** queries hit one shard.

src	dest	state
A	B	FOLLOWS
B	A	FOLLOWED_BY

```
-- followers of B
SELECT * FROM edges WHERE src = B AND state = 'FOLLOWED_BY';
-- people B follows
SELECT * FROM edges WHERE src = B AND state = 'FOLLOWS';
```

- No index on `dest` is needed any more, and the dataset shards cleanly on `src`.

3.4 Twitter's FlockDB

 Anchor: A **graph DB on a relational DB**. Edges: `source_id`, `destination_id`, `position`, `state`. **PK (source_id, state, position)**: every query and pagination is served from that one primary index.

Column	Type	Meaning
<code>source_id</code>	int64	Node the edge starts from
<code>destination_id</code>	int64	Node it points to
<code>position</code>	int64	Cursor and sort key (e.g. a timestamp for the following graph)
<code>state</code>	int8	Positive, negative, archived, ... (when an edge is deleted, change its state, which makes restoration possible)

- **Primary key:** `(source_id, state, position)` → equality on source and state, range and sort on position.
- **Unique index:** `(source_id, destination_id, state)`, which answers "does A follow B?" quickly.
- Data is partitioned by node, so no query requires cross-partition execution.
- **Pagination by position, not LIMIT/OFFSET**, so any page is equally fast:

```
SELECT * FROM edges
WHERE src = B AND state = 'FB' AND position > :last_seen
ORDER BY position ASC LIMIT 2;          -- O(log n + k)
```

Recall: Why write two rows per follow?

So both "who follows B" and "whom B follows" are lookups on `src = B`, on one shard, with no dest index and no scatter-gather.

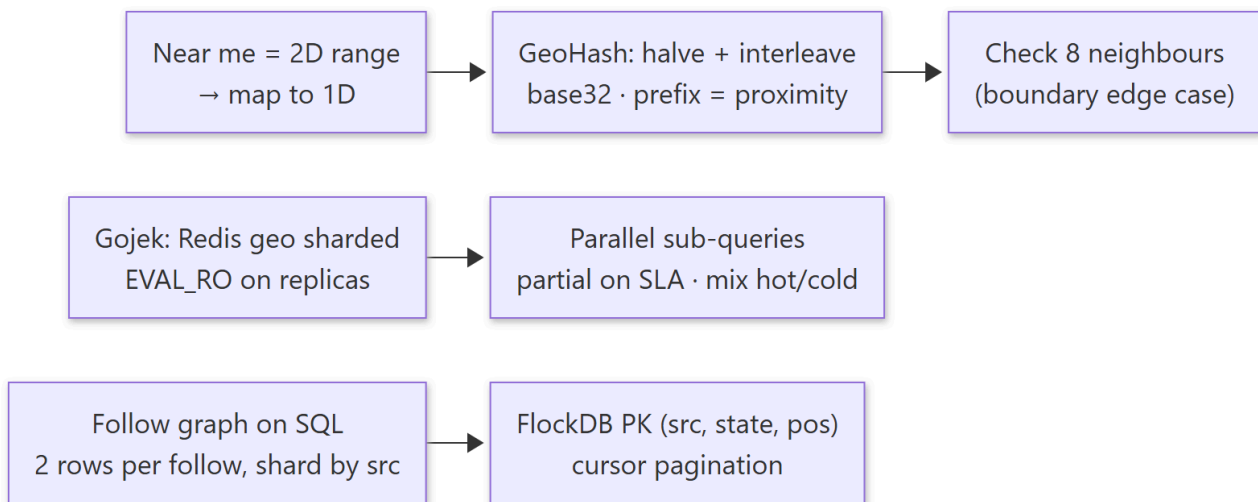
Recall: FlockDB's primary key and why?

`(source_id, state, position)`: equality on source and state picks one user's edge list of one kind, and position gives ordering and cursor pagination straight off the clustered index.

What these notes add beyond the slides

- **GeoHash precision:** 5 characters $\approx 4.9 \text{ km} \times 4.9 \text{ km}$, 6 characters $\approx 1.2 \text{ km} \times 0.6 \text{ km}$. Pick the prefix length from the search radius, then add the 8 neighbours.
- **Redis GEO is GeoHash inside:** `GEOADD` stores a 52-bit interleaved GeoHash as the score in a sorted set, and `GEOSEARCH` turns the radius into score ranges over the cell and its neighbours.
- **Two rows per follow need atomicity:** both rows live on *different* shards (`src = A` and `src = B`), so it isn't one local transaction. Use an outbox/async repair or idempotent retries; FlockDB applies writes idempotently and in order per edge.
- **Accurate follower counts:** keep a counter row per (user, state), updated alongside the edges, rather than `COUNT(*)` at read time.

One-page memory card



Flashcards (192)

Cover the answer column and answer out loud first.

01-01 · Foundational topics

#	Question	Answer
1	What is the "Framework of the Opposites"?	For every decision, consider the exact opposite (push vs pull, store vs don't store, build vs offload), evaluate all paths, then choose.
2	Why does a presence system use client heartbeats instead of the server polling clients?	Without a persistent connection the server cannot reach the client, so the client must push a periodic pulse.
3	What should the presence table store: a boolean or a timestamp?	The last heartbeat time (epoch). Online = now – last_hb is below a threshold (e.g. 30 s).
4	Storage for 1B users at 8 bytes per row?	About 8 GB (4 B user_id + 4 B last_hb).
5	How does "absence == offline" shrink the presence store?	Keep rows only for active users and expire them by TTL. 100k active × 8 B ≈ 800 KB instead of 8 GB.
6	Two ways to delete expired presence rows? Which is better?	A CRON job you run, or offloading to a datastore with native TTL (Redis/DynamoDB). Offloading is simpler.
7	Redis vs DynamoDB for presence: what decides?	Context: Redis is fast in-memory but costly and self-run; DynamoDB is durable, managed, multi-region, cheaper, with AWS lock-in.
8	Heartbeat every 10 s with 1M active users: DB write rate?	6M writes per minute (100k per second).
9	Why does connection pooling matter for tiny DB updates?	TCP setup/teardown per request dominates; reusing pre-established connections removes it.
10	What data structure implements a connection pool?	A bounded blocking queue (often a circular array) of connection objects; borrowers wait when it's empty.
11	Reasons for soft delete?	Recoverability, archival, audit, and no B+ tree rebalancing that a real DELETE can cause.
12	What is a cache stampede and the fix?	Many concurrent misses on one key all hit the DB. Request coalescing (single-flight): one request fetches, the rest wait for its result.

01-02 · Scaling, Delegation, Communication

#	Question	Answer
13	Vertical vs horizontal scaling: what does horizontal add?	Near-linear capacity increase and fault tolerance (vertical just makes one box bigger).
14	Why scale bottom-up?	Stateful layers (DB, cache) must handle the concurrent load from all API servers; scale them first.

#	Question	Answer
15	Three ways to scale a relational DB?	Vertical scaling, read replicas, sharding (plus a proxy to route).
16	What does a DB proxy like ProxySQL do?	Knows the topology; routes queries via rules (e.g. SELECT → replica hostgroup), pools connections, can cache.
17	The performance mantra for delegation?	What doesn't need to be done in realtime shouldn't be done in realtime.
18	Which tasks should be delegated to async workers?	Long-running tasks, heavy computation, batch writes, anything eventually consistent.
19	Message queue vs message stream?	Queue (SQS, RabbitMQ): each message goes to one consumer. Stream (Kafka, Kinesis): each consumer group reads every message.
20	Why use Kafka instead of one worker that counts and indexes?	Separation of concerns: separate consumer groups (counter, search) each process the same event.
21	Kafka's ordering guarantee?	Only within a partition; none across partitions.
22	Two Kafka partition limitations?	You can't reduce partitions; consumers in a group beyond the partition count sit idle.
23	Short polling vs long polling?	Short: server responds immediately, client repeats. Long: server holds the request until data is ready or timeout.
24	WebSockets vs SSE?	WebSockets: bidirectional persistent channel. SSE: server-to-client one-way stream over HTTP.
25	Typical load sequence for a realtime app?	Load UI → fetch initial state via REST → subscribe to live updates.

02-01 · Databases: locking, check-in, KV on SQL

#	Question	Answer
26	Pessimistic locking flow?	Acquire lock → read/update → release (on commit or rollback).
27	Why do databases need locks?	To protect data consistency and integrity under concurrent updates.
28	FOR UPDATE vs SKIP LOCKED vs NOWAIT on a locked row?	Wait · skip it and return other rows · fail immediately with an error.
29	Why use SKIP LOCKED for airline seat selection?	Users need any free seat; concurrent transactions each grab a different unlocked seat without queueing or double booking.
30	Name systems with "fixed inventory + contention".	Airline check-in, CoWIN slots, IRCTC, BookMyShow, flash sales.
31	How do you avoid an is_deleted column in the KV store?	Use a sentinel: expired_at = -1 marks a deleted row.

#	Question	Answer
32	How is PUT implemented in the KV store?	An upsert (MySQL REPLACE INTO / ON DUPLICATE KEY UPDATE; Postgres ON CONFLICT DO UPDATE), not GET-then-INSERT/UPDATE.
33	How does GET respect TTL?	WHERE key = k AND expired_at > NOW(). Expired rows are filtered at read time.
34	Why add LIMIT 1000 to the cleanup DELETE?	Avoid huge transactions that hold many locks, bloat logs, lag replicas and stall live traffic.
35	When are read replicas appropriate?	Read-heavy load where slightly stale reads are fine; send consistent reads to the master.
36	Master can't handle writes: what next?	Scale vertically first; then shard (range or hash) so each master owns an exclusive slice of keys.
37	Do FOR UPDATE locks block a plain SELECT in InnoDB/Postgres?	No. Plain reads use MVCC snapshots; only locking reads wait.

02-02 · NoSQL + Slack realtime

#	Question	Answer
38	What do most NoSQL DBs trade for scalability and availability?	Consistency (most are eventually consistent).
39	Does "NoSQL scales" imply "SQL doesn't"?	No. Anything you can shard scales, relational DBs included.
40	Document vs key-value store?	Document: JSON, complex queries, partial updates (MongoDB). KV: key-wise access, heavily partitioned, no complex queries (Redis, DynamoDB).
41	Why are column stores used for analytics?	They read only the columns a query touches, not whole rows.
42	Wide-column store in one line?	Columns grouped into column families stored together on disk across rows (Cassandra).
43	Slack insight 1?	A DM is just a channel with two members.
44	How are Slack messages sharded and why?	By workspace, so all of a workspace's history is on one shard (no cross-shard scrolling).
45	What does membership.checkpoint store and why?	Last message id read; the client fetches anything newer, which recovers missed realtime messages.
46	Persistence-first vs realtime-first chat examples?	Slack persists first; WhatsApp delivers realtime with async persistence; Zoom chat has no persistence.
47	Why multiplex everything over one WebSocket per user?	WebSockets are expensive persistent TCP connections and browsers limit them.
48	How do edge servers deliver across servers?	Each subscribes to Redis Pub/Sub channels (one per Slack channel) for its users; publishes fan out to every subscribed edge.

#	Question	Answer
49	What does a TCP connection's identity consist of?	Source IP, source port, destination IP, destination port, protocol.

03-01 · Load balancer, remote & distributed locks

#	Question	Answer
50	Why do LB servers keep config in memory?	A config-DB call per request would wreck latency; the in-memory copy is kept in sync.
51	How does config reach LB servers quickly, and what if that path fails?	Config DB → CDC → Redis Pub/Sub push; if Pub/Sub is down, LBs poll.
52	What does the LB orchestrator do?	Health-checks backends and LB servers, updates the backend list, scales LB servers using metrics from Prometheus.
53	How is the orchestrator itself kept alive?	Leader election: a master assigns exclusive work to workers; a new master is elected on failure.
54	How do you scale the LB tier itself?	Weighted DNS (e.g. CoreDNS) resolving one name to many LB server IPs.
55	How do two DNS servers provide HA without client changes?	They share a Virtual IP via VRRP; the secondary takes over the VIP (ARP) when the primary fails.
56	How do threads, processes and machines synchronise?	Mutex/semaphore; disk (lock files); remote lock manager.
57	Two requirements for a remote lock manager?	Atomic acquire and automatic expiry (TTL).
58	Redis command to acquire a lock with expiry?	SET key owner NX EX ttl (set-if-not-exists with TTL).
59	Why release a Redis lock via a Lua script?	Owner check + delete must be atomic so you never delete someone else's re-acquired lock.
60	How does Redlock acquire a lock?	SETNX on N independent Redis masters; success if a majority (e.g. 3 of 5) is acquired, else release all.
61	What does Redlock buy you and what does it cost?	No single point of failure and better correctness; lower throughput (consensus is slow).
62	Why do TTL-based locks need fencing tokens?	A paused holder can outlive its TTL while another client acquires the lock; monotonically increasing tokens let the resource reject stale writers.

03-02 · ID generation

#	Question	Answer
63	Why can't sharded DBs rely on auto-increment IDs?	Each independent shard's auto-increment collides with the others; the app must supply globally unique IDs.

#	Question	Answer
64	Why does epoch_ms alone fail as an ID?	Two machines (or two calls) in the same millisecond produce the same value.
65	How do you persist an ID counter cheaply and recover safely?	Flush every N increments; on restart begin at last_flushed + N (+1), skipping but never reusing IDs.
66	Why do ID schemes put time in the most significant bits?	Numeric order matches time order (sortable); lower bits only break ties.
67	What breaks monotonicity across machines?	Clock skew between machines.
68	Core lesson about distributed ID generation?	You can't distribute it and guarantee strict monotonicity, so real systems relax a constraint.
69	Amazon-style central ID service optimisation?	Hand out ID batches (ranges) per call; servers re-ask when exhausted.
70	Why avoid UUIDs as primary keys at scale?	128-bit random keys bloat indexes and scatter inserts; indexes stop fitting in memory.
71	How do Flickr ticket servers work and avoid SPOF?	MySQL auto-increment table bumped via upsert; two servers issue odd and even IDs.
72	INSERT ... ON DUPLICATE KEY UPDATE vs REPLACE INTO?	ODKU updates in place; REPLACE deletes then inserts (much slower). Both atomic.
73	Snowflake bit layout?	41 bits epoch ms, 10 bits machine id, 12 bits per-machine sequence (plus a 0 sign bit).
74	Instagram's ID layout and where it's generated?	41 bits ms since 2011, 13 bits logical shard, 10 bits sequence; a Postgres function at INSERT.
75	Why is since_id pagination efficient?	It seeks straight into the index (WHERE id > x LIMIT k) instead of skipping OFFSET rows.

04-01 · CDN, photo uploads, Gravatar

#	Question	Answer
76	How does a CDN serve a request?	Its domain maps to your origin; on a miss it fetches from the origin, caches the response and returns it.
77	Why shouldn't photo uploads go through the API server?	It doubles bandwidth and wastes API resources just relaying bytes.
78	How does a client upload to S3 without your servers relaying?	The service creates a pre-signed URL for an exact path; the client PUTs directly to S3.
79	Why store image_id instead of the full image URL on a post?	The S3 path is computable from user_id + image_id; saves space and lets the service validate ownership.
80	How are Instagram's private photos protected?	Short-lived signed CDN URLs; the CDN rejects invalid or expired signatures.
81	Why can a service verify a JWT without the auth service?	It checks the signature with the issuer's public key (asymmetric, e.g. RS256).

#	Question	Answer
82	When should you reach for public-key cryptography?	When system A must trust system B's statement without calling B.
83	How do you avoid sending huge photos to every device?	CDN on-the-fly transformation (e.g. ?w=360), with cached variants.
84	What is the Gravatar URL keyed by?	A hash of the user's email.
85	Two ways to model "one active photo per user"?	is_active flag flipped in one transaction (with a unique partial index), or active_photo_id denormalised on users.

04-02 · Hashtag service + unread indicator

#	Question	Answer
86	How does the hashtag page achieve super-fast reads?	One pre-computed document per tag (count + top posts), served by key from a partitioned DB, cache and CDN.
87	Post ids vs full post details in the hashtag doc?	Ids: small but N+1 enrichment on read. Full details: one fast read, more storage, risk of staleness.
88	What's the value of splitting read and write paths?	Each can be optimised independently: cache + thin API on reads; Kafka + batching on writes.
89	How do hashtag workers count efficiently?	Aggregate counts in memory and flush every N events or time window with one increment per tag.
90	How do you minimise stop-the-world during a flush?	Swap the active map under the lock (O(1)) and write the old map to the DB outside the lock.
91	Why add a hashtag-extraction adapter?	POST_PUBLISH is keyed by post; re-emitting per-hashtag events keyed by tag lets one consumer own each tag's counts.
92	What does the newly-unread badge count?	The number of distinct people with unread messages, not the number of messages.
93	Best index for counting distinct unread senders?	Covering composite (to, ts, from): equality, then range, then the read column.
94	Day-N design for the unread indicator?	ON_MSG_UNSENT events → workers SADD(dest, src) into a per-user Redis set; the badge is the set size; clear on read.
95	How is "unread" detected in the pre-computed approach?	The messaging service knows via WebSocket presence that delivery failed and emits ON_MSG_UNSENT.

05-01 · Distributed cache

#	Question	Answer
96	Core data structure of a cache server?	A hash table mapping key → typed object (type decides allowed operations).

#	Question	Answer
97	HTTP vs raw TCP (RESP) for a cache?	HTTP: easy tooling but heavy per-request overhead. RESP over TCP: minimal overhead, needs custom clients.
98	How does a cache know it's full?	Cap on key count, or track total bytes via an allocation wrapper (zmalloc/zfree).
99	LRU vs LFU: typical workloads?	LRU for recency-driven traffic (news, CDN); LFU for stable popularity (Wikipedia).
100	Why does Redis approximate LRU?	An exact doubly linked list costs memory and CPU; sampling keys into a small eviction pool is close enough.
101	Three ways to expire TTL keys?	Priority queue by expiry; lazy deletion on access; random sampling (Redis: 20 keys, repeat if > 25% expired).
102	How does single-threaded Redis run background work?	Time events in its event loop (serverCron, activeExpireCycle) interleaved with I/O.
103	How do multi-threaded caches (DiceDB) avoid lock contention?	Shared-nothing: one hash-table shard per core owned by a pinned data thread; I/O threads route by hash.
104	Problem with hash(key) % N routing?	Changing N remaps most keys ($\approx N/(N+1)$ of them).
105	How does consistent hashing assign keys?	Nodes and keys hash onto a ring; each key goes to the next node clockwise.
106	Steps to add a cache node with minimal transfer?	Snapshot the neighbour, load, replicate until lag ≈ 0 , brief pause, update ring, resume, delete moved data.
107	What problem do virtual nodes solve?	Skewed ring arcs: many positions per node spread load evenly.

05-02 · Dictionary without DB, Bitcask, WAL recovery

#	Question	Answer
108	How do you look up a word in a 1 TB file quickly without a DB?	Load a small word $\rightarrow$ (offset, length) index into memory and read only that byte range from S3.
109	Rough size of the dictionary index?	About 171k entries $\times \sim 16$ B ≈ 2.7 MB (more with 8-byte offsets, still tiny).
110	How is the weekly changelog applied efficiently?	Both are sorted, so merge them in $O(n)$ like merge sort, then rebuild the index.
111	How do you switch dictionary versions without garbage reads?	Upload to a new versioned path and flip a meta.json pointer; never overwrite files in place.
112	How do you store index and data in one portable file?	A fixed-length header holding the index and data offsets (plus metadata), followed by the index, then the data.
113	Why are log-structured stores fast for writes?	Append-only sequential writes avoid disk seeks.
114	How does an append-only store delete a key?	Append a tombstone (e.g. PUT(k, -1)); compaction removes it later.
115	Layout of a Bitcask entry?	CRC, timestamp, key size, value size, key, value.

#	Question	Answer
116	How does Bitcask achieve O(1) GET?	An in-memory hash index maps key → (file id, value position, size); one seek reads the value.
117	Bitcask's main limitation?	All keys must fit in memory.
118	What does merge/compaction do in Bitcask?	Rewrites immutable files keeping only each key's latest live value and updates the index atomically.
119	How do databases recover a torn write at the end of the log?	Scan entries verifying CRCs and truncate at the first mismatch.

06-01 · LSM trees, tiered storage, event ingestion

#	Question	Answer
120	Core idea of an LSM tree over Bitcask?	Buffer writes in an in-memory memtable (plus a WAL), flush periodically to immutable sorted files, so keys needn't fit in RAM.
121	What is an SSTable?	An immutable file of key-sorted entries with an index of key → offset.
122	Order of lookup for GET in an LSM?	Memtable, then SSTables newest to oldest; first hit wins.
123	Why flush the memtable to a new file each time?	Faster, one-shot sequential writes; avoids rewriting a large existing file.
124	How do Bloom filters speed up LSM reads?	Each SSTable's filter says "definitely not here" so the file is skipped; "maybe" means check it.
125	Bloom filter error types?	False positives possible, false negatives impossible.
126	Purpose of the WAL in an LSM?	Durability for memtable contents; replayed on restart and truncated after each flush.
127	B+ tree vs LSM workload fit?	B+ trees for read-heavy; LSM for write-heavy.
128	Name LSM-based storage engines.	RocksDB, LevelDB, BadgerDB (also Cassandra).
129	Idea behind multi-tiered order storage?	Move orders to cheaper tiers by age (hot MySQL → S3) while keeping them queryable via indexes.
130	How is cold data on S3 kept queryable?	Indexed file layout (index + data) served by index servers or Athena/Hive-style engines.
131	How do you ingest huge event volumes cheaply?	Write to rotating local log files and ship them as batches to Kafka/S3, not one sync write per event.
132	Why avoid video_id as the Kafka partition key for impressions?	A viral video would create a hot partition; spread events evenly.

06-02 · Live streaming + Designing S3

#	Question	Answer
133	Master vs playlist m3u8?	Master lists renditions (bandwidth/resolution → playlist) and doesn't change; a playlist lists .ts segments and grows by appends during live.
134	How does a player follow a live HLS stream?	It periodically re-fetches the playlist and plays newly appended segments.
135	SSAI vs CSAI?	SSAI stitches ad segments into the manifest server-side (seamless, hard to block); CSAI plays ads in the client on cues (glitchy, blockable).
136	Components of SSAI?	Origin with SCTE-35 markers, a manifest manipulator, an ad decision server (VAST), and the CDN.
137	How are private live streams protected at the CDN?	Signed tokens/certificates validated by the CDN per request.
138	Why use multiple CDNs?	Backup and performance: continuously measure and shift traffic in real time.
139	S3's four design goals?	Strong consistency, global namespace, multi-tenancy, cost efficiency.
140	What is a storage stamp?	A cluster of 10–20 racks (~30–50 PB) run at ≤ 70% utilisation, with headroom for rack failures.
141	What does the location service do?	Maps accounts/namespaces to stamps (via DNS) and manages cross-stamp replication.
142	Three layers inside a stamp?	Front-end (auth + routing + streaming), Partition (range ownership, throttling), Stream (replicated append-only DFS).
143	Why range partitioning in S3?	Keeps object locality and tenant isolation, and lets hot ranges be split.
144	Partition ownership rule?	Each partition is owned by exactly one partition server; a server can own many partitions.
145	How does S3-style storage get durability?	Sync replicas within the stamp, async replicas across data centres and regions.

07-01 · Recent searches, Cricbuzz, ES in production

#	Question	Answer
146	Which user behaviours shape the recent-searches design?	50% tap the search bar within 5 s (read-heavy, must be preloaded); 30% of searches come from recent searches (write-heavy).
147	How are recent searches stored for fast reads?	A pre-computed list of the last 10 queries per user in Redis, written synchronously.

#	Question	Answer
148	Where does the full search history go?	Async via Kafka into a partitioned NoSQL DB (search logs), archived to S3 after ~6 months.
149	How do you avoid a slow first load when a user's Redis entry was evicted?	Pre-warm recent searches into Redis when the app opens.
150	Why short polling instead of WebSockets for Cricbuzz commentary?	About one update per minute and identical data for everyone; cached polling is cheaper than millions of idle sockets.
151	How is the latest commentary served cheaply?	Redis holds the latest per match, or a CDN with max-age ≈ 5 s and prefetching near expiry.
152	How is commentary stored?	Ball-wise rows (match_id, innings, ball, text), upserted per ball.
153	Pain points of Day-0 Elasticsearch?	Painful recovery, limits on scale and experimentation, hard debugging, no auto replication or latency routing, costly big tenants, no throttling.
154	Key pieces of the revamped search platform?	Indexers to primary + backup-region ES, a search gateway, identity (JWT) and relevance re-ranking, multi-region failover.
155	ES routing strategy by tenant size?	Small tenant $\rightarrow$ one shard (tenant_id % shards); large tenant $\rightarrow$ spread over N shards (tenant_id + doc_id % N).
156	What problem does tenant-aware routing solve?	Hot shards and noisy neighbours from uneven tenant sizes.

07-02 · Task scheduler + flash sale

#	Question	Answer
157	Requirements of the distributed task scheduler?	One-off or cron tasks executed (as HTTP calls) within a 30 s SLA, minute granularity, multi-tenant, ~2B tasks/day.
158	Why must every scheduler component be horizontally scalable and fault tolerant?	The strict SLA at extreme scale leaves no room for bottlenecks or single points of failure.
159	How is the task store partitioned?	By tenant (customer or microservice).
160	What do task pullers do and how are they coordinated?	Pull due tasks and enqueue them; each owns a tenant range assigned via ZooKeeper, reassigned if it dies.
161	SQL used to pull due tasks concurrently?	WHERE scheduled_at $\leq$ now AND picked_at IS NULL ORDER BY scheduled_at LIMIT n FOR UPDATE SKIP LOCKED, then mark picked.
162	How are executors organised and scaled?	Separate queues/fleets by priority and task type; an orchestrator autoscales on queue length.
163	How are recurring (cron) tasks handled?	Expand the cron into the next N absolute run times inserted as jobs (idempotently), topping up on each pick.

#	Question	Answer
164	Why model flash-sale stock as one row per unit?	So concurrent buyers lock different rows (SKIP LOCKED) instead of all contending on one qty row.
165	What happens on payment failure in the flash sale?	The unit is released: picked_at and picked_by are reset to NULL.
166	How are abandoned carts handled?	A CRON job releases units picked more than ~12 min ago and not purchased.
167	Why can't add-to-cart and payment be one transaction?	They span services and time; there's no distributed transaction, so use steps with compensation.

08-01 · Impression counting + Dropbox sync

#	Question	Answer
168	Why can't per-minute unique counts be summed?	Users repeat across minutes; you need a set union, not a sum.
169	Why are exact hash sets impractical for impression counting at scale?	Each ad-minute can take MBs; hourly unions across thousands of customers run into TBs of processing.
170	What data structure estimates unique counts cheaply?	HyperLogLog (~12 KB per key, mergeable, approximate).
171	Redis HLL commands?	PFADD (add, O(1)), PFCOUNT (cardinality), PFMERGE (union of N HLLs).
172	Main limitation of HLL besides approximation?	You can't delete elements.
173	How do you answer unique viewers for an arbitrary range?	PFMERGE the per-minute HLLs for that range into a temp key, then PFCOUNT.
174	How does the impressions design keep Redis small?	Recent HLLs stay in Redis; bytes are copied periodically to DynamoDB and reloaded on demand.
175	Why archive raw view events to S3?	Replayability: if filter rules had a bug, events can be reprocessed.
176	How does Dropbox represent a file?	As a blocklist: hashes of 4 MB blocks, with blocks stored content-addressed.
177	Dropbox upload protocol?	Commit blocklist → metaserver replies with missing hashes → client uploads them to the block server → re-commit.
178	How do clients learn about changes?	A per-namespace incrementing version id; clients fetch everything after their last version, like a Kafka offset.
179	How does multi-versioning fall out of the design?	Old metadata rows keep old blocklists; the blocks still exist, so any version can be rebuilt.

08-02 · Geo-proximity, ride hailing, follow graph

#	Question	Answer
180	Why is geo-proximity hard with plain (x, y)?	It needs a 2D range query (intersecting two 1D ranges), $O(n)$ naively; 1D ranges are easy.
181	How does GeoHash encode a location?	Repeatedly halve the longitude and latitude ranges into bits, interleave them, and encode in base 32.
182	What does a shared GeoHash prefix mean?	The points fall in the same cell, so they're likely near each other (longer prefix = smaller cell).
183	GeoHash boundary edge case and fix?	Nearby points across a cell edge get different prefixes; also search the 8 neighbouring cells.
184	Why Redis for ride-hailing driver locations?	In-memory speed, built-in geo queries, multi-master multi-replica sharding.
185	Why did Gojek need EVAL_RO?	Redis rejects EVAL on replicas (scripts could write); a read-only variant lets filtering scripts run on replicas.
186	How does the matcher keep latency low?	Split the area into smaller GEOSEARCH queries run in parallel; return partial results if the SLA is breached.
187	How were hot shards avoided for driver locations?	Manually mix peak and low-traffic regions on each master (Redis sharding has no business context).
188	Why not a graph DB for follow/following?	No advanced graph algorithms needed; costly, hard to operate, and pagination is tricky.
189	Problem with sharding an edges(src, dest) table by src?	"Followers of B" filters on dest, so it must fan out across all shards.
190	How do you serve both follower and following queries from one shard?	Store two rows per follow, (A, B, FOLLOWS) and (B, A, FOLLOWED_BY), and shard by src.
191	FlockDB primary key and unique index?	PK (source_id, state, position); unique (source_id, destination_id, state).
192	How does FlockDB paginate efficiently?	Cursor on position (WHERE position > last ORDER BY position LIMIT k), not OFFSET.